Crash course: full guide

Reader: Rajju. Device: iPad, dark mode. Source of facts: the repo study guide. Points marked Not in source are not confirmed in that guide.

Blue text: a key fact to recall.

Green text: a decision rule. Apply it to the question.

Purple text: an exam tip about question wording.

Red text: a trap. A wrong answer that looks right.

Answer first

Choose each component from the constraints. Bedrock gives managed foundation models. SageMaker AI hosts and customizes models you own. Native AWS services such as Lambda, API Gateway, Step Functions, RDS, Aurora, DynamoDB, and S3 carry the request, hold the state, and store the evidence.

Decision rule: pick the model path first. Then pick the native services around it.

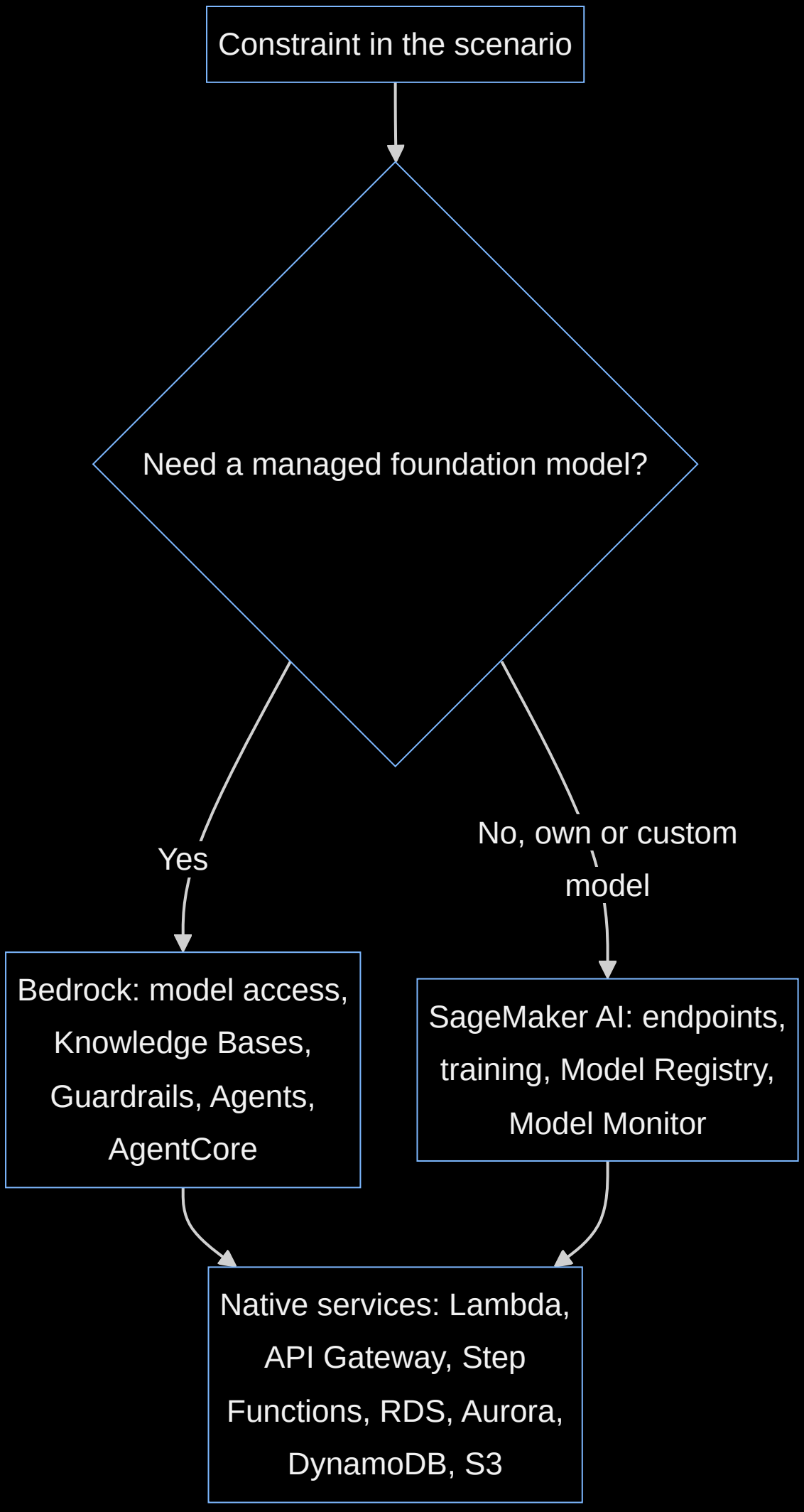

Constraint in the scenario
Need a managed foundation model?
Yes
No, own or custom model
Bedrock: model access, Knowledge Bases, Guardrails, Agents, AgentCore
SageMaker AI: endpoints, training, Model Registry, Model Monitor
Native services: Lambda, API Gateway, Step Functions, RDS, Aurora, DynamoDB, S3

Figure: the first split between Bedrock, SageMaker AI, and native services.

Amazon Bedrock model access

This part covers how a request reaches a model on Amazon Bedrock. It covers call APIs, capacity modes, routing, caching, logging, customization, model choice, errors, and private network paths. Entries marked Not in source are not confirmed in the repo guide. Figures are schematic.

Converse API

Converse sends chat and tool use through one request format for supported models.

- Methods or APIs: Converse for a full response. Converse needs permission for bedrock:InvokeModel. It runs on the bedrock-runtime endpoint. A request can carry inferenceConfig and additionalModelRequestFields.
- Pro: One request shape for many models keeps app code stable when the model changes. Tool definitions and guardrail settings sit in the same request.
- Con: Converse does not produce embeddings. A prompt from Prompt management cannot include additionalModelRequestFields in the request.
- Choose it when: the app needs chat, tool use, or multimodal input on a supported model.
- Do not choose it when: the call must return a vector. Use InvokeModel with an embedding model.
- Native pairing: Lambda or API Gateway calls Converse. The response trace object shows guardrail results when a guardrail runs.

Decision rule: Use Converse for chat and tool use. Use InvokeModel for embeddings and provider-specific bodies.

Trap: Converse does not produce embeddings. Send an embedding call to InvokeModel.

Exam tip: A scenario that names chat or tool use across models points to Converse. A scenario that names vectors points to InvokeModel.

Source: repo guide, Part 1A component table, Part 1B Lesson 3, Part 2 call methods table, Part 8A8 Converse unit.

ConverseStream

ConverseStream sends tokens as they arrive, so the client reads a stream.

- Methods or APIs: ConverseStream. The client must read the stream. The repo guide says it needs its own streaming permission. The permission name is Not in source.
- Pro: The first visible token appears sooner. Users see progress on long replies.
- Con: Total generation time does not change. A REST integration still stops at the 29-second default. The 29-second value is Not in source on the quotas page, so confirm it.
- Choose it when: a person reads the reply as it generates, and the path to the browser holds a stream.
- Do not choose it when: the caller cannot read a stream, or a batch job collects the output.
- Native pairing: WebSocket APIs or server-sent events carry the stream to the browser. A standard Lambda function runs at most 15 minutes per invocation.

Decision rule: Stream when a person watches the reply. Use WebSocket APIs or server-sent events for long streams.

Trap: A stream does not extend a REST timeout. A higher Lambda timeout does not fix a gateway timeout either.

Exam tip: A stream shortens the wait for the first token. A scenario that names a total time limit needs a different path.

Source: repo guide, Part 1B Lesson 2, Part 3 section 2.4, Part 1E visual units 11 and 32. Permission name: Not in source.

InvokeModel

InvokeModel sends the request body that one model provider expects.

- Methods or APIs: InvokeModel. A Claude body needs anthropic_version and max_tokens. Embedding calls always use InvokeModel with an embedding model.
- Pro: It reaches provider-specific fields. It is the named path for embeddings.
- Con: The app builds each provider body. A Titan body sent to a Claude model fails.
- Choose it when: the call makes an embedding, or a provider-specific field must go in the body.
- Do not choose it when: a chat or tool call can run on Converse for the same model.
- Native pairing: CloudTrail records InvokeModel calls for audit. Invocation logs record the request and the response. The repo guide names Titan Text Embeddings v2 and Cohere Embed as embedding models.

Decision rule: Use InvokeModel for vectors and provider-specific bodies.

Trap: With InvokeModel, tag user input for prompt attack checks. Untagged input is not checked for prompt attacks.

Exam tip: A body error after a model change points first to the body fields of that provider.

Source: repo guide, Part 1E visual unit 1, Part 2 call methods table, Part 3 section 2.3, Part 5 troubleshooting table, Part 8D guardrail unit.

InvokeModelWithResponseStream

InvokeModelWithResponseStream sends tokens as they arrive from an InvokeModel call.

- Methods or APIs: InvokeModelWithResponseStream. Invocation logging covers it. Prompt attack checks need tagged user input on it.
- Pro: Tokens reach the client early. It suits provider bodies that must stream.

- Con: The client must read the stream. Total generation time does not change. Stream parsing for each provider: Not in source.
- Choose it when: the client reads tokens as they arrive, and the provider body needs streaming.
- Do not choose it when: the app needs only the final result. Use InvokeModel.
- Native pairing: WebSocket APIs or server-sent events carry the stream to the browser. Guardrail input tags apply on this call path.

Decision rule: Stream only when the client reads tokens. Otherwise call InvokeModel.

Trap: A stream does not get past a REST gateway limit. Use a path that holds the stream open.

Exam tip: A stream is a perceived speed fix. It is not a total time fix.

Source: repo guide, Part 3 section 2.4, Part 1B Lesson 2, Part 8D guardrail unit. Stream parsing: Not in source.

CountTokens

CountTokens estimates the token count for a supported model before a call.

- Methods or APIs: CountTokens. The repo guide pairs it with Converse in its service tie-in. Request and response fields: Not in source.
- Pro: It checks the input size against the context window before the call. It helps avoid overflow.
- Con: It gives an estimate, and it works for supported models only. Exact field names: Not in source.
- Choose it when: a long input may fill the context window or leave no room for the output.
- Do not choose it when: the target model is not a supported model.
- Native pairing: Design guidance. A Lambda step counts tokens, compares the count with the model limit, then calls Converse.

Decision rule: Count tokens before the call when the input is near the limit.

Trap: Context overflow drops the end of a long input. The output then looks cut off or ignores the last section.

Exam tip: A cut-off output points to the token limit. Check the count before you change the model.

Source: repo guide, Part 1B Lesson 1, Part 1D tie-in, Part 5 section 4.1, Part 5 troubleshooting table. Field names: Not in source.

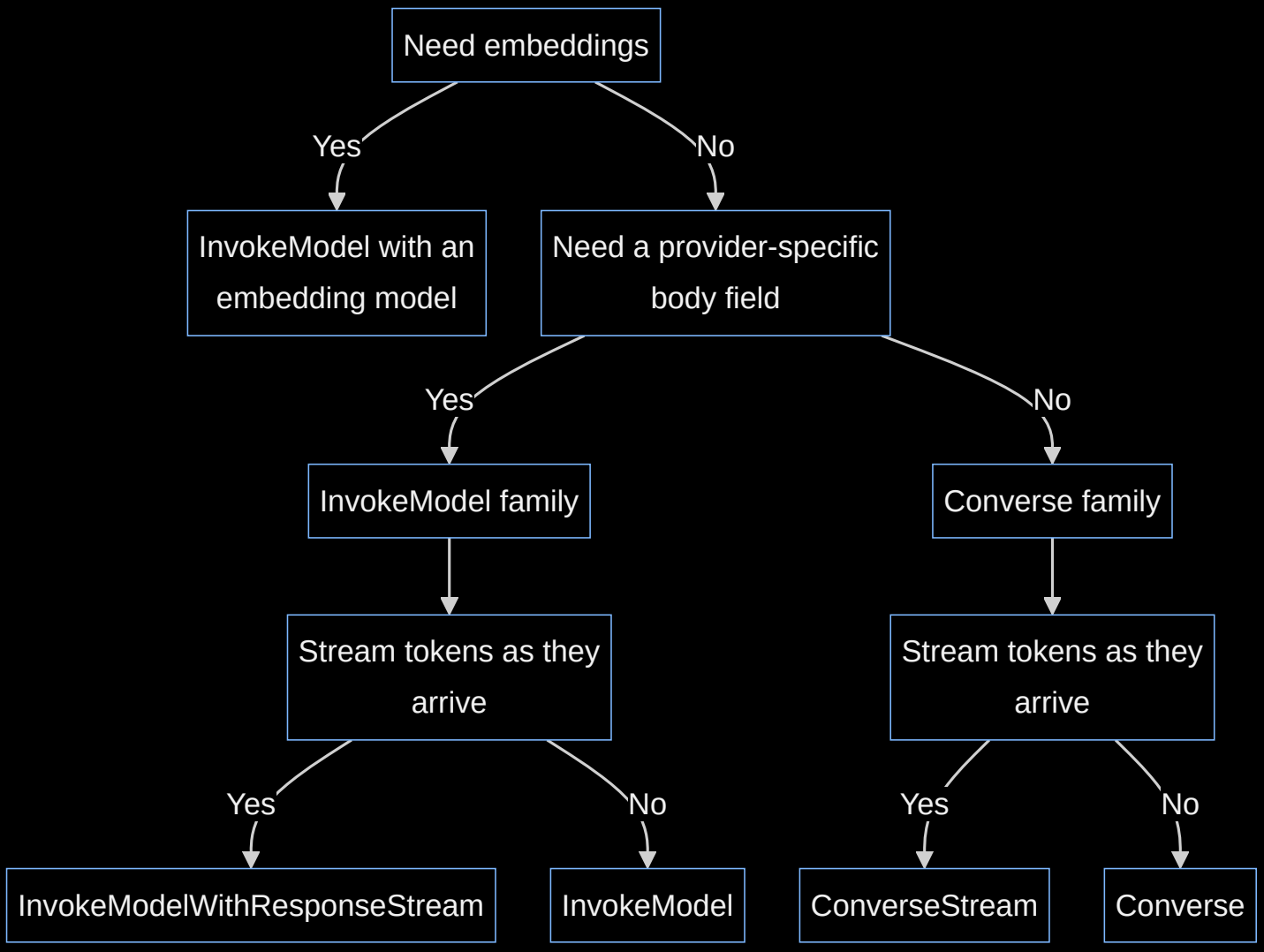


Figure: Converse or InvokeModel. Embeddings always take InvokeModel. Choose the stream variant of the chosen family when tokens must stream.

On-demand inference

On-demand inference charges per token and has no commitment.

- Methods or APIs: Calls to the base model ID use on-demand capacity.
- Pro: No commitment. Spiky or unknown traffic fits. It is the first mode to use in a proof of concept.

- Con: Quotas cap tokens per minute (TPM) and tokens per day (TPD). Throttled calls need retries. The throttle metric is InvocationThrottles.
- Choose it when: traffic is spiky or unknown, or the approach is still under proof.
- Do not choose it when: a steady, measured baseline must meet a latency target. Use Provisioned Throughput for that baseline.
- Native pairing: Lambda calls the model ID. Step Functions holds long flows, and each Lambda step stays short.

Decision rule: Start on demand. Buy reserved capacity only after the baseline is measured.

Trap: Reserved capacity bought before a proof of concept is a poor pattern. Prove the approach on real data first.

Exam tip: Spiky or unknown load points to on-demand. A steady, measured baseline points to reserved capacity.

Source: repo guide, Part 1B Lesson 2, Part 1E visual unit 2, Part 2 section 1.1, Part 3 section 2.2, Part 5 section 4.1.

Provisioned Throughput

Provisioned Throughput reserves model units for a steady rate, and it bills hourly.

- Methods or APIs: CreateProvisionedModelThroughput returns a provisionedModelArn. GetProvisionedModelThroughput shows the status until it reads InService. The provisioning mode is By Model Units. The purchase covers a base model or a custom model.
- Commitment terms: none, 1 month, or 6 months. A commitment needs an AWS support request for model units first.
- Model units: The count is set at purchase. Throughput per model unit: Not in source.
- Pro: Reserved capacity suits a steady baseline. It does not speed up one call.

- Con: A commitment does not fit spiky traffic. Batch inference does not support provisioned models. A capacity purchase does not change the speed of one call.
- Choose it when: traffic is steady, measured, and has a latency target.
- Do not choose it when: traffic is spiky, or the baseline is not yet measured.
- Native pairing: The app must call the provisioned model ARN. Mixed traffic uses on-demand for spikes. SageMaker AI endpoints host models you own, which is a different host.

Purchase steps

- Open the Bedrock console and choose Provisioned Throughput.
- Choose Purchase Provisioned Throughput, then select the base or custom model.
- Set the provisioning mode to By Model Units, then pick the commitment term.
- Enter the model units, confirm the purchase, and wait for In service.

Provisioned model ARN

Calls must use the provisioned model ARN. A call to the base model ID still uses on-demand capacity. The repo guide flags this as a common cause of throttling after a purchase.

Decision rule: Reserve model units only for a measured baseline. Send baseline calls to the provisioned model ARN.

Trap: More model units do not help a call that still uses the base model ID.

Exam tip: Throttled calls after a capacity purchase point to the model ID in the call. Check that ID first.

Source: repo guide, Part 3 section 2.2, Part 8B steps, Part 8D decision ladder. Throughput per model unit: Not in source.

Batch inference

Batch inference runs offline jobs with the input and output in S3.

- Methods or APIs: The create model invocation job operation creates the job. The API route is Not in source. The console asks for a service role, an S3 input location, and an S3 output location. The job time limit is a timeout duration in hours.
- S3 input: One or more JSONL files in S3. Each line is one record with a recordId and a modelInput. If recordId is missing, Bedrock adds one to the output.
- Record format: modelInput matches the InvokeModel body by default. With type Converse, it matches the Converse request body.
- S3 output: The results go to the S3 output location. Output order is not guaranteed to match input order.
- Limits: Quotas apply to records per file, records per job, and file size. The numeric values: Not in source.
- Pro: A lower price than on-demand for select models. Jobs run without a live client. The discount amount: Not in source.
- Con: Batch does not support Provisioned Throughput models. Batch with Converse does not support tool calling. The cache does not apply to batch.
- Choose it when: the work can wait for hours and the input sits in S3.
- Do not choose it when: a user waits for a chat reply.
- Native pairing: S3 holds the input and the output. For tool calls in batch, use the InvokeModel type.

Batch job steps

- Choose a service role for batch inference, or create a custom role.
- Create the job and choose the model.
- Point the job at an S3 input location and an S3 output location.

Decision rule: Use batch for offline work with S3 input. Use on-demand for any reply a user waits for.

Trap: Batch output order can differ from input order. Join each result to its input on recordId, not on position.

Exam tip: A user who waits for a reply needs real-time calls. Batch is wrong for that case.

Source: repo guide, Part 1F practical unit 4, Part 8A3 batch input unit, Part 5 section 4.1, Part 8C glossary.

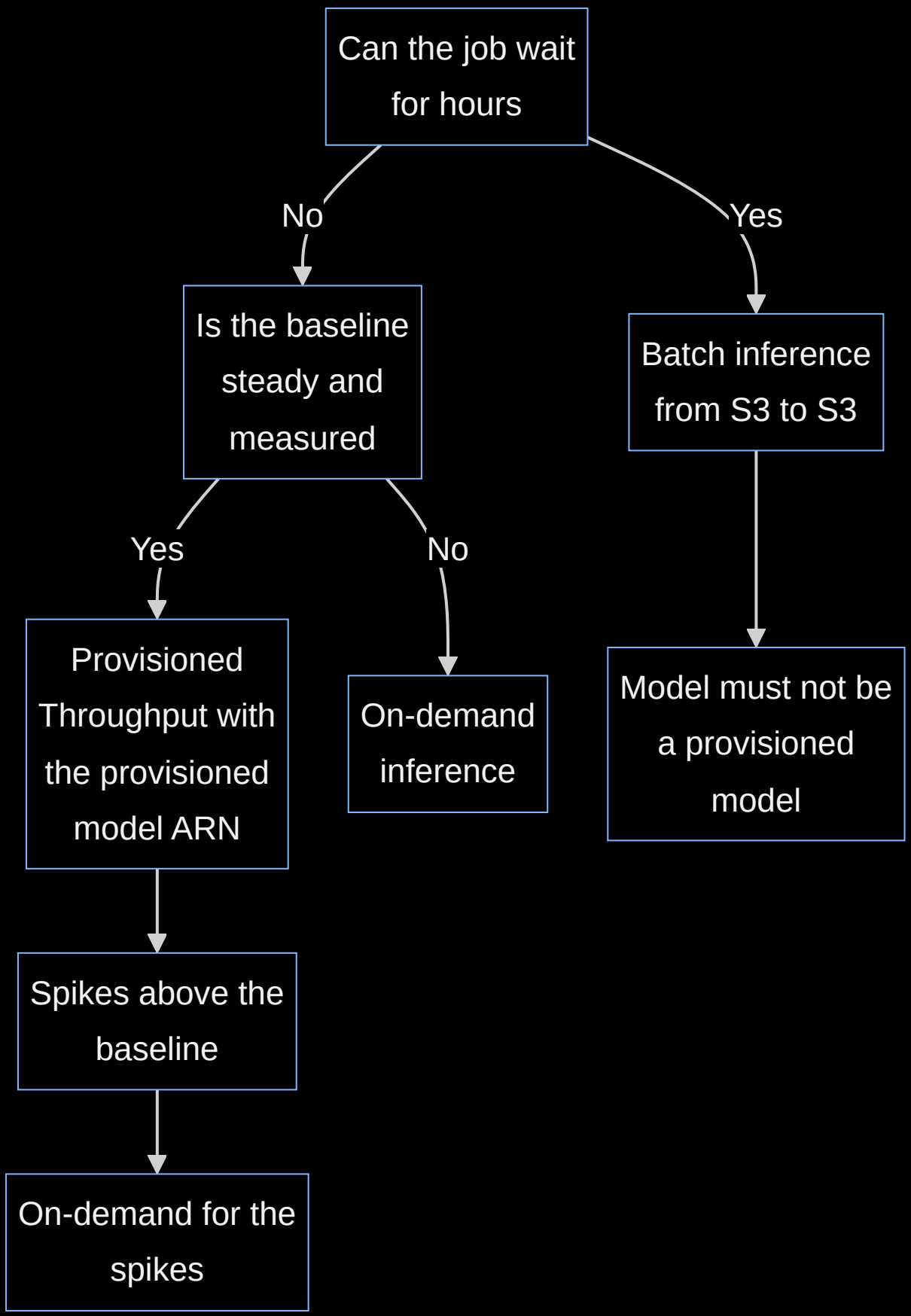


Figure: On-demand, Provisioned Throughput, or batch. A user who waits for a reply always takes a real-time path.

Cross-Region inference profiles

A cross-Region inference profile ID names a set of Regions. A request that uses it can run in any Region in that set.

- Methods or APIs: Call the profile ID, such as one with a us. or eu. prefix. A geographic profile routes inside its geography, such as the US or the EU. A global profile routes to any supported commercial Region.
- Pro: More throughput and availability within the same model family. Data stays on the AWS network and is encrypted in transit. The repo guide cites about 10 percent savings for global profiles.
- Con: Requests can run in another Region in the profile. A global profile can route outside a strict EU boundary. The profile does not give automatic failover for your app. The quota Region for a routed request: Not in source.
- Choose it when: the app needs more throughput or availability, and the residency rule allows the Regions in the profile.
- Do not choose it when: a strict single-Region rule applies. Cross-Region guardrail inference also routes to another Region in the same geography, so check that rule too.
- Native pairing: Retries and fallback in the application. CloudTrail logs the request in the source Region with additionalEventData.inferenceRegion.

Decision rule: A rule that names a geography needs a geographic profile. A rule that allows any commercial Region can use a global profile.

Trap: Cross-Region inference is not automatic failover. Add retries and fallback in the application.

Exam tip: An EU residency rule rules out a global profile. Pick the geographic EU profile.

Source: repo guide, Part 1E visual unit 3, Part 2 routing table, Part 4 section 3.2, Part 8A4 routing unit. Quota Region: Not in source.

Intelligent prompt routing

Intelligent prompt routing picks one of two models in one model family for each request, by predicted response quality and cost.

- Methods or APIs: The call shape for the router: Not in source. The repo guide names it as a Bedrock feature.
- Pro: Routes by predicted response quality and cost. Your code does not write the rule.
- Con: The official docs describe it as optimized for English prompts only. It chooses between two models, not more.
- Choose it when: the prompts are English, and both candidate models sit in one family.
- Do not choose it when: the prompts are not English, or your own routing policy is required.
- Native pairing: Lambda routing with AppConfig when you need your own rules and a model ID read at runtime.

Decision rule: Use intelligent routing for English prompts across two models in one family. Write your own routing rules for anything else.

Trap: A cross-Region profile does not choose a model by prompt complexity. Intelligent routing does that job within one model family.

Exam tip: Cost by predicted quality points to intelligent routing. Throughput and availability point to a profile.

Source: repo guide, Part 2 routing table, Part 5 section 4.1, Part 5 visual unit 54. Call shape: Not in source.

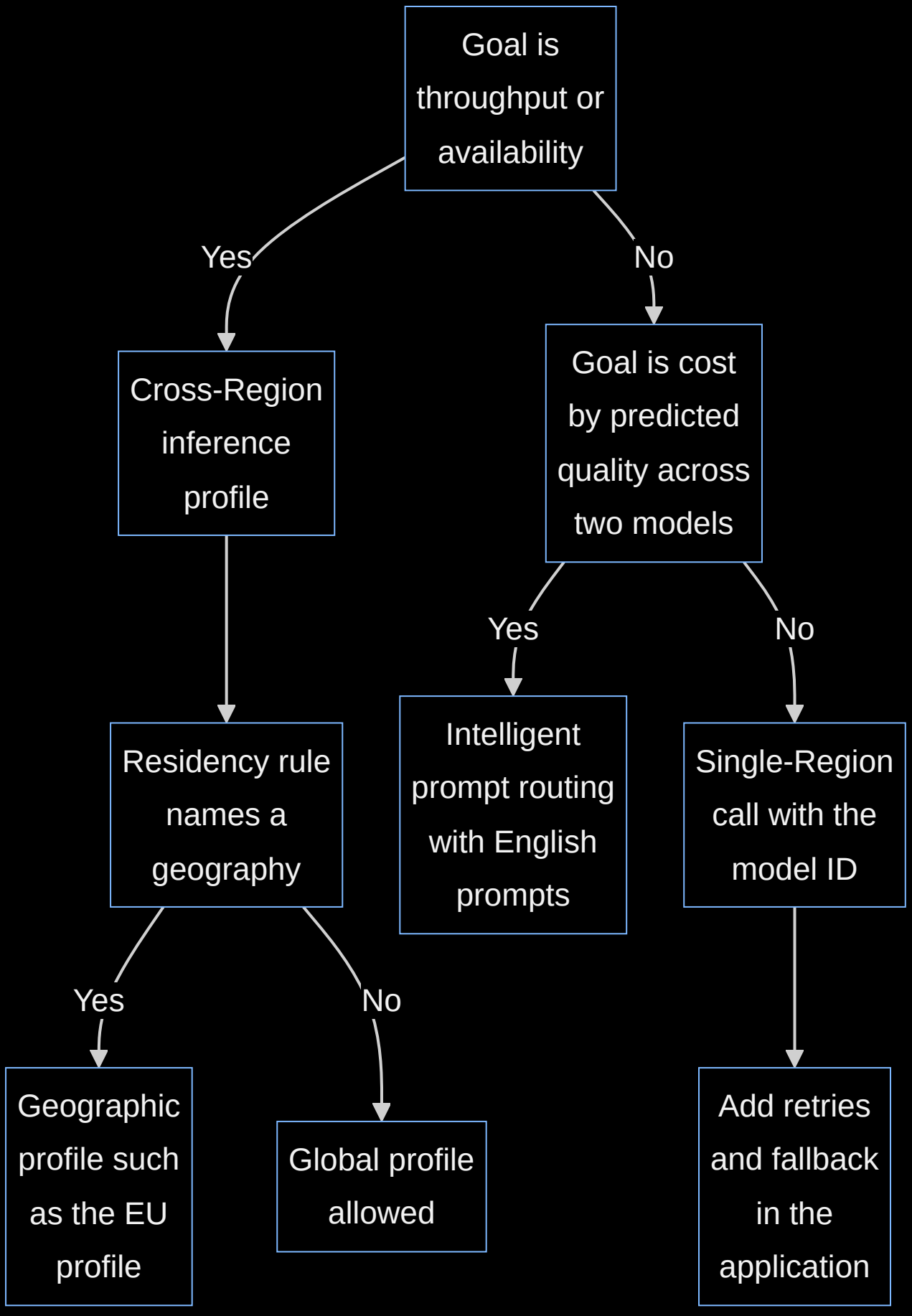


Figure: Cross-Region, single-Region, or intelligent routing. Cross-Region inference is not failover, so retries stay in the application.

Latency optimized inference

Not in source. The repo guide has no entry for a latency optimized inference mode.

- Methods or APIs: Not in source.
- Pro: Not in source.
- Con: Not in source.
- Choose it when: Not in source.
- Do not choose it when: Not in source.
- Native pairing: Not in source.

Decision rule: Set latency needs first when you choose a model. Confirm any latency mode on the official Bedrock pages before you rely on it.

Trap: A bigger model does not fix a slow search step. Find the slow step first.

Exam tip: Latency is the first filter in the order of decision. Read the latency clue before the cost clue.

Source: repo guide, Part 2 section on model axes, Part 5 section 4.2. Latency mode itself: Not in source.

Prompt caching

Prompt caching reuses an exact prompt prefix that sits before a cache checkpoint.

- Methods or APIs: In Converse, add a cachePoint block. In InvokeModel for Claude, add cache_control. Claude allows up to four checkpoints per request. Checkpoints process in this order: tools, system, then messages. The response reports cacheReadInputTokens and cacheWriteInputTokens. The inputTokens field excludes cached tokens.
- Minimum tokens: A checkpoint needs 512 to 4,096 tokens before it. The exact value varies by model.
- TTL: The default time to live is 5 minutes. Each cache hit resets it. Some models accept a one-hour TTL set with "ttl": "1h". On a model that supports

only five minutes, that ttl field causes a ValidationException.

- Pro: A repeated long prefix costs less. Cached tokens bill at the cache-read rate.
- Con: Cache writes cost extra, so the benefit needs repeat volume. The cache works for on-demand inference, not batch. It does not reserve throughput.
- Choose it when: the same long prefix repeats within the TTL window on on-demand calls.
- Do not choose it when: the prefix is short, changes on each call, or runs as batch.
- Native pairing: Lambda builds the prompt with the static prefix first and the user message last. The console has a caching toggle. Its menu path: Not in source.

Cache checkpoint placement

Put instructions, tools, and documents first. Put the changing user message last. A value that changes inside the prefix forces a miss on each call.

Decision rule: Add a checkpoint only when the static prefix is above the minimum and repeats within the TTL.

Trap: A timestamp or user name at the top of the prompt changes the prefix on each call. The cache then misses on every call.

Exam tip: The cache lowers the cost of a repeated prefix. It does not raise throughput. Reserved capacity is Provisioned Throughput.

Source: repo guide, Part 1E visual unit 4, Part 5 section 4.1, Part 8A5 caching unit. Console menu path: Not in source.

Semantic cache

A semantic cache returns a stored answer when a new request is close in meaning to a stored request. The repo guide presents it as a pattern built on embeddings.

- Methods or APIs: Embed each new request with an embedding model, such as one called through InvokeModel. Compare the vector with stored vectors. Return the stored answer on a hit. On a miss, call the model and store the pair. The store type: Not in source.
- Pro: A hit returns the stored answer with no model call. It works when many requests repeat with different words.
- Con: The threshold decides how close is close enough. Personal or unique requests rarely hit. The repo guide names no Bedrock semantic cache feature.
- Choose it when: many requests repeat in different words.
- Do not choose it when: requests are personal or unique, so hits stay rare.
- Native pairing: Titan Text Embeddings v2 or Cohere Embed for the vectors. The repo guide notes Cohere uses an input type for queries and passages.

Decision rule: Set the cache threshold on a golden set before you use a semantic cache.

Trap: Prompt caching is not a semantic cache. Prompt caching reuses an exact prefix at the model. A semantic cache returns a stored answer before the model runs.

Exam tip: Many paraphrases of one request point to a semantic cache. A long repeated prompt prefix points to prompt caching.

Source: repo guide, Part 1E visual units 5 and 25, Part 5 section 4.1, Part 1B Lesson 3. Cache store type: Not in source.

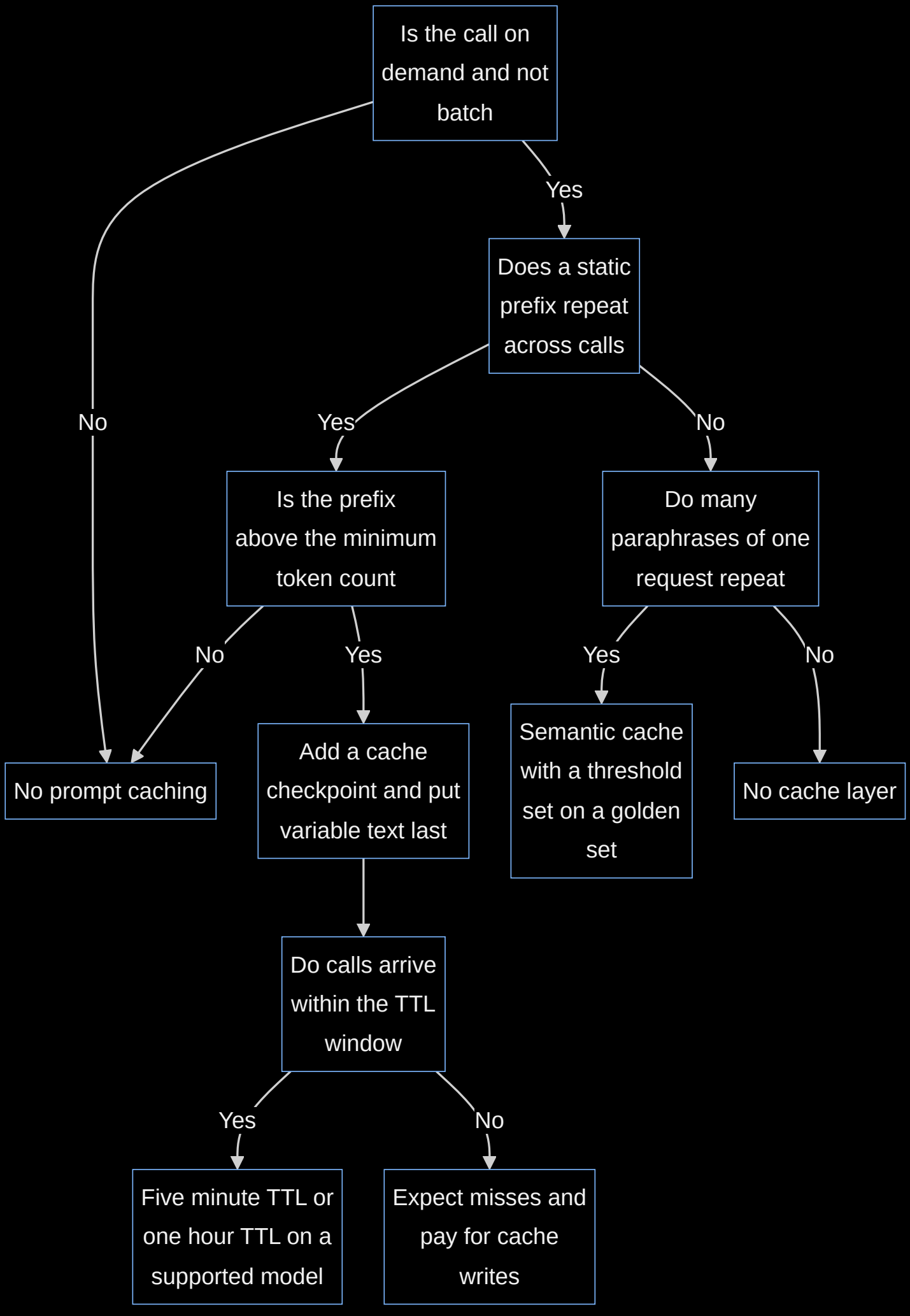


Figure: Cache or not. Prompt caching and a semantic cache solve different repeats.

Token economics

Cost = (input tokens / 1,000,000 × input price) + (output tokens / 1,000,000 × output price)

- Illustrative prices: 3 USD per million input tokens and 15 USD per million output tokens. These are not current AWS prices.
- Worked example: 4,000 input tokens cost 0.012 USD. 1,000 output tokens cost 0.015 USD. The total is 0.027 USD.
- Output ratio: At these illustrative prices, one output token costs five times one input token.
- Cached input: Total input equals inputTokens plus cacheReadInputTokens plus cacheWriteInputTokens. Cached reads bill at the cache-read rate. Cache write and read rates: Not in source.
- Batch: A lower price than on-demand for select models. The discount: Not in source.
- Provisioned Throughput: Billed hourly, with a commitment of none, 1 month, or 6 months.
- Price table: Not in source for this build. Check the Bedrock pricing page for current prices.

Decision rule: Count input and output tokens separately. Cap output length first when replies run long.

Trap: More capacity does not fix a cost that comes from long prompts. Shorten the prefix or cache it.

Exam tip: Cost from a long repeated prompt points to caching. Cost from steady load points to reserved capacity.

Source: repo guide, Part 5 section 4.1, Part 5 visual units 39 and 40. Cache rates, batch discount, prices: Not in source.

Model invocation logging

Model invocation logging records requests and responses for Converse, ConverseStream, InvokeModel, and InvokeModelWithResponseStream.

- Methods or APIs: Turn on invocation logging in the Bedrock settings. Choose an S3 destination with a bucket policy, a CloudWatch Logs log group, or both. The screen path for the toggle: Not in source.
- Record shape: S3 delivery is gzipped JSON. Each entry records the identity ARN, request metadata, and token counts. Athena and S3 Select can query the S3 delivery.
- Retention: Logs stay until you delete the logging configuration. A Bedrock retention period: Not in source.
- Pro: The exact payload is available for audit and debugging. Token counts come with each call.
- Con: Invocation logs are off by default. Logs can hold prompts, completions, and sensitive data. Logs keep the original request even when a guardrail masks the output.
- Choose it when: an audit needs the words sent to the model and the words returned.
- Do not choose it when: strict compliance rules bar stored content. Disable logging in that case.
- Native pairing: KMS encrypts the log store. S3 Object Lock gives write-once retention. CloudTrail gives identity and timing. X-Ray finds the slow step. Logs Insights searches CloudWatch Logs delivery.

Setup steps

- Choose an S3 destination, or choose a CloudWatch Logs log group.
- Add a bucket policy to the log bucket, and replace the example values.
- Turn on invocation logging in the Bedrock settings.

Decision rule: Use invocation logs for content. Use CloudTrail for who called and when.

Trap: CloudTrail does not show row content. Invocation logs keep the original request even when the output is masked.

Exam tip: Metrics alone cannot rebuild what the model received. A scenario that asks for the content points to invocation logs.

Source: repo guide, Part 1F practical unit 5, Part 1E visual unit 16, Part 4 section 3.2. Screen path and retention: Not in source.

Custom models

Customization changes the model or its input. Each rung of the ladder changes more and costs more.

- Methods or APIs: The repo guide places fine-tuning on SageMaker AI. A Bedrock documentation link covers fine-tuning and continued pre-training as reference only. Fine-tuning changes weights with labeled examples. Continued pre-training trains on unlabeled text to teach a domain. The API calls: Not in source.
- LoRA: Freezes the base weights and trains small adapter matrices. The repo guide describes LoRA for SageMaker AI. A Bedrock LoRA path: Not in source.
- Distillation: A smaller student model learns one task from a larger teacher. The Bedrock distillation API: Not in source.
- Pro: Stable behavior at high volume. A distilled student is cheaper and faster for its task.
- Con: Fine-tuning does not hold facts that change. Tuned models need approval before deployment.
- Choose it when: prompts and RAG fail, and the behavior is stable at high volume.
- Do not choose it when: the facts change. Use RAG for changing facts.
- Native pairing: SageMaker AI fine-tunes and hosts the custom model. Each tuned version goes into Model Registry. Model Cards record intended use and evaluation results.

Decision rule: Try prompts, then RAG, then fine-tuning. Fine-tune only for stable behavior at high volume.

Trap: Fine-tuning on documents that change is wrong. Knowledge goes to retrieval. Behavior goes to weights.

Exam tip: A scenario with changing facts points to RAG, not to tuning.

Source: repo guide, Part 1B Lesson 7, Part 2 customization ladder, Part 8A reference terms. Bedrock LoRA and distillation API: Not in source.

Model choice on five axes

A model choice reads five axes: capability, cost per token, latency, context window, and Regional availability.

- Methods or APIs: Not in source for an axis API. Each axis uses the limits and prices of each model. The price table: Not in source.
- Axis detail: Capability covers reasoning, multilingual work, code, and vision. Cost has separate input and output prices. Latency covers time to first token and tokens per second. The context window holds instructions, history, retrieved text, and the request. Regional availability asks whether the model runs in your Region.
- Pro: A fixed checklist makes the choice repeatable.
- Con: The repo guide sets no weights for the five axes. Weights: Not in source.
- Choose it when: a team compares candidate models for one workload.
- Do not choose it when: the result is accepted without a test on your own data.
- Native pairing: AppConfig holds the model ID, and Lambda reads it at runtime. A change to the ID rolls out with a deployment strategy.

Decision rule: Order of decision is latency needs, then reasoning difficulty, then cost. Test the final choice on your own data.

Trap: The flagship model is not the default. It can break a latency budget. A cheap model that is not offered in your Region is not a choice.

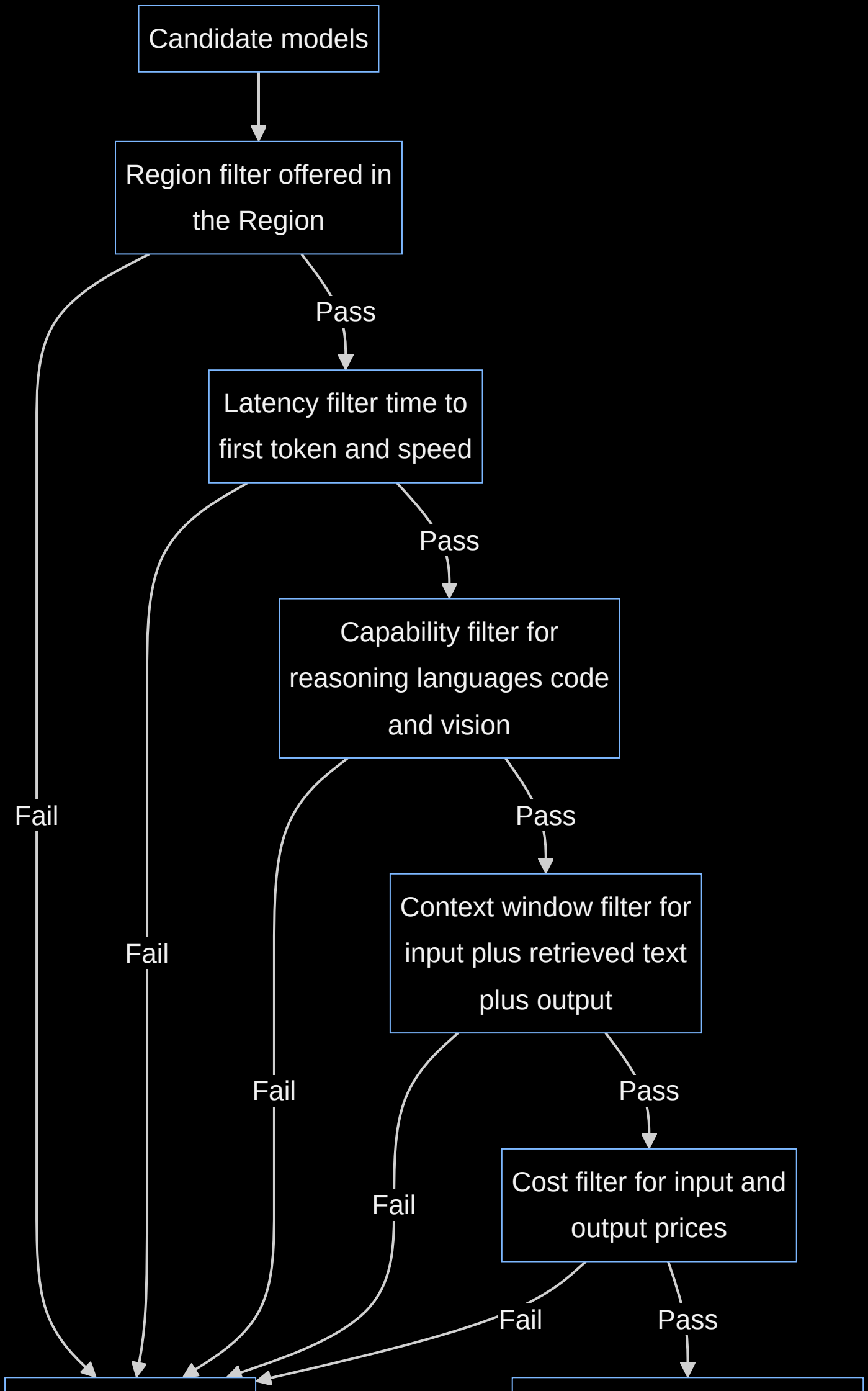
Exam tip: Check the Region before the price. A low price in the wrong Region fails the filter.

Source: repo guide, Part 2 model axes table, Part 1B Lessons 1 and 2, Part 2 visual unit 22. Weights and prices: Not in source.

Model choice as a constraint filter

A model choice is a filter. Remove each candidate that breaks one hard constraint. Keep the cheapest survivor.

The tree applies each hard filter in turn. A failed filter drops the candidate, and the survivor then gets a test on your own data.


Candidate models
Region filter offered in the Region
Pass
Fail
Latency filter time to first token and speed
Pass
Fail
Capability filter for reasoning languages code and vision
Pass
Fail
Context window filter for input plus retrieved text plus output
Pass
Fail
Cost filter for input and output prices
Pass
Fail

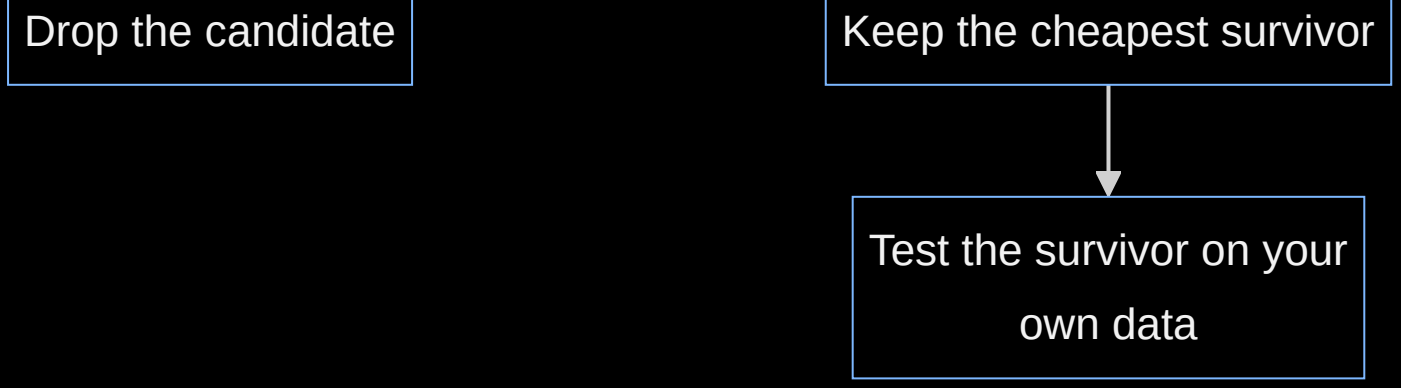


Figure: Model choice as a sequence of hard filters. Region and latency run first. Cost decides only among survivors.

Decision rule: Check Region availability as a pass or fail filter. Then weigh latency needs, reasoning difficulty, and cost, in that order. Test the final choice on your data.

Trap: A model that passes the cost filter but fails the Region filter is not a candidate. Do not rank it.

Exam tip: Read each constraint in the scenario. Each constraint removes candidates before cost decides.

Source: repo guide, Part 2 visual unit 22, Part 2 model axes table, Part 1B Lesson 2.

Model lifecycle and versions

A model registry groups the versions of a model and tracks the approval status of each version.

- Methods or APIs: update_model_package sets ModelApprovalStatus to Approved or Rejected. In the Studio console, the approval control sits on the Deploy tab of the model version.
- Rollback: A rollback points the endpoint to an earlier approved version. Keep that version in the registry.
- Bedrock model version IDs and retirement dates: Not in source.
- Pro: Each release has a version and an approval record. A rollback has a target.
- Con: Registry approval adds a step before deployment. The registry covers SageMaker AI models. Its Bedrock counterpart: Not in source.

- Choose it when: a custom model or an endpoint needs versions and rollback.
- Do not choose it when: a prompt or model changes in place with no saved version. Version it first.
- Native pairing: AppConfig rolls back a model ID change when an alarm fires. CloudTrail records InvokeModel and Converse calls for the audit trail.

Decision rule: Keep the last approved version before each deploy. Roll back to it when a gate fails.

Trap: A model or prompt that changes in place has no safe rollback point.

Exam tip: A rollback clue needs a previous approved version. Without one, add the versioning step first.

Source: repo guide, Part 2 lifecycle table, Part 1C registry, Part 1E visual unit 56, Part 8B steps. Bedrock version IDs: Not in source.

Bedrock error codes and fixes

An error code shows who must act. A quota error needs a slower rate, a capacity error needs a retry, and an input error needs a code fix.

- ThrottlingException, HTTP 429: The account exceeded a quota. Retry with exponential backoff and random jitter. Request a quota increase, or buy Provisioned Throughput for sustained traffic.
- ServiceUnavailable, HTTP 503, and overloaded_error, HTTP 529: Capacity errors, not quota errors. Retry with backoff and jitter. Consider cross-Region inference.
- ValidationError, HTTP 400: The input breaks a constraint. Fix the request. Do not retry.
- AccessDeniedException, HTTP 403: IAM permissions are missing or expired. Fix the policy or the role.
- ResourceNotFound, HTTP 404: The model ID or resource name is wrong. Fix the identifier.

- Body error: A provider body sent to the wrong model fails. A Claude body needs anthropic_version and max_tokens.
- Connection drops: NAT gateways, interface VPC endpoints, and Network Load Balancers drop idle TCP connections after 350 seconds. Enable TCP keep-alive in the SDK client. Set the OS keep-alive interval below 350 seconds. Both settings are needed.
- Region-only AccessDeniedException: The troubleshooting table in the repo guide points to IAM resource ARNs for that Region. A later check found that the error page lists no Region-based cause. Confirm model availability in the Region and the ARNs.

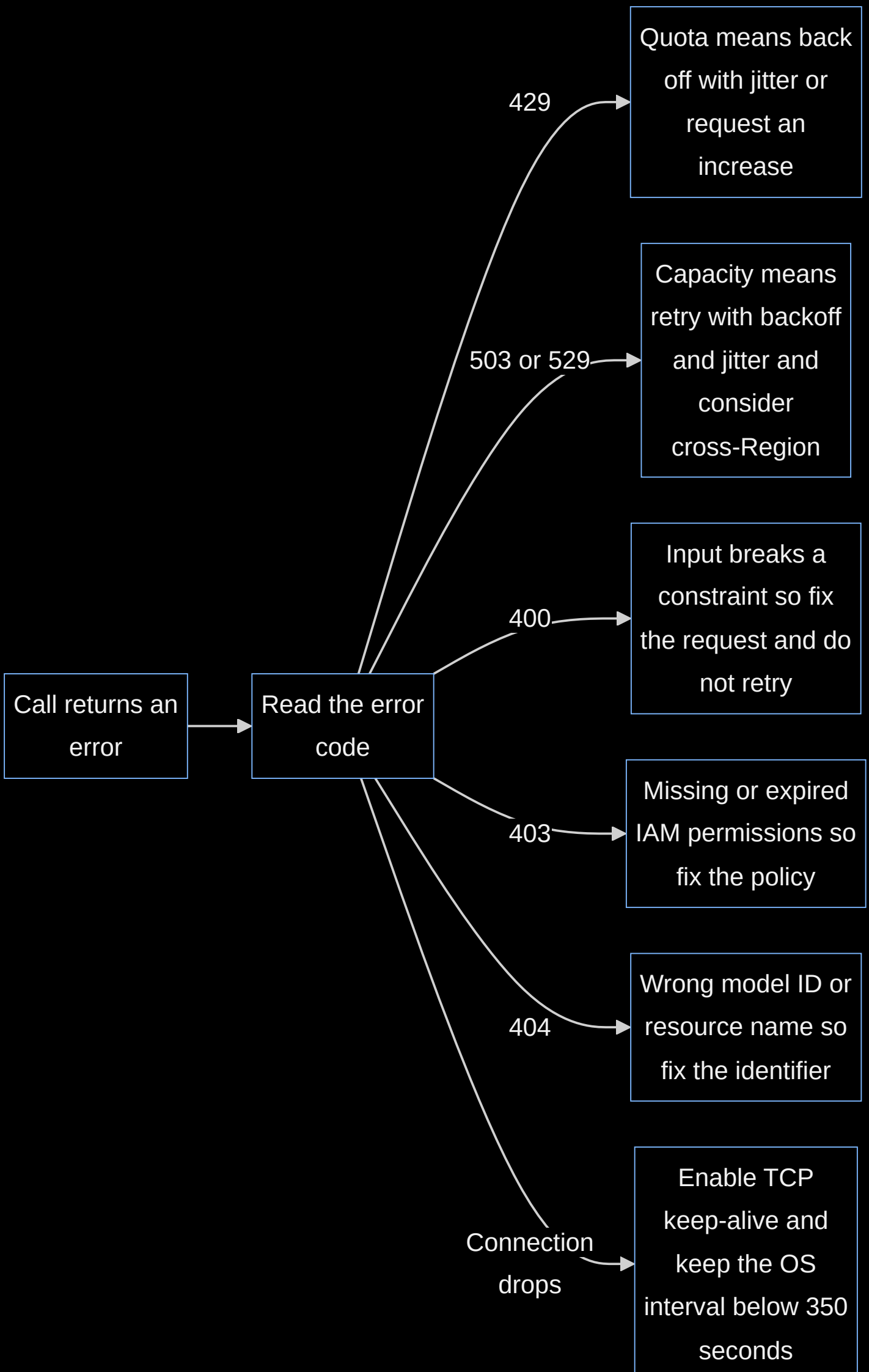

Call returns an error
Read the error code
429
Quota means back off with jitter or request an increase
503 or 529
Capacity means retry with backoff and jitter and consider cross-Region
400
Input breaks a constraint so fix the request and do not retry
403
Missing or expired IAM permissions so fix the policy
404
Wrong model ID or resource name so fix the identifier
Connection drops
Enable TCP keep-alive and keep the OS interval below 350 seconds

Figure: Error triage by code. Quota, capacity, input, and permission errors each need a different fix.

Decision rule: Retry 429, 503, and 529 with backoff and jitter. Do not retry a 400 error. Fix the request first.

Trap: A tight retry loop is never correct. Retries need jitter, a maximum attempt count, and a fallback.

Exam tip: A 429 quota error and a 503 capacity error need different fixes. Quota needs more limit or capacity. Capacity needs a retry or a Region change.

Source: repo guide, Part 8A3 error code unit, Part 3 section 2.4, Part 5 section 4.1, Part 5 troubleshooting table. Region-only cause: Not in source.

PrivateLink planes for Bedrock

VPC interface endpoints through AWS PrivateLink keep API traffic on the AWS network. Create one endpoint for each Bedrock plane the workload uses.

- Planes: bedrock-runtime serves model invocations, such as Converse calls. bedrock-agent and bedrock-agent-runtime serve agents. bedrock is the control plane.
- Methods or APIs: VPC interface endpoints. Endpoint service names for each plane: Not in source.
- Pro: Traffic stays off the public internet for the planes you create endpoints for.
- Con: One runtime endpoint does not cover agent calls. Each plane needs its own endpoint. Endpoint cost: Not in source.
- Choose it when: a rule keeps API traffic off the public internet.
- Do not choose it when: a NAT gateway is planned as the private path. A NAT gateway gives outbound internet access. It is not a private path to Bedrock.
- Native pairing: IAM least privilege controls access. KMS encrypts data at rest. CloudTrail records API calls. AWS WAF filters web requests but does not make the network path private.

Decision rule: Create one interface endpoint for each plane the workload calls.

Trap: One runtime endpoint does not cover agent calls. Check each plane.

Exam tip: Encryption and private paths answer different risks. KMS does not change the network path. PrivateLink does not encrypt data at rest.

Source: repo guide, Part 1E visual unit 17, Part 4 section 3.2, Part 8D security controls unit. Endpoint names and cost: Not in source.

Amazon Bedrock application layer

Every production call passes through an entry point, a compute step, a safety check, and a model.

Source: repo guide, Part 1A, Architecture 1

This part covers the Bedrock services that an application calls, configures, or tests.

Knowledge Bases

Knowledge Bases for managed RAG

Knowledge Bases is managed RAG. It syncs a data source, chunks and embeds it, retrieves, and can return citations.

- Methods or APIs: StartIngestionJob, GetIngestionJob, Retrieve, RetrieveAndGenerate, KnowledgeBaseDocuments.
- Pro: You sync sources, then call one API. The service runs both the ingest path and the query path.
- Con: Each stage follows the service design. A custom retrieval algorithm needs hand-built RAG.
- Choose it when: Private documents need citations and the team wants the least custom effort.

- Do not choose it when: A stated requirement needs control of each stage.
- Native pairing: S3 holds documents. Lambda runs custom chunking. IAM roles scope access per knowledge base.

Decision tree: Knowledge Bases or hand-built RAG

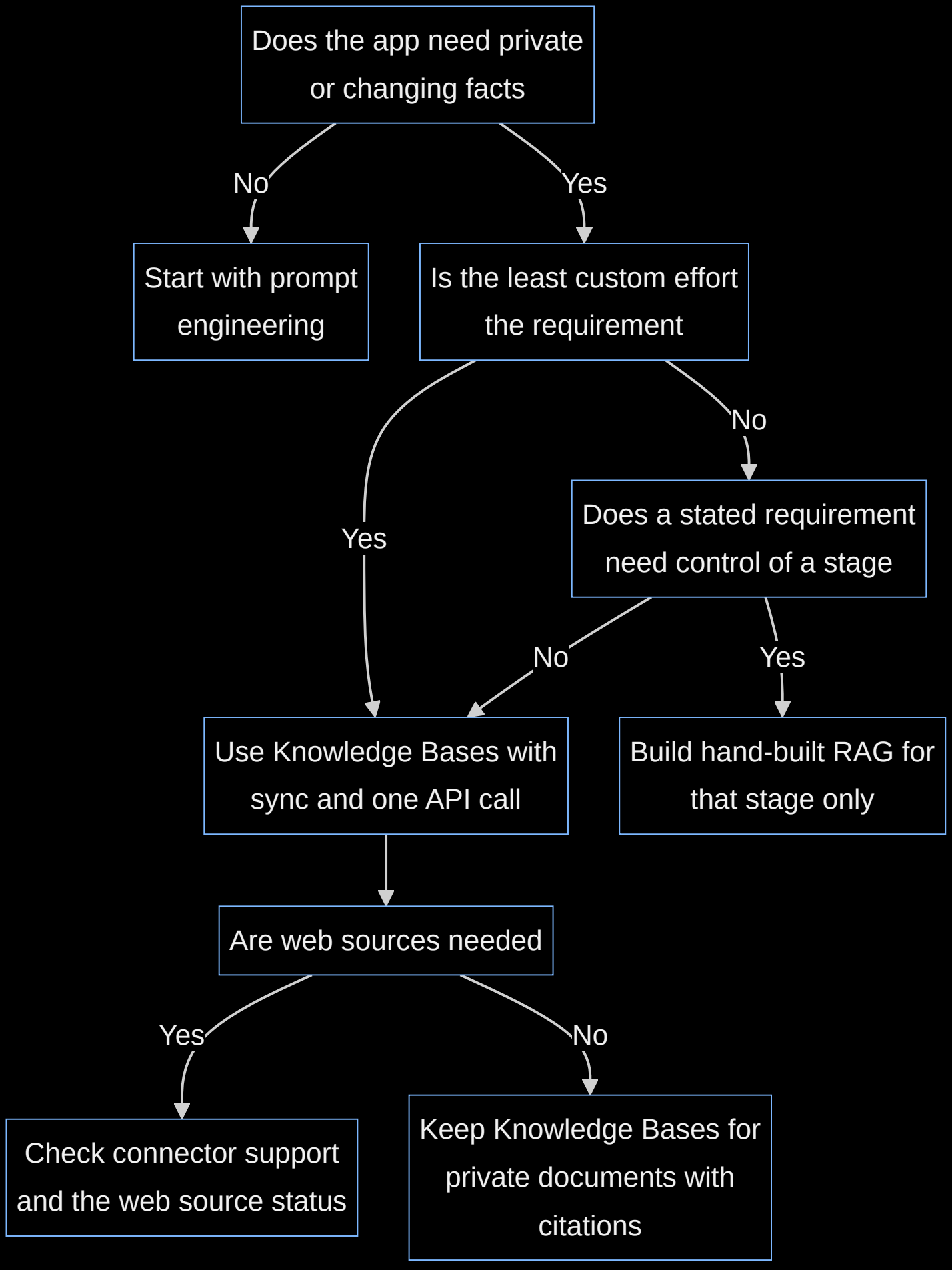


Figure: Knowledge Bases or hand-built RAG. Least effort and private cited facts favor the managed service.

Decision rule: Choose Knowledge Bases for private, cited facts unless a stated requirement forces hand-built RAG.

Trap: A hand-built pipeline is wrong when the requirement is the least custom effort.

Exam tip: The words managed or least effort point to Knowledge Bases. The words custom retrieval point to hand-built RAG.

Source: repo guide, Part 1A component table, Part 2 section 1.5

Data sources, sync, and direct ingestion

A sync reads a data source and updates the index. Direct ingestion adds, updates, or deletes single files.

- Methods or APIs: StartIngestionJob starts a sync. GetIngestionJob tracks it. KnowledgeBaseDocuments handles direct ingestion.
- Pro: Sync is incremental. Only added, modified, or deleted documents process after the last sync.
- Con: The source names no automatic S3 event trigger for sync jobs. That trigger is Not in source.
- Choose it when: A schedule starts a batch sync. Choose direct ingestion for single-file changes.
- Do not choose it when: Urgent changes must be searchable at once. A quarterly sync alone does not meet that need.
- Native pairing: An S3 event invokes Lambda asynchronously. The function then calls the sync API.

Decision tree: sync or ingest

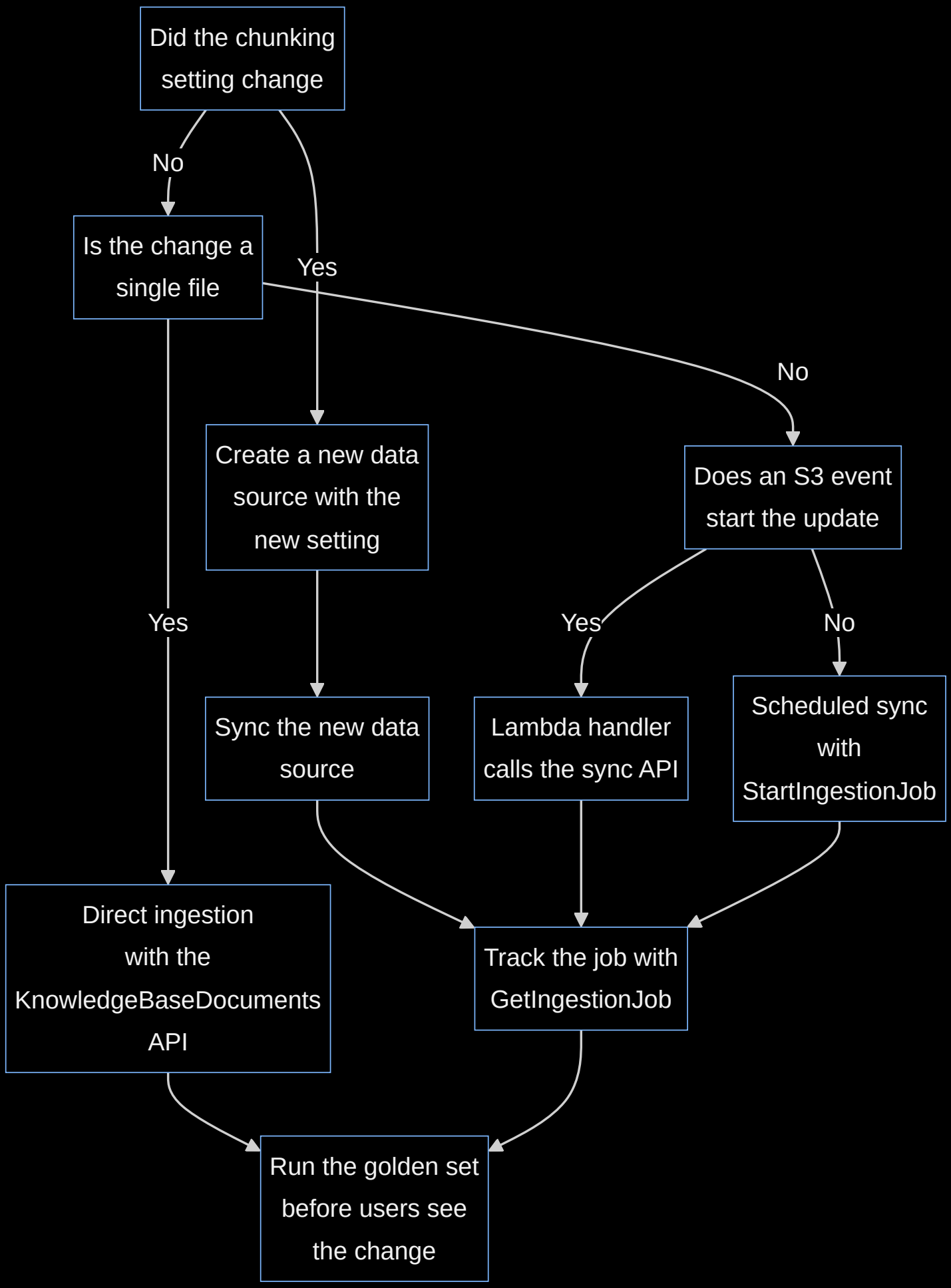


Figure: Sync or ingest. A re-sync is a change, so it needs the same release gate.

Decision rule: Run a scheduled sync as the backstop. Add an event handler for urgent document changes.

Trap: A knowledge base answers from its last sync. A file change stays out of results until a sync or a direct ingestion call processes that file.

Exam tip: The phrase urgent document change points to an event handler plus a scheduled sync.

Source: repo guide, Part 2 visual unit 59, Part 8A5 freshness unit

Chunking strategies

A chunk is the unit of text that retrieval returns. A chunk must hold one complete answer.

- Methods or APIs: Default, none, fixed size, hierarchical, semantic, and custom chunking with a transformation Lambda function.
- Pro: Default chunking makes chunks of about 300 tokens. Hierarchical chunking returns larger parents.
- Con: Semantic chunking costs more to process. Hierarchical chunking is not recommended with an S3 vector bucket.
- Choose it when: Short, self-contained text suits fixed size. Manuals and contracts suit hierarchical. Tables suit custom Lambda chunking. Inference: the source ties table handling to a custom transformation Lambda function.
- Do not choose it when: The split breaks one complete answer, a table, or a figure caption.
- Native pairing: You set the strategy when you create the data source. A numeric chunk size for any one model is Not in source.

Decision rule: Pick the strategy from the document shape. Set it when you create the data source.

Trap: A table split across chunks can lose its values. Each part can look correct alone.

Exam tip: Words such as change the chunking point to a new data source, then a sync.

Source: repo guide, Part 1B lesson 6, Part 1E visual unit 6

Embedding models: Titan Text Embeddings v2 and Cohere Embed

An embedding maps text to a list of numbers. Texts with similar meaning get vectors that point the same way.

- Methods or APIs: InvokeModel for embeddings. Titan Text Embeddings v2 outputs 1024 (default), 512, or 256 dimensions. Cohere Embed takes input_type search_query or search_document.
- Pro: Titan v2 gives a size choice. Cohere separates a query from a passage through input_type.
- Con: More dimensions cost more storage and compute. Smaller sizes may lower quality. Converse does not produce embeddings.
- Choose it when: Use search_query for a user query and search_document for a stored passage. Test Titan sizes on a golden set.
- Do not choose it when: The call uses Converse. Every embedding call uses InvokeModel.
- Native pairing: Knowledge Bases embeds chunks and queries. Multimodal console steps name Titan Multimodal Embeddings G1 or Cohere Embed v3.

Decision rule: Test both dimension sizes on a golden set. Keep the smallest size that passes.

Trap: A bigger embedding model does not fix a bad chunk. Fix the chunk first.

Exam tip: Pick search_query or search_document by the role of the text, not by the model name.

Source: repo guide, Part 1B lesson 3, Part 1E visual unit 25

Vector stores for Knowledge Bases

A vector store holds embeddings and returns the stored vectors closest to a query vector.

- Methods or APIs: OpenSearch Serverless, Aurora with pgvector, and S3 Vectors. Neptune and DynamoDB are not Knowledge Base stores in the source.
- Pro: OpenSearch Serverless combines filters and vectors. S3 Vectors needs no cluster to run.
- Con: An always-on cluster costs money even when few queries arrive. Neptune suits graph traversal, not plain search. ElastiCache is not a durable store of record.
- Choose it when: Frequent, interactive search fits an always-on store. A large, rarely queried corpus fits S3 Vectors.
- Do not choose it when: The store must be the durable record and the candidate is ElastiCache.
- Native pairing: The ingest path writes chunk embeddings to the store. Check store docs for filter timing.

Decision rule: Pick the store by query rate, latency, and whether the team already runs a database.

Trap: ElastiCache is a cache, not a durable store. Check durability before you use it as a store.

Exam tip: A rare, very large corpus points to S3 Vectors. Frequent, interactive search points to an always-on store.

Source: repo guide, Part 1A component table, Part 1E visual units 24 and 63

Retrieve and RetrieveAndGenerate

Retrieve returns chunks without generation. RetrieveAndGenerate returns a generated answer with source citations.

- Methods or APIs: Retrieve for chunks only. RetrieveAndGenerate for a generated answer. The console test window shows chunks and their sources.
- Pro: Retrieve lets you read the chunks before you trust the answer. RetrieveAndGenerate returns the answer and citations in one call.
- Con: Whether RetrieveAndGenerate always returns citations is Not in source for this build. Citations are attribution, not proof of truth.
- Choose it when: To read chunks only, use Retrieve. To return a cited answer, use RetrieveAndGenerate.
- Do not choose it when: The app needs its own prompt around the chunks. Use Retrieve when the app builds its own prompt. The source does not describe that prompt step: Not in source.
- Native pairing: The app places retrieved text in a Converse or InvokeModel prompt.

Decision rule: Read the chunks first. Use Retrieve to test the retriever before you test the answer.

Trap: A new generator model does not fix broken retrieval. Check the chunks first.

Exam tip: A test that returns chunks points to Retrieve. A test that returns an answer points to RetrieveAndGenerate.

Source: repo guide, Part 1F practical unit 2, Part 2 visual unit 57

Reranking and hybrid search

Hybrid search adds keyword matching to vector search. A reranker scores each candidate against the request.

- Methods or APIs: Hybrid search and reranker are named. Their API calls and model IDs are Not in source.
- Pro: Keyword search finds exact codes and citation strings. Vector search finds related meaning. The reranker orders candidates.

- Con: Reranking fixes order only. It does not add a chunk that retrieval missed.
- Choose it when: Keyword-heavy requests drift, add hybrid search. Related but wrong top results need a reranker. Vocabulary gaps need query expansion.
- Do not choose it when: Retrieval never returned the right text. Fix chunking or the store first.
- Native pairing: Retrieve candidates, merge them, rerank, then keep the top few. The unit counts are schematic.

Decision tree: retrieval fix by symptom

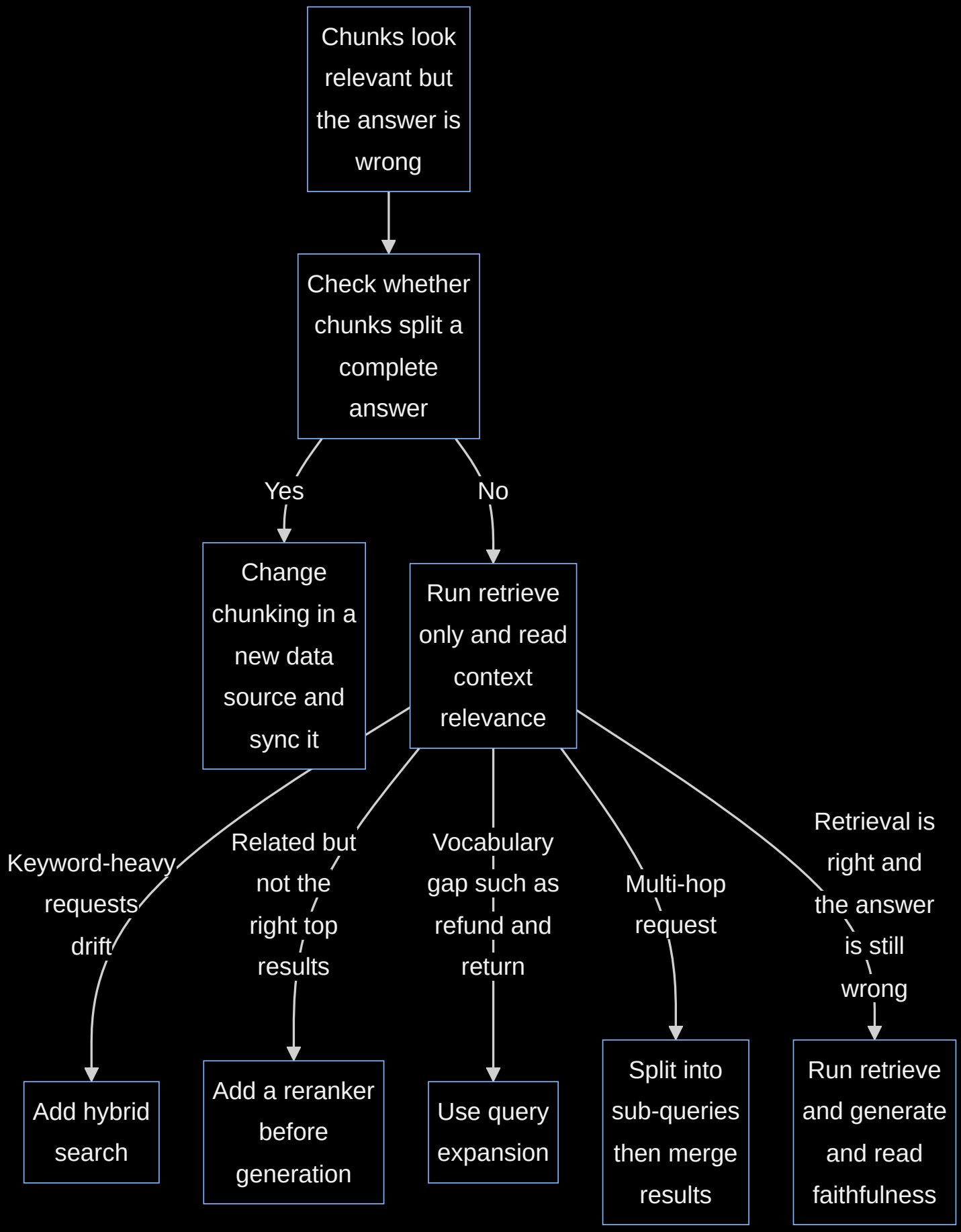


Figure: Retrieval fix by symptom. Check the chunks first, then the ranking, then the generator.

Decision rule: Rerank before generation. Add hybrid search when exact terms matter to retrieval.

Trap: Similarity is not relevance. A reranker cannot add a chunk that retrieval never returned.

Exam tip: Exact codes and citation strings signal keyword search. Paraphrases signal vector search.

Source: repo guide, Part 1B lesson 4, Part 1E visual unit 7

Metadata filters and tenant isolation

Metadata filters narrow a search. Tenant data stays apart through the platform, not through prompt text.

- Methods or APIs: Metadata filters. Filter syntax: Not in source. One knowledge base and one IAM role for each tenant.
- Pro: Separate knowledge bases and roles keep tenant data apart at the platform layer. Filters split results by region or product line.
- Con: The repo guide does not show filter syntax or the filter rules of each store.
- Choose it when: Tenants or regions need separate results, and the store supports the filter.
- Do not choose it when: A prompt instruction is the only control. A prompt that says ignore other tenants is not isolation.
- Native pairing: IAM roles scope each tenant role to its own knowledge base.

Decision rule: Isolate tenants with separate knowledge bases and IAM roles. Add metadata filters for finer splits.

Trap: The model receives only what retrieval returns. Prompt text cannot separate tenant data.

Exam tip: Words such as structural or platform point to IAM roles and separate knowledge bases.

Source: repo guide, Part 1E visual unit 61, Part 2 section 1.6

Retrieve-only and retrieve-and-generate evaluation (Knowledge Base metrics)

A retrieve-only job isolates the retriever. A retrieve-and-generate job scores the full RAG answer.

- Methods or APIs: Retrieve-only metrics are context relevance and context coverage. Retrieve-and-generate metrics are correctness, completeness, helpfulness, logical coherence, faithfulness, citation precision, citation coverage, harmfulness, stereotyping, and refusal.
- Pro: Each metric measures one stage. Faithfulness measures how well responses avoid claims the retrieved text does not support.
- Con: Context coverage needs a ground truth in the prompt dataset. The repo guide conflicts on citation precision and citation coverage.
- Choose it when: Answers are wrong and retrieval is in doubt. Run retrieve only first.
- Do not choose it when: The claim is tone or brand voice. Use human review.
- Native pairing: Setup steps for these job types are Not in source.

Decision rule: Read context relevance first. Read faithfulness when retrieved text is good but the answer ignores it.

Trap: Faithfulness does not check outside facts. Citation precision and citation coverage are Not in source in one section.

Exam tip: A good final answer can hide weak retrieval. Measure the retriever and the generator separately.

Source: repo guide, Part 8A6 knowledge base evaluation unit, Part 6 section 5.1 RAG evaluation

Web sources note

The repo guide says web crawler connectors cannot be created on customer-managed knowledge bases after September 30, 2026.

- Methods or APIs: Amazon Bedrock Managed Knowledge Base for web sources. Check connector support for your knowledge base type.

- Pro: The guide names the managed knowledge base as the path for web sources.
- Con: Part 9 of the repo guide lists the web crawler end date as Not in source.
- Choose it when: Web content is a source, and the managed path supports it.
- Do not choose it when: The plan depends on a web crawler connector without a current status check.
- Native pairing: Web source setup steps are Not in source.

Decision rule: Check the current web source status before you design a web crawl.

Trap: One section gives the date. Part 9 says the date is Not in source. Confirm it on the Knowledge Base documentation.

Exam tip: A date in a requirement can change the right service. Check the date first.

Source: repo guide, Part 2 section 1.5, Part 9 open checks

Guardrails

Amazon Bedrock Guardrails and its six policies

A guardrail checks input and output with six policy types. Each policy stops one kind of failure.

- Methods or APIs: The six policies listed below. The Converse response includes a trace object with guardrail results.
- Pro: One guardrail covers input and output with separate switches. A draft is editable. A version is a frozen snapshot.
- Con: A word list cannot stop every phrasing of a subject. Guardrail creation steps are Not in source.
- Choose it when: The failure matches one policy in the decision tree below.

- Do not choose it when: The job is data quality, field extraction, or PII in S3 at rest. Guardrails does not scan a whole bucket.
- Native pairing: The guardrail checks input before the model call and output after it. Invocation logs keep the original request.

Six policies and the failure each one stops

Denied topics stop a whole subject. Word filters stop exact strings and miss rephrased requests. Content filters stop harmful categories, and the prompt attack category applies to input. Sensitive information filters stop PII entities by redacting or blocking them. Contextual grounding checks stop answers that the source does not support, but they do not test harm. Automated Reasoning checks stop claims that break formal policy rules.

Decision tree: which guardrail policy for which failure

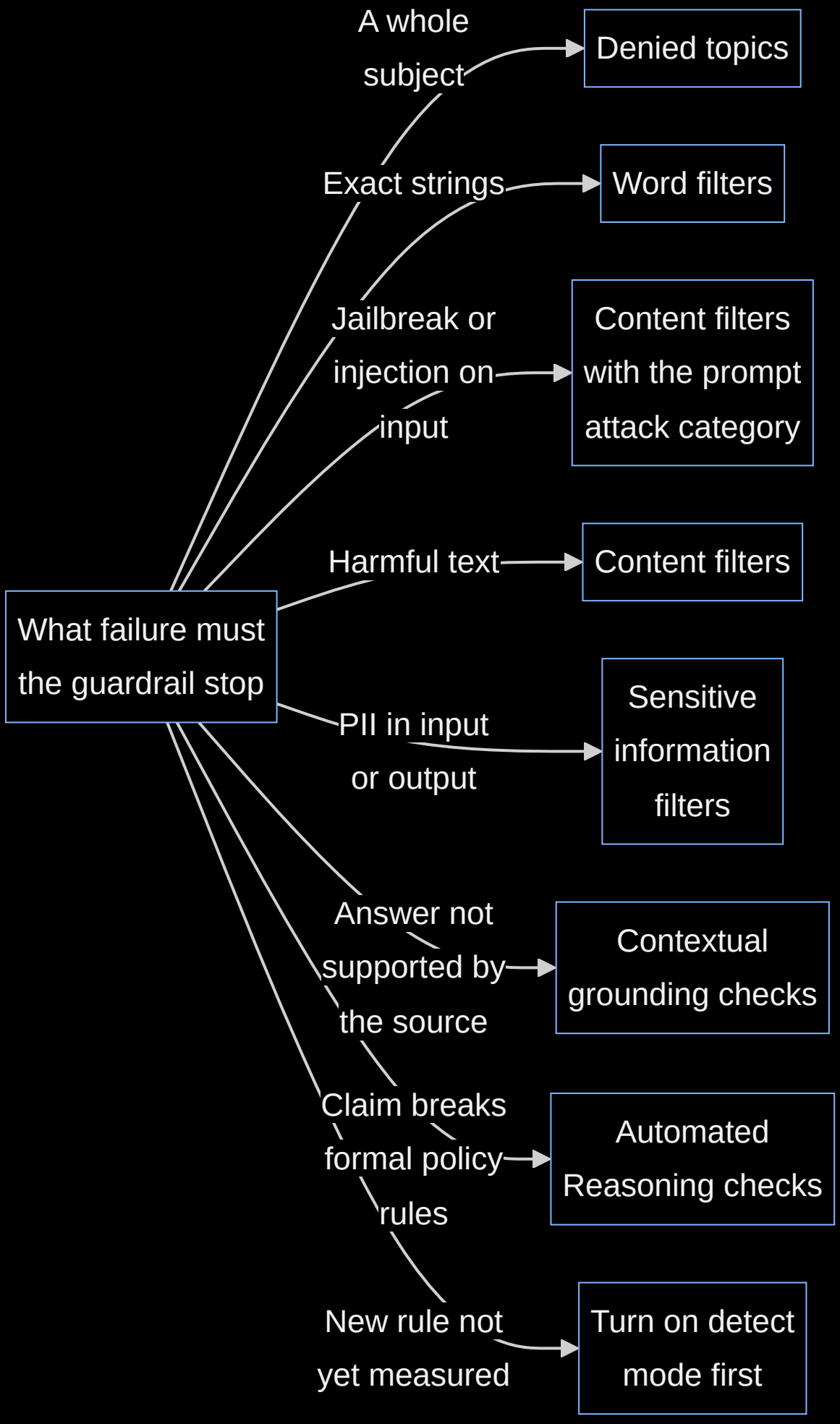


Figure: Which guardrail policy for which failure. Match one failure to one policy.

Decision rule: Match the failure to one policy. Then set the input and output checks separately.

Trap: Guardrails block, mask, or filter content. Guardrails do not validate data quality or extract invoice fields.

Exam tip: A whole subject points to denied topics. Exact strings point to word filters.

Source: repo guide, Part 4 section 3.1, Part 4 visual units 34 and 36

ApplyGuardrail

ApplyGuardrail checks content from any model. It does not call the model.

- Methods or APIs: ApplyGuardrail returns an allow or block result for the content it receives.
- Pro: One safety check works on text from any model, and it also covers models outside Bedrock.
- Con: ApplyGuardrail checks text only. The app still calls the model and handles the result.
- Choose it when: The model is outside Bedrock, or the app screens text at a separate step.
- Do not choose it when: Bedrock runs the model and the guardrail must travel with the Converse request.
- Native pairing: Lambda or API Gateway can call ApplyGuardrail before a model call and after it.

Decision rule: Use ApplyGuardrail for text from any model. Use the Converse request when Bedrock runs the model.

Trap: A non-Bedrock model does not need Converse to use ApplyGuardrail.

Exam tip: The words any model or outside Bedrock point to ApplyGuardrail.

Source: repo guide, Part 3 section 2.3, Part 3 visual unit 33

Detect mode

Detect mode runs the classifier and reports the verdict. It does not refuse the message.

- Methods or APIs: Detect mode, and the Block action. A blocked message needs the Block action.
- Pro: You measure a new rule on real traffic before the rule blocks anything.
- Con: A message in detect mode is not refused, so it still reaches the model. Where the verdict reports go: Not in source.
- Choose it when: A new rule needs measured verdicts before you block with it.
- Do not choose it when: The rule must stop the message now. Use the Block action.
- Native pairing: Switch the policy action to Block after the verdicts are right.

Decision rule: Run detect mode to measure a new rule. Use the Block action after the verdicts are right.

Trap: Detect mode is not a block. A new rule in detect mode lets the message through.

Exam tip: The words measure first point to detect mode.

Source: repo guide, Part 4 section 3.1 detect mode

Cross-Region guardrail inference

Cross-Region guardrail inference routes requests to another Region in the same geography.

- Methods or APIs: A guardrail setting that allows cross-Region inference. The setting name: Not in source.
- Pro: Requests can route to another Region in the same geography.
- Con: Input prompts and outputs can move to that Region. A strict single-Region rule conflicts with that routing.

- Choose it when: The residency rule allows the geography, and the team accepts routing across its Regions.
- Do not choose it when: A strict single-Region residency rule applies.
- Native pairing: Model calls use cross-Region inference profiles. A global profile can route outside a strict EU boundary.

Decision rule: Check the guardrail scope against the residency rule before you choose cross-Region guardrail inference.

Trap: A global profile does not meet a strict EU residency rule. Use a geographic EU profile.

Exam tip: A single-Region rule with routing wording signals a conflict. Check the scope first.

Source: repo guide, Part 4 section 3.2, Part 8A4 cross-Region routing unit

Guardrail enforcement with IAM and SCPs

A condition on the bedrock:GuardrailIdentifier key can require a guardrail on each request.

- Methods or APIs: The bedrock:GuardrailIdentifier IAM condition key. Service control policies that apply across accounts.
- Pro: The rule applies across accounts. Application code changes do not remove it.
- Con: A service control policy sets only the most a member account can do. A user still needs an identity policy.
- Choose it when: Every team must call the guardrail on every request.
- Do not choose it when: One application is the only caller. Application code alone can be skipped.
- Native pairing: Deploy the guardrail with a StackSet. Deny model calls that omit the guardrail identifier in an SCP.

Decision rule: Enforce a required guardrail in IAM or an SCP when many accounts or teams call the model.

Trap: A guardrail that only the application code calls can be skipped. Enforce it at IAM.

Exam tip: Every call and every account point to an SCP or an IAM condition key.

Source: repo guide, Part 4 section 3.3, Part 8A5 StackSets and SCP unit

Automated Reasoning checks

Automated Reasoning checks test whether a response agrees with formal policy rules.

- Methods or APIs: Automated Reasoning checks, one of the six guardrail policy types.
- Pro: Formal rules can catch a claim that breaks a stated policy.
- Con: No prompt injection detection. English only. No streaming. The repo guide says they run in detect mode only.
- Choose it when: A claim must agree with formal rules, the text is English, and the output is not streamed.
- Do not choose it when: The main risk is a prompt attack, or the output must stream.
- Native pairing: Use content filters together with Automated Reasoning checks.

Decision rule: Use Automated Reasoning for formal rules. Pair it with content filters.

Trap: Automated Reasoning checks do not stop prompt injection. The prompt attack category does, on tagged input.

Exam tip: Formal rules point to Automated Reasoning. Jailbreak wording points to the prompt attack category.

Source: repo guide, Part 4 section 3.1, Part 8D red-teaming unit

Prompt attack filter

The prompt attack category detects jailbreak and injection patterns on input.

- Methods or APIs: The prompt attack category in content filters. Tag user input when you call InvokeModel or InvokeModelWithResponseStream.
- Pro: It targets attack patterns on the input before the model call.
- Con: Untagged input is not checked for prompt attacks. The filter does not evaluate tool results or tool definitions. Prompt leakage is detected on the Standard tier only.
- Choose it when: The app accepts user text that could carry jailbreak or injection patterns.
- Do not choose it when: A denied topic is the only control. A denied topic does not detect prompt attacks.
- Native pairing: Run adversarial test sets after each guardrail or prompt change.

Decision rule: Keep the prompt attack filter on for input. Tag user input on InvokeModel calls.

Trap: If the prompt attack filter is off, jailbreaks reach the model. No control stops every injection.

Exam tip: Words such as absolute or prevents all signal a wrong claim.

Source: repo guide, Part 4 section 3.1, Part 8D red-teaming unit

Sensitive information filters and PII detection

Sensitive information filters find PII entities in model input and output, then redact or block them.

- Methods or APIs: Sensitive information filters, Amazon Comprehend PII detection, and Amazon Macie for S3 objects at rest.
- Pro: Each place that PII can appear gets its own control. Comprehend real-time entity detection takes single documents up to 100 KB.
- Con: Macie does not watch live model traffic. The PII-only Comprehend limit is Not in source.

- Choose it when: PII must be redacted or blocked at the model call. Use Comprehend for text before the call. Use Macie for data at rest.
- Do not choose it when: The data sits in S3 and no model call happens. Use Macie.
- Native pairing: Invocation logs can hold PII. Encrypt the logs with KMS and restrict access.

Decision rule: Detect and redact PII at each place it appears, with a control for each place.

Trap: Invocation logs keep the original request even when the output is masked.

Exam tip: Data at rest in S3 points to Macie. The model call boundary points to Guardrails.

Source: repo guide, Part 4 section 3.2 privacy, Part 4 visual units 35 and 38

Prompt management and Prompt Flows

Prompt management

Prompt management stores prompts with variables, versions, and variants. An application deploys a prompt by version.

- Methods or APIs: A prompt with a name, a message, and an optional system prompt. Variables use double curly braces.
- Pro: A version is a saved snapshot that you can roll back to. CloudTrail tracks use. Approval workflows gate publication.
- Con: A base inference parameter set as an additional field does not override the prompt value. A prompt from Prompt management cannot include additionalModelRequestFields in the request.
- Choose it when: Prompts change often, need rollback, or need approval before release.

- Do not choose it when: A prompt changes in place. An in-place change has no safe rollback point.
- Native pairing: A prompt node in Prompt Flows uses a prompt from Prompt management. Lambda checks expected output.

Variants change the message, model, or inference configuration. Versions are saved snapshots. Variables use double curly braces.

Decision rule: Keep every released prompt as a version. Deploy by version. Roll back to the last approved version.

Trap: A prompt that changes in place has no safe rollback point.

Exam tip: Two wordings compared side by side point to variants. A frozen snapshot points to versions.

Source: repo guide, Part 1E visual unit 26, Part 1F practical unit 6

Prompt Flows

Prompt Flows links prompts, models, Knowledge Bases, and AWS services into one visual workflow.

- Methods or APIs: A console visual builder with prompt, Knowledge Base, and Lambda nodes. The repo guide also calls it Amazon Bedrock Flows.
- Pro: A fixed chain needs no workflow code. A flow can call Lambda.
- Con: Each model call in a flow is billed. Complex branching in code fits Lambda code or Step Functions better.
- Choose it when: The steps are fixed and predictable.
- Do not choose it when: A person must approve, the workflow waits for days, or retries must be durable.
- Native pairing: A Knowledge Base node runs retrieval inside the chain. A Lambda node calls other AWS services.

Decision tree: Prompt Flows or Step Functions

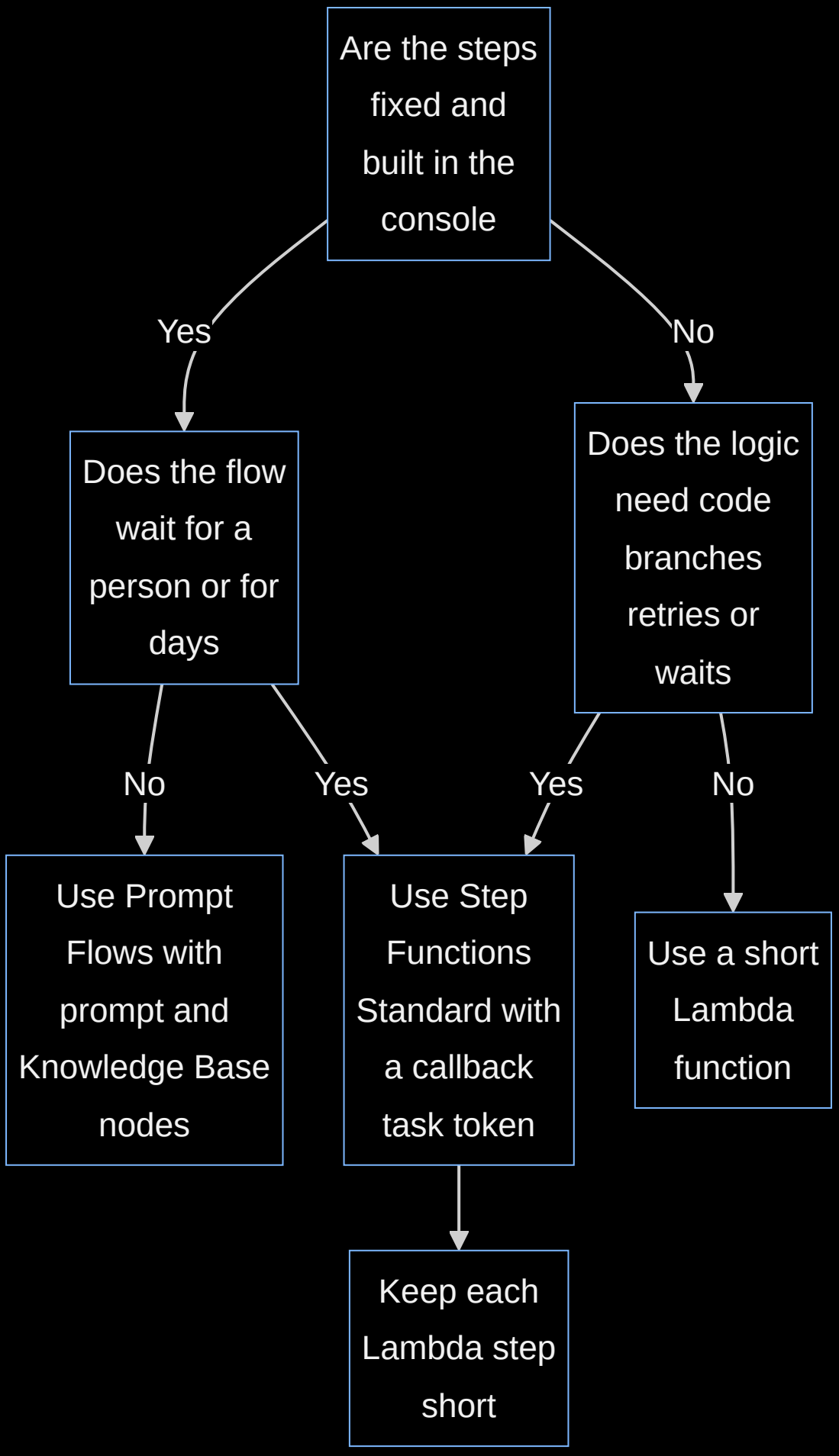


Figure: Prompt Flows or Step Functions. Fixed console chains use Prompt Flows. Waits and approvals use Step Functions.

Decision rule: Use Prompt Flows for a fixed chain built in the console. Use Step Functions for waits, retries, and approval.

Trap: A Lambda function that waits for a person hits the 15-minute limit. Use Step Functions Standard for multi-day waits.

Exam tip: A pause for a person, or a wait of hours or days, points to Step Functions.

Source: repo guide, Part 1E visual unit 27, Part 2 section 1.6

Bedrock Data Automation

Bedrock Data Automation

Bedrock Data Automation converts documents, images, video, and audio into structured formats through one API.

- Methods or APIs: One API for documents, images, video, and audio. Output can be customized. Blueprint mechanics: Not in source.
- Pro: Built-in safeguards include visual grounding and confidence scores.
- Con: Blueprint steps are Not in source. It does not check data quality rules or detect PII.
- Choose it when: Fields must come from documents, images, audio, or video.
- Do not choose it when: The job is data quality rules or speech to text. Use Glue Data Quality or Amazon Transcribe.
- Native pairing: It runs in the Process stage of the data pipeline. Glue Data Quality validates, and Comprehend adds PII detection.

Decision rule: Extract fields with Bedrock Data Automation. Validate with Glue Data Quality. Detect PII with Comprehend.

Trap: Guardrails filter content. Guardrails do not extract invoice fields.

Exam tip: Field extraction from a document points to Data Automation. Unsafe text points to Guardrails.

Source: repo guide, Part 2 section 1.3, Part 1E visual unit 23

Bedrock evaluations

Decision tree: evaluation method for the claim

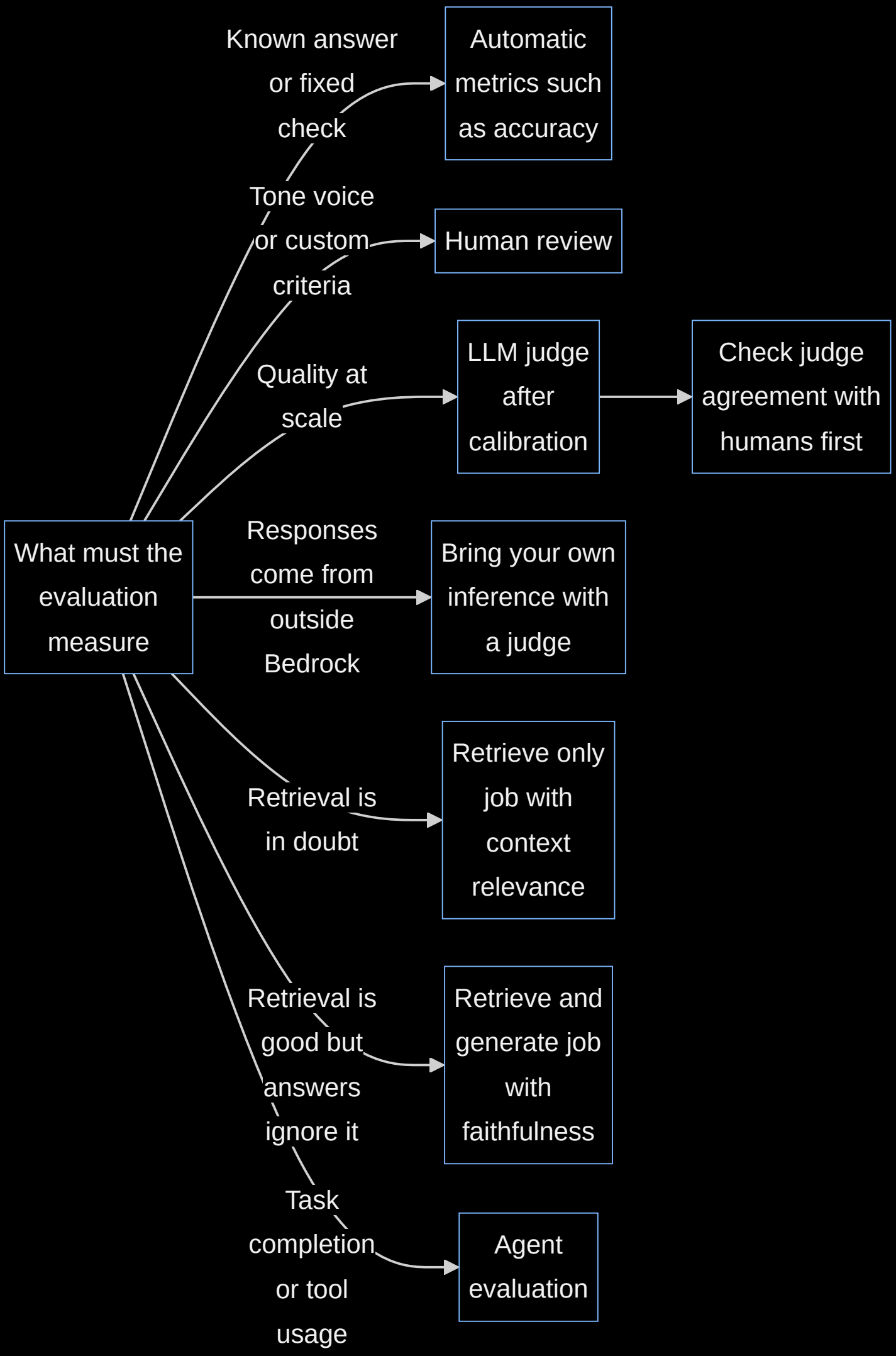

What must the evaluation measure
Known answer or fixed check
Automatic metrics such as accuracy
Tone voice or custom criteria
Human review
Quality at scale
LLM judge after calibration
Check judge agreement with humans first
Responses come from outside Bedrock
Bring your own inference with a judge
Retrieval is in doubt
Retrieve only job with context relevance
Retrieval is good but answers ignore it
Retrieve and generate job with faithfulness
Task completion or tool usage
Agent evaluation

Figure: Evaluation method for the claim. Pick the cheapest method that measures the claim.

Automatic evaluation

Automatic evaluation scores outputs with built-in metrics when a known answer or a fixed check exists.

- Methods or APIs: Built-in metrics named in the repo guide include accuracy and toxicity. Accuracy needs ground-truth labels. BLEU and ROUGE: Not in source.
- Pro: Objective and repeatable checks. The repo guide calls this the cheap option.
- Con: Does not measure tone, style, or voice.
- Choose it when: The claim has a known answer or a fixed check.
- Do not choose it when: The claim is about tone, voice, or custom criteria.
- Native pairing: Bedrock model evaluation jobs run on your dataset. Setup steps are Not in source.

Decision rule: Use automatic metrics for claims with known answers. Escalate only for what the cheap method cannot judge.

Trap: Automatic accuracy does not measure tone. Use human review for brand voice.

Exam tip: A known answer and an objective check point to automatic. Brand voice points to human review.

Source: repo guide, Part 6 section 5.1 evaluation matrix, Part 8A BLEU and ROUGE entry

LLM as a judge

An LLM judge scores another model output against criteria. Calibrate the judge before you trust its scores.

- Methods or APIs: A judge model in a model evaluation job. Metrics: correctness, faithfulness, harmfulness, style, and custom metrics.
- Pro: Scores quality at scale for RAG and agent evaluation.

- Con: An uncalibrated judge gives opinions, not measurements. A changed judge model changes the scores.
- Choose it when: Quality needs a score at scale, after calibration.
- Do not choose it when: The judge has no calibration. Brand voice needs human judgment.
- Native pairing: The judge scores your Bedrock dataset. For outside responses, use bring your own inference.

Decision rule: Calibrate the judge against human ratings on a fixed golden set. Pin the judge model ID.

Trap: An uncalibrated judge gives opinions, not measurements.

Exam tip: The phrase quality at scale with faithfulness points to a calibrated judge.

Source: repo guide, Part 6 section 5.1 evaluation matrix, Part 1E visual units 13 and 14

Judge calibration

Judge calibration compares judge scores with human ratings on the same fixed golden set.

- Methods or APIs: Rate a golden set with humans. Run the same set through the judge. Measure agreement. Pin the judge model ID.
- Pro: Turns judge opinions into a measure that you can defend.
- Con: Low agreement means the judge is not valid for that claim. Agreement counts only on the same prompts.
- Choose it when: A judge score drives a release gate or a report.
- Do not choose it when: The judge has not been checked against human ratings.
- Native pairing: The golden set feeds the release gate. A new judge model gives new scores, so the gate reruns.

Decision rule: Trust a judge only after agreement with human ratings on the same golden set.

Trap: A new judge model gives new scores. Compare old and new scores only after calibration.

Exam tip: Pin the judge model ID. Otherwise the score history changes without a change in the app.

Source: repo guide, Part 6 section 5.1 judges need calibration, Part 1E visual unit 13

Human evaluation

Human evaluation scores subjective quality, such as relevance, brand voice, and custom criteria.

- Methods or APIs: Human judgment in the evaluation matrix. A human evaluation job API: Not in source.
- Pro: Covers tone and voice that metrics cannot measure.
- Con: The source treats human review as the escalation step after cheaper methods.
- Choose it when: Brand voice or custom criteria need human judgment.
- Do not choose it when: A known-answer check settles the claim.
- Native pairing: Human ratings calibrate the judge on a fixed golden set.

Decision rule: Use human review for brand voice, tone, and custom criteria.

Trap: Automatic accuracy metrics cannot judge brand voice.

Exam tip: Brand voice points to human review, not to automatic accuracy.

Source: repo guide, Part 6 section 5.1 evaluation matrix, Part 6 visual unit 44

Bring your own inference

A judge job can score responses that come from outside Bedrock. You supply the responses in the prompt dataset.

- Methods or APIs: A model evaluation job with a judge model. Bedrock skips the model invoke step and scores the data that you supply.
- Pro: Scores non-Bedrock models and external RAG sources. Can score the final response that users see.
- Con: A check of only the model call misses errors in the code around it. An end-to-end evaluation feature: Not in source.
- Choose it when: Responses already come from your own system.
- Do not choose it when: You need Bedrock to generate the responses for the test.
- Native pairing: The same metrics apply to your responses.

Decision rule: To score a full application, store the final response that users see in the dataset.

Trap: A check of only the model call misses errors in the code around it.

Exam tip: Responses that already exist in your own system point to bring your own inference.

Source: repo guide, Part 8A4 bring-your-own-inference unit, Part 6 section 5.1

Agent evaluation

Agent evaluations measure task completion and tool usage.

- Methods or APIs: Agent evaluation methods appear in the Bedrock evaluation documentation. Metric names: Not in source.
- Pro: Measures the result of an agent run, not only one model output.
- Con: The repo guide names no agent evaluation metric. Step-by-step setup is Not in source.
- Choose it when: The app is an agent and the claim is task completion or tool use.
- Do not choose it when: The claim is retrieval quality only.
- Native pairing: Traces show each step, the chosen tool, and the observation.

Decision rule: Score agents on task completion and tool usage. Use traces to find the failed step.

Trap: A passing test on DRAFT does not prove that the alias works.

Exam tip: Task completion and tool usage are the named measures. Pick agent evaluation for those two.

Source: repo guide, Part 6 section 5.1 safety tests, Part 6 visual unit 68

Amazon Bedrock Agents

Amazon Bedrock Agents

An agent reasons about the next step, calls a tool, observes the result, and repeats until a stop condition is met.

- Methods or APIs: invoke_agent, action groups with Lambda functions, a linked knowledge base, and enableTrace for step records.
- Pro: The service runs the agent loop and the tool calls. Trace output records each step.
- Con: Amazon Bedrock Agents is no longer open to new customers. The repo guide marks its rows for existing use only.
- Choose it when: Existing use continues on this service.
- Do not choose it when: A new build starts. The source names Amazon Bedrock AgentCore as the alternative.
- Native pairing: Lambda action groups call the external API. DynamoDB holds customer memory with a time to live.

Decision rule: For a new build, choose AgentCore. Keep Bedrock Agents for existing use only.

Trap: A prompt that says stop after three steps is not a control. Enforce the limit in the workflow or the tool layer.

Exam tip: Words such as new customer or new build point to AgentCore.

Source: repo guide, Part 1A component table, Part 3 section 2.1

invoke_agent

invoke_agent takes agentId, agentAliasId, sessionId, and inputText. It returns a stream.

- Methods or APIs: invoke_agent. The enableTrace setting turns on a trace record for each step.
- Pro: Callers send the alias ID. The alias can move to a new version without a caller change.
- Con: The sessionId is the memory key for one conversation. The agent session TTL is between 60 and 5400 seconds.
- Choose it when: A caller invokes the agent through an alias.
- Do not choose it when: A production caller sends DRAFT traffic.
- Native pairing: Lambda runs each action group call. Lambda and Step Functions bound the time of each step.

Decision rule: Send agentAliasId in production. Turn on enableTrace when you need the step record.

Trap: The agent session is not the memory store. It expires.

Exam tip: sessionId names one conversation. Memory for weeks needs a store with a time to live.

Source: repo guide, Part 3 section 2.1 invoke_agent, Part 3 visual unit 49

DRAFT, versions, and aliases

Edits go to DRAFT. Callers reach a version only through an alias that points to it.

- Methods or APIs: Prepare the agent after an edit. Create a version from the prepared DRAFT. Point the alias to the version.
- Pro: Callers keep the same agent ID and alias ID when the version changes.

- Con: A console test on DRAFT does not prove the alias works. The prepare and alias console steps are Not in source.
- Choose it when: Any agent change goes live.
- Do not choose it when: Production callers send DRAFT traffic.
- Native pairing: The release path names the alias or AppConfig as the traffic mover.

Decision tree: agent alias release

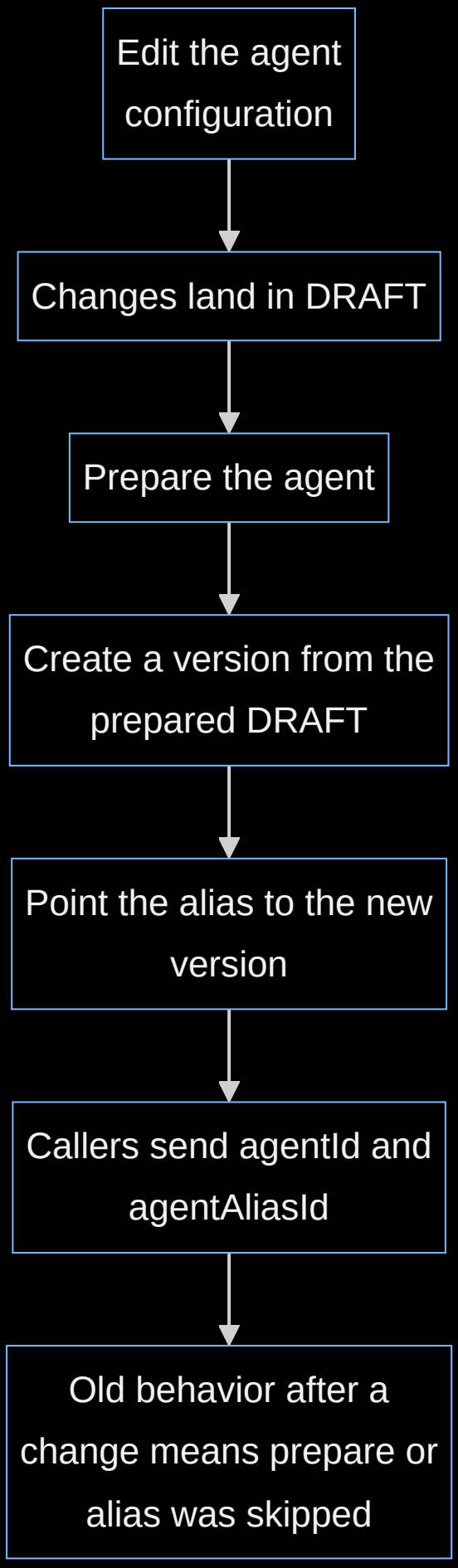


Figure: Agent alias release. Callers keep their IDs while the version changes.

Decision rule: Prepare, create a version, then move the alias. Production callers never use DRAFT.

Trap: An agent that runs without prepare keeps old behavior. Prepare the agent, then move the alias.

Exam tip: Old behavior after an edit points to prepare or alias. Check the alias version first.

Source: repo guide, Part 3 section 2.1, Part 1E visual unit 8

Action groups and tool executors

An action group links an agent to a Lambda function that performs an action for the agent.

- Methods or APIs: Action groups in the agent build. The Lambda function acts as the executor for each action.
- Pro: The agent chooses the action. Your code performs it.
- Con: A model cannot enforce token refresh or field removal. The token lifetime for any specific API is Not in source.
- Choose it when: The agent must call an external API through the executor. A database call through an action group is Not in source.
- Do not choose it when: Two agents must work together. An action group does not link two agents.
- Native pairing: The executor holds the OAuth bearer token, refreshes it, and removes fields the agent must not see.

Decision rule: Put credentials, token refresh, and response filtering in the executor Lambda function.

Trap: A prompt that asks the model to refresh a token or hide fields does not enforce either rule.

Exam tip: An action group is a tool link. Words such as link two agents point to collaboration.

Source: repo guide, Part 8A4 action groups unit, Part 7 traps table

Session memory and customer memory

An agent session expires. Customer memory that must last weeks belongs in a store with a key and a time to live.

- Methods or APIs: sessionId for session memory. A DynamoDB table keyed by customer or session ID. Native TTL removes old records.
- Pro: DynamoDB gives single-digit millisecond lookup by key. It encrypts data at rest.
- Con: The agent session TTL is between 60 and 5400 seconds. A session cannot hold customer memory for weeks.
- Choose it when: Memory must outlive one conversation.
- Do not choose it when: Context only needs to last one conversation.
- Native pairing: AgentCore Memory gives agents short-term and long-term context. DynamoDB holds customer memory.

Decision rule: Keep one conversation in the session. Keep customer memory in DynamoDB with native TTL.

Trap: The agent session is not the memory store. It expires.

Exam tip: Memory for weeks points to DynamoDB with TTL. One conversation points to the session.

Source: repo guide, Part 3 section 2.1 session memory, Part 3 visual unit 49

Multi-agent collaboration

A supervisor agent reads the request and routes each part to a collaborator agent.

- Methods or APIs: A supervisor agent and collaborator agents. Each collaborator uses its own action group.

- Pro: Each collaborator owns its tools. The supervisor classifies intent in natural language and does not call the tools itself.
- Con: A DRAFT collaborator cannot take delegations. Prepare and alias the collaborator first.
- Choose it when: Separate parts of the work need separate tools. Design guidance.
- Do not choose it when: One agent with one action group covers the work.
- Native pairing: Each collaborator calls its own action group, which runs as a Lambda function.

Decision rule: Use a supervisor when each part of the work needs a different set of tools.

Trap: Delegation is not an action group. An action group does not link two agents.

Exam tip: The phrase link two agents points to the built-in supervisor and collaborator pattern.

Source: repo guide, Part 3 section 2.1 multi-agent collaboration, Part 3 visual unit 30

Amazon Bedrock AgentCore

AgentCore overview

AgentCore runs agents built with any framework behind one interface. AgentCore is not a model.

- Methods or APIs: Runtime, Memory, Gateway, Identity, Policy, and Observability. Browser and Code Interpreter: Not in source.
- Pro: One service covers run, memory, tools, identity, rules, and traces.
- Con: The repo guide confirms six components. Browser and Code Interpreter are Not in source.
- Choose it when: A new agent needs hosting, governed tools, and traces.

- Do not choose it when: You need a model. AgentCore hosts and governs the agent around the model.
- Native pairing: The agent path runs an action group or an MCP tool through a gateway.

Decision tree: AgentCore component for each need

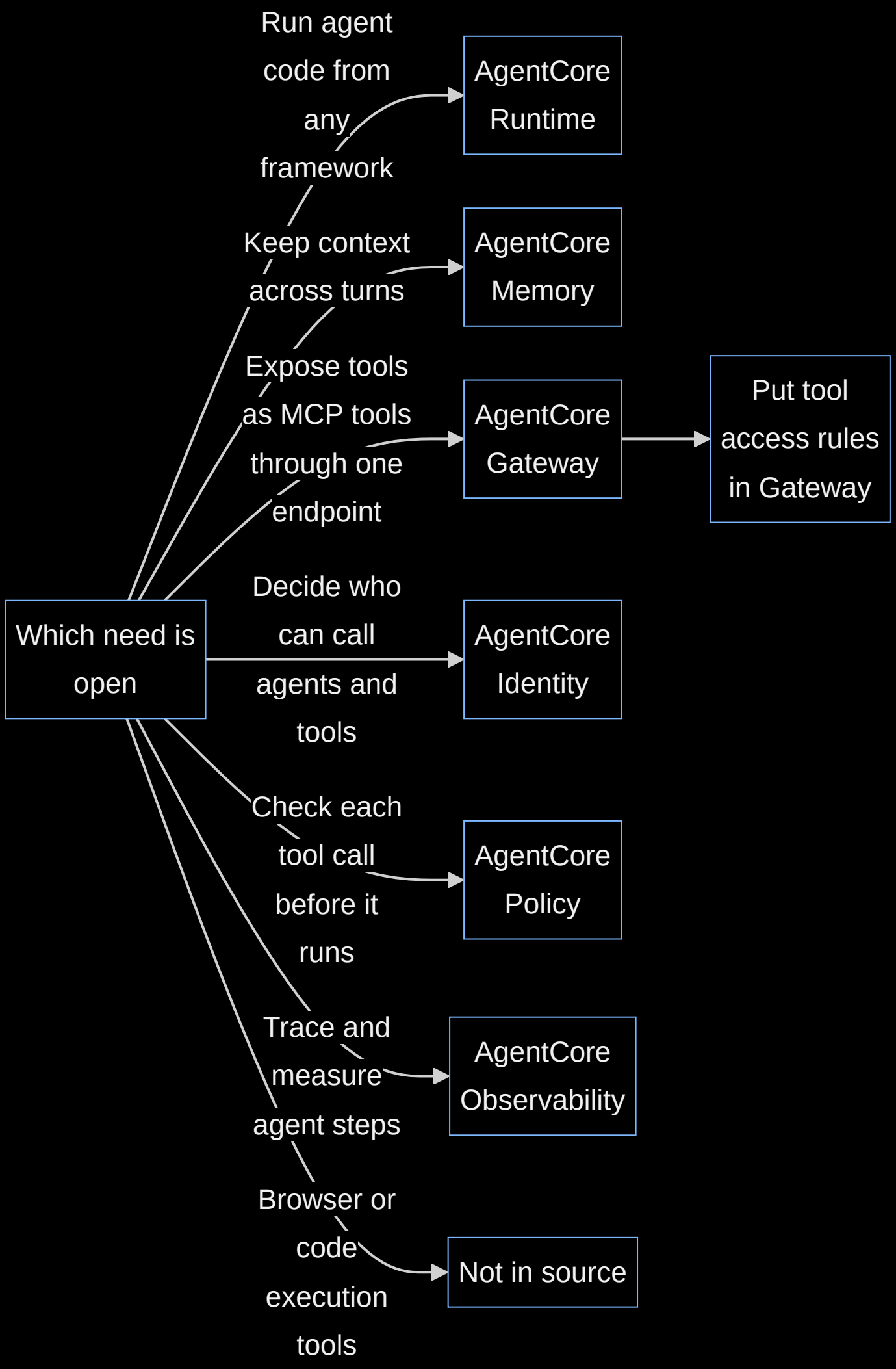


Figure: AgentCore component for each need. Match the need to one component.

Decision rule: Choose AgentCore for a new agent that needs governed tools and traces.

Trap: AgentCore is not a model. It hosts and governs the agent around the model.

Exam tip: A place to run, memory, tools, identity, rules, and traces points to AgentCore as one service.

Source: repo guide, Part 1A architecture 2 and component table, Part 1E visual unit 9

AgentCore Runtime

Runtime hosts the agent code from any framework.

- Methods or APIs: Hosts agent code and remote MCP servers, which use Streamable HTTP. The repo guide names Strands Agents and AWS Agent Squad.
- Pro: Works with any framework. An open-source framework gives code-level control.
- Con: Heavy, stateful MCP servers need a different host. That host is Not in source.
- Choose it when: Agent code or a remote MCP server needs a managed host.
- Do not choose it when: The tool is light, stateless, and bursty. Lambda hosts that tool better.
- Native pairing: The agent calls its MCP tools through a gateway.

Decision rule: Host remote MCP servers on Runtime with Streamable HTTP. Host light, stateless tools on Lambda.

Trap: A remote MCP server on AgentCore Runtime does not use another transport here.

Exam tip: The words remote MCP server with AgentCore Runtime point to Streamable HTTP.

Source: repo guide, Part 1A component table, Part 3 section 2.1 MCP hosting

AgentCore Memory

Memory gives an agent short-term and long-term context.

- Methods or APIs: The Memory service. Method names and retention settings: Not in source.
- Pro: Keeps context between turns across two time scales.
- Con: The repo guide gives no API or retention details.
- Choose it when: The agent needs context across turns.
- Do not choose it when: Business records must last for weeks. The source names DynamoDB with a time to live for customer memory.
- Native pairing: A session ID keys one conversation. DynamoDB pairs with agent memory for customer history.

Decision rule: Use Memory for agent context. Use DynamoDB with TTL for customer memory that lasts weeks.

Trap: The agent session expires. Memory that must outlive it needs a store with a time to live.

Exam tip: Context between turns points to Memory. Weeks of customer history point to DynamoDB.

Source: repo guide, Part 1A component table, Part 3 section 2.1 memory

AgentCore Gateway

Gateway exposes tools as MCP tools through one endpoint. It is a fully managed AI gateway.

- Methods or APIs: One gateway endpoint for agent traffic. Protocol details for MCP tool servers: Not in source.

- Pro: One entry point connects agents to tools, other agents, and models. It includes observability and auditing.
- Con: A gateway does not replace the guardrail. Enforce the guardrail at IAM too.
- Choose it when: Many agents or teams need one controlled path to tools and models.
- Do not choose it when: The source gives no rule for one private tool. Inference: a gateway suits shared tools.
- Native pairing: Tool Lambda functions or MCP servers sit behind the gateway. Policy checks each tool call.

Decision rule: Put tool access rules in Gateway. Keep the tool code behind it.

Trap: A gateway does not replace the guardrail. Enforce both.

Exam tip: One endpoint for tools points to Gateway. A check on each tool call points to Policy.

Source: repo guide, Part 1A component table, Part 3 section 2.3

AgentCore Identity

Identity controls who and what can call agents and tools.

- Methods or APIs: The Identity service for agents and tools. Inbound authentication mechanics: Not in source.
- Pro: The source says it covers both who and what can call an agent or tool.
- Con: The repo guide gives no method names.
- Choose it when: A caller must be checked before an agent or tool runs.
- Do not choose it when: The check applies to each tool call. Use Policy for that check.
- Native pairing: IAM Identity Center gives workforce users federated access. It is a separate service.

Decision rule: Use Identity for who can call. Use Policy for what each tool call may do.

Trap: IAM Identity Center and AgentCore Identity have similar names. Check which one handles agent callers.

Exam tip: Who can call points to Identity. Each tool call points to Policy.

Source: repo guide, Part 1A component table, Part 3 section 2.2

AgentCore Policy

Policy checks each tool call through Gateway before the tool runs.

- Methods or APIs: A policy check on each tool call. Rule syntax and language: Not in source.
- Pro: The check runs before each tool action, not only at sign-in.
- Con: The repo guide gives no rule syntax or policy examples.
- Choose it when: Each tool call needs a rule check before it runs.
- Do not choose it when: The only need is a caller sign-in. Use Identity.
- Native pairing: Policy sits after Gateway and before the Lambda or MCP tool.

Decision rule: Put each tool-call rule in Policy. Run it before the tool.

Trap: A prompt that says stop is not a control. Enforce tool rules in Policy or in the tool layer.

Exam tip: A check on each tool call before it runs points to Policy.

Source: repo guide, Part 1A architecture 2, Part 1E visual unit 9

AgentCore Observability

Observability records traces and metrics for agent steps.

- Methods or APIs: Trace records for each step, the chosen tool, and the observation. Metrics for agent steps.
- Pro: Traces show the slow or wrong step. Metrics show time and cost.

- Con: Metrics alone cannot rebuild what the model received.
- Choose it when: An agent step is slow or wrong, and you need that step.
- Do not choose it when: You need the full payload. Use invocation logs with access control.
- Native pairing: X-Ray traces downstream calls. CloudWatch holds metrics, alarms, and invocation logs.

Decision rule: Use traces to find the slow or wrong step. Use logs to see the payload.

Trap: Metrics alone cannot rebuild what the model received.

Exam tip: Which step points to traces. What was sent points to logs.

Source: repo guide, Part 1A component table, Part 1E visual units 14 and 15

AgentCore Browser and AgentCore Code Interpreter

Not in source. The repo guide does not name these components.

- Methods or APIs, pros, cons, and native pairing: Not in source.

Decision rule: Confirm these components on the official AgentCore pages before any design uses them.

Trap: A component name that the repo guide does not confirm is not a confirmed feature.

Exam tip: Treat unknown component names as Not in source, not as true features.

Not in source

MCP tools and Streamable HTTP

Model Context Protocol (MCP) tools

MCP is a standard for agents to find and call tools. Tools announce what they do.

- Methods or APIs: An MCP client and an MCP server that exposes tools. Protocol details beyond Streamable HTTP: Not in source.
- Pro: One protocol replaces a custom client for each tool.
- Con: MCP is not a model and not a guardrail. It is not a safety control.
- Choose it when: Agents call tools from several servers through one standard.
- Do not choose it when: The need is a safety check, a model choice, or a data check.
- Native pairing: AgentCore Gateway exposes tools as MCP tools. Runtime hosts remote servers. Lambda hosts light tools.

Decision rule: Use MCP for the tool connection. Use Guardrails or Policy for safety checks.

Trap: MCP is the tool connection standard. It is not a safety control.

Exam tip: A standard for agents to find and call tools points to MCP. A check on each call points to Policy.

Source: repo guide, Part 1E visual unit 62, Part 2 section 1.5

Streamable HTTP transport

A remote MCP server on AgentCore Runtime uses Streamable HTTP.

- Methods or APIs: Streamable HTTP for remote MCP servers. Other transports: Not in source.
- Pro: One named transport for remote servers on Runtime.
- Con: The repo guide names no other transport.

- Choose it when: A remote MCP server runs on AgentCore Runtime.
- Do not choose it when: The tool is light, stateless, and bursty. Use Lambda.
- Native pairing: Runtime hosts the server. Gateway exposes its tools to the agent.

Decision rule: A remote MCP server on AgentCore Runtime uses Streamable HTTP.

Trap: A different transport for a remote MCP server on Runtime breaks the stated requirement.

Exam tip: The words remote MCP with AgentCore Runtime point to Streamable HTTP.

Source: repo guide, Part 3 section 2.1 MCP hosting, Part 1E visual unit 50

Amazon SageMaker AI

This part covers the SageMaker AI components that the repo guide names. Each entry gives the purpose, the limits, and the decision rule. Where the repo guide gives no detail, the entry says Not in source.

Real-time endpoint

A real-time endpoint serves one request at a time with low latency. You deploy a model to SageMaker AI hosting and get an endpoint.

- Methods or APIs: Not in source for endpoint API names.
- Pro: Fits steady traffic. The repo guide names autoscaling for this endpoint type.
- Pro: Fits interactive work where a user waits for the answer.
- Con: The repo guide says idle traffic often suits the serverless endpoint.
- Con: A job that runs 45 minutes is the wrong pick for this endpoint type.
- Choose it when: traffic is steady and each caller waits for a reply.

- Do not choose it when: the work is a long job. Use an asynchronous endpoint.
- Native pairing: API Gateway or Lambda can front the endpoint. See the gateway entry below.

Decision rule: Pick a real-time endpoint for steady traffic when a user waits for each reply.

Trap: A real-time endpoint for a 45-minute job is the wrong pick. The repo guide flags this as a trap.

Exam tip: Steady traffic and a user who waits point to real-time. A long job points to asynchronous.

Source: repo guide, Part 3 Domain 2 table (SageMaker endpoints). Part 8A8 unit N3.

Serverless endpoint (serverless inference)

A serverless endpoint serves intermittent traffic and scales to zero. Its memory runs from 1 GB to 6 GB.

- Methods or APIs: Not in source for API names. Provisioned Concurrency is the named setting.
- Pro: Scales to zero.
- Pro: Provisioned Concurrency keeps capacity warm for predictable bursts.
- Con: The first request after idle time can have a cold start.
- Con: The cold start duration is Not in source.
- Choose it when: traffic is intermittent and a cold start on the first request is acceptable.
- Do not choose it when: traffic is steady. The repo guide names the real-time endpoint for steady traffic.
- Native pairing: The gateway chain in the Lambda and API Gateway entry applies. A serverless call pattern is Not in source.

Decision rule: Pick a serverless endpoint for intermittent traffic when a cold start is acceptable.

Trap: The serverless endpoint scales to zero. The first request after idle time can start cold.

Exam tip: Intermittent traffic and scales to zero point to serverless. Steady traffic points to real-time.

Source: repo guide, Part 1C Amazon SageMaker AI, Hosting. Part 3 Domain 2 table.

Asynchronous endpoint (asynchronous inference)

An asynchronous endpoint queues long requests. Its payload comes from S3, and the result goes to S3.

- Methods or APIs: create model, create endpoint configuration, and create endpoint operations. The endpoint configuration holds the asynchronous settings.
- Pro: Supports processing times up to one hour.
- Pro: The caller reads the result from the S3 output location.
- Con: Latency is near real-time, not real-time.
- Con: The caller sends the request payload to S3 first.
- Con: Processing longer than one hour is Not in source for this endpoint type.
- Choose it when: a job runs up to one hour and the result can wait in S3.
- Do not choose it when: a user waits for each reply with low latency. Use the real-time endpoint.
- Native pairing: S3 holds the input and the output. The repo guide does not name an event or polling step for the result. Not in source.

Create an asynchronous endpoint (repo guide steps)

- Create a model with the create model operation.
- Create an endpoint configuration with the create endpoint configuration operation.

- Set the asynchronous settings.
- Create the HTTPS endpoint with the create endpoint operation.
- Send the request payload to S3.
- Read the result from the S3 output location.

Not in source for the other console fields on this page.

Decision rule: Pick an asynchronous endpoint when the job runs up to one hour and S3 holds the result.

Trap: A real-time endpoint for a 45-minute job is the wrong pick. Use the asynchronous endpoint.

Exam tip: Queue, S3 output, and a one-hour job point to the asynchronous endpoint.

Source: repo guide, Part 1C Amazon SageMaker AI, Hosting. Part 3 Domain 2 table. Part 1F practical unit 10.

Endpoint type decision tree

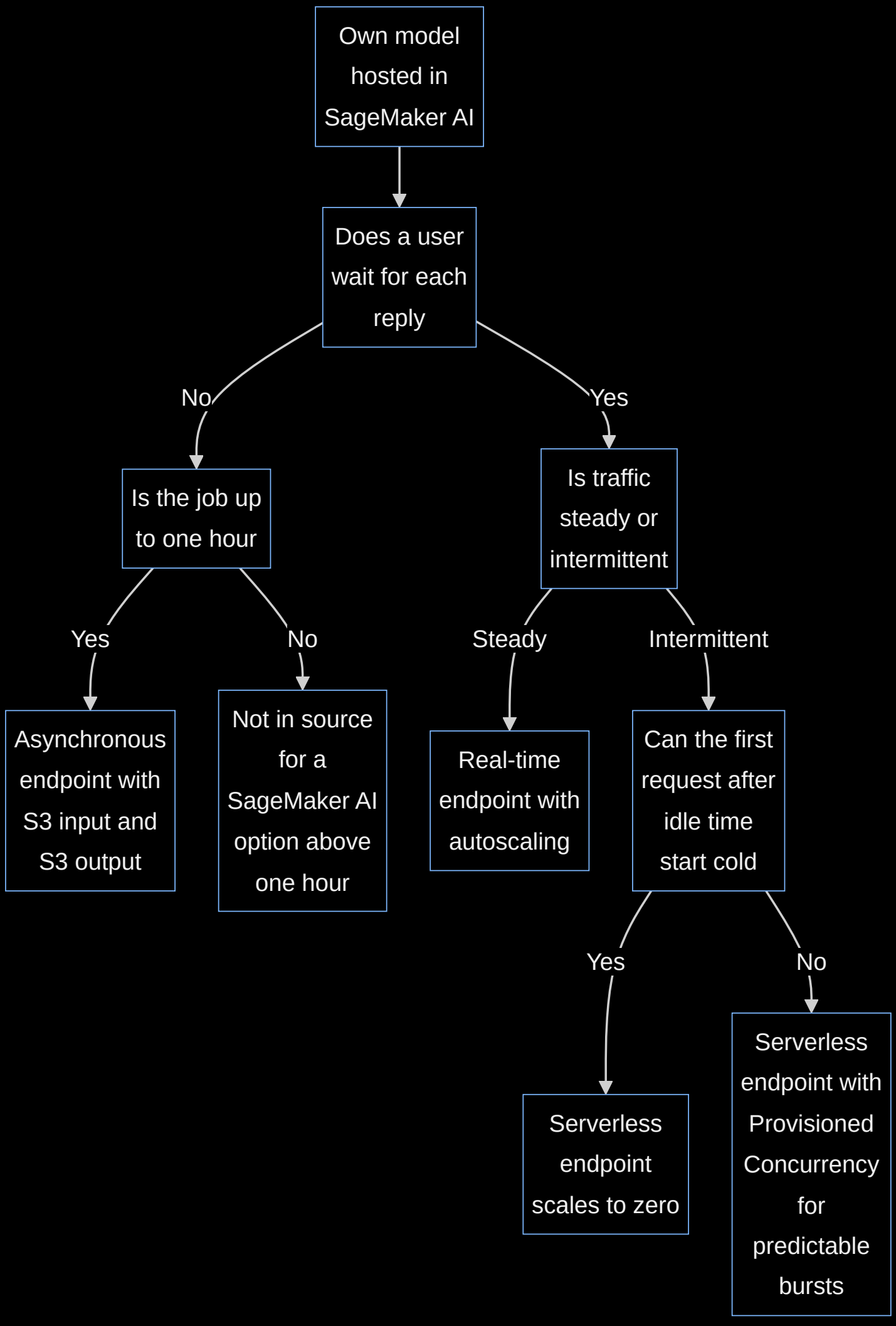


Figure: Endpoint type for a model you host in SageMaker AI. Each answer leads to one endpoint type.

Batch transform

Not in source. The repo guide does not name a SageMaker batch transform job.

- Methods or APIs: Not in source.
- Pro: Not in source.
- Con: Not in source.
- Choose it when: Not in source. For offline Bedrock work, the repo guide names batch inference from S3 to S3.
- Do not choose it when: a user waits for a reply. The repo guide lists a batch job for interactive chat as a misconception.
- Native pairing: Not in source for SageMaker. The repo guide pairs Bedrock batch inference with S3 input and output.

Decision rule: For offline Bedrock work, the repo guide names batch inference. For a SageMaker batch transform, the answer is Not in source.

Trap: The repo guide uses batch for Bedrock batch inference and for SageMaker Processing jobs. Neither one is a batch transform entry.

Exam tip: Offline work that can wait points to batch. A user who waits for a reply points to real-time.

Not in source

Deployment patterns for endpoints

The repo guide names autoscaling and blue-green and rolling deployments for real-time endpoints. It gives no steps for these patterns.

- Methods or APIs: Not in source for deployment API names and traffic settings.
- Pro: The repo guide names autoscaling for real-time endpoints that serve steady traffic.

- Pro: Each release keeps a version that can roll back. A rollback points the endpoint to an earlier approved version.
- Con: Blue-green and rolling steps are Not in source. Do not assume traffic split values.
- Con: Autoscaling policy values are Not in source. The repo guide lists Auto Scaling for GenAI workloads as a gap.
- Choose it when: a release needs a rollback target and the endpoint must grow and shrink with traffic.
- Do not choose it when: the model or prompt changes in place. That change has no safe rollback point.
- Native pairing: AppConfig can roll out a model ID change in steps. A CloudWatch alarm rolls it back. The repo guide calls this the model switch pattern.

Decision rule: Keep the previous approved version. Point the endpoint back to it when a release fails.

Trap: A model that changes in place has no safe rollback point. Register each version first.

Exam tip: No code change or no redeploy points to a configuration rollout, not to a new model version.

Source: repo guide, Part 3 Domain 2 table (SageMaker endpoints). Part 1C AWS AppConfig and SageMaker AI. Part 2 task 1.2. Part 9 gaps.

SageMaker Processing

The repo guide places SageMaker Processing in the Process stage, for batch jobs.

- Methods or APIs: Not in source.
- Pro: The repo guide names it for batch data jobs.
- Con: Job settings, instance types, and outputs are Not in source.

- Choose it when: a batch data job runs in the Process stage of data preparation.
- Do not choose it when: the input is speech or media. The repo guide names Amazon Transcribe for speech and Bedrock Data Automation for documents, images, audio, and video.
- Native pairing: The Validate stage names AWS Glue Data Quality, Lambda checks, and CloudWatch metrics. The Enhance stage names Amazon Comprehend and Lambda.

Decision rule: For a batch data job, the repo guide names SageMaker Processing. For speech or media, it names other services.

Trap: The repo guide shows no Processing job API. Do not assume one.

Exam tip: Match the data type first. Speech goes to Transcribe, and documents and media go to Bedrock Data Automation.

Source: repo guide, Part 2 Domain 1 task 1.3 (data validation and processing in four stages).

Training and fine-tuning jobs

SageMaker AI fine-tunes a model. LoRA trains small adapters on frozen base weights. Distillation trains a smaller student from a teacher.

- Methods or APIs: Not in source for training job API names, instance types, and hyperparameters.
- Pro: Fine-tuning changes model weights with labeled examples.
- Pro: LoRA freezes the base weights and trains small adapter matrices. The adapter is much smaller than the base model.
- Pro: You can store several adapters and load the one that fits the task.
- Con: A tuned adapter still holds facts from its training data.
- Con: Training run times and costs are Not in source.
- Choose it when: prompts and RAG fail, and the behavior must stay stable. Distillation fits a narrow task at high volume.

- Do not choose it when: the facts change or need citations. Use RAG for those facts.
- Native pairing: Register each tuned version in Model Registry. Deploy the approved version to a SageMaker AI endpoint. Track its approval status.

Decision rule: Try prompts first, then RAG for changing facts. Fine-tune only for stable behavior that prompts cannot hold.

Trap: Fine-tuning on documents that change is wrong. Knowledge goes to retrieval. Behavior goes to weights.

Exam tip: Stable behavior at high volume points to fine-tuning or distillation. Changing documents with citations point to RAG.

Source: repo guide, Part 1C Amazon SageMaker AI, Customization. Part 1B Lesson 7. Part 2 task 1.2 customization ladder.

Training versus customization decision tree

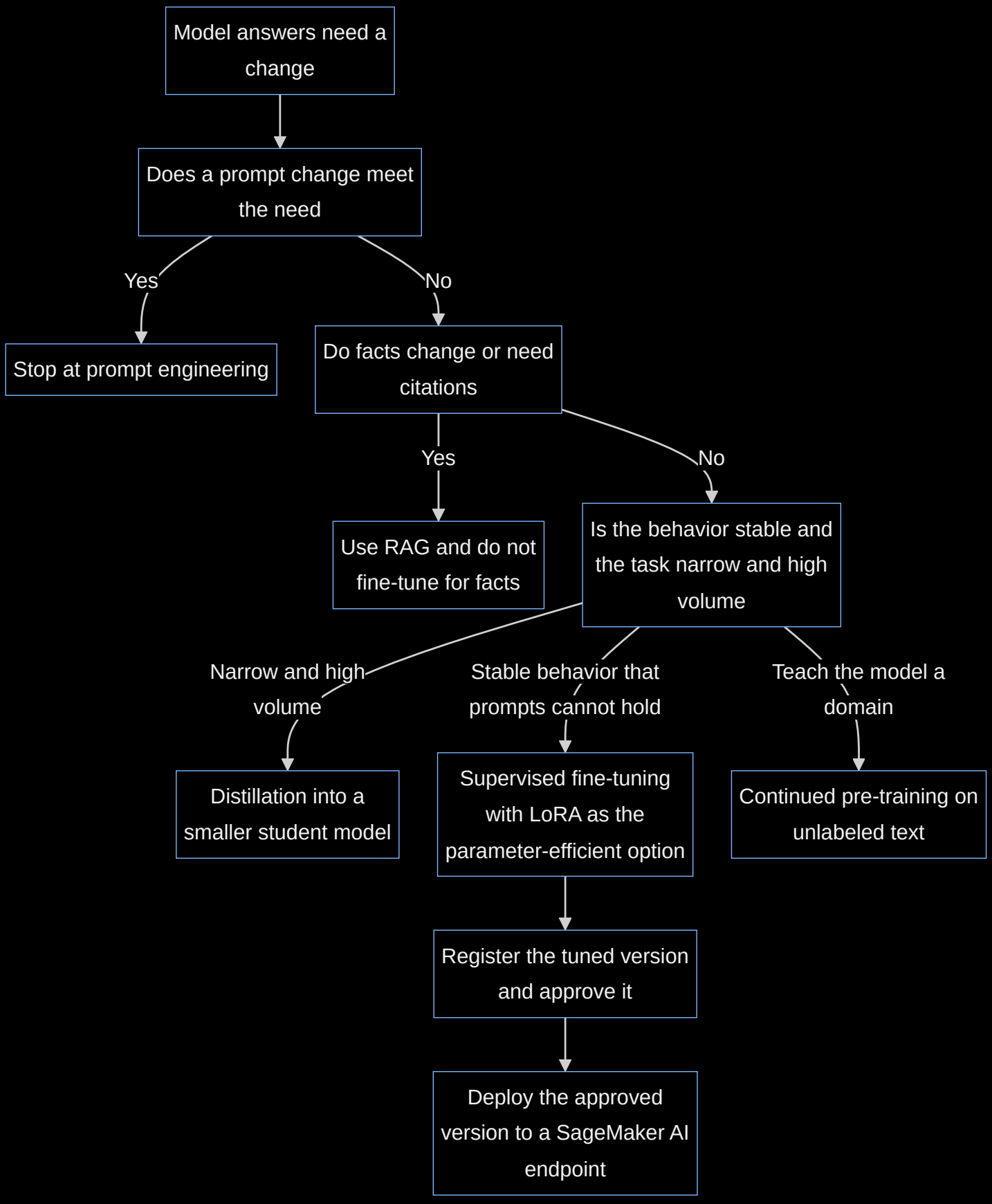


Figure: Customization rungs. Move up only when the cheaper rung fails a measured test.

SageMaker Pipelines

Pipelines can require approval before deployment.

- Methods or APIs: Not in source.

- Pro: Approval gates the deployment step.
- Pro: With a project template, an approved version can start CI/CD deployment.
- Con: Pipeline steps and run settings are Not in source.
- Choose it when: a release needs an approval gate before deployment.
- Do not choose it when: the wait for a person lasts hours or days. Use Step Functions Standard for that wait.
- Native pairing: Model Registry holds the approval status. Step Functions uses a task token for a durable human approval pause.

Decision rule: Gate deployment on approval status in Model Registry. For long human waits, use Step Functions Standard.

Trap: The repo guide names the approval gate only. Pipeline step names are Not in source.

Exam tip: Approval before deployment points to an approval gate. A wait measured in hours or days points to Step Functions Standard.

Source: repo guide, Part 1C Amazon SageMaker AI, Registry. Part 8B Approve a model version. Part 3 Domain 2 human approval unit.

Model Registry

Model Registry groups model versions. Each version has an approval status.

- Methods or APIs: update_model_package, with ModelApprovalStatus set to Approved or Rejected.
- Pro: Groups versions and tracks the approval status of each one.
- Pro: A registry can share versions across accounts.
- Pro: A rollback points the endpoint to an earlier approved version.
- Con: Steps for cross-account sharing are Not in source.
- Con: The registry tracks versions. Hosting happens on an endpoint.

- Choose it when: a model needs version records, an approval status, and a rollback target.
- Do not choose it when: the model changes in place. A changed model has no safe rollback point.
- Native pairing: Model Cards link to the model package. Pipelines can gate deployment on approval. Endpoints deploy the approved version.

Approve a model version

- Open the Studio console.
- Choose Models.
- Choose the Registered models tab.
- Choose the model group.
- Select the model version.
- On the Deploy tab, choose the approval status dropdown.
- Select Approved or Rejected.

The API call for the same change is update_model_package with ModelApprovalStatus set to Approved or Rejected. Approved can start CI/CD deployment when a project template is used.

Decision rule: Approve a version before an endpoint deploys it. Keep the previous approved version for rollback.

Trap: The repo guide names only Approved and Rejected as status values. Do not assume other status values.

Exam tip: Approval status and rollback point point to Model Registry. Intended use points to Model Cards, and data access points to Lake Formation.

Source: repo guide, Part 1C Amazon SageMaker AI, Registry. Part 2 task 1.2 Model lifecycle. Part 8B Approve a model version.

SageMaker Model Cards

A versioned record of intended use, limits, and evaluation results.

- Methods or APIs: Not in source for a Model Card API. The repo guide says to check the Model Card JSON schema for the fields.
- Pro: Documents intended use and limits for audit.
- Pro: Links to the model package in Model Registry.
- Con: Describes the model. It does not control who reads data.
- Con: A model card alone does not explain one decision. Add citations and the trace.
- Choose it when: an audit asks for intended use, limits, or evaluation results of a model version.
- Do not choose it when: the question asks which rows or columns a team can read. Use Lake Formation.
- Native pairing: Model Registry holds the model package. Lake Formation controls data access. CloudTrail records API calls for audit.

Decision rule: Use Model Cards for what a model is for. Use Lake Formation for who reads which data.

Trap: Model Cards describe the model. Lake Formation controls data access. The two answers differ.

Exam tip: Intended use and risk rating point to Model Cards. A salary column or a team limited to its own rows points to Lake Formation.

Source: repo guide, Part 1A component table. Part 4 Domain 3 Visual unit 64 and governance mapping.

SageMaker Model Monitor

Model Monitor watches drift in structured features and predictions. It does not score free-text answer quality.

- Methods or APIs: Not in source. Model Monitor and judge evaluation job steps are Not in source.
- Pro: Watches statistical drift in structured features and predictions.

- Pro: Covers one signal. The other signals come from CloudWatch, X-Ray, and invocation logs.
- Con: The repo guide says the SageMaker documentation states Model Monitor is no longer open to new customers. Check that status before you rely on it.
- Con: Free-text answer quality needs a judge or a golden set.
- Choose it when: the signal is drift in structured inputs or predictions, and the account can use the feature.
- Do not choose it when: the question asks whether an answer is accurate, grounded, or well written. Use a judge or a golden set.
- Native pairing: CloudWatch metrics track tokens, invocations, and throttles. X-Ray traces the slow step. Invocation logs hold the exact prompts and completions.

Monitoring signal decision tree

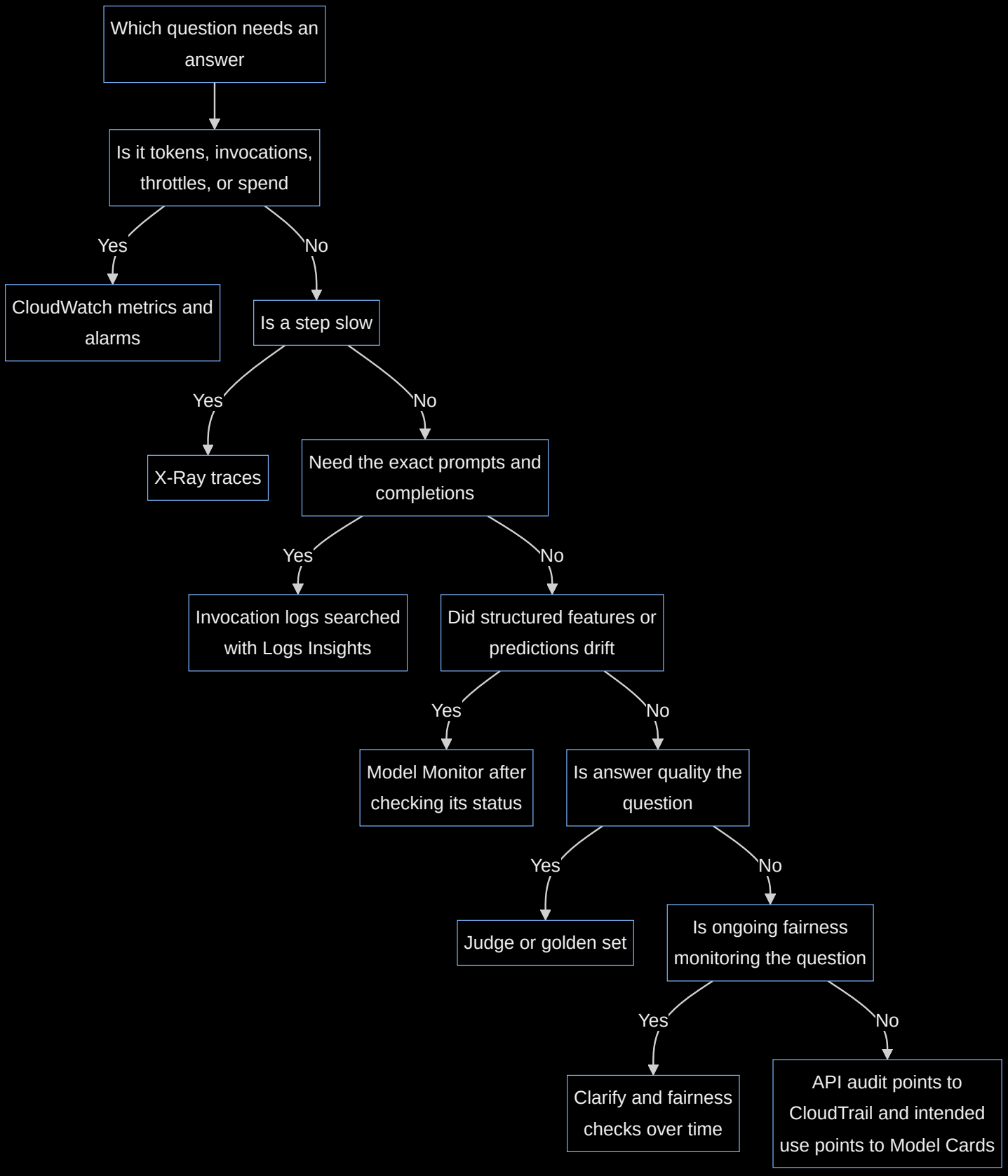


Figure: Signal choice for a question about a production system. A question that names one idea points to one signal.

Decision rule: Match the question to one signal. Drift in structured features points to Model Monitor. Answer quality points to a judge or a golden set.

Trap: Model Monitor does not score free-text quality. Metrics alone also cannot rebuild what the model received.

Exam tip: Drift in structured features is one signal. A slow step, a cost spike, and exact prompts each point to another signal.

Source: repo guide, Part 1B Lesson 11 Operations signals. Part 1D tie-in row Monitoring signals. Part 5 Domain 4 table. Part 1C Amazon SageMaker AI, Monitoring.

SageMaker Clarify

Clarify measures bias in data and models. It also explains how input features affect predictions.

- Methods or APIs: Not in source.
- Pro: Measures bias in data and in models.
- Pro: Explains how input features affect predictions.
- Pro: Can create governance reports.
- Con: The repo guide sources this entry to sample notebooks and launch material. The official developer guide page did not load in this build.
- Con: Pipeline or monitoring steps for Clarify are Not in source.
- Choose it when: a question asks for bias measurement or the effect of input features on predictions.
- Do not choose it when: the question asks only for fairness after launch. A launch test is a snapshot. Monitor fairness over time and act on each alert.
- Native pairing: Not in source for a named service pairing. The repo guide pairs fairness checks with a remediation workflow on each alert.

Decision rule: Use Clarify to measure bias or feature influence. Monitor fairness after launch, not only before it.

Trap: A one-time bias test before launch does not prove fairness later. Data drift and demographic drift change behavior.

Exam tip: Bias in data or the model points to Clarify. Explaining one loan decision needs citations and a trace, not a model card alone.

Source: repo guide, Part 8A reference terms (Clarify entry, sourced to sample notebooks and launch material). Part 4 Domain 3 Visual unit 66.

SageMaker JumpStart

Not in source. The repo guide lists JumpStart only in its gaps.

- Methods or APIs: Not in source.
- Pro: Not in source.
- Con: Not in source.
- Choose it when: Not in source.
- Do not choose it when: Not in source. Check the official page before you rely on it.
- Native pairing: Not in source.

Decision rule: JumpStart has no rule in the repo guide. Not in source. Confirm its role on the official page.

Trap: The repo guide lists JumpStart as a gap. Do not infer its features from the name.

Exam tip: The repo guide has no JumpStart unit. Choose from the services the repo guide covers first.

Not in source

SageMaker Ground Truth

Not in source. The repo guide does not name this service.

- Methods or APIs: Not in source.
- Pro: Not in source.
- Con: Not in source.
- Choose it when: Not in source.

- Do not choose it when: Not in source. Check the official page before you rely on it.
- Native pairing: Not in source.

Decision rule: Ground Truth has no rule in the repo guide. Not in source.

Trap: The repo guide has no labeling service entry. Do not assume one from the word label.

Exam tip: When a question names a service the repo guide does not cover, check the official page first.

Not in source

SageMaker Canvas

Not in source. The repo guide does not name this service.

- Methods or APIs: Not in source.
- Pro: Not in source.
- Con: Not in source.
- Choose it when: Not in source.
- Do not choose it when: Not in source. Check the official page before you rely on it.
- Native pairing: Not in source.

Decision rule: Canvas has no rule in the repo guide. Not in source.

Trap: The repo guide has no Canvas entry. Do not assume its features from the name.

Exam tip: Confirm any Canvas detail on the official page before you rely on it.

Not in source

SageMaker Neo

Not in source. The repo guide lists Neo only in its gaps.

- Methods or APIs: Not in source.
- Pro: Not in source.
- Con: Not in source.
- Choose it when: Not in source.
- Do not choose it when: Not in source. Check the official page before you rely on it.
- Native pairing: Not in source.

Decision rule: Neo has no rule in the repo guide. Not in source.

Trap: Do not infer what Neo does from its name. The repo guide gives no detail.

Exam tip: Neo is a gap in the repo guide. Confirm it on the official page.

Not in source

SageMaker Unified Studio

Not in source for depth. The repo guide lists Unified Studio only in its gaps.

- Methods or APIs: Not in source.
- Pro: Not in source.
- Con: Not in source for its features.
- Choose it when: Not in source.
- Do not choose it when: Not in source. Check the official page before you rely on it.
- Native pairing: The repo guide Model Registry steps start in the Studio console. How that console relates to Unified Studio is Not in source.

Decision rule: Unified Studio has no depth rule in the repo guide. Not in source.

Trap: Do not treat the Studio console in the registry steps as a full description of Unified Studio.

Exam tip: Unified Studio depth is Not in source. Do not answer from it.

Not in source

Built-in XGBoost algorithm

XGBoost is a gradient-boosted trees algorithm for supervised learning. SageMaker AI provides it as a built-in algorithm.

- Methods or APIs: Not in source for API names and hyperparameters.
- Pro: Handles regression, binary and multiclass classification, and ranking.
- Pro: Tabular data with labels often suits gradient-boosted trees.
- Con: The repo guide marks the definition as general, not an AWS claim.
- Con: Training settings are Not in source.
- Choose it when: labeled tabular data needs regression, classification, or ranking.
- Do not choose it when: the data is not tabular with labels. The repo guide rule is to pick the simplest model that fits the data type.
- Native pairing: Not in source for a named pairing with Lambda or S3 for this algorithm.

Decision rule: For labeled tabular data, gradient-boosted trees often fit. The repo guide names the built-in XGBoost algorithm for them.

Trap: One-hot encoding a column with thousands of values makes the feature set too large. Use a compact encoding.

Exam tip: Labeled tabular data points to gradient-boosted trees. Check the encoding of any column with many distinct values.

Source: repo guide, Part 8A5 unit P6 (XGBoost and category encoding). The XGBoost definition is general, not an AWS claim.

SageMaker AI endpoints versus Bedrock

SageMaker AI endpoints host models. Bedrock hosts the managed foundation models. Pick by who owns the model.

- Methods or APIs: The repo guide names Converse, InvokeModel, and ConverseStream for Bedrock. For SageMaker AI it names endpoint types only.
- Pro (SageMaker AI): Hosts your own model artifact in S3.
- Pro (SageMaker AI): Covers custom or fine-tuned models as real-time, serverless, or asynchronous endpoints.
- Pro (Bedrock): Covers managed foundation models with on-demand, Provisioned Throughput, and batch options.
- Con (SageMaker AI): Instance types and sizing are Not in source.
- Con (Bedrock): The repo guide points to a SageMaker AI endpoint for your own model artifact in S3.
- Choose it when: you own the model artifact and need hosting in SageMaker AI.
- Do not choose it when: a managed foundation model meets the need. Use Bedrock.
- Native pairing: API Gateway, Lambda or EventBridge, and then Bedrock or SageMaker AI form one gateway chain.

When to host your own model decision tree

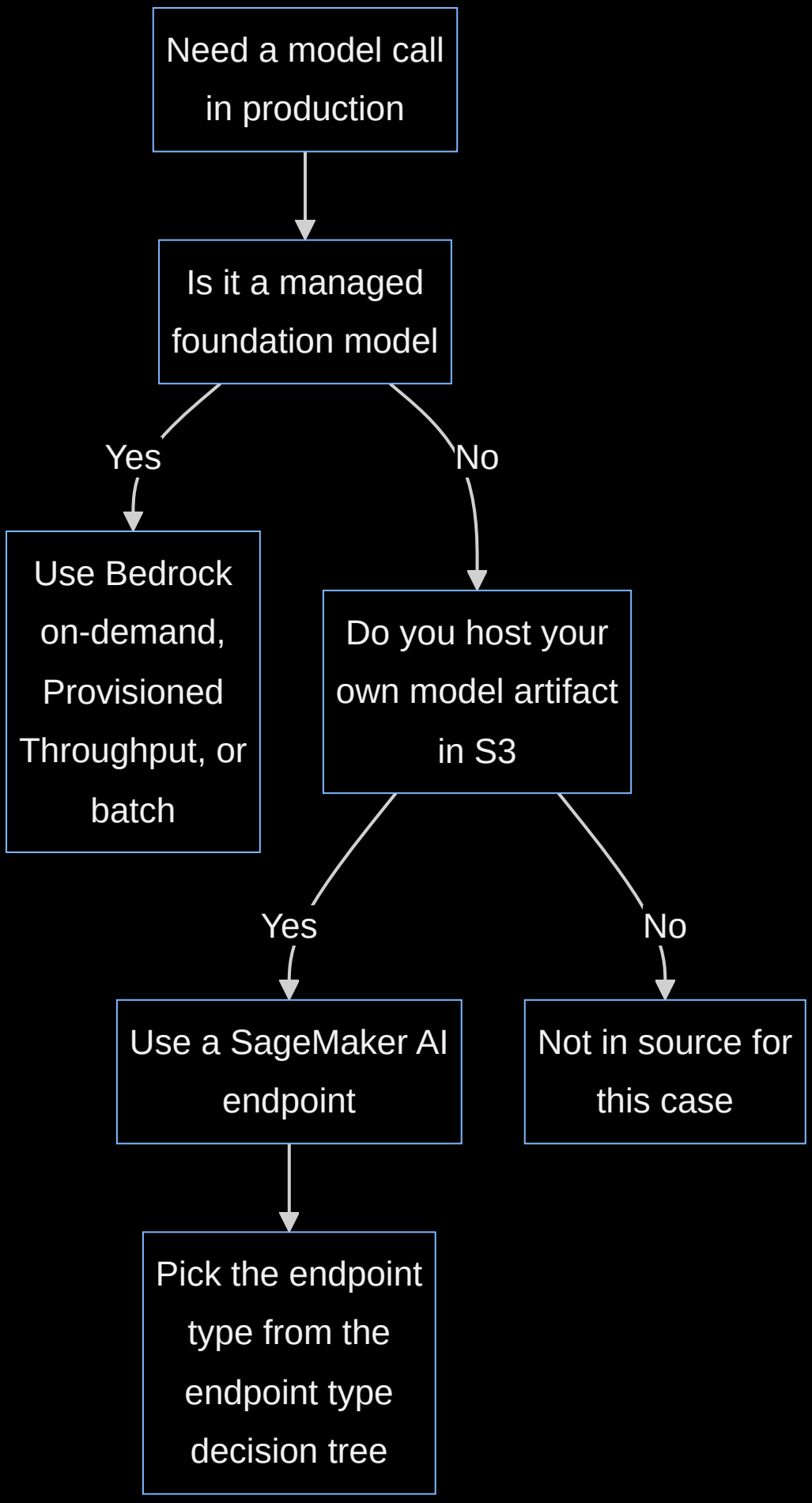


Figure: Bedrock or a SageMaker AI endpoint. Pick by who owns the model, as the repo guide states.

Decision rule: Own the model artifact and need hosting, use a SageMaker AI endpoint. Need a managed foundation model, use Bedrock.

Trap: AppConfig controls settings and does not host the model. Choose between Bedrock and a SageMaker AI endpoint for the model itself.

Exam tip: Own artifact, custom model, or fine-tuned model points to SageMaker AI. Managed foundation model points to Bedrock.

Source: repo guide, Part 1C Amazon SageMaker AI. Part 3 Domain 2 deployment patterns table. Part 8A8 unit N3. Part 1D tie-in table.

Lambda and API Gateway in front of SageMaker endpoints

The repo guide puts API Gateway, then Lambda or EventBridge, then Bedrock or SageMaker AI in one controlled entry.

- Methods or APIs: The endpoint call itself is Not in source. API Gateway creates REST, HTTP, and WebSocket APIs.
- Pro: One gateway enforces guardrail identifiers, IAM policy, uniform logs, and team cost tags.
- Pro: Clients keep one contract. The Lambda function owns the model choice.
- Con: The REST integration default timeout is 29 seconds. The repo guide says to confirm this value on the API Gateway quotas page.
- Con: Raising the Lambda timeout does not change the gateway limit.
- Con: A standard Lambda function runs at most 15 minutes in one invocation. Waits in Lambda bill for the full duration.
- Choose it when: the call finishes inside the gateway limit and clients need one contract, logs, and policy.
- Do not choose it when: the call runs past the gateway limit. Use an asynchronous endpoint with S3 output, or a WebSocket API for a stream.
- Native pairing: For long flows, put the steps in Step Functions and keep each Lambda step short.

Gateway path decision tree

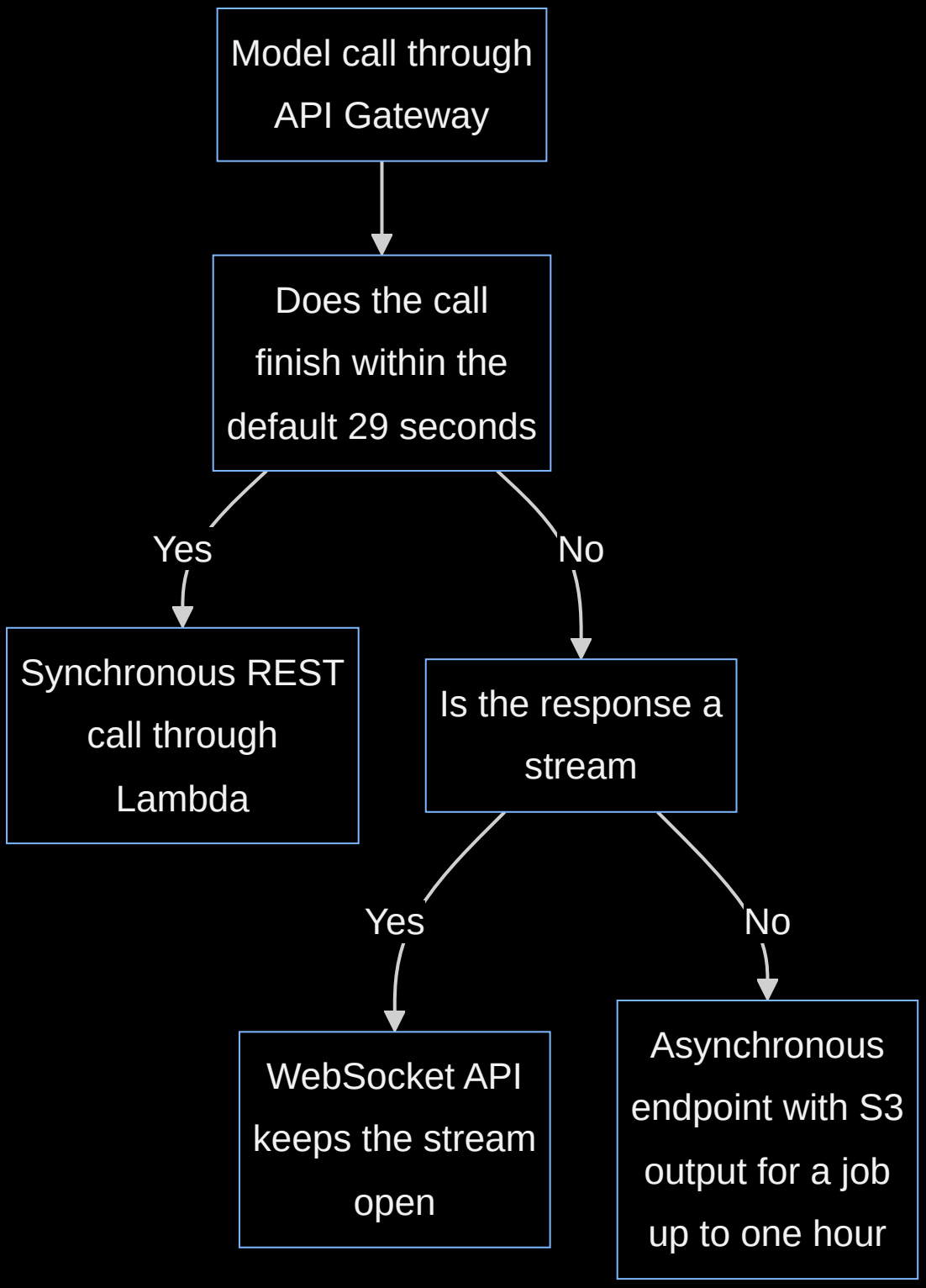


Figure: Path for a model call through API Gateway. The 29 second default needs confirmation on the API Gateway quotas page.

Decision rule: Send a SageMaker AI call through API Gateway only when it fits the gateway limit. Otherwise use an asynchronous endpoint.

Trap: Streaming does not extend a REST timeout. Raising the Lambda timeout does not fix a gateway timeout.

Exam tip: A gateway timeout points to an asynchronous job or a WebSocket stream. A larger Lambda timeout does not solve it.

Source: repo guide, Part 3 Domain 2 units on Lambda limits and long work, streaming and the gateway timeout, and enterprise gateway and access. Part 8A8 unit N6.

Native AWS services for compute, integration, and data

This part covers the native AWS services that feed a model call, carry its events, or hold its data. Each entry states what the repo guide says. A claim the guide does not confirm carries the mark Not in source.

Source: repo guide, Parts 1A, 1B, 1C, 1D, 2, 3, 4, 7, 8A4 to 8A8, 8B, 8D, 9

Decision trees for native services

Start at the top of each tree. Follow the need to one service, then read that service entry for its limits.

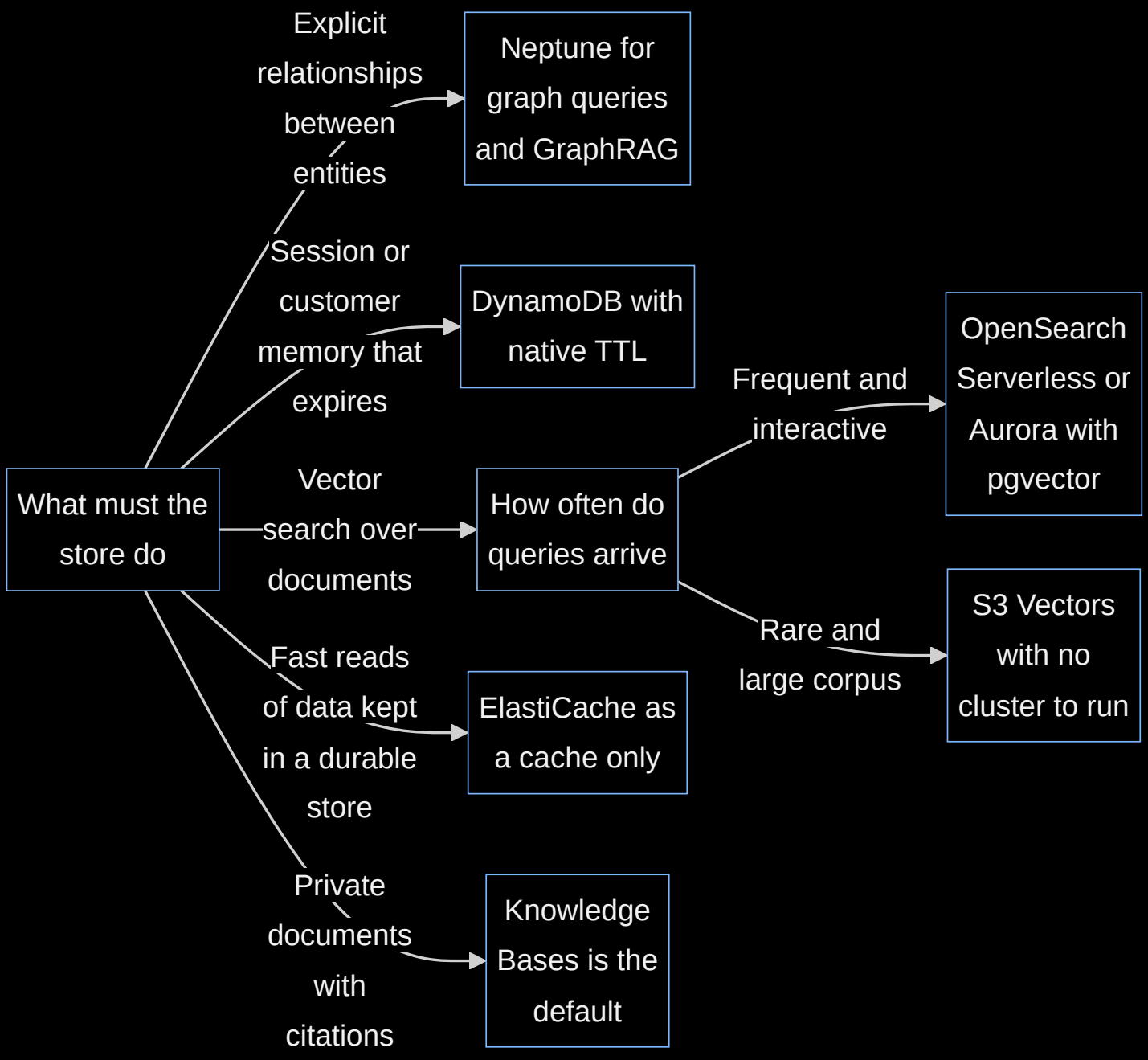


Figure: Which data store. Follow the need to one store, then check the store limits in its entry.

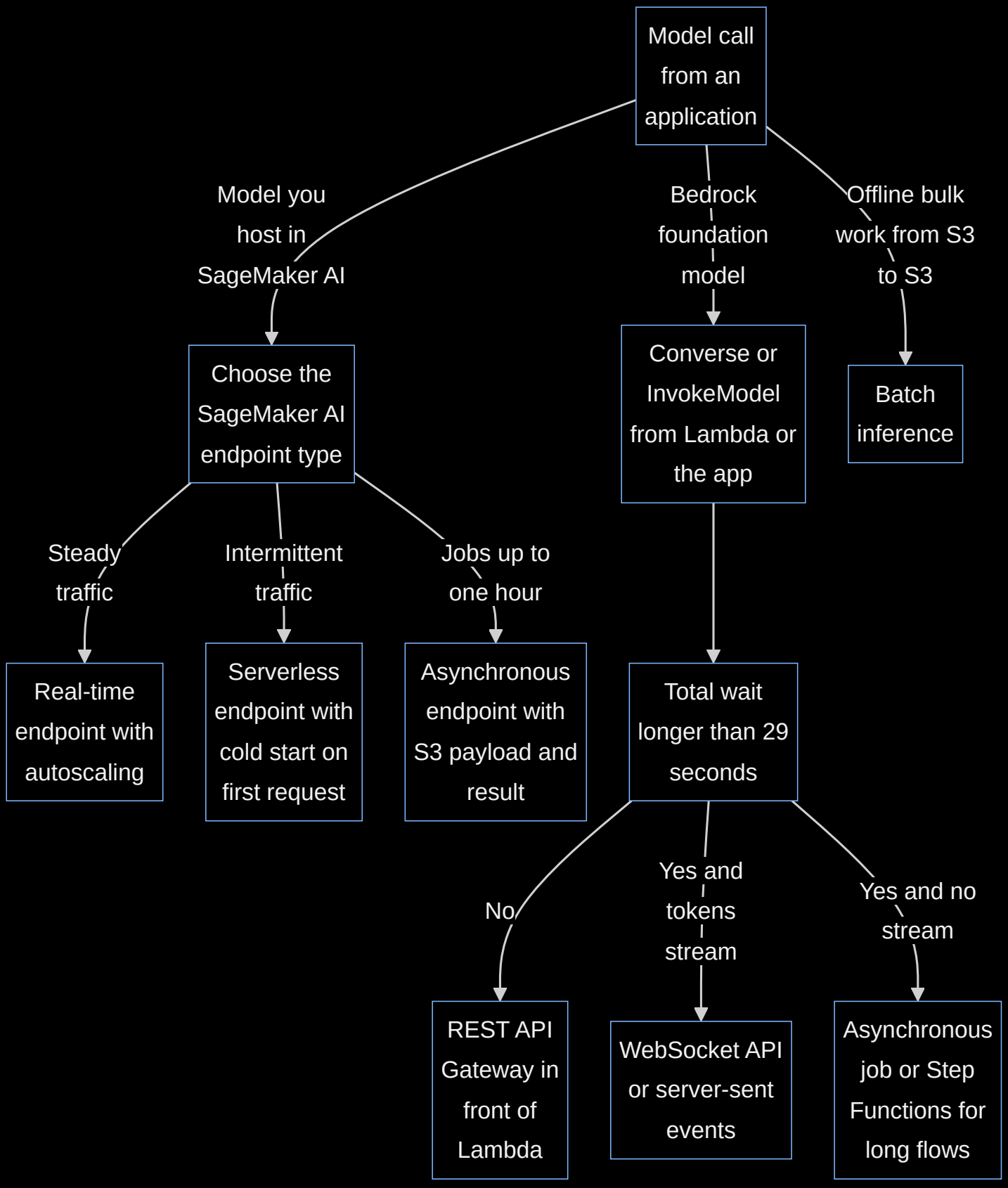


Figure: Which compute for model calls. The 29-second REST limit and the 15-minute Lambda limit set the branches.

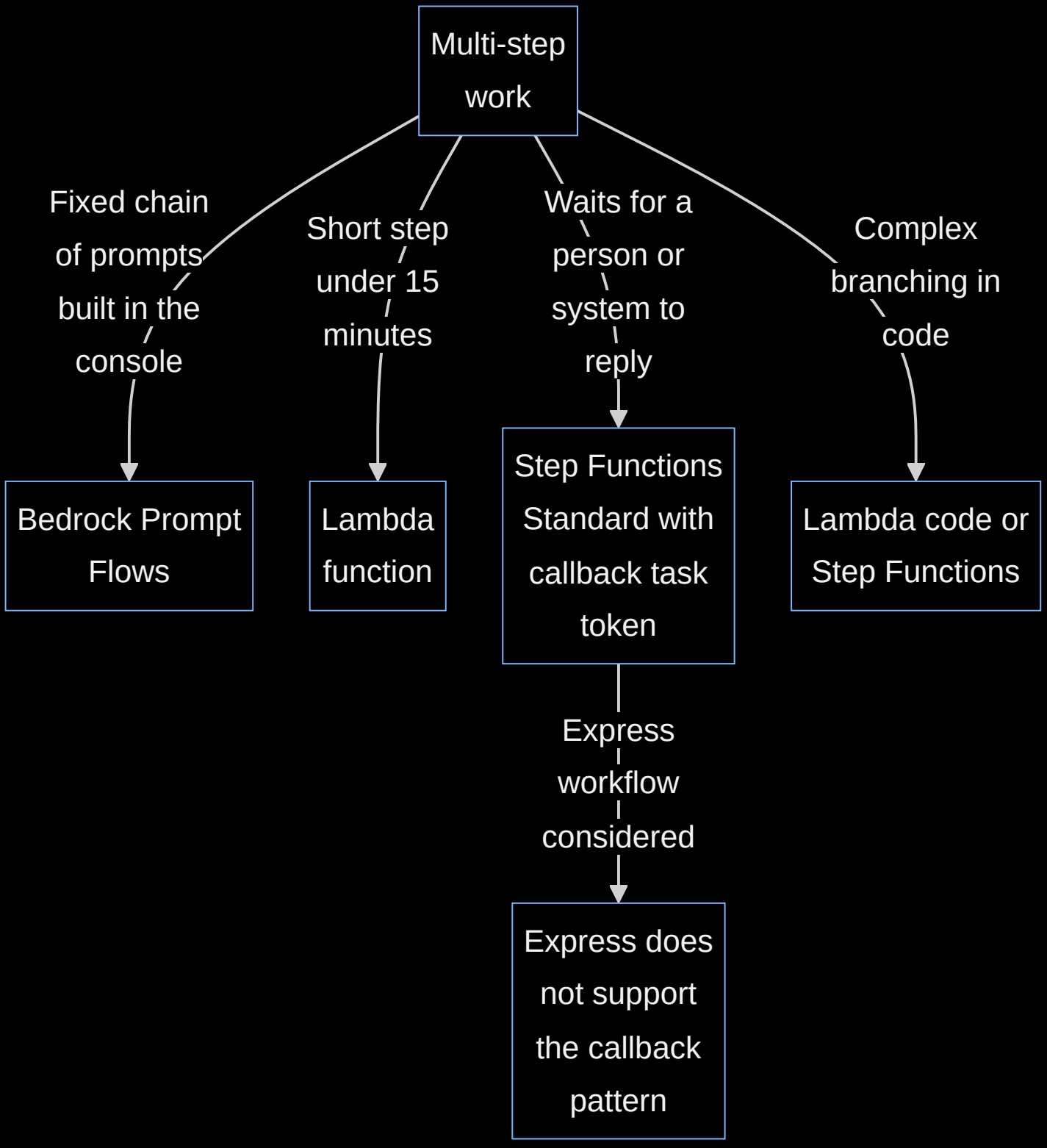


Figure: Which orchestration tool. Waits and approvals move the work to Step Functions Standard.

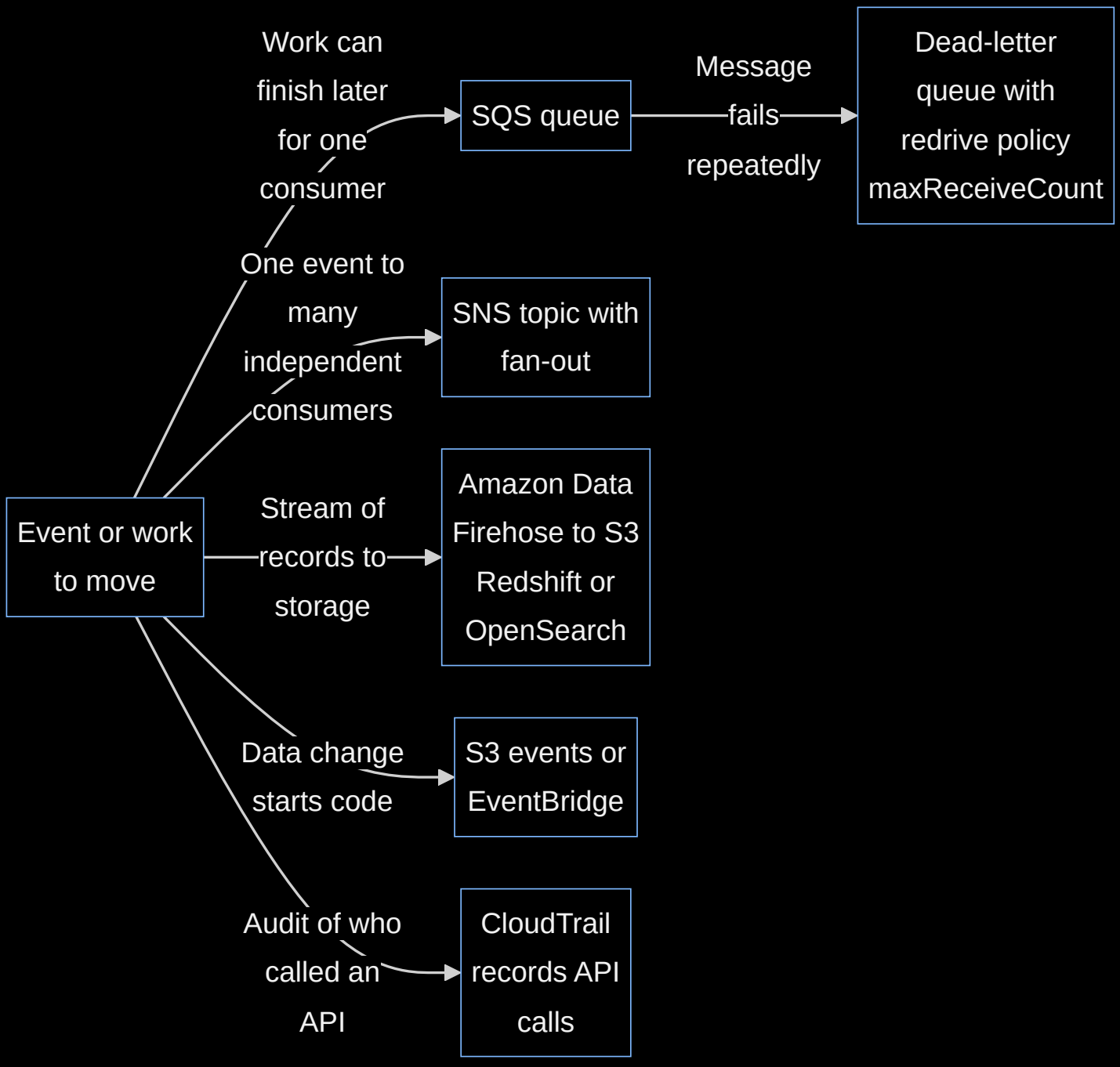


Figure: Which messaging or event service. CloudTrail is an audit record, not a data trigger.

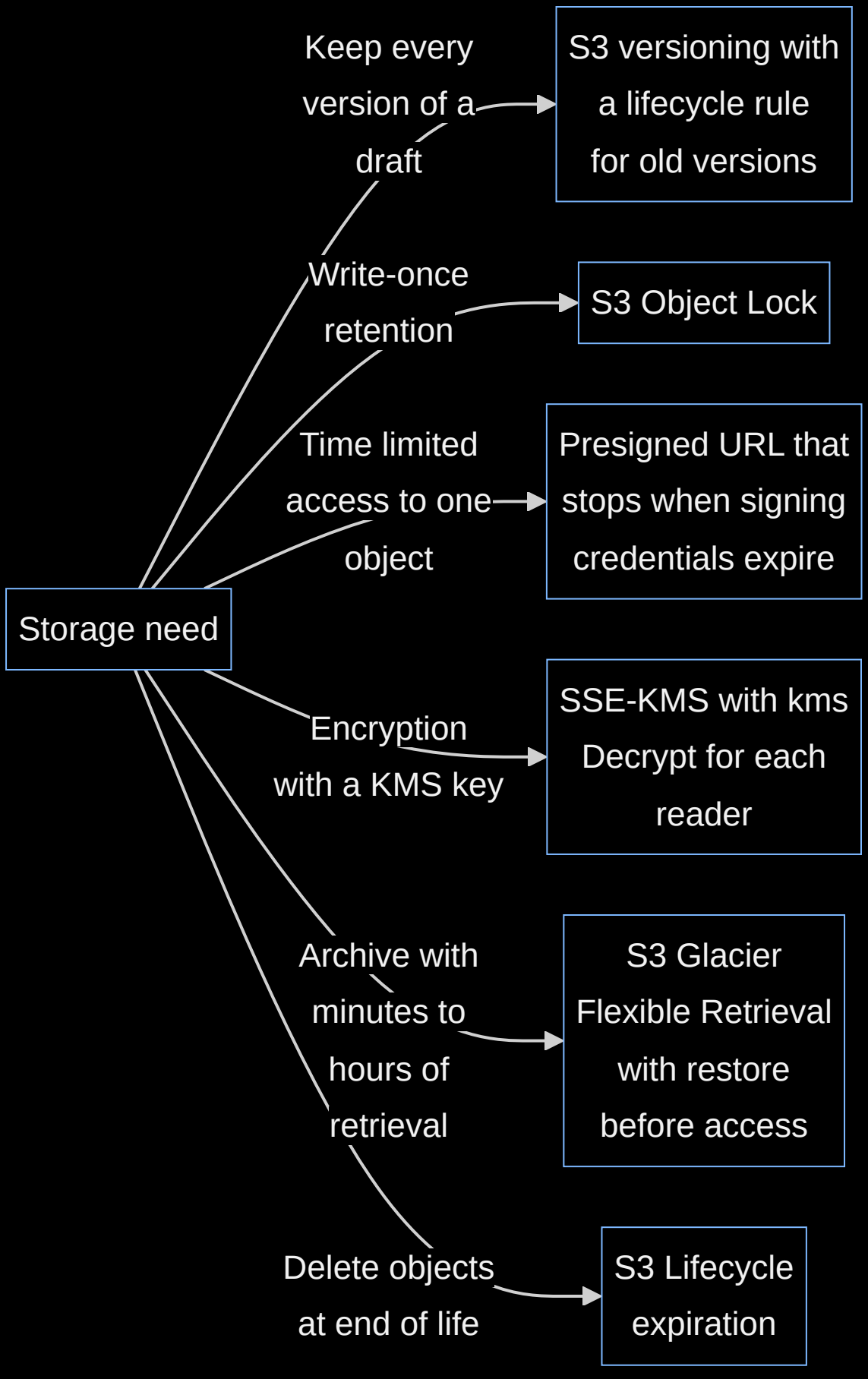


Figure: Which S3 control. Each control covers one storage need.

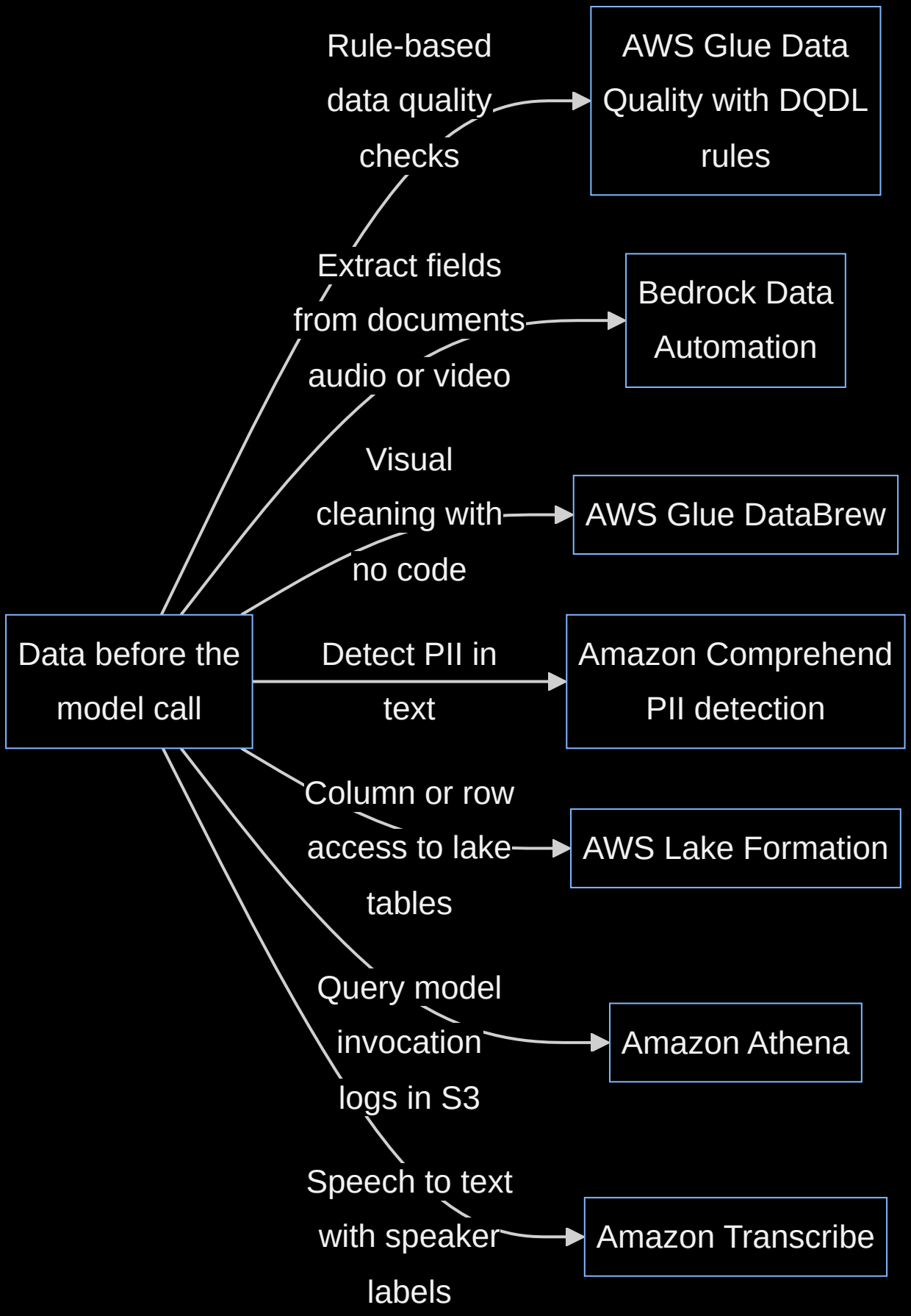


Figure: Which data preparation tool. One tool covers one stage. Guardrails are not on this tree.

Decision rule: Match the need to one tree first. Then read the trap in that service entry before you rely on it.

Trap: A tree shows one path. It does not show the limits of each service.

Source: repo guide, Parts 1A, 1D, 2 (sections 1.3, 1.4, 2.2, 2.4), 8A5, 8A6, 8A8

Compute and orchestration. The entries below cover Lambda, Step Functions, and API Gateway. Together they form the request path that a model call uses.

AWS Lambda

Short compute steps. Standard functions run up to 15 minutes.

- Methods or APIs: update-function-configuration, the --timeout CLI flag, and KMSKeyARN for a customer managed key on environment variables.
- Pro: Holds short steps, such as format checks, custom chunking, and action group executors.
- Pro: Reads the model ID from AppConfig at runtime. A model switch needs no code release.
- Con: A standard function ends at 15 minutes. Waits in Lambda bill for the full duration.
- Con: Anyone who can read the function configuration can read its environment variables.
- Choose it when: each step is short and stateless, and it ends well inside 15 minutes.
- Do not choose it when: a step waits for a person, or a flow runs past 15 minutes. Use Step Functions.
- Native pairing: Lambda sits behind API Gateway. An action group in Bedrock Agents links the agent to a Lambda function. S3 events can invoke it.

Decision rule: Keep each Lambda step short. Move a long or waiting flow to Step Functions.

Trap: A Lambda function that waits for a person pays for the wait and hits the 15-minute limit.

Exam tip: When the wording says a step waits for approval, a Lambda function alone is the wrong pick.

Timeout path

The guide gives this console path for the timeout.

- Open the Lambda console and choose the function.
- Choose the Configuration tab, then General configuration.
- Choose Edit. For Timeout, enter a value from 1 to 900 seconds.
- Choose Save. The CLI flag is --timeout. The default is 3 seconds.

The maximum is 900 seconds. For longer work, use asynchronous jobs or Step Functions. Lambda Managed Instances functions invoked asynchronously allow up to 90 minutes. Do not assume that for a standard function.

Environment variables and secrets

- Lambda stores environment variables encrypted at rest. Set a customer managed key with KMSKeyARN.
- Keys start with a letter and have at least two characters. They use only letters, numbers, and the _ character.
- The variables total 4 KB at most.
- An update with update-function-configuration replaces the whole Variables structure. Include every existing value.
- Keep non-secret settings, such as a bucket name or log level, in environment variables.
- Read credentials from AWS Secrets Manager at run time. The guide recommends it for database credentials, API keys, and tokens.
- Give the function an execution role. Keep long-term access keys out of environment variables.

Decision rule: Store secrets in Secrets Manager. Use environment variables only for settings that are not secret.

Trap: Encryption at rest does not make a value a secret. The guide still recommends Secrets Manager for credentials.

S3 event triggers and loop risk

- Amazon S3 invokes the function asynchronously when an object is created or deleted.
- Set the trigger in the bucket notification, or let the Lambda console add it.
- The function resource-based policy must allow S3 to invoke it.
- If the function uses the AWS SDK to manage S3, its execution role needs S3 permissions.
- Write function output to a second bucket. Or limit the trigger to one prefix that holds incoming objects.

Decision rule: Write output outside the bucket and prefix that trigger the function.

Trap: A function that writes to the bucket that triggers it can run in a loop. The official page gives this warning.

Source: repo guide, Parts 1B and 1C, Part 2 sections 2.1 and 2.2, Part 8A6 environment variable and S3 event units, Part 8B timeout steps

AWS Step Functions

Durable workflows with waits, retries, and callback task tokens. Standard runs up to one year.

- Methods or APIs: callback task token, TimeoutSeconds, and Catch. Retry fields: IntervalSeconds, MaxAttempts, BackoffRate, JitterStrategy, MaxDelaySeconds. Error names: States.Timeout, States.Runtime, States.ALL.
- Pro: Standard keeps state for waits and runs up to one year.
- Pro: A callback task token pauses a flow until a person or system responds.
- Pro: Standard keeps execution history for up to 90 days. The history gives the audit trail.
- Con: Express workflows do not support the callback pattern.
- Con: Each Lambda step inside the flow still has the 15-minute limit. Keep each step short.

- Choose it when: a wait lasts hours or days, or a person must approve before the flow continues.
- Do not choose it when: the steps are a fixed chain built in the console. Use Bedrock Prompt Flows.
- Native pairing: Step Functions calls Lambda steps and waits on callback task tokens. Its timeouts bound agent loops.

Decision rule: Use Step Functions Standard for waits measured in hours or days. Use a callback task token for approval.

Retries, catchers, and timeouts

- A Task state reports States.Timeout when it runs longer than TimeoutSeconds.
- The whole execution also reports States.Timeout when it runs longer than its own TimeoutSeconds.
- Retry defaults are IntervalSeconds 1, MaxAttempts 3, and BackoffRate 2.0. The JitterStrategy default is NONE.
- MaxDelaySeconds caps the wait between retries.
- Catch sends the error to a Next state. When both fields exist, Retry runs first.
- States.Runtime is not retriable. A Retry or Catch on States.ALL does not catch it.

Decision rule: Retry transient errors. Fix input errors in the flow, because States.Runtime is not retriable.

Trap: A Catch on States.ALL does not handle States.Runtime. Fix the input so the runtime error does not occur.

Exam tip: A timeout in the wording matches States.Timeout. States.Runtime is a separate error that retries do not fix.

Console path

- Open the Step Functions console and create a state machine from a template.

- Select states on the canvas to review their settings.
- Start an execution. Review the execution details and the state output.
- Choose the IAM role for the state machine. Use the IAM console when the role needs changes.

The fetched page does not show the callback task token steps. Those steps are Not in source for the console path.

Source: repo guide, Parts 1A and 1B, Part 2 sections 2.2 and 2.4, Part 8A6 workflow error unit, Part 1F Step Functions unit, Part 7 traps table

Amazon API Gateway

Creates, publishes, maintains, monitors, and secures REST, HTTP, and WebSocket APIs.

- Methods or APIs: REST, HTTP, and WebSocket APIs, and usage plans. Throttling limit numbers: Not in source.
- Pro: One stable contract for clients. The Lambda function behind it owns the model choice.
- Pro: Throttling and usage plans sit at the front of the request path.
- Con: A REST integration stops at 29 seconds by default. Raising the Lambda timeout does not change this limit.
- Con: The guide states the 29-second default in Parts 1A and 2.2. Part 9 marks that value Not in source, because the quotas page did not show it.
- Choose it when: clients need one entry point, with throttling and logs, in front of Lambda and Bedrock.
- Do not choose it when: a REST path must hold a model call longer than 29 seconds. Use WebSocket or an asynchronous job.
- Native pairing: API Gateway sits in front of a Lambda function that calls Bedrock. WebSocket APIs carry token streams to a browser.

The guide treats a GenAI gateway as a design pattern built from API Gateway and Lambda. Its exact product form is Not in source.

Long model calls

- REST path: the default is 29 seconds. A slow model call loses its connection at that point.
- WebSocket path: the connection stays open, and tokens stream as they arrive.
- Asynchronous job: submit the request, receive a job ID, then poll or receive the result later.
- Streaming does not change the timeout. ConverseStream and InvokeModelWithResponseStream still stop at the 29-second limit on REST.

Decision rule: Use WebSocket streams or an asynchronous job when a model call can pass 29 seconds.

Trap: A 90-second model call fails at the gateway. Raising the Lambda timeout does not fix it.

Exam tip: When the wording says a long model call fails at the front door, the limit is the gateway limit.

Source: repo guide, Part 1A, Part 2 sections 2.2 to 2.4, Part 4 section 3.1, Part 8A8 API Gateway unit, Part 9 open checks

Integration and events. The entries below cover SQS, SNS, EventBridge, Firehose, and Kinesis. They move work between services and carry records to storage.

Amazon SQS

Holds asynchronous work that can finish later. A dead-letter queue (DLQ) holds messages that fail repeatedly.

- Methods or APIs: redrive policy with maxReceiveCount. Redrive moves messages from the DLQ back to the source queue.
- Pro: A failed message leaves the main queue, so it does not block the work behind it.
- Pro: Use SQS for asynchronous model work that can finish later.

- Con: A DLQ does not fix the failing code. It isolates the failed messages so you can find the cause.
- Con: A dead-letter queue must sit in the same account and Region as its source queue. Give the dead-letter queue a longer retention period than its source queue.
- Choose it when: a job can finish later and failed messages need inspection.
- Do not choose it when: a user waits for the reply. Use a synchronous Bedrock call.
- Native pairing: SNS can deliver to SQS queues. Each queue scales and fails on its own.

Decision rule: Set maxReceiveCount, then route each failed message to a DLQ for a root-cause check.

Trap: A DLQ does not fix the failing code. It only isolates the failed messages.

Exam tip: When the wording says a message must leave the main queue, look for a redrive policy and a DLQ.

Source: repo guide, Part 2 section 2.4, Part 8A4 failed messages unit, Part 8A5 data and messaging unit

Amazon SNS

Sends one message from one topic to many subscribers.

- Methods or APIs: topic and subscriber. Other API names: Not in source.
- Pro: Fan-out to independent consumers. Each queue scales and fails on its own.
- Pro: Subscribers include SQS queues, Lambda, Amazon Data Firehose, and HTTP endpoints.
- Con: Filtering, retry, and delivery settings: Not in source.
- Choose it when: one event must reach several independent consumers.
- Do not choose it when: only one consumer reads the event. This is inference from the fan-out definition.

- Native pairing: SNS delivers to SQS queues, Lambda, and Firehose. Each SQS queue scales and fails on its own. Other subscriber failure rules: Not in source.

Decision rule: Use SNS fan-out when several consumers must act on one event on their own.

Trap: Each queue scales and fails on its own. Check each subscriber, not only the topic.

Exam tip: When the wording says fan-out to many consumers, the match is SNS, not one SQS queue.

Source: repo guide, Part 8A5 data and messaging unit

Amazon EventBridge

Not in source as a service definition. The guide names EventBridge in two places only.

- Methods or APIs: Not in source.
- Pro: The enterprise gateway path lists EventBridge before Bedrock or SageMaker AI.
- Pro: The data trigger table lists S3 events and EventBridge as the correct trigger for data.
- Con: Rules, buses, and targets: Not in source.
- Choose it when: a data change or an event must start code, and the guide names it for that job.
- Do not choose it when: the need is an audit of who called an API. Use CloudTrail.
- Native pairing: The guide shows API Gateway, then Lambda or EventBridge, then Bedrock or SageMaker AI. Wiring detail: Not in source.

Decision rule: For a data change that starts code, use S3 events or EventBridge.

Trap: CloudTrail is not a data trigger. CloudTrail records API calls for audit and does not show row content.

Exam tip: When the wording names a trigger for data, look for S3 events or EventBridge.

Source: repo guide, Part 2 section 2.3, Part 7 traps table, Part 1A component table

Amazon Data Firehose

Delivers streaming records to S3, Redshift, OpenSearch, and other destinations.

- Methods or APIs: Not in source.
- Pro: One delivery service covers S3, Redshift, OpenSearch, and other destinations.
- Pro: The guide lists Firehose as an SNS subscriber.
- Con: Buffer, transform, and error settings: Not in source.
- Choose it when: records arrive as a stream, and the destination is S3, Redshift, or OpenSearch.
- Do not choose it when: one request needs a reply now. This is inference. Use API Gateway and Lambda for a reply.
- Native pairing: SNS delivers to Firehose. Firehose delivers to S3 and OpenSearch.

Decision rule: Pick Firehose when records arrive as a stream and must land in a listed destination.

Trap: The guide gives no Firehose buffer or transform settings. Mark them Not in source until a page confirms them.

Exam tip: The words streaming records and a destination in S3 or OpenSearch point to Firehose.

Source: repo guide, Part 8A5 data and messaging unit

Amazon Kinesis

Not in source. The guide names Kinesis Data Streams only as a gap.

- Methods or APIs: Not in source.
- Pro: Not in source.
- Con: Not in source.
- Choose it when: Not in source. The guide gives no use case for Kinesis.
- Do not choose it when: Not in source. This is inference only: the guide covers Firehose delivery, so check the Kinesis page before you choose either one.
- Native pairing: Not in source.

Decision rule: Not in source. Do not choose Kinesis on facts from this guide alone.

Trap: Not in source. Do not assume stream limits or behavior from general knowledge.

Exam tip: Not in source. Check the official Kinesis page before you rely on a Kinesis claim.

Not in source

Data preparation and analytics. The entries below cover Glue, DataBrew, Athena, and Lake Formation. They check, clean, query, and restrict data before or around a model call.

AWS Glue

AWS Glue Data Quality runs rule-based quality checks written in DQDL. The Data Catalog is a source registry for governance. Full lineage also needs citations on each output.

- Methods or APIs: DQDL rules and the Data Catalog. DynamicFrame: Not in source.

- Pro: The validate stage uses Glue Data Quality rules to check data before processing.
- Pro: The Data Catalog serves as the lineage and source registry for governance.
- Con: ETL jobs with DynamicFrame: Not in source.
- Con: DQDL rule syntax and rule examples: Not in source.
- Choose it when: a stage must check data against written rules before the model uses it.
- Do not choose it when: the need is content safety on a prompt or a response. Use Bedrock Guardrails.
- Native pairing: The pipeline in the guide runs Glue Data Quality first, then Bedrock Data Automation, then format, then Comprehend and Lambda.

Decision rule: Validate with Glue Data Quality rules before processing. Keep one tool for one stage.

Trap: Guardrails do not validate data quality or extract fields.

Exam tip: When the wording says rule-based data quality, the match is Glue Data Quality with DQDL.

Source: repo guide, Part 1A component table, Part 2 section 1.3, Part 4 section 3.3

AWS Glue DataBrew

Cleans data with more than 250 visual transforms, with no code. Its output goes to S3.

- Methods or APIs: visual transforms. Transform names and recipe steps: Not in source.
- Pro: No code. More than 250 visual transforms.
- Con: The guide names S3 as the output. Other output targets: Not in source.
- Con: Scheduled jobs and profile jobs: Not in source.
- Choose it when: a team cleans data with visual steps and no code.

- Do not choose it when: the check is a set of written rules. Use Glue Data Quality.
- Native pairing: DataBrew writes cleaned output to S3. Pairing with Lambda or Glue jobs: Not in source.

Decision rule: Pick DataBrew for visual cleaning with no code. Read the cleaned output from S3.

Trap: DataBrew is a cleaning tool. Written rule checks belong to Glue Data Quality with DQDL.

Exam tip: The words visual transforms and no code point to DataBrew.

Source: repo guide, Part 8A5 data and messaging unit (What is AWS Glue DataBrew)

Amazon Athena

Queries model invocation logs that Bedrock delivers to S3 as gzipped JSON.

- Methods or APIs: query syntax and table setup: Not in source.
- Pro: Searches invocation logs by date and user for a long audit trail.
- Pro: Each log entry records the identity ARN, request metadata, and token counts.
- Con: CloudWatch Logs Insights serves operations needs. Athena fits the long audit trail instead.
- Con: Partitioning and query cost: Not in source.
- Choose it when: an audit needs model calls by date and user, and the logs sit in S3.
- Do not choose it when: an operations need is fast search in CloudWatch. Use Logs Insights.
- Native pairing: Athena reads the S3 log bucket that model invocation logging writes. S3 Select can also read it.

Decision rule: Use S3 with Athena for the long audit trail. Use CloudWatch Logs Insights for operations.

Trap: Logs in CloudWatch Logs Insights serve operations needs. A long audit trail needs S3 with Athena.

Exam tip: When the wording asks who called a model by date and user, look for Athena on S3 logs.

Source: repo guide, Part 8A5 invocation logs unit, Part 4 section 3.3

AWS Lake Formation

Column and row access for lake tables.

- Methods or APIs: permission model details: Not in source.
- Pro: Column-level access. One team sees a salary column and another team does not.
- Pro: Row-level access. A team sees only its own rows.
- Con: Lake Formation does not replace CloudTrail or Model Cards.
- Con: The permission model details are Not in source. Confirm them before you rely on a design.
- Choose it when: lake tables need column or row limits for each team.
- Do not choose it when: the need is a record of what the model is. Use SageMaker Model Cards.
- Native pairing: Lake Formation controls data access. CloudTrail records API calls. Model Cards record intended use and results.

Decision rule: Use Lake Formation for who reads which rows and columns. Use Model Cards for what a model is.

Trap: Model Cards describe the model. Lake Formation controls who reads the data.

Exam tip: When the wording says one team sees a column and another does not, the match is Lake Formation.

Source: repo guide, Part 1A component table, Part 4 sections 3.2 and 3.3, Part 4 Lake Formation visual unit, Part 9 gaps

Data stores. The entries below cover OpenSearch, Aurora with pgvector, DynamoDB, Neptune, ElastiCache, S3, and S3 Vectors. Each store fits one access pattern.

Amazon OpenSearch Service and OpenSearch Serverless

OpenSearch Serverless is managed vector search that combines filters and vectors.

- Methods or APIs: Faiss engine, FP16 scalar quantization (SQfp16), and the collection types Time series, Search, and Vector search.
- Pro: Combines metadata filters with vector search.
- Pro: Knowledge Bases lists OpenSearch Serverless as a vector store. The guide names it for frequent, interactive search.
- Pro: Vector quantization cuts index memory. FP16 halves memory. Binary stores 1 bit for each dimension.
- Con: More shards do not make search faster. Each shard uses CPU and memory.
- Con: Collection type descriptions and shard rules for Serverless: Not in source.
- Choose it when: vector search must filter by metadata and serve frequent, interactive queries.
- Do not choose it when: the corpus is very large and queries are rare. Use S3 Vectors.
- Native pairing: Knowledge Bases can use it as the vector store. Indexing compute units store indexes in S3 for search compute units.

Shards and sizing for OpenSearch Service

- By default, OpenSearch Service creates five primary shards, and each has one replica. That is ten shards for each index.
- You cannot easily change the primary shard count of an existing index. Decide it before indexing the first document.

- Keep each shard between 10 and 30 GiB for search latency. Keep each shard between 30 and 50 GiB for write-heavy log analytics.
- Estimate primary shards as (source data plus room to grow) times (1 plus indexing overhead), divided by the desired shard size.
- Example from the guide: 66 GiB that does not grow, with 30 GiB shards, gives about three shards.
- Keep no more than 25 shards for each GiB of Java heap on a node.

Decision rule: Size the index from current data and growth. Then check the shard count against node heap.

Trap: More shards do not make search faster. Many small shards can cause performance problems.

Vector quantization

- A float32 dimension uses 4 bytes. Quantization stores fewer bits for each dimension.
- FP16 scalar quantization (SQfp16) halves memory with the Faiss engine. It needs OpenSearch 2.13 or later. The source is an OpenSearch project page, not an AWS page.
- Binary quantization stores 1 bit for each dimension. That is 32 times smaller than float32. The 16 and 8 times figures for 2-bit and 4-bit come from a secondary AWS blog.
- Design arithmetic: one 1,536-dimension float32 vector is 6,144 bytes. Ten million vectors need about 60 GB.
- Design arithmetic: ten million binary vectors need about 1.9 GB.
- Rescore the top candidates with full-precision vectors or a reranker.

Decision rule: Use binary quantization for a first pass over a large index. Rescore for final ranking.

Trap: Binary quantization without rescoring can lose accuracy. Measure the loss on your data. A general loss figure is Not in source.

Exam tip: When the wording asks to cut vector memory, look for quantization.

Source: repo guide, Part 2 section 1.4, Part 8A6 shard unit, Part 8D vector quantization unit, Part 8A7 official diagram captions, Part 1A component table

Amazon Aurora with pgvector, and Amazon RDS

Aurora with pgvector gives vector search in a relational database.

- Methods or APIs: pgvector. Aurora PostgreSQL edition name: Not in source. RDS API names: Not in source.
- Pro: Vector search runs in a relational database the team already runs.
- Pro: Knowledge Bases lists Aurora as a vector store.
- Con: Sizing, scaling, and vector index limits for Aurora: Not in source.
- Con: The guide names no Amazon RDS use case. RDS details: Not in source.
- Choose it when: the team already runs a relational database and needs vector search there.
- Do not choose it when: the need is a graph of relationships. Use Neptune. Or the corpus is very large with rare queries. Use S3 Vectors.
- Native pairing: Knowledge Bases stores vectors in Aurora. Relational data next to a model call: Not in source.

Decision rule: Pick Aurora with pgvector when the team already runs the relational database.

Trap: The guide gives no Lambda connection pattern for Aurora or RDS. Do not state one as fact.

Exam tip: When the wording says the team already runs a relational database, Aurora with pgvector fits.

Source: repo guide, Part 1A component table, Part 1D tie-in, Part 2 section 1.4, Part 9 gaps

Amazon DynamoDB

Holds session and customer memory with a key and a time to live (TTL).

- Methods or APIs: native TTL and a gateway VPC endpoint for DynamoDB. Partition and sort key rules: Not in source.
- Pro: Single-digit millisecond lookup by a key such as customer ID or session ID.
- Pro: Native TTL removes old records. Memory that lasts weeks fits here.
- Pro: The table encrypts data at rest.
- Con: Cross-Region reads are eventually consistent. Strongly consistent reads must stay in one Region.
- Con: Vector search on DynamoDB: Not in source. The guide names only one table design with metadata and embeddings.
- Choose it when: memory must outlive the agent session and needs a key lookup.
- Do not choose it when: the agent session alone holds the memory. The session expires.
- Native pairing: Bedrock Agents session TTL is 60 to 5400 seconds. DynamoDB holds the memory that lasts longer. AppSync pairs with DynamoDB for real-time editing.

Global tables

- A global table holds replicas in several Regions.
- Writes replicate to all replicas, usually within seconds.
- Conflicts resolve by last writer wins.
- Transactions apply only within the Region where they start.

Network path

- A gateway VPC endpoint for DynamoDB adds a route to the route table. It needs no internet gateway and no NAT device.
- The endpoint has no extra charge. It does not serve on-premises networks, peered VPCs, or transit gateways.
- A timeout to DynamoDB from a private subnet often means a missing route, not a security group rule.

Decision rule: Use DynamoDB with native TTL for memory that must last weeks. Keep the agent session for short context.

Trap: Global tables do not give cross-Region transactions. Keep writes in one Region when strong consistency matters.

Exam tip: When the wording says memory must outlast the session, the match is DynamoDB with TTL.

Source: repo guide, Part 1A component table, Part 2 sections 1.4 and 2.1, Part 3 session memory visual unit, Part 8A5 network and data units

Amazon Neptune

Graph queries for GraphRAG over explicit relationships.

- Methods or APIs: graph queries. Query language and API names: Not in source.
- Pro: Traverses explicit relationships between entities for GraphRAG.
- Con: For plain document search, the guide treats Neptune as a wrong pick.
- Con: Cost, sizing, and graph loading: Not in source.
- Choose it when: the need depends on explicit relationships between entities.
- Do not choose it when: the need is plain document search. Use a vector store.
- Native pairing: The guide lists Neptune in its vector store table for GraphRAG. Pairing with Bedrock calls: Not in source.

Decision rule: Pick Neptune when the need is relationship traversal. Pick a vector store for plain documents.

Trap: Neptune for plain document search is a wrong pick. Neptune is for graph queries.

Exam tip: When the wording says traverse or explicit relationships, the match is Neptune.

Source: repo guide, Part 1A component table, Part 2 section 1.4, Part 7 traps table

Amazon ElastiCache

An in-memory cache. It is not a durable store of record.

- Methods or APIs: node-based Valkey clusters with durability enabled. Other API names: Not in source.
- Pro: Fast in-memory reads.
- Con: Speed does not make a service durable.
- Con: Durable only with durability enabled on node-based Valkey clusters. Other engine settings: Not in source.
- Choose it when: the design needs fast reads of data that a durable store holds.
- Do not choose it when: the data must be a store of record or a vector store. Use OpenSearch Serverless, Aurora with pgvector, or S3 Vectors.
- Native pairing: Inference: ElastiCache fits as a cache layer beside a durable store. The guide gives no pairing steps: Not in source.

Decision rule: Use ElastiCache as a cache only. Name a durable store for the data.

Trap: ElastiCache as the durable vector store is a wrong pick. Use OpenSearch Serverless, Aurora with pgvector, or S3 Vectors.

Exam tip: Fast and in memory do not mean durable. Look for a store of record in the wording.

Source: repo guide, Part 2 section 1.3 look-alike table, Part 2 section 1.4, Part 7 traps table

Amazon S3

Stores objects. Versioning keeps every version. Object Lock gives write-once-read-many retention.

- Methods or APIs: PUT and GetObject in a versioning-enabled bucket, DELETE with versionId, presigned URLs, SSE-KMS, S3 Lifecycle expiration, and S3 Select.
- Pro: Versioning restores an earlier draft.

- Pro: Object Lock gives write-once-read-many retention for audit records.
- Pro: A presigned URL grants time-limited access to one object without changing the bucket policy.
- Con: Versioning is off by default. After it is enabled, the bucket can only be suspended.
- Con: Each stored version is billed.
- Con: S3 Glacier Flexible Retrieval needs a restore before each object can be read.
- Choose it when: the design needs draft history, audit retention, a time-limited download, or an archive.
- Do not choose it when: users edit the same content live. Versioning is not a live collaboration tool.
- Native pairing: An S3 object event can invoke Lambda. Invocation logs land in S3 for Athena. S3 documents feed the Knowledge Bases ingest path.

Versioning and Object Lock

- An overwrite creates a new version. The previous version stays.
- A delete adds a delete marker. The data stays until you delete that version.
- Add a lifecycle rule that expires noncurrent versions after a set age.
- Object Lock sets a retain-until-date. In the diagram example from the guide, a 2-year duration sets 2029-01-01. Shortening it to 1 year does not reduce that date.
- When a hold is released, the date is fixed at the release time plus the duration.

Presigned URLs, KMS, and archive

- A URL created with the console lasts from 1 minute to 12 hours. A URL created with the CLI or SDK lasts up to 7 days.
- A presigned URL stops working when its signing credentials expire.
- SSE-KMS needs the caller to have kms:Decrypt. Disabling or changing the key policy revokes access.

- S3 Glacier Flexible Retrieval needs minutes to hours of retrieval. The minimum storage duration is 90 days.
- Each archived object adds 40 KB of metadata.

Decision rule: Use versioning for history, Object Lock for write-once retention, and presigned URLs for one object.

Trap: A presigned URL does not outlive its signing credentials. Revoke the key or the role to stop access.

Exam tip: When the wording says keep every version, look for versioning. When it says write-once, look for Object Lock.

Source: repo guide, Part 8A5 versioning and storage units, Part 8A6 S3 event unit, Part 8A7 official diagram captions, Part 4 section 3.2

Amazon S3 Vectors

Vector storage for large, infrequent search with no cluster to run.

- Methods or APIs: S3 vector bucket. Other API names: Not in source.
- Pro: No cluster to run.
- Pro: Knowledge Bases lists S3 Vectors as a vector store.
- Con: Steady, interactive search with high query volume fits an always-on store better.
- Con: Hierarchical chunking is not recommended with an S3 vector bucket.
- Con: Latency figures and quotas: Not in source.
- Choose it when: the corpus is large and queries are infrequent.
- Do not choose it when: queries are frequent and interactive. Use OpenSearch Serverless or Aurora with pgvector.
- Native pairing: Knowledge Bases can use an S3 vector bucket. For custom structure, a custom transformation Lambda function keeps tables and figures whole.

Decision rule: Choose S3 Vectors for large, rare searches. Choose an always-on store for steady, interactive search.

Trap: The cheapest store is a wrong pick when the latency need is strict.

Exam tip: The words large corpus, infrequent queries, and no cluster point to S3 Vectors.

Source: repo guide, Part 1A component table, Part 1D tie-in, Part 2 section 1.4, Part 8A5 knowledge base freshness and chunking unit

---

Security, operations, cost, and supporting AI services

This part covers the identity, network, data protection, operations, cost, and supporting AI services around each model call. Facts marked Not in source are not confirmed in the repo guide.

AWS IAM

IAM policies set resource boundaries on every tool call. A permissions boundary sets the most that a user or role can do.

- Methods or APIs: Not in source for API names. The source names execution roles, resource-based policies, and permissions boundaries.
- Pro: The effective permission is the overlap of the identity policy and the boundary. An explicit deny in either policy wins.
- Con: A boundary grants nothing alone. It does not limit a resource-based policy that grants access to a role session.
- Choose it when: a workload needs a role with narrow actions. Give the Lambda function an execution role.
- Do not choose it when: a value is a secret. Store the secret in Secrets Manager and grant the role read access.
- Native pairing: For S3 events, the function resource-based policy must allow S3 to invoke it. Knowledge Bases can use one IAM role per tenant.

Decision rule: Grant each workload only the actions and resources it needs. Use a permissions boundary to cap what a role can get.

Trap: A permissions boundary does not grant access. An allow in the identity policy and in the boundary are both needed.

Exam tip: Least privilege and cap point to IAM. Across all accounts points to an SCP.

Source: repo guide, Part 3 section 2.1, Part 4 task 3.2, Part 8A5 permissions boundary units, and Part 8A6 S3 event units.

AWS IAM Identity Center

A permission set is a template of IAM policies. Identity Center assigns it to users or groups in one or more accounts.

- Methods or APIs: Not in source for API names. The source gives a default role session of one hour, with a maximum of 12 hours. The default portal session is eight hours, with a maximum of 90 days.
- Pro: Staff sign in once to a portal with a company identity. Sessions are short-lived roles, so no long-term access key is needed.
- Con: A permission set grants access through roles. It does not replace the resource policy of a knowledge base or a guardrail.
- Choose it when: workforce users need access to AWS accounts. Create a narrow permission set for each team.
- Do not choose it when: the users are end users of your app. Use Cognito for app sign-in.
- Native pairing: AgentCore Gateway routes agent traffic through one endpoint. Identity Center gives workforce users federated access.

Decision rule: Use Identity Center for staff access. Use Cognito for app users. Use a narrow permission set, not the administrator set.

Trap: The administrator set for everyone breaks least privilege. Assign a narrower set for daily work.

Exam tip: Company identity and staff point to Identity Center. Users of your app point to Cognito.

Source: repo guide, Part 8A4 gap units on workforce access, and Part 3 visual units on the enterprise gateway.

Amazon Cognito

A user pool authenticates users and issues JSON web tokens. An identity pool exchanges a trusted token for temporary AWS credentials.

- Methods or APIs: Not in source for API names. The source names user pools and identity pools.
- Pro: A user pool handles sign-in for your app users.
- Con: The source does not cover Cognito pricing, quotas, or sign-in flows. Not in source.
- Choose it when: app users must sign in. Use a user pool for sign-in.
- Do not choose it when: staff need AWS account access. Use IAM Identity Center.
- Native pairing: Not in source for a named service. The identity pool gives temporary AWS credentials for direct AWS access.

Decision rule: Use a user pool for sign-in. Use an identity pool when the client needs direct AWS access.

Trap: A user pool issues JSON web tokens, not AWS credentials. An identity pool exchanges a trusted token for AWS credentials.

Exam tip: Sign-in points to a user pool. Temporary AWS credentials point to an identity pool.

Source: repo guide, Part 8A reference terms on Amazon Cognito, and Part 8C glossary.

Amazon Verified Permissions

Verified Permissions is an authorization service. It evaluates policies written in the Cedar policy language.

- Methods or APIs: Not in source for API names. The application asks whether a request is allowed.
- Pro: Authorization rules sit in written policies that the service evaluates.
- Con: The source does not state how the service combines with IAM. Not in source.
- Choose it when: the application must decide if a request is allowed under written policies.
- Do not choose it when: you need user sign-in. Use Cognito for sign-in.
- Native pairing: Not in source.

Cedar policy language

Cedar is the policy language that Verified Permissions uses to decide if a request is allowed.

The source names no Cedar syntax details. Not in source.

Decision rule: Use Verified Permissions for per-request allow decisions. Use IAM for AWS API access.

Trap: The source does not state how Verified Permissions and IAM combine. Not in source.

Exam tip: Policies written in Cedar point to Verified Permissions.

Source: repo guide, Part 8A reference terms on Amazon Verified Permissions, and Part 8C glossary on Cedar.

AWS KMS

SSE-KMS encrypts S3 objects with a KMS key. KMS keys encrypt data at rest.

- Methods or APIs: kms:Decrypt is the permission that a caller needs to read an SSE-KMS object.
- Pro: A change to the key policy revokes access to the encrypted data.
- Con: Encryption does not change the network path. Encryption answers the storage risk only. Check data terms for training use.

- Choose it when: data at rest needs a key that you control. Use it for invocation logs, S3 objects, and Lambda environment variables.
- Do not choose it when: you need a private network path. Use PrivateLink for that need.
- Native pairing: Lambda environment variables accept a customer managed key through KMSKeyARN. The source diagram shows KMS keys for invocation logs and CloudTrail records.

S3 Object Lock

S3 Object Lock gives write-once-read-many retention. S3 Lifecycle expiration deletes objects at the end of their lifetime.

Pro: Object Lock keeps a record unchanged for a set time. Con: A shorter duration does not reduce the retain-until-date. A 2-year duration sets a 2029-01-01 date, and a change to 1 year keeps that date.

Decision rule: Use Object Lock when a record must stay unchanged for a set time. Use Lifecycle when an object must expire.

Trap: A shorter duration does not shorten retention. The retain-until-date stays at the later date.

Decision rule for KMS: Use KMS for encryption at rest. Use key policies and IAM to control who decrypts.

Trap: A presigned URL does not outlive its signing credentials. Revoke the key or the role to stop access.

Exam tip: Encrypt at rest points to KMS. Write-once wording points to Object Lock.

Source: repo guide, Part 4 task 3.2, Part 8A5 SSE-KMS units, Part 8A6 Lambda variable units, and Part 8A7 Object Lock timelines.

AWS Secrets Manager

AWS recommends Secrets Manager instead of Lambda environment variables for database credentials, API keys, and authorization tokens.

- Methods or APIs: Not in source for API names. The source says to read credentials from Secrets Manager at run time.
- Pro: The function reads the secret at run time, so the value stays out of the function configuration.
- Con: Rotation, pricing, and quotas are Not in source.
- Choose it when: code needs a password, key, or token.
- Do not choose it when: the value is a plain setting, such as a bucket name or log level. Keep those in environment variables.
- Native pairing: AppConfig can use Secrets Manager as a free-form configuration store. Lambda reads the value at run time.

Decision rule: Put secrets in Secrets Manager. Keep non-secret settings in environment variables or AppConfig.

Trap: Encryption at rest does not make an environment variable a secret. The source still recommends Secrets Manager.

Exam tip: Words such as API key or database password point to Secrets Manager.

Source: repo guide, Part 8A6 gap units on Lambda environment variables, and Part 1C AWS AppConfig.

AWS Systems Manager Parameter Store

AppConfig lists Systems Manager Parameter Store as a store for free-form configuration data.

- Methods or APIs: Not in source.
- Pro: Not in source. The source names it only as an AppConfig store.
- Con: Limits, tiers, and encryption details are Not in source.
- Choose it when: the AppConfig free-form data lives in Parameter Store.
- Do not choose it when: the value is a credential. The source recommends Secrets Manager for credentials.

- Native pairing: An AppConfig configuration profile can read from Parameter Store as one of its stores.

Decision rule: For credentials, use Secrets Manager. For AppConfig free-form data, Parameter Store is one listed store.

Trap: Parameter Store limits are Not in source. Do not state a size or tier from memory.

Exam tip: Parameter Store appears in the source only as an AppConfig store.

Source: repo guide, Part 1C AWS AppConfig and Part 1F console steps for AppConfig. Parameter Store details are Not in source.

AWS Organizations and service control policies

A service control policy sets the most that a member account can do. An SCP grants no permissions.

- Methods or APIs: A condition on the bedrock:GuardrailIdentifier key can require a guardrail on each request.
- Pro: One rule applies to every member account. The IAM layer still applies when application code changes or skips the check.
- Con: SCPs do not affect the management account or service-linked roles. A user still needs an identity policy that allows the action.
- Choose it when: a rule must hold across accounts, such as a required guardrail identifier.
- Do not choose it when: one application in one account needs the rule. Use IAM in that account.
- Native pairing: Organizations groups accounts into organizational units and gives one consolidated bill. StackSets can deploy the guardrail that the SCP requires.

Decision rule: Use an SCP with IAM condition keys for an organization-wide rule. Test the SCP on one organizational unit first.

Trap: A guardrail that only the application calls can be skipped. Enforce it in IAM.

Exam tip: Across all accounts or every team points to an SCP, not to one application setting.

Source: repo guide, Part 4 task 3.3, Part 8A5 gap units on SCPs and Organizations, and Part 3 visual units on SCPs and condition keys.

AWS CloudFormation and StackSets

CloudFormation creates and deletes a collection of resources as one unit, called a stack, from a template file.

- Methods or APIs: Not in source for API names. The source names stacks and StackSets.
- Pro: StackSets create, update, or delete stacks across many accounts and Regions in one operation.
- Con: The source does not describe StackSet limits or permission setup. Not in source.
- Choose it when: one template must deploy the same stack to many accounts.
- Do not choose it when: a stack in one account is enough. Not in source for this comparison.
- Native pairing: Deploy the guardrail with a StackSet. Then deny calls that omit the guardrail identifier in an SCP.

Decision rule: Use CloudFormation for repeatable stacks. Use StackSets when one template must reach many accounts.

Trap: A StackSet deploys the guardrail. It does not block calls by itself. Add the SCP.

Exam tip: The same template in many accounts points to StackSets.

Source: repo guide, Part 8A reference terms on CloudFormation, Part 8A5 gap units on StackSets, and Part 0 target candidate.

AWS PrivateLink and VPC endpoints

A VPC interface endpoint gives private IP access to an AWS service. Traffic from the VPC stays on the AWS network.

- Methods or APIs: The Bedrock planes are bedrock-runtime for invocations, bedrock-agent and bedrock-agent-runtime for agents, and bedrock for the control plane.
- Pro: API traffic stays on the AWS network and off the public internet.
- Con: Each plane needs its own endpoint. One runtime endpoint does not cover agent calls.
- Choose it when: the workload must keep API traffic on the AWS network.
- Do not choose it when: the need is outbound internet access. A NAT gateway gives that access, and it is not a private path to Bedrock.
- Native pairing: In the safety path, PrivateLink sits below the model calls beside KMS keys for data at rest.

Gateway endpoint for DynamoDB

A gateway VPC endpoint for DynamoDB adds a route to the route table.

It needs no internet gateway and no NAT device. It has no extra charge. It does not serve on-premises networks, peered VPCs, or transit gateways.

Decision rule: Use an interface endpoint for private API calls. Use a gateway endpoint route for DynamoDB from a private subnet.

Trap: A timeout to DynamoDB from a private subnet often means a missing route, not a security group rule.

Exam tip: Private network path points to PrivateLink. Data at rest points to KMS.

Source: repo guide, Part 4 task 3.2, Part 1E PrivateLink visual unit, Part 8A5 DynamoDB endpoint units, and Part 8D security hardening unit.

NAT gateways and idle timeout

A NAT gateway gives outbound internet access. It is not a private path to Bedrock.

- Methods or APIs: Not in source for NAT API names and placement rules.
- Pro: A private workload gets outbound internet access.
- Con: NAT gateways, interface VPC endpoints, and Network Load Balancers drop idle TCP connections after 350 seconds.
- Choose it when: a private workload needs outbound internet access.
- Do not choose it when: the need is a private path to Bedrock. Use PrivateLink.
- Native pairing: Not in source for a named service pairing. The 350 second idle limit also applies to interface VPC endpoints.

Fix for idle drops: Enable TCP keep-alive in the SDK client. Set the OS keep-alive interval below 350 seconds. You need both settings.

Decision rule: For long idle connections, set both SDK keep-alive and an OS keep-alive interval below 350 seconds.

Trap: One keep-alive setting is not enough. The source requires both settings.

Exam tip: Dropped idle connections point to the 350 second limit and to keep-alive settings.

Source: repo guide, Part 8A3 error codes and connection timeouts, and Part 8D security hardening unit. Placement rule is Not in source.

Network ACLs and security groups

A network ACL filters at the subnet edge. A security group filters at the instance.

- Methods or APIs: Not in source for API names.
- Pro: A security group is stateful, so it allows return traffic for an allowed request.

- Con: A network ACL is stateless. It checks rules in order by rule number, from lowest to highest, and the first match applies.
- Choose it when: you need a subnet-edge rule with explicit rule order. Use a security group for per-instance rules.
- Do not choose it when: you need return traffic handled for you. A security group handles that traffic.
- Native pairing: Not in source for Bedrock or Lambda specifics.

Decision rule: Use a security group for instance access. Use a network ACL for subnet-edge rules that depend on rule order.

Trap: A network ACL stops at the first matching rule. A later allow rule does not override an earlier deny.

Exam tip: Rule number order points to a network ACL. Return traffic handled for you points to a security group.

Source: repo guide, Part 8A5 gap units on network ACLs and security groups.

AWS WAF

WAF filters web requests. It does not make a network path private.

- Methods or APIs: Not in source for rule configuration.
- Pro: The source names WAF as a filter for web requests.
- Con: Rule configuration and rule names are Not in source.
- Choose it when: the need is to filter web requests.
- Do not choose it when: the need is a private network path. Use PrivateLink.
- Native pairing: Not in source. The source pairs API Gateway throttling and usage plans with the defense layers instead.

Decision rule: Use WAF to filter web requests. Use PrivateLink for a private network path.

Trap: WAF does not make a network path private. Do not use it as a private path control.

Exam tip: Filter web requests points to WAF. Keep traffic off the internet points to PrivateLink.

Source: repo guide, Part 8D security hardening unit, and Part 9 gaps, where AWS WAF rules are Not in source.

AWS Outposts

Outposts runs AWS infrastructure on your premises for data that must stay on site.

- Methods or APIs: The same AWS programming interfaces apply on Outposts.
- Pro: Data processed on Outposts stays at the site for the rule that requires it.
- Con: Hardware, capacity, and pricing details are Not in source.
- Choose it when: a rule says the data stays on site.
- Do not choose it when: no on-site rule applies. Use a Region for most workloads.
- Native pairing: Not in source for a named model or service pairing.

Decision rule: Use a Region for most workloads. Use Outposts when a rule keeps the data on site.

Trap: A Region in the cloud does not keep data on your premises.

Exam tip: Data must stay on site points to Outposts. The same APIs do not change where data sits.

Source: repo guide, Part 1A component table, and Part 1E visual units on Outposts versus a Region.

Amazon Macie

Macie discovers sensitive data in S3 objects at rest.

- Methods or APIs: Not in source for API names.
- Pro: It finds sensitive data that is already stored in S3.
- Con: Macie does not watch live model traffic.

- Choose it when: sensitive data sits in S3 and needs discovery.
- Do not choose it when: the data is model input or output. Use Comprehend before the call, or Guardrails at the call.
- Native pairing: S3 at rest, with KMS keys for data at rest in the safety path.

Decision rule: Macie for S3 at rest. Comprehend for text before a call. Guardrails at the model call.

Trap: Guardrails does not scan a whole bucket. Macie does not watch live model traffic.

Exam tip: Sensitive data in S3 points to Macie.

Source: repo guide, Part 4 task 3.2 privacy table, and Part 1B lesson on safety layers.

Amazon Comprehend

Comprehend detects entities and PII in text. Real-time entity detection takes single documents up to 100 KB.

- Methods or APIs: Real-time entity detection and PII detection. The source names both.
- Pro: The source design runs Comprehend as a pre-filter before the model call.
- Con: Sentiment and syntax take 5 KB. The PII-only limit is Not in source as a separate line.
- Choose it when: text needs a PII check before the model call.
- Do not choose it when: the need is a whole S3 bucket. Use Macie.
- Native pairing: The source pipeline runs a Comprehend PII pre-filter, then Guardrails on input, then the model.

Decision rule: Comprehend checks text before the model call. Guardrails redact or block PII at the model call.

Trap: The 100 KB limit applies to single documents in real-time entity detection. The PII-only limit is Not in source.

Exam tip: A pre-filter for PII before a call points to Comprehend.

Source: repo guide, Part 1A component table, Part 4 task 3.2 privacy table, and Part 1A architecture 3 on the safety path.

Amazon Textract

Textract detects typed and handwritten text. It extracts text, forms, and tables.

- Methods or APIs: Queries, AnalyzeExpense for invoices and receipts, AnalyzeID for ID documents, and Analyze Lending.
- Pro: The source names it for document input before a model call.
- Con: Pricing, quotas, and accuracy figures are Not in source.
- Choose it when: a document needs text, form, or table extraction before the model call.
- Do not choose it when: you need document fields across documents, images, audio, and video. Bedrock Data Automation covers that need.
- Native pairing: Not in source for a named pairing. Its output feeds the model call.

Decision rule: Use Textract for text, forms, and tables. Use Data Automation for field extraction across media types.

Trap: Guardrails does not extract fields. The source names Data Automation or a parser for document fields.

Exam tip: Invoices and receipts point to AnalyzeExpense. ID documents point to AnalyzeID.

Source: repo guide, Part 8A reference terms on Amazon Textract, and Part 8A5 gap units on Data Automation.

Amazon Transcribe

Transcribe converts speech to text with speaker labels.

- Methods or APIs: Not in source for API names.
- Pro: The source names speaker labels as a feature.

- Con: Pricing and language coverage are Not in source.
- Choose it when: the input is speech and the next step needs text.
- Do not choose it when: you need fields from audio. Bedrock Data Automation covers fields from audio and video.
- Native pairing: Transcribe runs in the process stage. Comprehend enhances the text in the next stage.

Decision rule: Speech to text with speaker labels points to Transcribe. Fields from audio point to Data Automation.

Trap: Transcribe returns text. The source then sends that text to Comprehend for PII detection.

Exam tip: Speech and speaker labels point to Transcribe.

Source: repo guide, Part 2 section 1.3 data validation, Part 1A component table, and Part 8A5 gap units on Data Automation.

Amazon Translate

Translate changes the language of text. It does not detect PII.

- Methods or APIs: Not in source for API names.
- Pro: It changes the language of text.
- Con: It does not detect or protect PII.
- Choose it when: the task is translation of text.
- Do not choose it when: the task is PII detection or protection. Use Comprehend for detection.
- Native pairing: Use Comprehend to detect PII in the text.

Decision rule: Translate for language. Comprehend for PII.

Trap: Translate changes language, not PII. It is not a privacy control.

Exam tip: A language change does not solve a privacy need. Read what the service returns first.

Source: repo guide, Part 1A component table, and Part 2 visual units on service distractors.

Amazon Rekognition

Rekognition is a vision service.

- Methods or APIs: Not in source.
- Pro: Not in source. The source names only its category.
- Con: Features, limits, and pricing are Not in source. The source does not present it as a PII control.
- Choose it when: the task is vision. Not in source for specific use cases.
- Do not choose it when: the need is PII detection in text. Use Comprehend.
- Native pairing: Not in source.

Decision rule: Use Rekognition for a vision task. Use Comprehend for PII in text.

Trap: The source does not present Rekognition as a PII control. Check what the service returns before you choose it.

Exam tip: Image words point to Rekognition. PII in text points to Comprehend.

Source: repo guide, Part 8D security hardening unit. Other Rekognition details are Not in source.

Amazon Kendra

Kendra is an enterprise search service over documents. It is no longer open to new customers.

- Methods or APIs: Not in source for API names.
- Pro: The source names it for enterprise search over documents.
- Con: New customers cannot open it. AWS points new work to Bedrock Knowledge Bases.

- Choose it when: Not in source for existing use. The source gives only the new-customer status.
- Do not choose it when: new work needs enterprise search. Use Knowledge Bases.
- Native pairing: Knowledge Bases is the documented alternative. A sync and one API call cover its flow.

Decision rule: For new enterprise search, choose Knowledge Bases. The source names it as the alternative to Kendra.

Trap: Kendra looks right for enterprise search, but it is not open to new customers.

Exam tip: Read the status note first for any service that the source marks as no longer open.

Source: repo guide, Part 1A component table, and Part 2 visual units on service distractors.

Amazon CloudWatch

CloudWatch provides metrics, alarms, anomaly detection, and Logs Insights. Anomaly detection builds a band from history and alarms when a value leaves it.

- Methods or APIs: Metric names include InputTokenCount, OutputTokenCount, Invocations, and InvocationThrottles. Logs Insights searches invocation logs by time, caller identity, or request metadata.
- Pro: Anomaly alarms catch spikes that a fixed threshold misses.
- Con: A fixed threshold alarm misses slow growth. CloudWatch Synthetics and Auto Scaling for GenAI workloads are Not in source.
- Choose it when: the signal is rate, cost, or throttling over time.
- Do not choose it when: you need the exact prompt text. Search the invocation logs for that text.
- Native pairing: AppConfig alarms watch CloudWatch metrics during a rollout. X-Ray traces the slow step.

Decision rule: Metrics for rate and cost. X-Ray for the slow step. Invocation logs for exact content.

Trap: Metrics alone cannot rebuild what the model received. Keep the payload logs under access control.

Exam tip: A spike that a fixed threshold misses points to anomaly detection.

Source: repo guide, Part 1A operations path, Part 5 task 4.3 monitoring signals, and Part 1B operations signals lesson.

AWS X-Ray

X-Ray traces requests and downstream calls across services.

- Methods or APIs: Not in source for SDK or API names. The source says to trace calls and log token counts per call.
- Pro: It finds the slow step in a request path.
- Con: It does not hold the exact prompt text. Invocation logs hold that content.
- Choose it when: a request is slow and the slow step is not known.
- Do not choose it when: you need the exact prompt or response. Use invocation logs.
- Native pairing: The source places X-Ray on the operations path beside the request path and the invocation logs.

Decision rule: Use X-Ray to find the slow step. Then read the invocation logs for the exact content.

Trap: A trace alone does not prove what the model received. Read the invocation logs.

Exam tip: Which step is slow points to X-Ray.

Source: repo guide, Part 1A architecture 1 and architecture 5, Part 3 section 2.4, and Part 5 task 4.3.

AWS CloudTrail

CloudTrail records API calls for audit. It shows who called which API, when, and from where.

- Methods or APIs: Not in source for API names. The source names InvokeModel and Converse as recorded calls.
- Pro: It records Amazon Bedrock API calls for the audit trail.
- Con: It does not show row content or prompt text.
- Choose it when: the need is the identity and timing of an API call.
- Do not choose it when: you debug an answer. Use CloudWatch Logs and invocation logs.
- Native pairing: Invocation logs hold the request and response. KMS keys protect both records.

Decision rule: CloudTrail for who called and when. Invocation logs for what was sent.

Trap: CloudTrail is not a data trigger. The source uses S3 events and EventBridge for triggers.

Exam tip: Who and when points to CloudTrail. Words such as prompt or response point to invocation logs.

Source: repo guide, Part 4 task 3.3, Part 6 task 5.2 debugging tools, and Part 7 traps list.

AWS AppConfig

AppConfig holds runtime settings and rolls them out to a running application. The application picks up the change without a code release.

- Methods or APIs: Feature flags, free-form configuration, validators, deployment strategies, and alarms. The source names these parts.
- Pro: A model ID can change at run time. The Lambda function reads the ID, so the function needs no code release.
- Con: AppConfig controls settings only. It does not route traffic by prompt complexity, and it does not host the model.

- Choose it when: a running app reads a setting that changes, such as a model ID.
- Do not choose it when: routing rules must run by prompt complexity. Use Lambda routing with rules you write.
- Native pairing: Lambda reads the setting, and CloudWatch alarms watch the rollout. The stores are the hosted store, Secrets Manager, Parameter Store, and Amazon S3.

Deployment strategy, validators, and alarms

A deployment strategy sets how fast a change reaches the application. Validators check syntax and meaning before deployment.

Alarms watch CloudWatch metrics during a deployment. If an alarm fires, AppConfig rolls the change back. Feature flags release a capability to some users first. Console rollout field names are Not in source.

Decision rule: Use AppConfig for a setting that changes without a code release. Use an alarm to roll back a bad change.

Trap: AppConfig does not host the model. It does not route by prompt complexity.

Exam tip: Without a code release points to AppConfig. Custom rules point to Lambda code.

Source: repo guide, Part 1C AWS AppConfig, Part 1F console steps for AppConfig, and Part 1A release path.

AWS CodeDeploy

Not in source. The repo guide lists AWS CodeDeploy only as a gap.

- Methods or APIs, pros, cons, and choice rules: Not in source.
- Native pairing: Not in source.

For runtime settings, the source uses AppConfig deployment strategies and alarms.

Decision rule: Not in source for CodeDeploy. For runtime settings, use AppConfig deployment strategies.

Trap: Not in source. Do not state CodeDeploy features from memory.

Exam tip: Treat CodeDeploy as a gap. Check the AWS page before you rely on it.

Not in source

AWS CodePipeline

Not in source. The repo guide does not name CodePipeline.

- Methods or APIs, pros, cons, and choice rules: Not in source.
- Native pairing: Not in source. The source uses CI/CD only in the SageMaker Model Registry context. There, an approved version can start a deployment through a project template.

Decision rule: Not in source for CodePipeline. For release gates, use the golden set, quality gate, and canary that the source names.

Trap: Not in source. Do not map CI/CD to CodePipeline from memory.

Exam tip: Treat CodePipeline as a gap. Name the release gate that the source states instead.

Not in source

AWS Amplify

Amplify is the source choice for a web app with managed hosting and sign-in.

- Methods or APIs: Not in source.
- Pro: Managed hosting and sign-in for a web app.
- Con: Features beyond hosting and sign-in are Not in source.
- Choose it when: a web team needs hosting and sign-in.
- Do not choose it when: the need is real-time collaborative editing. Use AppSync with DynamoDB.

- Native pairing: Not in source for a named service pairing.

Decision rule: Amplify for a web app with hosting and sign-in. AppSync for real-time editing.

Trap: Amplify does not replace AppSync for real-time collaborative editing.

Exam tip: Managed hosting and sign-in points to Amplify.

Source: repo guide, Part 3 section 2.5 application tools, and Part 3 visual units on application tools.

AWS AppSync

AppSync with DynamoDB is the source choice for real-time collaborative editing.

- Methods or APIs: Not in source.
- Pro: The source names it for real-time collaborative editing.
- Con: Limits and pricing are Not in source.
- Choose it when: several users edit the same content in real time.
- Do not choose it when: you need draft history. Use S3 versioning for the history.
- Native pairing: DynamoDB stores the data. The source names this pair.

Decision rule: AppSync with DynamoDB for live editing. S3 versioning for draft history.

Trap: Versioning is not a live collaboration tool. Use a sync service for live edits.

Exam tip: Live editing points to AppSync. Draft history points to S3 versioning.

Source: repo guide, Part 3 section 2.5, and Part 8A5 gap units on versioning.

Amazon Q Developer

Amazon Q Developer is a generative AI assistant. It helps you understand, build, extend, and operate AWS applications.

- Methods or APIs: In an IDE, code chat and inline code completions. Not in source for API names.
- Pro: The source names code help in the IDE, code changes, and test generation.
- Con: It serves staff who build AWS applications. It does not serve end users of your app. The official page names an IDE plugin support end date, and the date is Not in source.
- Choose it when: developers need code help in the IDE.
- Do not choose it when: your customers need a chat feature. Build a Bedrock application for them.
- Native pairing: Q Developer is built on Amazon Bedrock.

Decision rule: Q Developer for staff who write AWS code. A Bedrock application for your own end users.

Trap: Amazon Q Developer and Amazon Q Business have similar names. One writes code. The other does company chat.

Exam tip: Code help in the IDE points to Q Developer. Company chat points to Q Business.

Source: repo guide, Part 3 section 2.5, and Part 8A8 topic units on Amazon Q Developer.

Amazon Q Business

Q Business gives company chat over company data. It is no longer open to new customers.

- Methods or APIs: Not in source for API names.
- Pro: The source names it for chat over company data.
- Con: New customers cannot open it. Amazon Quick is the documented alternative.
- Choose it when: Not in source for new work. Check the status note first.
- Do not choose it when: developers need code help. Use Q Developer.

- Native pairing: Not in source.

Decision rule: For company chat, check Amazon Quick, the documented alternative, before you choose Q Business.

Trap: Q Business is not a code tool. Its name is close to Q Developer.

Exam tip: Company chat over company data points to Q Business. Read the status note before you choose it.

Source: repo guide, Part 1A component table, and Part 3 section 2.5.

Spot instances

A Spot Instance uses spare EC2 capacity at a price below On-Demand.

- Methods or APIs: Not in source for API names.
- Pro: A lower price than On-Demand for work that can restart.
- Con: EC2 can interrupt the instance when it needs the capacity. The interruption notice is two minutes.
- Choose it when: a batch job or background work can restart.
- Do not choose it when: a user waits for a reply. Use real-time calls.
- Native pairing: Not in source for a named service pairing. Savings Plans do not apply to Spot.

Decision rule: Use Spot for restartable batch work. Use on-demand or real-time calls for a waiting user.

Trap: Spot is wrong for a user waiting on a reply. Spot is EC2 capacity, not Bedrock batch inference.

Exam tip: Interruptible and restartable work points to Spot.

Source: repo guide, Part 8A5 gap units on Spot instances, and Part 5 task 4.1 cost.

Savings Plans

Savings Plans do not apply to Spot. The source has no Savings Plan fact for inference.

- Methods or APIs: Not in source.
- Pro: Not in source for inference.
- Con: Savings Plans do not apply to Spot instances.
- Choose it when: Not in source for inference.
- Do not choose it when: you want an inference discount. Not in source.
- Native pairing: Not in source.

Decision rule: For inference cost, use the levers in the source, such as Provisioned Throughput for steady load.

Trap: A one-year Savings Plan for inference is Not in source. Do not state it as a fact.

Exam tip: Check each commitment discount claim for inference against the source before you use it.

Source: repo guide, Part 8A5 gap units on Spot instances, and Part 8D decision ladders unit. Inference Savings Plans are Not in source.

AWS Cost Explorer

The source says cost allocation tags and application inference profiles appear in Cost Explorer and the Cost and Usage Report.

- Methods or APIs: Cost Explorer features are Not in source.
- Pro: Per-team cost is visible when cost allocation tags exist. The source names this use.
- Con: Per-request metadata does not appear in Cost Explorer.
- Choose it when: finance needs cost split by team or feature on the bill.
- Do not choose it when: you need per-call detail. Use request metadata in invocation logs.
- Native pairing: AWS Organizations gives one consolidated bill for all accounts.

Decision rule: A bill split by team or feature points to cost allocation tags and Cost Explorer.

Trap: The Domain 4 gap list names Cost Explorer as Not in source. Check the console before you rely on its features.

Exam tip: A bill-level split points to Cost Explorer. Per-call detail points to invocation logs.

Source: repo guide, Part 5 Domain 4 gaps, Part 8A5 gap units on cost attribution and Organizations, and Part 9 gaps.

Cost allocation tags versus request metadata

Request metadata attaches up to 16 key-value pairs to a call. The pairs appear in model invocation logs, not on the bill.

- Methods or APIs: Request metadata on each call. Application inference profiles route to cost allocation tags.
- Pro: Request metadata gives per-call detail for analysis.
- Con: Request metadata is not a cost allocation tag. It does not appear in Cost Explorer or the Cost and Usage Report.
- Choose it when: the team needs per-call detail in logs.
- Do not choose it when: finance needs a bill line per feature. Use an application inference profile per feature.
- Native pairing: Reconcile invocation logs to the bill at the model and usage-type level.

Decision rule: Request metadata for logs. An application inference profile per feature for the bill.

Trap: Token metrics tagged per feature do not put cost on the bill. Per-request tags do not reach the bill.

Exam tip: Bill-level attribution points to an application inference profile, not to request metadata.

Source: repo guide, Part 8A5 gap units on cost attribution.

Well-Architected Generative AI Lens

The Generative AI Lens covers LLM applications. It excludes model training and advanced customization.

- Methods or APIs: Not in source for lens tools.
- Pro: Six pillars apply: sustainability, cost optimization, performance efficiency, reliability, security, and operational excellence.
- Con: The full list of design principles is Not in source.
- Choose it when: a team reviews a GenAI workload design across the six pillars.
- Do not choose it when: the work is model training. The lens excludes training.
- Native pairing: Not in source for a named service pairing.

Decision rule: Use the Generative AI Lens to review an application design. Use another source for model training.

Trap: The lens does not cover model training. Do not apply it to training design.

Exam tip: Six pillars wording points to the Well-Architected Framework, not to one service.

Source: repo guide, Part 8A3 Well-Architected Generative AI Lens unit, and Part 2 task 1.1.

Decision trees

Which privacy service

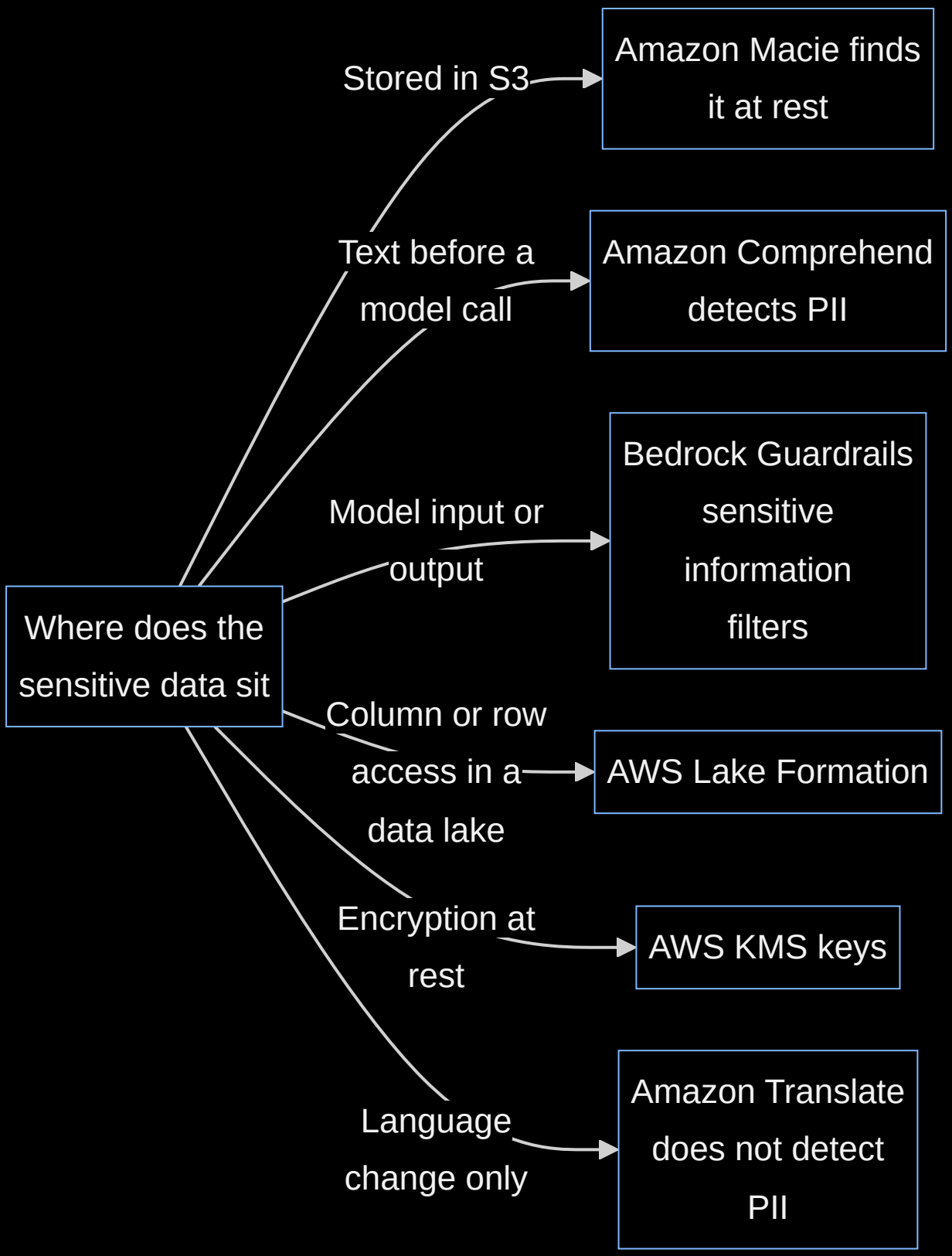


Figure: Privacy service choice by where the sensitive data sits.

Which network path

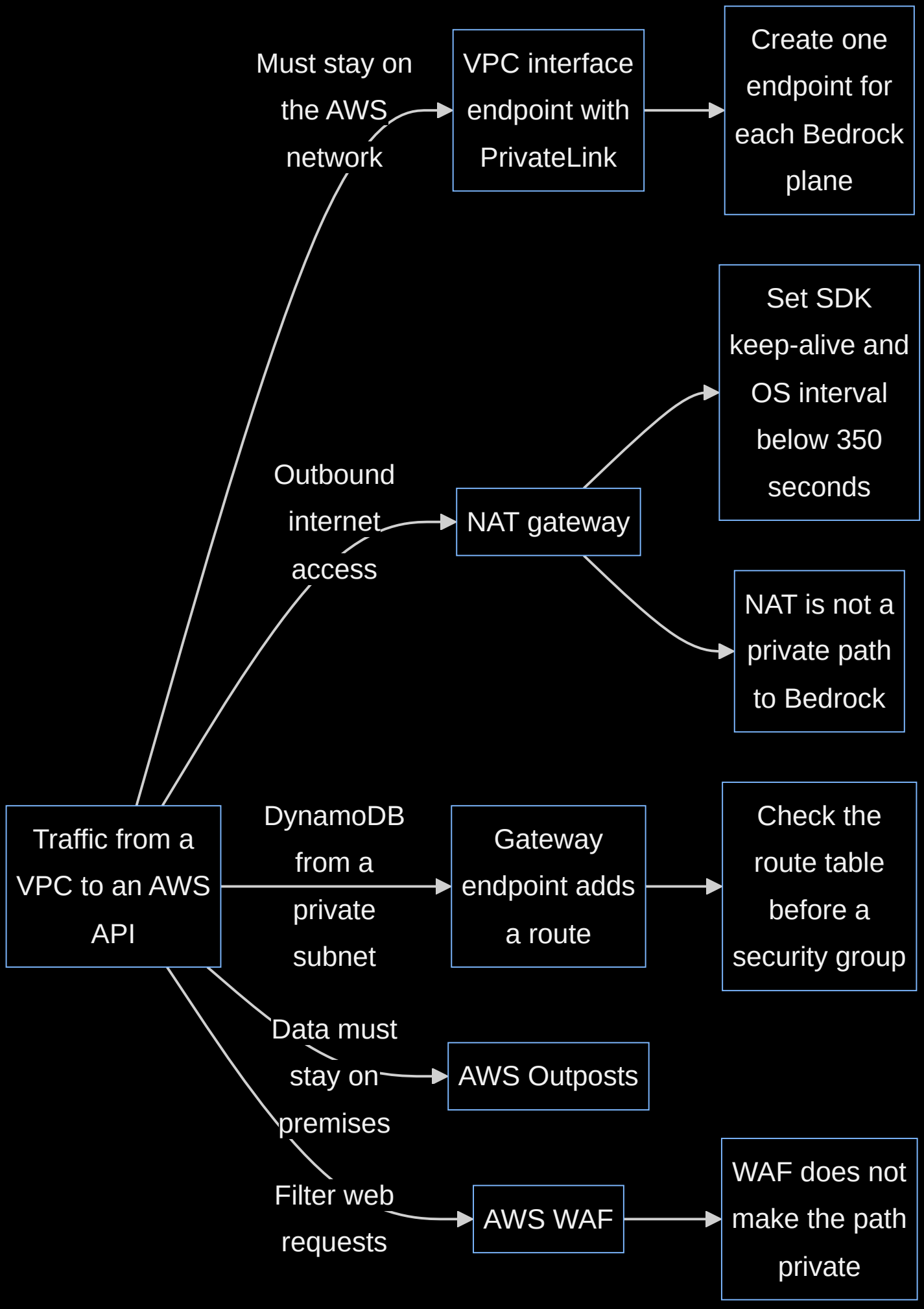


Figure: Network path choice for traffic from a VPC.

Which identity mechanism

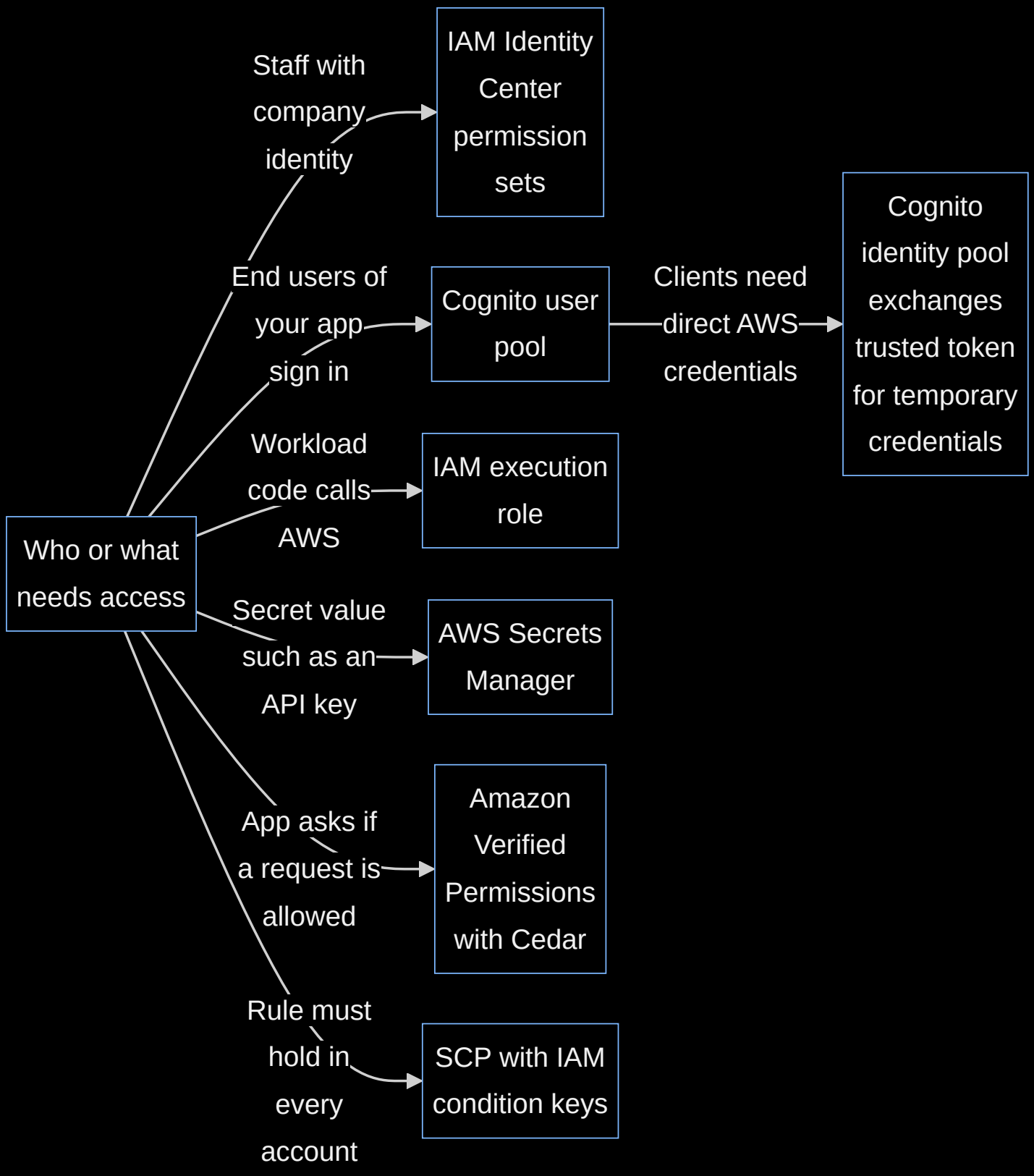


Figure: Identity mechanism choice by who or what needs access.

Which monitoring signal

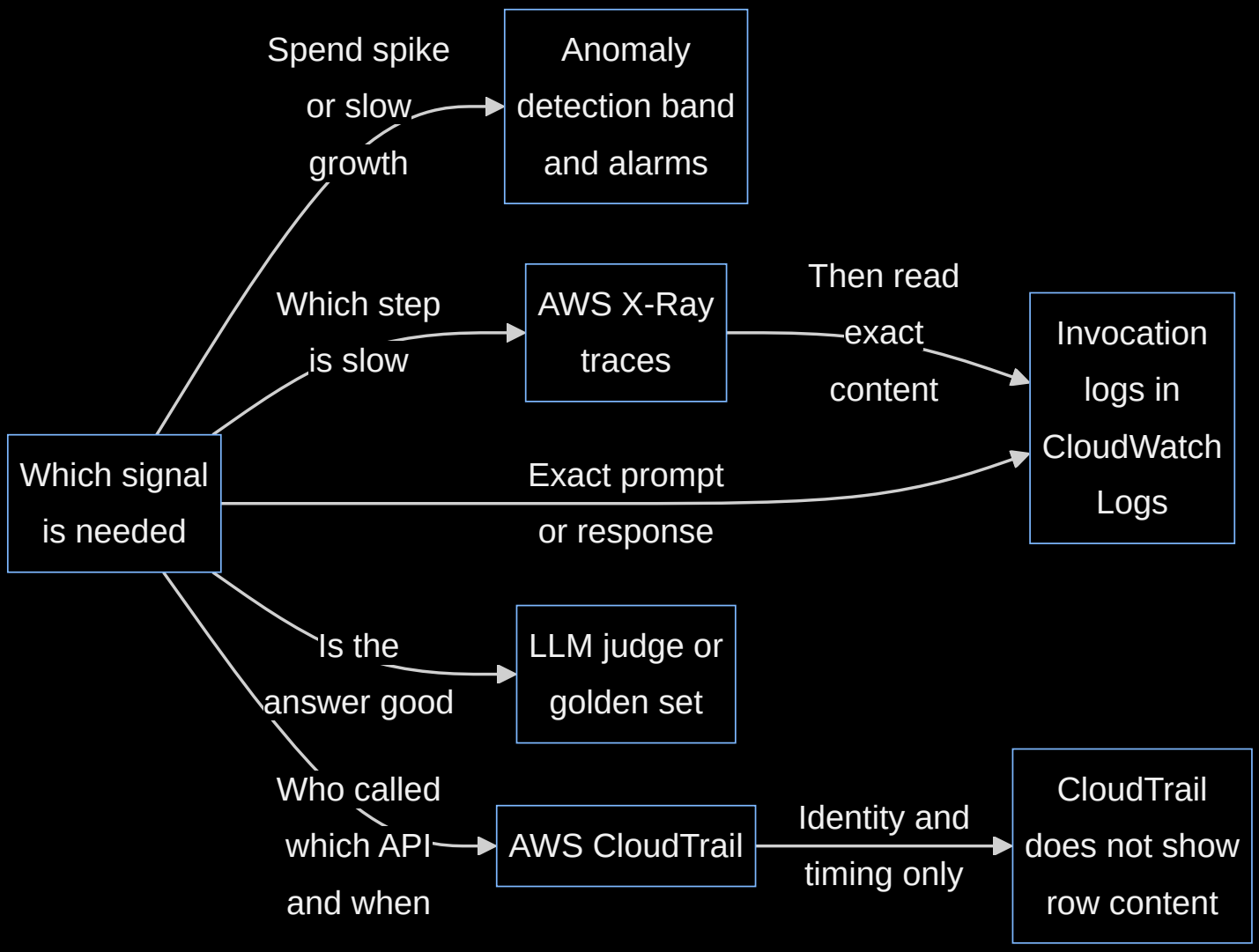


Figure: Monitoring signal choice by what the signal must show.

Which cost lever

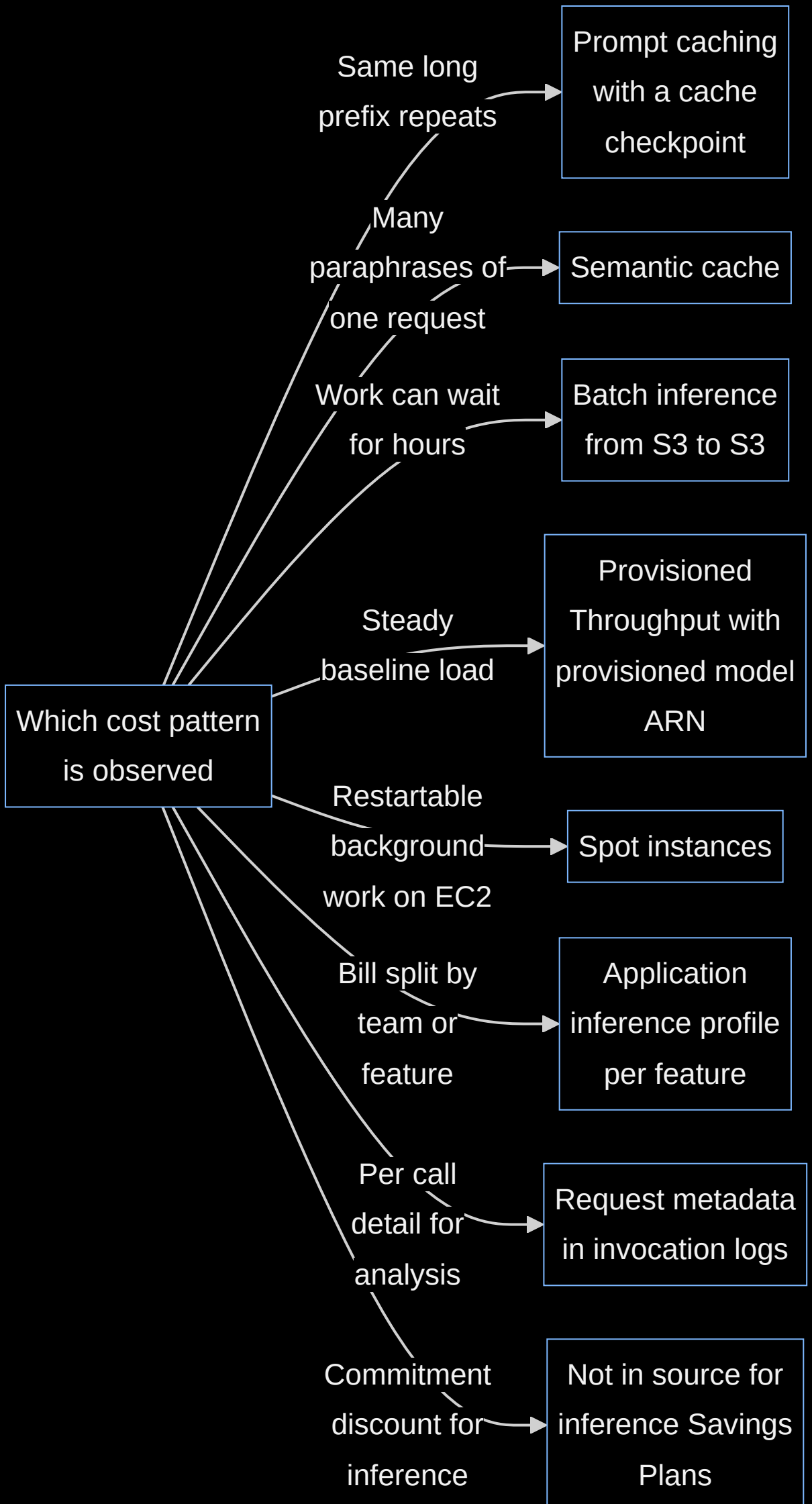

Which cost pattern is observed
Same long prefix repeats
Prompt caching with a cache checkpoint
Many paraphrases of one request
Semantic cache
Work can wait for hours
Batch inference from S3 to S3
Steady baseline load
Provisioned Throughput with provisioned model ARN
Restartable background work on EC2
Spot instances
Bill split by team or feature
Application inference profile per feature
Per call detail for analysis
Request metadata in invocation logs
Commitment discount for inference
Not in source for inference Savings Plans

Figure: Cost lever choice by the observed cost pattern.

Which supporting AI service

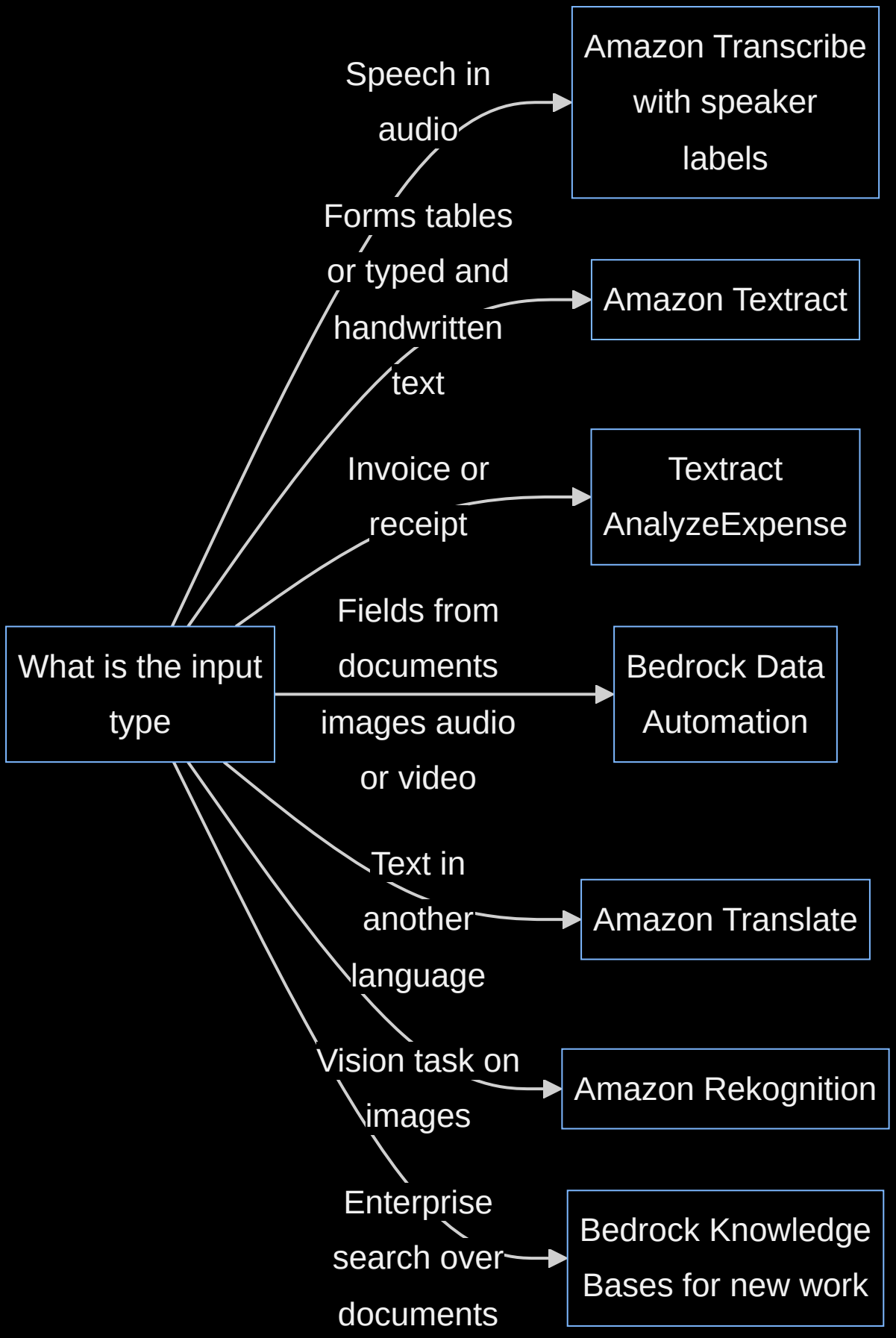


Figure: Supporting AI service choice by input type.

Domain 1 · Foundation model integration, data management, and compliance · Weight 31 percent

Domain 1 asks how to get the correct model, data, and controls into a working system on AWS. Read the six task statements in order.

Source: repo guide, Part 2 Domain 1.

Task 1.1 Design the solution first, then prove it

Design from the constraints. Build a proof of concept. Use on-demand inference first. Use the Well-Architected Framework and the Generative AI Lens for consistent designs.

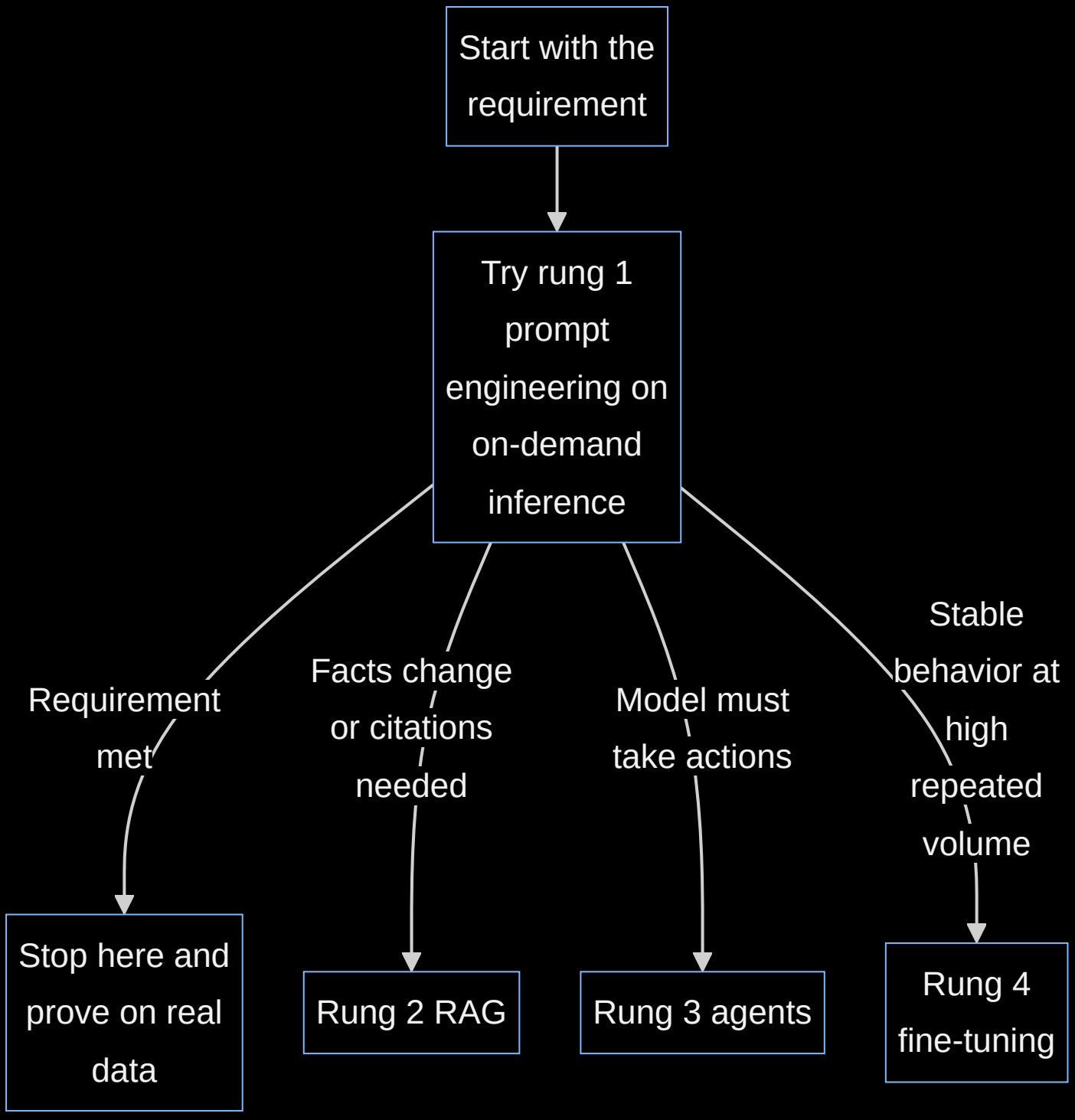


Figure: Climb from the cheapest rung. Stop at the first rung that meets the need.

Approach ladder: four rungs

Rungs are prompt engineering, RAG, agents, and fine-tuning. Each rung must earn its cost.

- Methods or APIs: none named.
- Pro: Lower rungs cost less and change less.
- Con: Each higher rung adds cost and change.

- Choose it when: Format or tone needs prompts. Current or cited facts need RAG. Actions on tools need agents. Stable high-volume behavior needs fine-tuning.
- Do not choose it when: A lower rung already meets the requirement.
- Native pairing: None named for this choice. RDS: Not in source.

Decision rule: Stop at the first rung that meets the requirement. Knowledge goes to retrieval. Behavior goes to weights.

Trap: Fine-tuning on documents that change is wrong. Use RAG.

Exam tip: Words about current documents or citations point to RAG.

Source: repo guide, Part 2 Domain 1, task 1.1.

On-demand inference for the proof of concept

Use on-demand inference first. Commit capacity after the approach works on real data.

- Methods or APIs: none named.
- Pro: On-demand pays per token with no commitment.
- Con: A known steady baseline may suit reserved capacity (task 2.2).
- Choose it when: Traffic is unknown or the design is still a proof of concept.
- Do not choose it when: A steady baseline is already proven.

Decision rule: Prove the approach on real data, then commit capacity.

Trap: Buying Provisioned Throughput or hardware before a proof of concept is a distractor pattern.

Exam tip: A plan to buy capacity before any proof of concept is the wrong order.

Source: repo guide, Part 2 Domain 1, task 1.1.

Task 1.2 Choose a model on five axes

Order of decision: latency needs, then reasoning difficulty, then cost. Test the final choice on your own data.

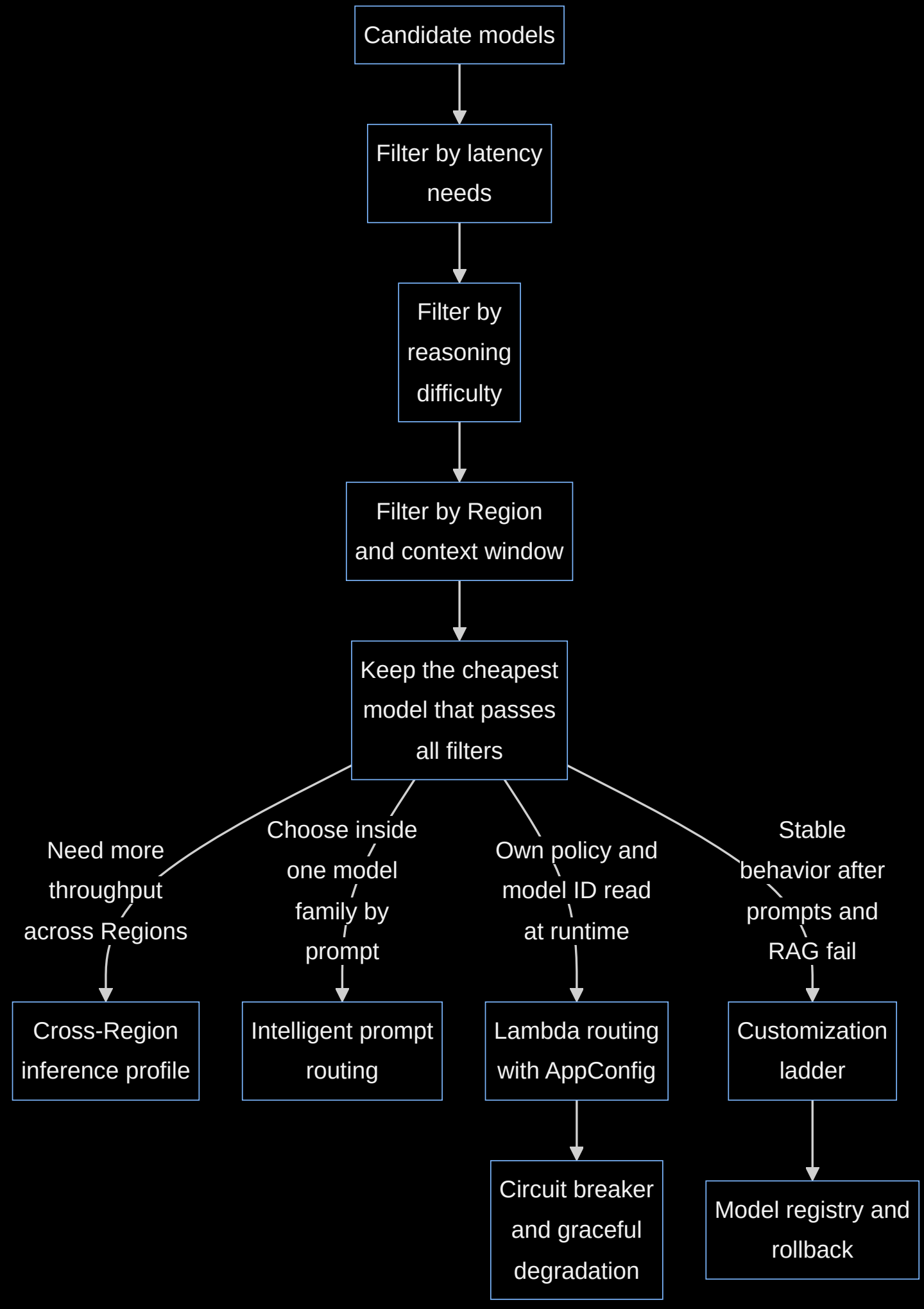


Figure: Filter on hard constraints, then route by mechanism.

Five model axes as filters

Check capability, cost per token, latency, context window, and Regional availability.

- Methods or APIs: none named.
- Pro: Hard limits leave one survivor.
- Con: The flagship model is not the default. It can break a latency budget.
- Choose it when: A hard limit applies on latency, Region, context size, or capability.
- Do not choose it when: Cost alone decides.

Decision rule: Check latency needs first, then reasoning difficulty, then cost.

Trap: A model that is cheap but unavailable in your Region is not a choice.

Exam tip: A cheap model that the Region does not offer fails the Regional availability axis.

Source: repo guide, Part 2 Domain 1, task 1.2.

Routing: cross-Region inference profile and intelligent prompt routing

A cross-Region profile routes for throughput and availability, not by prompt complexity.

- Methods or APIs: Inference profile ID with a us or eu prefix. Intelligent prompt routing.
- Pro: More throughput across Regions within one model family.
- Con: Not automatic failover. Intelligent prompt routing is optimized for English prompts only.
- Choose it when: Throughput or Regional availability is the need. Intelligent prompt routing picks between two models in one family.
- Do not choose it when: Residency applies. A global profile can leave the boundary. Use a geographic profile.

- Native pairing: Retries and fallback belong in the application. Source names no service for them.

Decision rule: A residency rule points to a geographic profile, which keeps routing inside its listed Regions.

Trap: A cross-Region profile does not choose a model by prompt complexity.

Exam tip: Routing by prompt difficulty points to intelligent prompt routing or a rule you write.

Source: repo guide, Part 2 Domain 1, task 1.2.

Lambda routing with AppConfig and survival patterns

Read the model ID from configuration. Switch models without a redeploy.

- Methods or APIs: AWS AppConfig with Lambda and API Gateway. Circuit breaker. Graceful degradation.
- Pro: A circuit breaker stops calls to a failing model.
- Con: You write and own the routing rules.
- Choose it when: You need your own policy and a model ID read at runtime.
- Do not choose it when: A managed routing mechanism already meets the rule.
- Native pairing: AppConfig with Lambda and API Gateway.

Decision rule: A need to switch models with no redeploy points to configuration-driven routing only.

Trap: A redeploy-free switch does not replace retries and fallback.

Exam tip: Changing configuration without code changes maps to AppConfig.

Source: repo guide, Part 2 Domain 1, task 1.2.

Converse, ConverseStream, and InvokeModel

Converse uses one request format for chat and tool use. InvokeModel sends the body that one provider expects.

- Methods or APIs: Converse. ConverseStream. InvokeModel. Claude bodies need anthropic_version and max_tokens.
- Pro: Converse covers text, images, documents, and video on supported models.
- Con: Converse is not an embeddings API.
- Choose it when: Chat and tools across models use Converse. Embeddings use InvokeModel.
- Do not choose it when: Embeddings go through Converse. Streaming does not shorten total generation time.
- Native pairing: Streaming runs through WebSocket APIs or server-sent events (task 2.4). CloudTrail records Bedrock API calls, including InvokeModel and Converse.

Decision rule: Embeddings always use InvokeModel.

Trap: A Titan body sent to Claude fails.

Exam tip: One request shape for chat and tools points to Converse.

Source: repo guide, Part 2 Domain 1, task 1.2.

Structured output schema

A schema is a contract. A request for JSON is a hope.

- Methods or APIs: The exact schema field name is Not in source. Check the API reference.
- Pro: The response matches the schema before code reads it.
- Con: Support depends on the model and the API.
- Choose it when: Code must read the output.
- Do not choose it when: The output is free text for a person.
- Native pairing: Validate the output at the boundary. Source names Lambda checks for validation (task 1.3).

Decision rule: Add the schema to the request. Validate at the boundary anyway.

Trap: A prompt that asks for JSON does not guarantee valid JSON.

Exam tip: A need for machine-readable output points to a schema.

Source: repo guide, Part 2 Domain 1, task 1.2.

Customization ladder and distillation

Fine-tuning changes model parameters. Distillation trains a smaller student from a larger teacher.

- Methods or APIs: Supervised fine-tuning. LoRA on SageMaker AI. Distillation.
- Pro: LoRA is parameter-efficient. Distillation builds a cheaper student for a narrow task.
- Con: Fine-tuning does not fix facts that change.
- Choose it when: Prompts and RAG fail for stable behavior. A narrow, high-volume task suits distillation.
- Do not choose it when: Facts change. Use RAG.
- Native pairing: Fine-tune on SageMaker AI.

Decision rule: Climb the customization ladder in order. Fine-tune only when prompts and RAG fail.

Trap: Cascade routing without a signal cannot decide.

Exam tip: Stable high-volume behavior points to fine-tuning or distillation. Changing facts point to RAG.

Source: repo guide, Part 2 Domain 1, task 1.2.

Model lifecycle and registry

Every release should be a version that you can roll back.

- Methods or APIs: SageMaker Model Registry. CloudTrail records InvokeModel and Converse calls.
- Pro: The registry groups versions and tracks approval status.
- Con: A rollback needs a previous approved version.

- Choose it when: A model changes over time and each change needs approval.
- Do not choose it when: The model changes in place with no version.
- Native pairing: CloudTrail gives the audit trail. Deployment choices appear in task 2.2.

Decision rule: Register each version. Approve it before deployment.

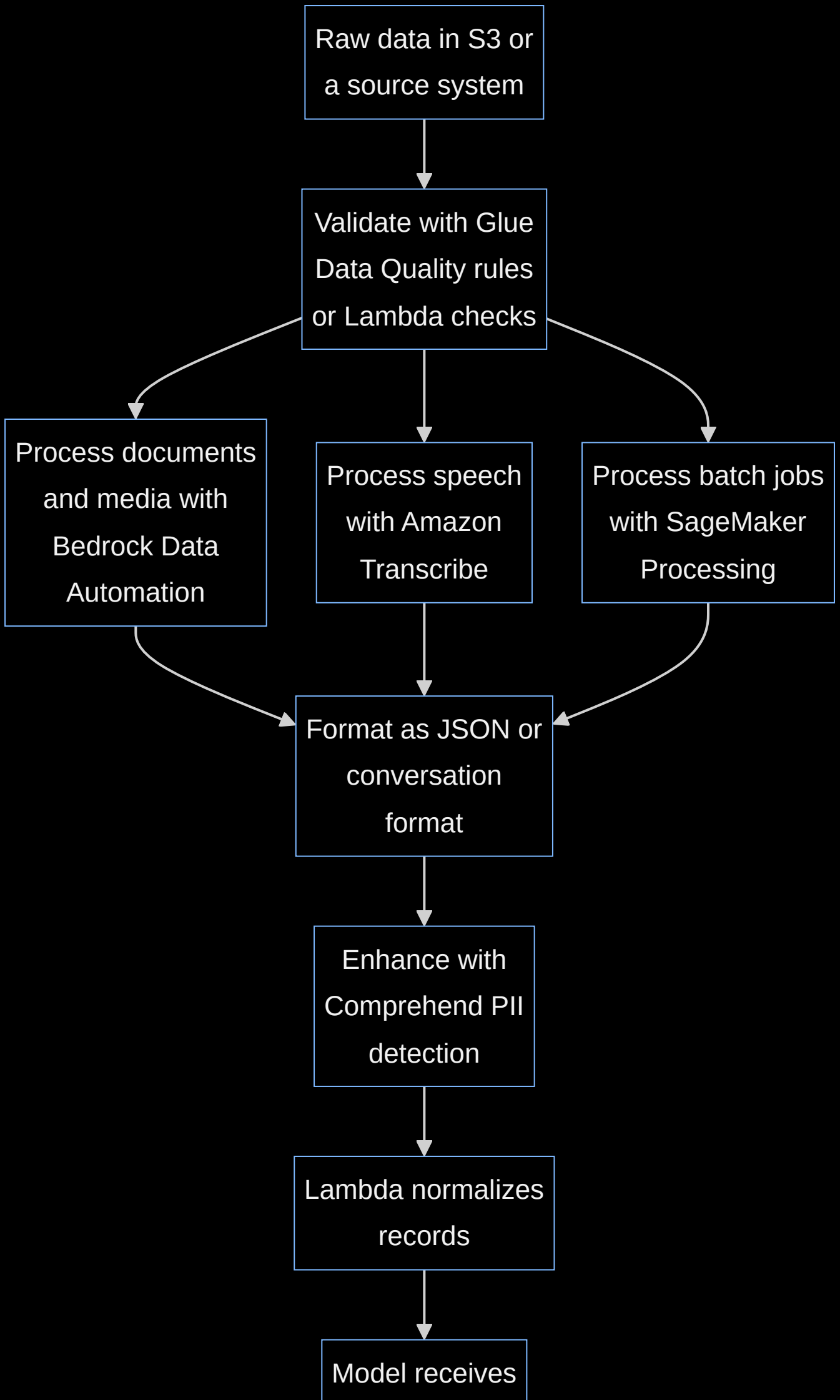
Trap: A model or prompt that changes in place has no rollback point.

Exam tip: An audit trail of model calls points to CloudTrail.

Source: repo guide, Part 2 Domain 1, task 1.2.

Task 1.3 Validate and process data before the model

Clean the data in stages. Each stage uses one tool for one job.


Raw data in S3 or a source system
Validate with Glue Data Quality rules or Lambda checks
Process documents and media with Bedrock Data Automation
Process speech with Amazon Transcribe
Process batch jobs with SageMaker Processing
Format as JSON or conversation format
Enhance with Comprehend PII detection
Lambda normalizes records
Model receives

clean records

Figure: Four stages run before the model.

Validate with AWS Glue Data Quality

Glue Data Quality checks data against rules written in DQDL.

- Methods or APIs: DQDL rules. Lambda checks. CloudWatch metrics.
- Pro: Rule-based checks run before any model call.
- Con: Guardrails do not validate data quality.
- Choose it when: Rule-based data quality is the need.
- Do not choose it when: The need is a safety policy on text. Use Bedrock Guardrails.
- Native pairing: Lambda checks run in the same stage. Documents for ingest sit in S3.

Decision rule: Run the data quality check before processing and the model.

Trap: Guardrails block, mask, or filter content. They do not validate data quality.

Exam tip: Rule-based data quality points to Glue Data Quality with DQDL.

Source: repo guide, Part 2 Domain 1, task 1.3.

Process and enhance: Data Automation, Transcribe, SageMaker Processing, and Comprehend

Bedrock Data Automation extracts fields from documents, images, audio, and video. Comprehend detects PII.

- Methods or APIs: Bedrock Data Automation. SageMaker Processing for batch jobs. Amazon Transcribe for speech. Comprehend PII detection.
- Pro: Transcribe adds speaker labels to speech output. Comprehend finds PII in text.
- Con: Each tool has one job. Translate changes language and does not detect PII.

- Choose it when: Fields come from documents or media: Data Automation. Speech with speaker labels: Transcribe. Offline batch work: SageMaker Processing. PII in text: Comprehend.
- Do not choose it when: The need is a safety policy (Guardrails) or only a language change (Translate).
- Native pairing: S3 holds the source files for ingest. Lambda normalizes records after processing.

Decision rule: Pick the process tool by input type. Detect PII before records reach the model.

Trap: A guardrail does not extract invoice fields. Use Data Automation.

Exam tip: A need to find personal data in text points to Comprehend.

Source: repo guide, Part 2 Domain 1, task 1.3.

Chunking by document type

A chunk must hold one complete answer. The document shape decides the chunk shape.

- Methods or APIs: Fixed size. Hierarchical. Semantic. Custom with Lambda.
- Pro: Hierarchical keeps child chunks for matching and parent chunks for context.
- Con: Semantic chunking costs more to process.
- Choose it when: FAQs use fixed size. Long documents with headings use hierarchical. Tables or figures use custom with Lambda.
- Do not choose it when: A table would split at a chunk boundary.
- Native pairing: Custom chunking runs in Lambda. Documents come from S3.

Decision rule: Keep each table or figure whole.

Trap: A table split across chunks loses its values. Each half can look correct alone.

Exam tip: Tables, figures, or headings in the source point to the chunk shape.

Source: repo guide, Part 2 Domain 1, task 1.3.

Look-alike services: Kendra, ElastiCache, Guardrails, and Translate

Each look-alike stores or returns a different thing. Read what the service stores first.

- Methods or APIs: none named.
- Pro: Bedrock Knowledge Bases is the documented alternative to Kendra.
- Con: Amazon Kendra is no longer open to new customers. ElastiCache is not a durable store of record.
- Choose it when: Guardrails checks text against safety policy. ElastiCache serves fast in-memory reads, once durability is checked.
- Do not choose it when: ElastiCache is the only store of record.
- Native pairing: ElastiCache is durable only with durability enabled on node-based Valkey clusters.

Decision rule: Pick the service whose stored data and output match the need.

Trap: Speed does not make a service durable.

Exam tip: A familiar service name can still fail the job test.

Source: repo guide, Part 2 Domain 1, task 1.3.

Task 1.4 Pick a vector store and an embedding size

Choose the store by query rate, latency, and whether you already run a database.

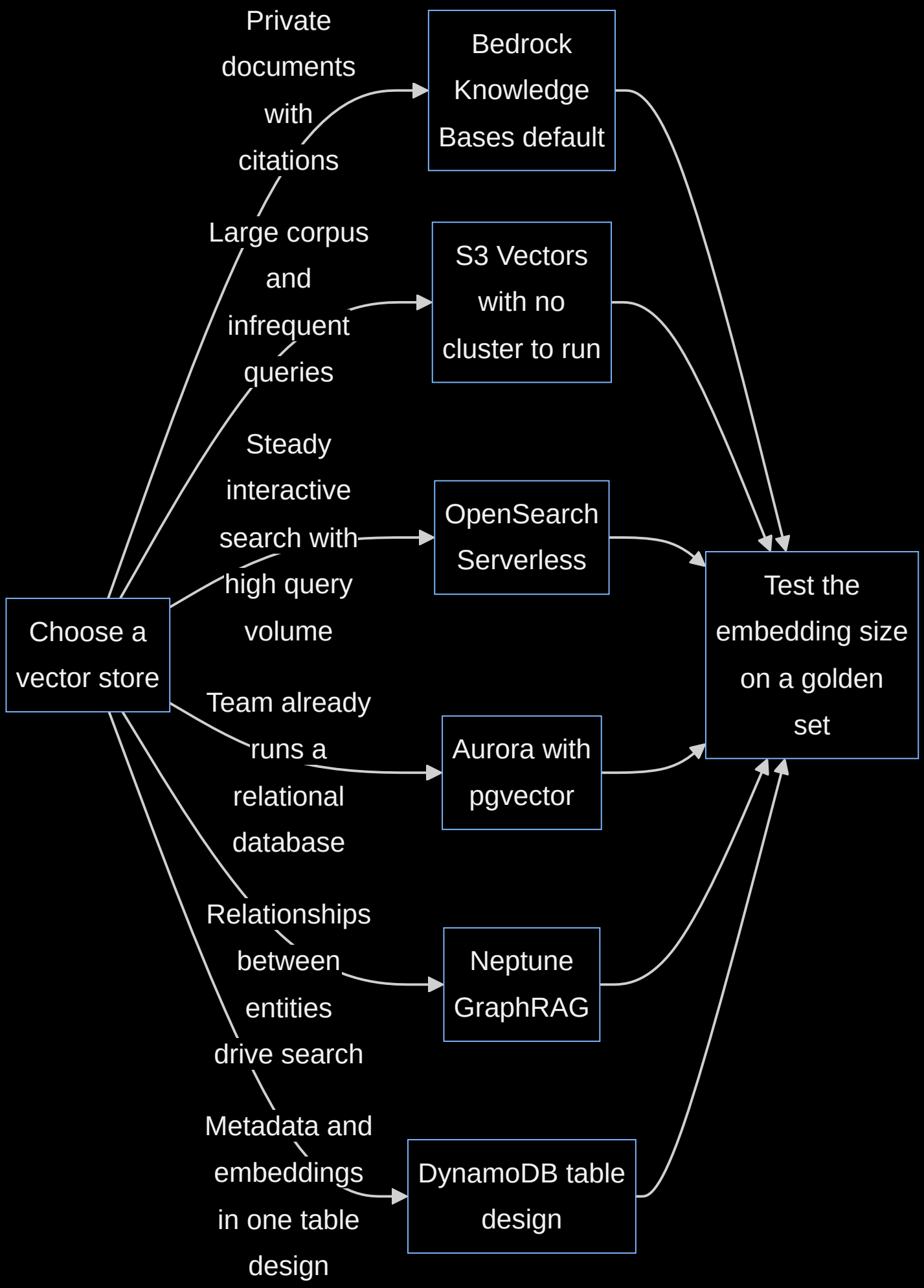


Figure: Pick the store for the query pattern, then test size.

Bedrock Knowledge Bases as the default store

Knowledge Bases is the default for private documents with citations. It is managed RAG.

- Methods or APIs: StartIngestionJob. RetrieveAndGenerate.
- Pro: Managed RAG with citations.
- Con: Less control per stage than hand-built RAG.
- Choose it when: Private documents need citations and the least custom effort.
- Do not choose it when: A stated requirement needs a custom retrieval algorithm.
- Native pairing: S3 holds the documents that a data source points to.

Decision rule: Choose Knowledge Bases for private documents that need citations.

Trap: A hand-built pipeline is wrong when the need is least effort.

Exam tip: A stated need for citations from private documents points here.

Source: repo guide, Part 2 Domain 1, task 1.4.

Always-on stores versus S3 Vectors

OpenSearch Serverless scales with usage. Aurora with pgvector runs in a relational database. S3 Vectors suits rare searches.

- Methods or APIs: OpenSearch Serverless. pgvector on Aurora. S3 Vectors.
- Pro: OpenSearch combines filters and vectors. S3 Vectors has no cluster to run.
- Con: An always-on store costs money even when few queries arrive.
- Choose it when: Steady, interactive search with high query volume: an always-on store. A large corpus with rare queries: S3 Vectors. The team already runs Aurora: pgvector.
- Do not choose the cheapest store when the latency need is strict.

- Native pairing: Aurora is named in source. RDS: Not in source. Storage detail of S3 Vectors beyond its name: Not in source.

Decision rule: Match the store to the query rate and the latency need.

Trap: The cheapest store is wrong when the latency need is strict.

Exam tip: A need to avoid running a cluster points to S3 Vectors.

Source: repo guide, Part 2 Domain 1, task 1.4.

DynamoDB table design and Neptune GraphRAG

DynamoDB holds metadata and embeddings in one table design. Neptune traverses relationships between entities.

- Methods or APIs: DynamoDB key lookup. Neptune GraphRAG.
- Pro: Neptune fits relationship traversal.
- Con: Neptune is for graph traversal, not plain document search.
- Choose it when: Relationships between entities drive the answer, or metadata and embeddings share one table.
- Do not choose it when: The need is plain document search.
- Native pairing: DynamoDB is named in source for metadata (task 1.4) and memory (task 2.1).

Decision rule: Pick Neptune only when relationships between entities drive the search.

Trap: Neptune is a distractor for plain document search.

Exam tip: Words about graph traversal point to Neptune.

Source: repo guide, Part 2 Domain 1, task 1.4.

Embedding models, input type, and dimensions

Titan Text Embeddings v2 outputs 1024 (default), 512, or 256 dimensions.

- Methods or APIs: InvokeModel for embeddings. Cohere Embed input_type: search_query for query text, search_document for passages.

- Pro: Smaller dimensions cost less.
- Con: Smaller dimensions may lower quality.
- Choose it when: Query and passage use the matching input type. Choose the smallest size that passes the golden set.
- Do not choose it when: The size is guessed.
- Native pairing: Vectors feed the stores above.

Decision rule: Pick the smallest dimension that passes the golden set.

Trap: Do not guess the size. Measure it.

Exam tip: The input type for a query and for a passage is part of the retrieval setup.

Source: repo guide, Part 2 Domain 1, tasks 1.4 and 1.5.

Task 1.5 Build RAG end to end and diagnose by stage

Diagnose by stage. Check the chunks first, then the ranking, then the generator.

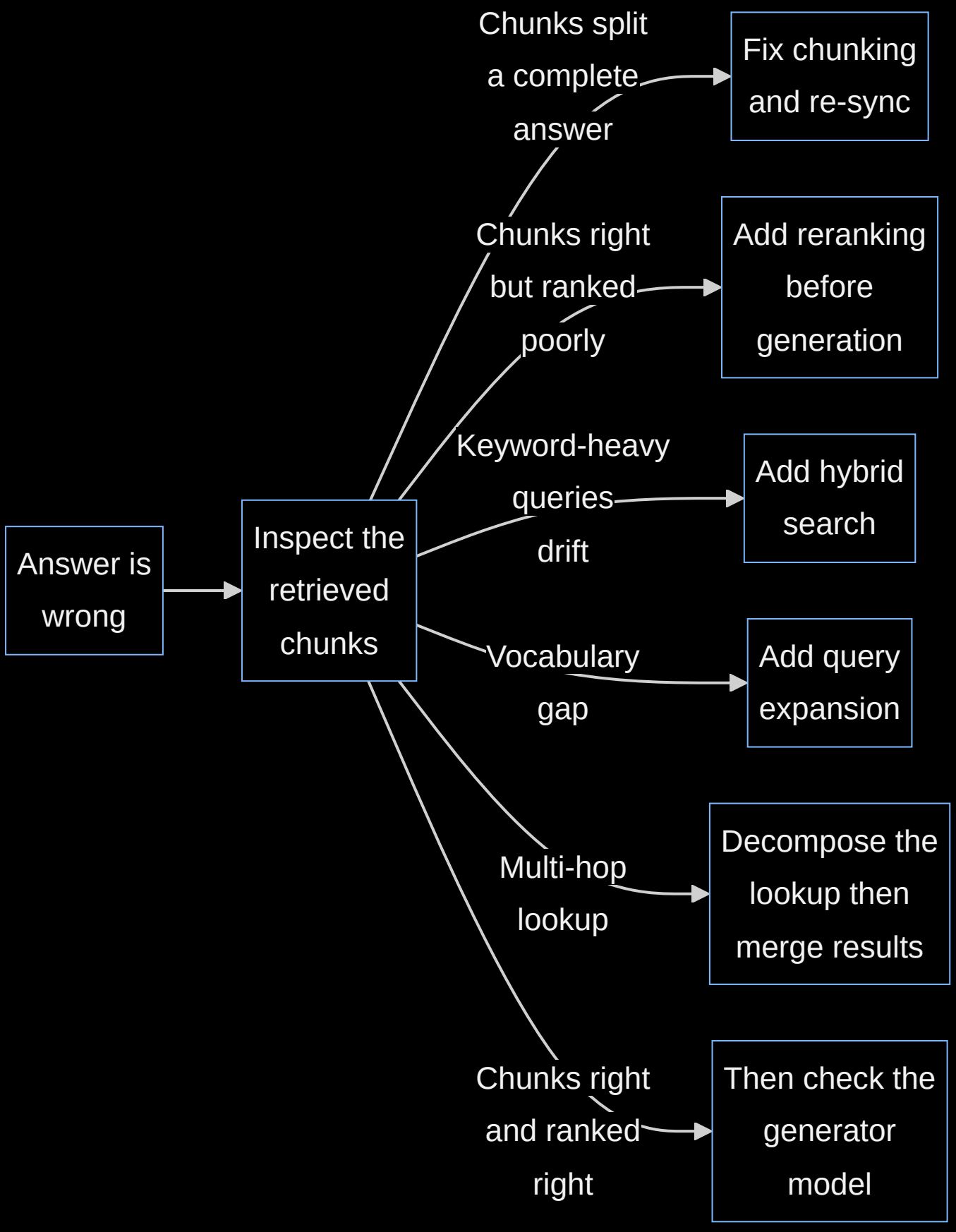


Figure: Find the failed stage first.

RAG ingest path and query path

The ingest path runs ahead of time. The query path runs for each request.

- Methods or APIs: none named.

- Pro: Each stage has one place to check.
- Con: A failure in one stage looks like a failure in the answer.
- Choose it when: Answers must come from private documents on each request.
- Do not choose it when: The model must act on systems. Use agents (task 2.1).
- Native pairing: Documents in S3 feed the ingest path. Custom chunking can run in Lambda.

Decision rule: Diagnose the failed stage before you change any model.

Trap: Changing the generator does not fix broken retrieval.

Exam tip: Right-looking chunks with wrong answers point to chunking or ranking.

Source: repo guide, Part 2 Domain 1, task 1.5.

Hybrid search, reranking, and retrieval fixes

Hybrid search combines keyword and vector signals. A reranker orders candidates for relevance.

- Methods or APIs: none named.
- Pro: Keywords find exact codes. Vectors find related meaning. Expansion handles a vocabulary gap. Decomposition handles a multi-hop lookup.
- Con: A reranker does not add missing chunks. Each chunk adds tokens and cost.
- Choose it when: Exact codes or citation strings need hybrid search. Related but not right results need a reranker. Several linked facts need decomposition, then merged results.
- Do not choose it when: The chunks are wrong. Fix chunking first.
- Native pairing: Source names no reranker or decomposition service. Not in source.

Decision rule: Rerank before generation. Fewer chunks reach the model.

Trap: Similarity is not relevance. Bigger models do not fix broken retrieval or chunking.

Exam tip: Words about exact codes point to keyword signals. Related meaning points to vectors.

Source: repo guide, Part 2 Domain 1, task 1.5, retrieval fixes table.

Knowledge Bases or hand-built RAG

Knowledge Bases runs the RAG pipeline. You sync sources, then call one API.

- Methods or APIs: StartIngestionJob. KnowledgeBaseDocuments API for single files on S3 and custom sources.
- Pro: Least custom effort.
- Con: Less control per stage. Web crawler connectors cannot be created on customer-managed knowledge bases after September 30, 2026.
- Choose it when: Private documents need citations and the least custom effort.
- Do not choose it when: A stated requirement needs a custom retrieval algorithm.
- Native pairing: For web sources, use Amazon Bedrock Managed Knowledge Base. The September 30, 2026 cutoff has passed as of October 21, 2026. Confirm on the Knowledge Base documentation.

Decision rule: Choose Knowledge Bases for private documents with citations and minimal custom effort.

Trap: A hand-built pipeline is wrong when the need is least custom effort.

Exam tip: A custom retrieval algorithm is the stated signal for hand-built RAG.

Source: repo guide, Part 2 Domain 1, tasks 1.4 and 1.5.

Knowledge Base data source: sync or ingest

A data source tells the knowledge base where its documents live. A sync updates the index.

- Methods or APIs: StartIngestionJob. KnowledgeBaseDocuments API.
- Pro: A scheduler can call StartIngestionJob. An event handler can ingest single files.
- Con: Chunking is set on the data source.
- Choose it when: Updates arrive on a schedule: sync. Updates arrive as events: your handler or the direct API.
- Do not choose it when: A chunking change goes into the existing data source.
- Native pairing: The S3 bucket is the document source for the data source.

Decision rule: To change chunking, create a new data source. Then sync it.

Trap: Changing chunking on an existing data source is the wrong move.

Exam tip: A stated chunking change points to a new data source plus a sync.

Source: repo guide, Part 2 Domain 1, task 1.4.

Citations and grounding

RetrieveAndGenerate returns an answer with source citations. Contextual grounding scores each answer.

- Methods or APIs: RetrieveAndGenerate. Contextual grounding check.
- Pro: Citations give attribution for audits. Auditors can check the scores.
- Con: Citations are attribution, not proof of truth.
- Choose it when: An audit needs the source behind each answer.
- Do not choose it when no audit or source attribution is needed. Do not build a citation layer by hand when the need is least effort.
- Native pairing: Knowledge Bases returns citations with answers. Check the API reference for fields.

Decision rule: Use citations for attribution. Use grounding scores for review.

Trap: Citations are attribution, not proof of truth.

Exam tip: An audit need for source support points to citations and grounding scores.

Source: repo guide, Part 2 Domain 1, task 1.5.

Task 1.6 Govern prompts and route long workflows

A prompt changes behavior like code does. Store it, version it, test it, and roll it back.

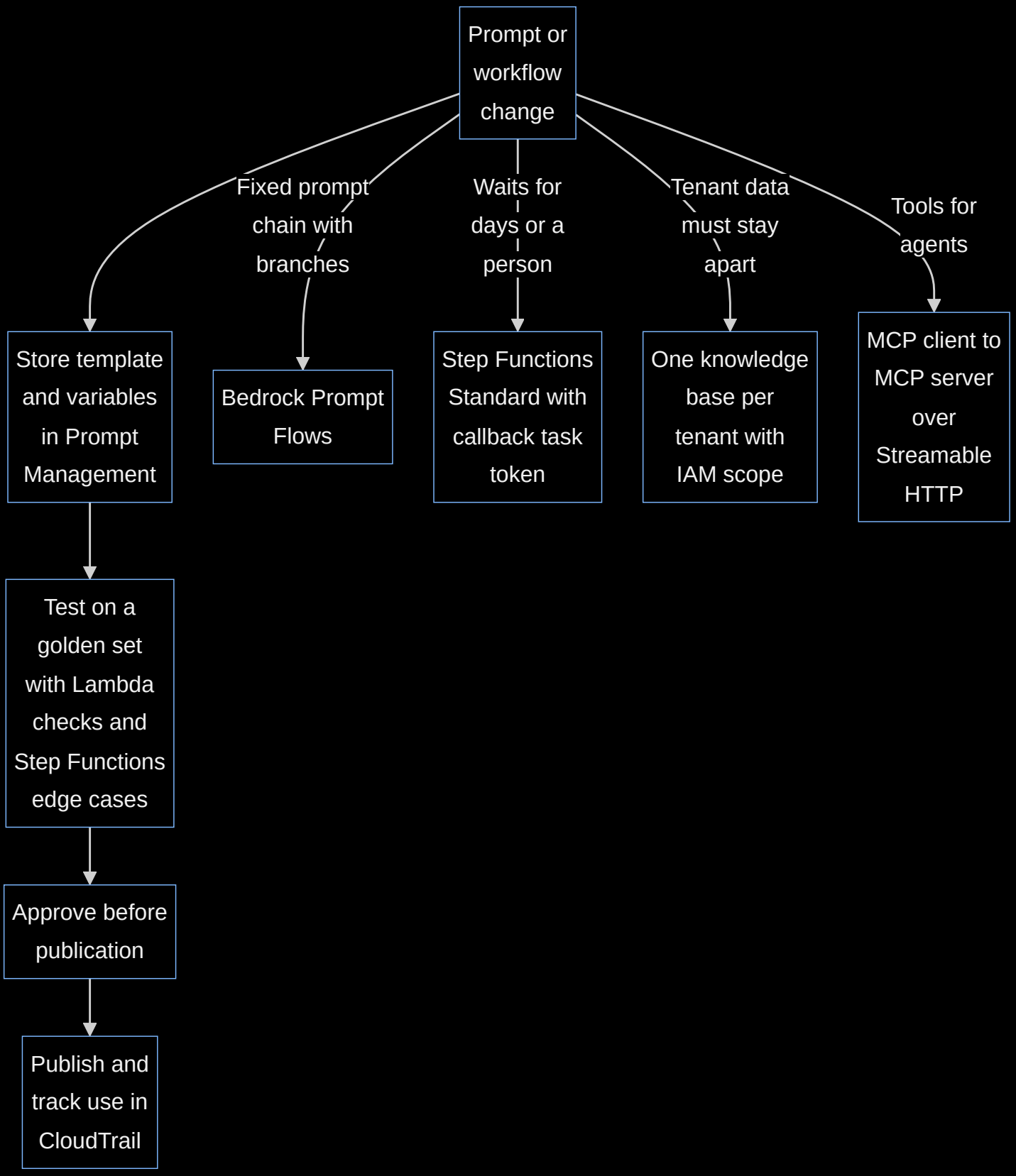


Figure: Version and approve prompts. Route by predictability and waits.

Prompt Management and version control

Store role definitions and variables in Amazon Bedrock Prompt Management.

- Methods or APIs: Bedrock Prompt Management. Bedrock Guardrails for responsible AI rules. CloudTrail tracks use.
- Pro: Versioned templates give a rollback point. Approval gates publication.

- Con: An untracked prompt has no rollback point.
- Choose it when: Prompts change over time and need approval before release.
- Do not choose it when: A prompt changes in place with no version.
- Native pairing: Lambda checks expected output in the test step. Step Functions runs edge cases.

Decision rule: Approve each prompt version before publication. Keep the older version for rollback.

Trap: A prompt that changes in place has no safe rollback point.

Exam tip: Words about approvals and rollback point to versioned prompts.

Source: repo guide, Part 2 Domain 1, task 1.6.

Prompt Flows versus Step Functions Standard

Prompt Flows is a fixed chain of nodes with branches, built in the console. Step Functions Standard is a durable workflow with waits.

- Methods or APIs: Bedrock Prompt Flows. Callback task tokens. Step Functions Standard.
- Pro: Prompt Flows gives a console chain for predictable steps. Standard runs for up to one year and holds durable waits.
- Con: A flow is not a durable workflow and does not hold waits.
- Choose it when: Steps are predictable: Prompt Flows. A workflow waits about two days for a person: Step Functions Standard.
- Do not choose it when: Waits, retries, or approvals appear in a flow. Express does not support the callback pattern (task 2.2).
- Native pairing: Put long flows in Step Functions. Keep each Lambda step short.

Decision rule: Predictable steps point to a flow. Waits and approvals point to Step Functions Standard.

Trap: A Lambda that waits for a person hits the 15-minute limit.

Exam tip: Multi-day approval points to Step Functions Standard.

Source: repo guide, Part 2 Domain 1, task 1.6.

Tenant and Region isolation

Isolation must be structural. A prompt that says to ignore other tenants is not isolation.

- Methods or APIs: IAM roles scoped to each knowledge base. Metadata filters for Region or product line.
- Pro: One knowledge base per tenant. The model receives only what retrieval returns.
- Con: Each tenant needs its own knowledge base and IAM role.
- Choose it when: Tenant data must stay apart. A Region or product line filter applies.
- Do not choose it when: The only control is an instruction in the prompt.
- Native pairing: IAM scopes each role to its own knowledge base. Domain 3 covers data scope (task 3.2).

Decision rule: Use separate knowledge bases and IAM roles for tenant separation.

Trap: Prompt instructions are not isolation.

Exam tip: A need for tenant separation points to platform structure, not to wording.

Source: repo guide, Part 2 Domain 1, tasks 1.4 and 1.6.

MCP tools

Model Context Protocol (MCP) sets one standard for agents to find and call tools.

- Methods or APIs: Streamable HTTP for remote MCP servers.
- Pro: One protocol replaces a custom client for each tool.

- Con: MCP is not a model choice or a guardrail.
- Choose it when: Agents need one standard tool connection.
- Do not choose it when: A safety control is the need. Use guardrails and IAM.
- Native pairing: Light, stateless tools can run on Lambda (task 2.1). Remote servers use Streamable HTTP.

Decision rule: Use MCP for tool connection. Use guardrails for safety.

Trap: MCP is the tool connection standard. It is not a safety control.

Exam tip: A need for a common tool protocol points to MCP.

Source: repo guide, Part 2 Domain 1, tasks 1.5 and 1.6 (tools for agents).

Decision signals for Domain 1

- Facts that change, or a need for cited answers, point to retrieval, not fine-tuning.
- Stable behavior at high repeated volume points to fine-tuning or distillation, after prompts and retrieval fail.
- A stated need for least custom build effort points to a managed option such as Knowledge Bases.
- A hard limit on Region, latency, or context size removes candidates before cost is compared.
- A residency rule points to a geographic profile, not a global profile.
- A need to switch models with no redeploy points to configuration-driven routing.
- Rule-based data quality points to Glue Data Quality. Field extraction from media points to Data Automation.
- A need to find personal data in text points to Comprehend, not to a translation service.
- Wrong answers while retrieved chunks look right point to chunking or ranking, not to the generator.

- Steady interactive search points to an always-on store. A large corpus with rare searches points to S3 Vectors.
- A need for machine-readable output points to a schema contract, checked at the boundary.
- A wait for a person, or a multi-day approval, points to Step Functions Standard with a callback task token.

Domain 2 · Implementation and integration · Weight 26 percent

Domain 2 covers agents, deployment, enterprise integration, API patterns, and application tools. The fact-check for this domain is still open.

Source: repo guide, Part 3 Domain 2.

Task 2.1 Build agents with limits and versioned releases

An agent repeats reason, act, and observe until a stop condition.

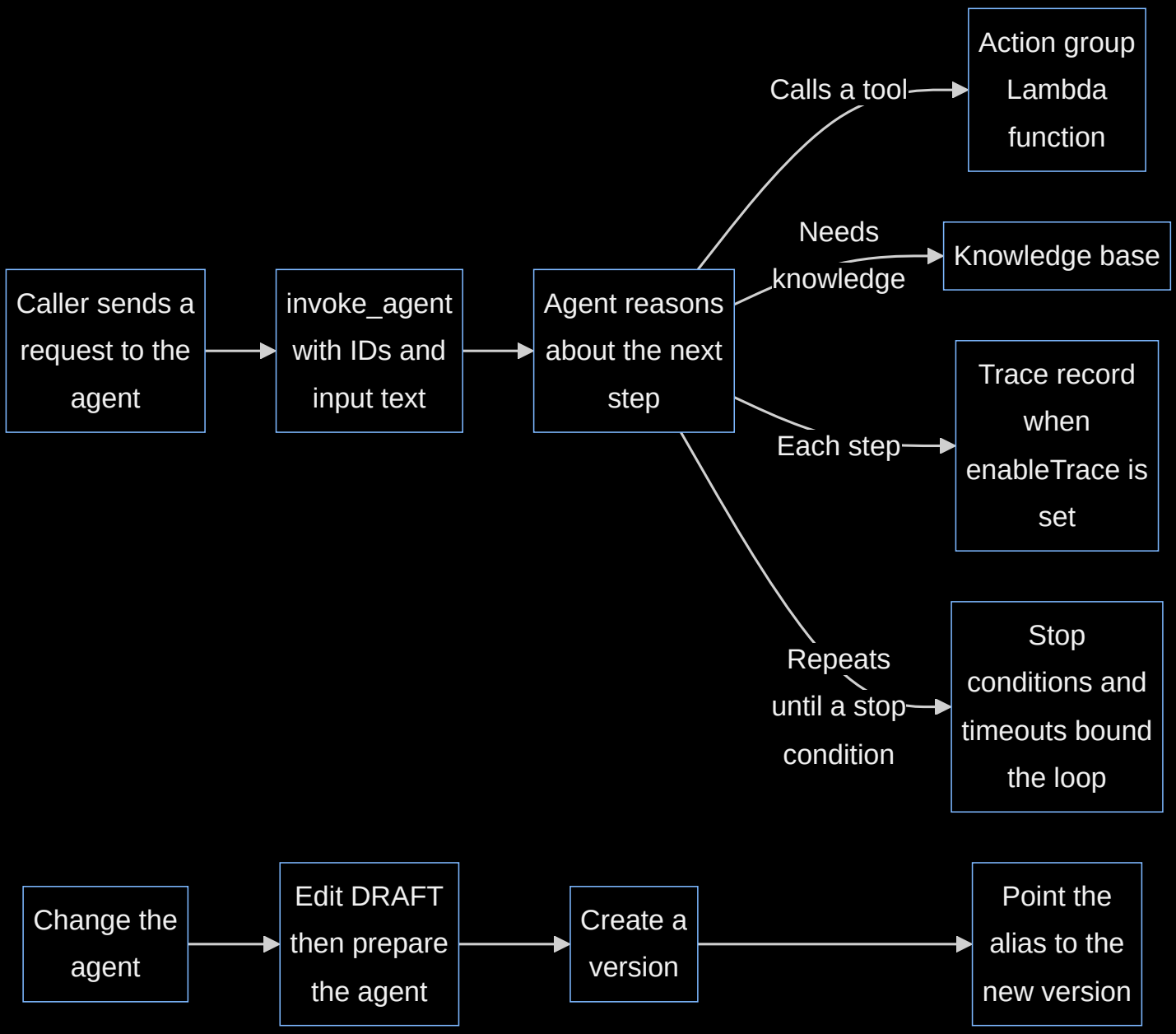


Figure: The loop calls tools. Changes move through DRAFT, a version, and an alias.

Bedrock Agents anatomy: invoke_agent and action groups

The invoke_agent call takes an agent ID, an alias ID, a session ID, and input text. It returns a stream.

- Methods or APIs: invoke_agent with agentId, agentAliasId, sessionId, and inputText. enableTrace sets a trace record for each step.
- Pro: One call starts a conversation. Trace records show each step.
- Con: Amazon Bedrock Agents is no longer open to new customers. The docs name AgentCore as the alternative.
- Choose it when: The model calls an action group or a knowledge base in a loop, and the account can open the service.

- Do not choose it when: The account cannot open the service. Use AgentCore.
- Native pairing: An action group runs on a Lambda function. Lambda limits apply to each step (task 2.2).

Decision rule: Keep one sessionId for each conversation. It is the memory key for that conversation.

Trap: After a change, prepare the agent and move the alias. Otherwise callers run the old version.

Exam tip: Words about tracing each step point to enableTrace.

Source: repo guide, Part 3 Domain 2, task 2.1.

DRAFT, versions, and aliases

Configuration changes go to DRAFT. Callers reach a new version only after an alias points to it.

- Methods or APIs: Prepare the agent. Create a version. Point the alias to it. Callers send agentId and agentAliasId.
- Pro: A version is a frozen snapshot. Moving the alias moves traffic without a caller change.
- Con: Console tests run on DRAFT. They do not prove alias traffic works.
- Choose it when: A change must reach production in a controlled step.
- Do not choose it when: Callers would use DRAFT directly.

Decision rule: Test the alias, not only DRAFT, before you call a change released.

Trap: A console test on DRAFT does not prove the alias works.

Exam tip: A tool that works in the console but not in production points to an older alias target.

Source: repo guide, Part 3 Domain 2, task 2.1.

Session memory and customer memory

An agent session expires. Customer memory must outlive the session.

- Methods or APIs: DynamoDB key lookup. Native TTL removes expired records. The agent session TTL is between 60 and 5400 seconds.
- Pro: Lookup by customer or session ID returns in single-digit milliseconds. Data is encrypted at rest.
- Con: The agent session is not a memory store. It expires.
- Choose it when: Memory must last weeks. Use DynamoDB with a key and a time to live.
- Do not choose it when: The session alone must carry memory across visits.
- Native pairing: DynamoDB is the native store named in source for this memory.

Decision rule: Store customer memory in DynamoDB with a key and a time to live.

Trap: The agent session is not the memory store. It expires.

Exam tip: A need for memory that lasts weeks points to DynamoDB with native TTL.

Source: repo guide, Part 3 Domain 2, task 2.1.

Agent loop limits: stop, time, access, and failure

Limits belong in the workflow or the tool layer, not only in the prompt.

- Methods or APIs: Stop conditions. Lambda and Step Functions timeouts. IAM policies on each tool call. Circuit breakers.
- Pro: Each limit type has one control. A circuit breaker stops repeated failures to one tool.
- Con: A prompt that says stop is not a control.
- Choose it when: The loop must end after a step count, a time limit, or repeated failures.

- Do not choose it when: The only limit is text in the prompt.
- Native pairing: Step Functions and Lambda timeouts bound time. IAM bounds tool access.

Decision rule: Bound steps, time, access, and failures in the workflow or tool layer.

Trap: A prompt that says stop is not a control.

Exam tip: Repeated failure to one tool points to a circuit breaker.

Source: repo guide, Part 3 Domain 2, task 2.1.

Multi-agent collaboration with a supervisor

A supervisor reads the request and routes each part to a collaborator. The supervisor does not call the tools itself.

- Methods or APIs: Bedrock multi-agent collaboration. Supervisor agent. Collaborator agents.
- Pro: Each collaborator owns its tools. The supervisor classifies intent in natural language.
- Con: Delegation is not an action group. A DRAFT collaborator cannot take delegations.
- Choose it when: Separate parts of one request need separate tools.
- Do not choose it when: One agent with one action group covers the need.
- Native pairing: Each collaborator calls its tools through its own action group.

Decision rule: Prepare and alias each collaborator before it takes delegations.

Trap: Delegation is not an action group.

Exam tip: Routing parts of one request to specialists points to a supervisor and collaborators.

Source: repo guide, Part 3 Domain 2, task 2.1.

Open-source frameworks and MCP hosting

Strands Agents and AWS Agent Squad are open-source frameworks named in source.

- Methods or APIs: Strands Agents. AWS Agent Squad. Streamable HTTP for remote MCP servers.
- Pro: Frameworks give code-level control. Managed agents need less code.
- Con: A framework needs more code to build and keep.
- Choose it when: Code-level control is needed. Light, stateless, bursty MCP tools go on Lambda.
- Do not choose it when: The console is enough. Choose a managed agent.
- Native pairing: Lambda hosts light MCP tools. AgentCore Runtime hosts a remote MCP server with Streamable HTTP. Host for heavy, stateful servers: Not in source for this build.

Decision rule: Light, stateless, bursty tools go on Lambda. Remote servers on AgentCore Runtime use Streamable HTTP.

Trap: A remote MCP server on AgentCore Runtime does not use another transport.

Exam tip: A need for code-level control points to a framework. A console-only need points to a managed agent.

Source: repo guide, Part 3 Domain 2, tasks 2.1 and 2.2.

Task 2.2 Match deployment to traffic shape and duration

Model capacity is bought in shapes. Match the purchase to the traffic.

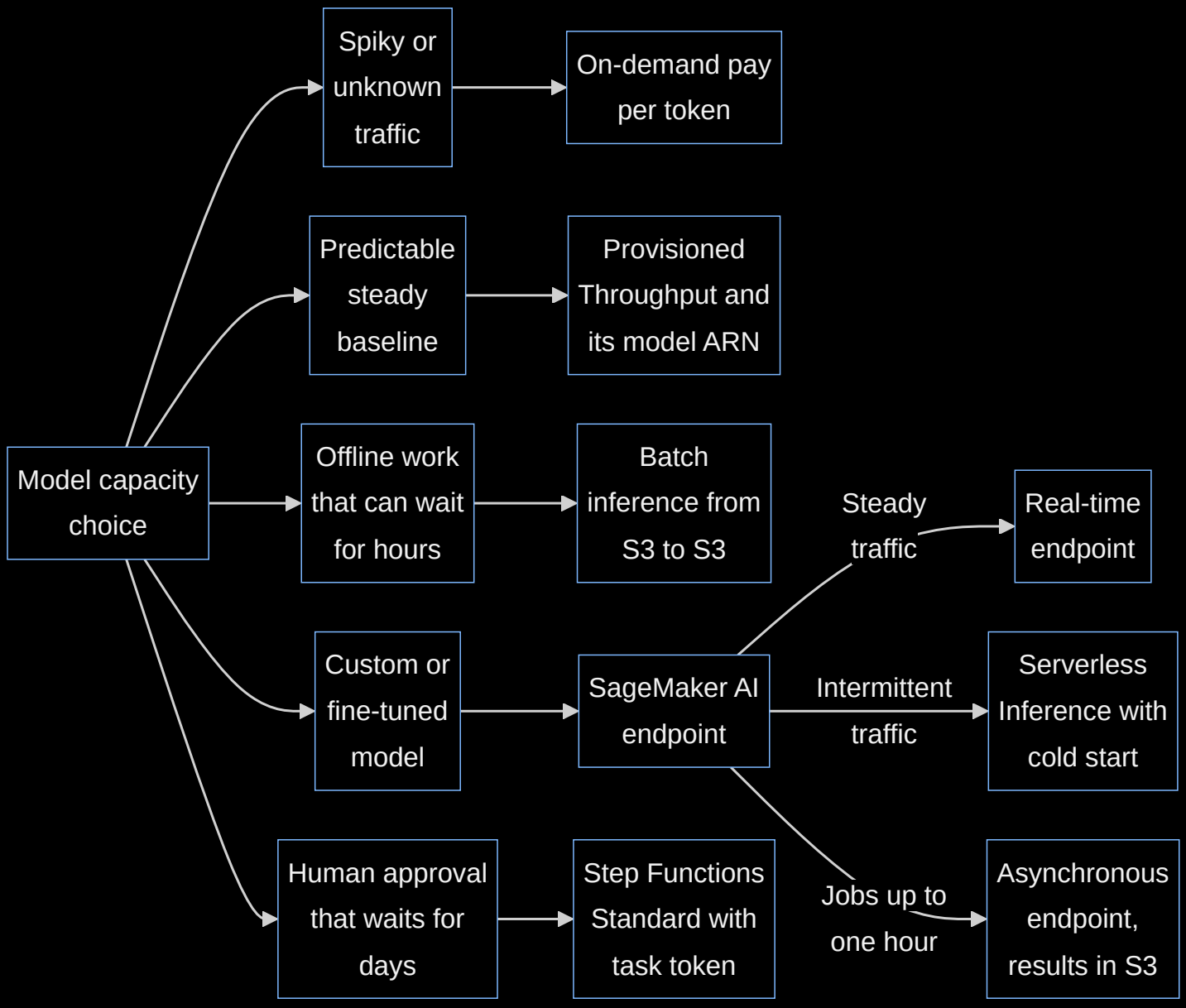


Figure: Choose the purchase or endpoint by traffic, job length, and wait.

On-demand, Provisioned Throughput, and batch inference

On-demand suits spiky traffic. Provisioned Throughput suits a steady baseline. Batch suits offline work.

- Methods or APIs: On-demand. Provisioned Throughput with reserved model units. Batch inference. The provisioned model ARN for calls.
- Pro: On-demand has no commitment and bills per token. Batch costs less than on-demand.
- Con: Provisioned Throughput bills hourly with a commitment term. The exact batch discount is Not in source.
- Choose it when: Spiky or unknown traffic uses on-demand. A steady baseline uses Provisioned Throughput. Offline sets that can wait for hours

use batch.

- Do not choose it when: Traffic is spiky with no steady baseline.
- Native pairing: Batch runs offline from S3 to S3.

Decision rule: Match the purchase to the shape of the load. Mixed traffic uses a reserved baseline and on-demand spikes.

Trap: Calls to the base model ID still use on-demand capacity after a purchase. Call the provisioned model ARN.

Exam tip: Words about a steady baseline point to reserved capacity. Offline sets point to batch.

Source: repo guide, Part 3 Domain 2, task 2.2.

SageMaker AI endpoints: real-time, serverless, and asynchronous

Real-time endpoints fit steady traffic. Asynchronous endpoints fit long jobs up to one hour.

- Methods or APIs: Real-time endpoints. Serverless Inference. Asynchronous inference.
- Pro: Real-time endpoints autoscale and support blue-green and rolling deployments. Serverless scales to zero.
- Con: Serverless has a cold start on the first request. Asynchronous jobs are queued.
- Choose it when: Custom or fine-tuned models. Steady traffic uses real-time. Intermittent traffic uses serverless. Jobs up to one hour use asynchronous.
- Do not choose it when: A long job would run on a real-time endpoint.
- Native pairing: Asynchronous results go to S3.

Decision rule: Pick the endpoint by traffic pattern and job length.

Trap: A long job on a real-time endpoint is the wrong pick. Use asynchronous inference.

Exam tip: Words about long jobs point to asynchronous inference. Rare calls point to serverless.

Source: repo guide, Part 3 Domain 2, task 2.2.

Lambda limits, Step Functions waits, and human approval

A standard Lambda invocation runs at most 15 minutes. A task token pauses Step Functions until a person responds or a timeout ends the wait.

- Methods or APIs: Lambda Managed Instances, invoked asynchronously. Callback task token. Step Functions Standard.
- Pro: Step Functions keeps durable state. Standard keeps execution history for up to 90 days.
- Con: Express workflows do not support the callback pattern. Waits in Lambda bill for the full duration.
- Choose it when: Work is one short step inside a longer flow. A wait lasts hours or days, so use Standard.
- Do not choose it when: A wait sits inside one Lambda invocation.
- Native pairing: API Gateway REST integrations default to 29 seconds (task 2.4). Confirm on the API Gateway quotas page.

Decision rule: Put long flows in Step Functions. Keep each Lambda step short.

Trap: Do not assume the 90-minute case applies to a standard function.

Exam tip: A wait inside a function points to Step Functions. A human approver with a timeout points to a task token.

Source: repo guide, Part 3 Domain 2, task 2.2.

AgentCore components around one agent

AgentCore runs agents built with any framework behind one interface.

- Methods or APIs: Runtime. Memory. Gateway. Identity. Observability. Policy checks each tool call.
- Pro: One service covers hosting, memory, tools, identity, and traces.

- Con: AgentCore is not a model. It hosts and governs the agent around the model.
- Choose it when: One agent needs hosting, memory, MCP tools, identity, and traces.
- Do not choose it when: The only need is a model choice.
- Native pairing: Gateway exposes tools as MCP tools. Identity controls who can call agents and tools.

Decision rule: Pick AgentCore when the need covers hosting and governance around one agent.

Trap: AgentCore is not a model.

Exam tip: Who may call an agent or a tool points to Identity.

Source: repo guide, Part 3 Domain 2, task 2.2.

Task 2.3 Put one controlled gateway in front of models

One gateway enforces guardrail identifiers, IAM policy, uniform logs, and team cost tags.

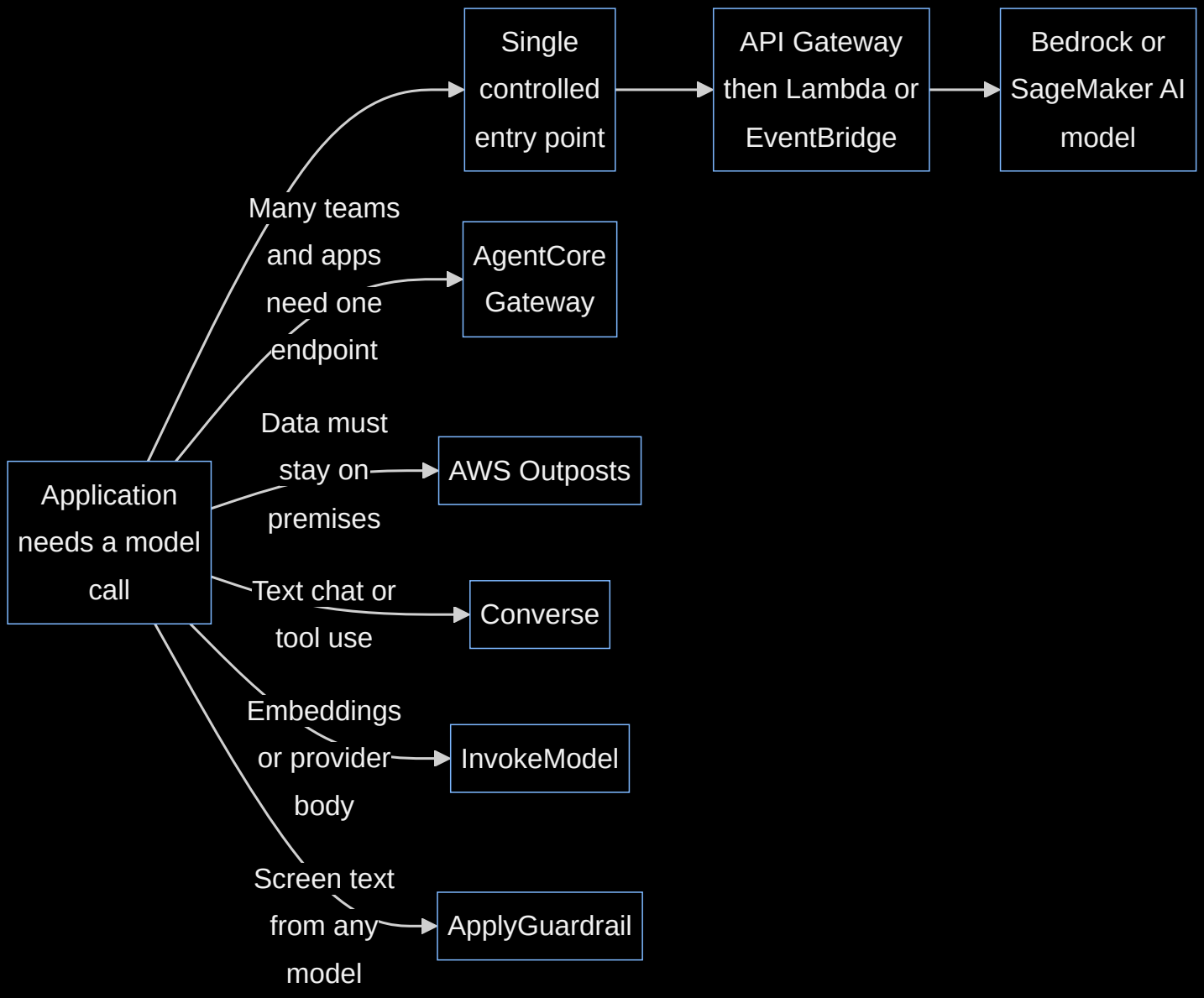


Figure: Route each call through one entry point.

Enterprise gateway with API Gateway, Lambda or EventBridge, and models

Put one controlled entry in front of the models. The gateway gives one place for policy, logs, and cost tags.

- Methods or APIs: API Gateway. Lambda or EventBridge. AgentCore Gateway. Identity Center. Converse for chat and tools, InvokeModel for embeddings (task 1.2).
- Pro: One gateway enforces guardrail identifiers and IAM policy. Logs and team cost tags stay uniform.
- Con: A gateway does not replace the guardrail. Converse is not the embeddings API.
- Choose it when: Many teams or apps share the same models.

- Do not choose it when: Data must stay on premises. Do not rely on a Region for that rule. Use AWS Outposts.
- Native pairing: API Gateway, then Lambda or EventBridge, then Bedrock or SageMaker AI. AgentCore Gateway includes observability and auditing.

Decision rule: Put one gateway in front of shared models. Guardrail identifier and IAM policy apply on each call.

Trap: A gateway does not replace the guardrail. Enforce the guardrail at IAM as well.

Exam tip: Words about on-premises data rules point to Outposts.

Source: repo guide, Part 3 Domain 2, task 2.3.

ApplyGuardrail on any model

ApplyGuardrail applies a Bedrock safety policy to content from any model. It does not call the model.

- Methods or APIs: ApplyGuardrail. The result is allow or block.
- Pro: Works on models outside Bedrock. Input and output screening are separate checks.
- Con: The app still makes the model call.
- Choose it when: A model outside Bedrock needs the same policy check.
- Do not choose it when: Input and output are treated as one check.
- Native pairing: The gateway enforces the guardrail identifier on each call (entry above).

Decision rule: Screen input and output with separate ApplyGuardrail checks.

Trap: A non-Bedrock model does not need Converse to use ApplyGuardrail.

Exam tip: Words about a model outside the platform point to ApplyGuardrail.

Source: repo guide, Part 3 Domain 2, tasks 2.3 and 3.1.

Task 2.4 Stream tokens and retry without overload

Streaming shortens the wait for the first visible token. It does not extend a REST timeout.

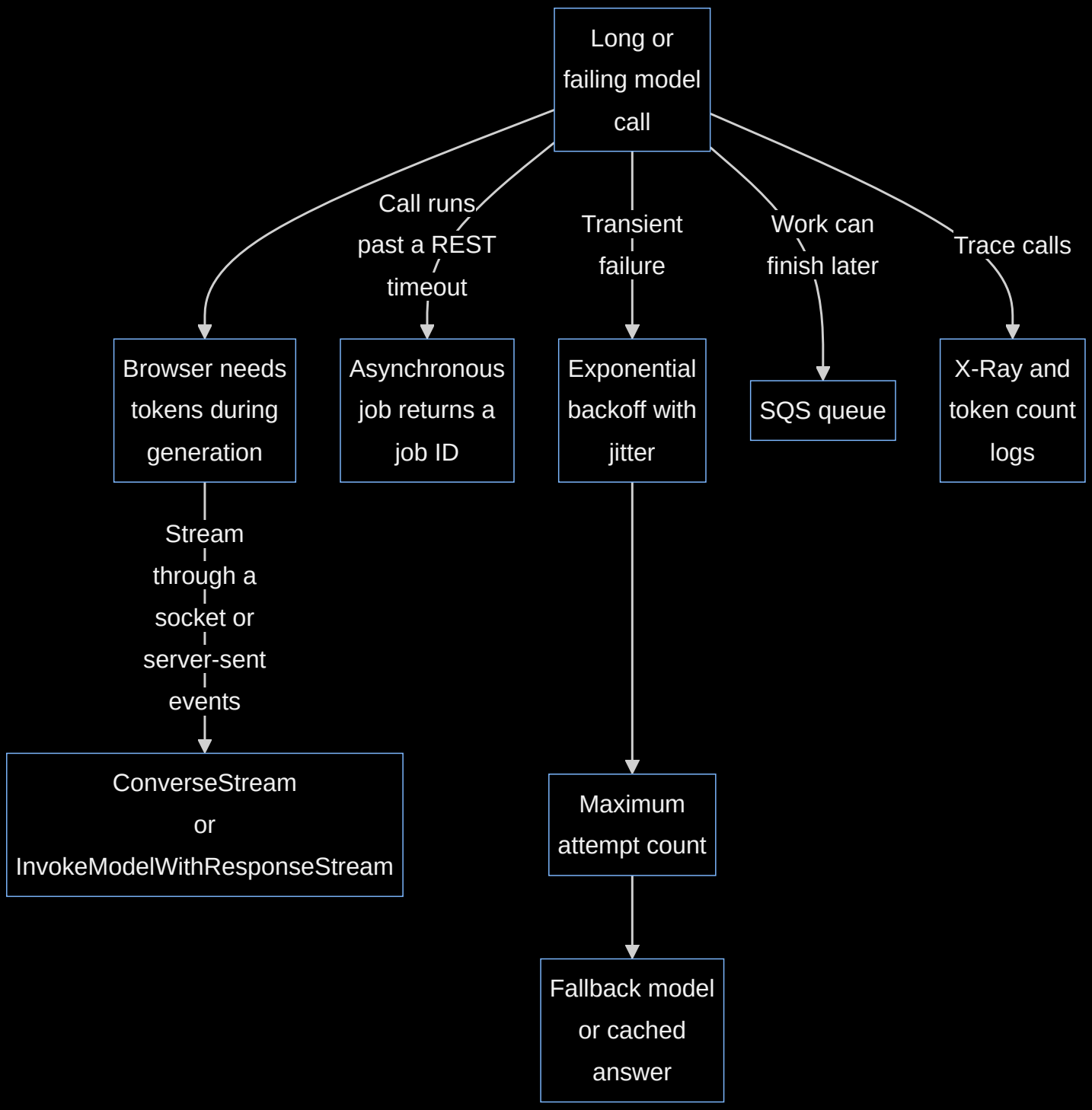


Figure: Hold long streams on a path that can carry them. Retry with jitter, then fall back.

Streaming to the browser

ConverseStream and InvokeModelWithResponseStream send tokens as they arrive.

- Methods or APIs: ConverseStream. InvokeModelWithResponseStream. WebSocket APIs. Server-sent events.

- Pro: The first visible token appears sooner. WebSocket APIs hold a long stream.
- Con: A REST integration still stops at the 29-second default. The source lists that value as an open check.
- Choose it when: A browser must show tokens while the model generates. Use a path that holds the stream.
- Do not choose it when: A REST path must carry a call longer than its limit. Use an asynchronous job.
- Native pairing: An asynchronous job returns a job ID, then the client polls or receives the result.

Decision rule: Pick a path that can hold the stream. A REST path cannot hold it past the limit.

Trap: A model call that runs past the gateway limit fails there. Raising the Lambda timeout does not fix it.

Exam tip: Words about a faster first token point to streaming. A fixed timeout points to a socket or a job.

Source: repo guide, Part 3 Domain 2, tasks 2.3 and 2.4.

Call models reliably: backoff, jitter, limits, and fallback

A failed call can succeed later. Retry with exponential backoff and jitter.

- Methods or APIs: Bedrock APIs for synchronous calls. SQS for asynchronous work. CountTokens before the call. InvocationThrottles in CloudWatch. X-Ray traces.
- Pro: Jitter spreads retries across clients. A maximum attempt count stops the loop.
- Con: A tight retry loop makes an overload worse.
- Choose it when: A call fails transiently. Work can finish later: SQS. Routing by content: Step Functions.
- Do not choose it when: The retry is immediate and repeated with no wait.

- Native pairing: SQS queues asynchronous work. Step Functions handles content-based routing.

Decision rule: Use backoff, jitter, a maximum attempt count, then a fallback model.

Trap: A tight retry loop is never correct.

Exam tip: Many clients retrying at once point to jitter.

Source: repo guide, Part 3 Domain 2, task 2.4.

Task 2.5 Pick the application tool by user and task

Match the tool to the person and the task.

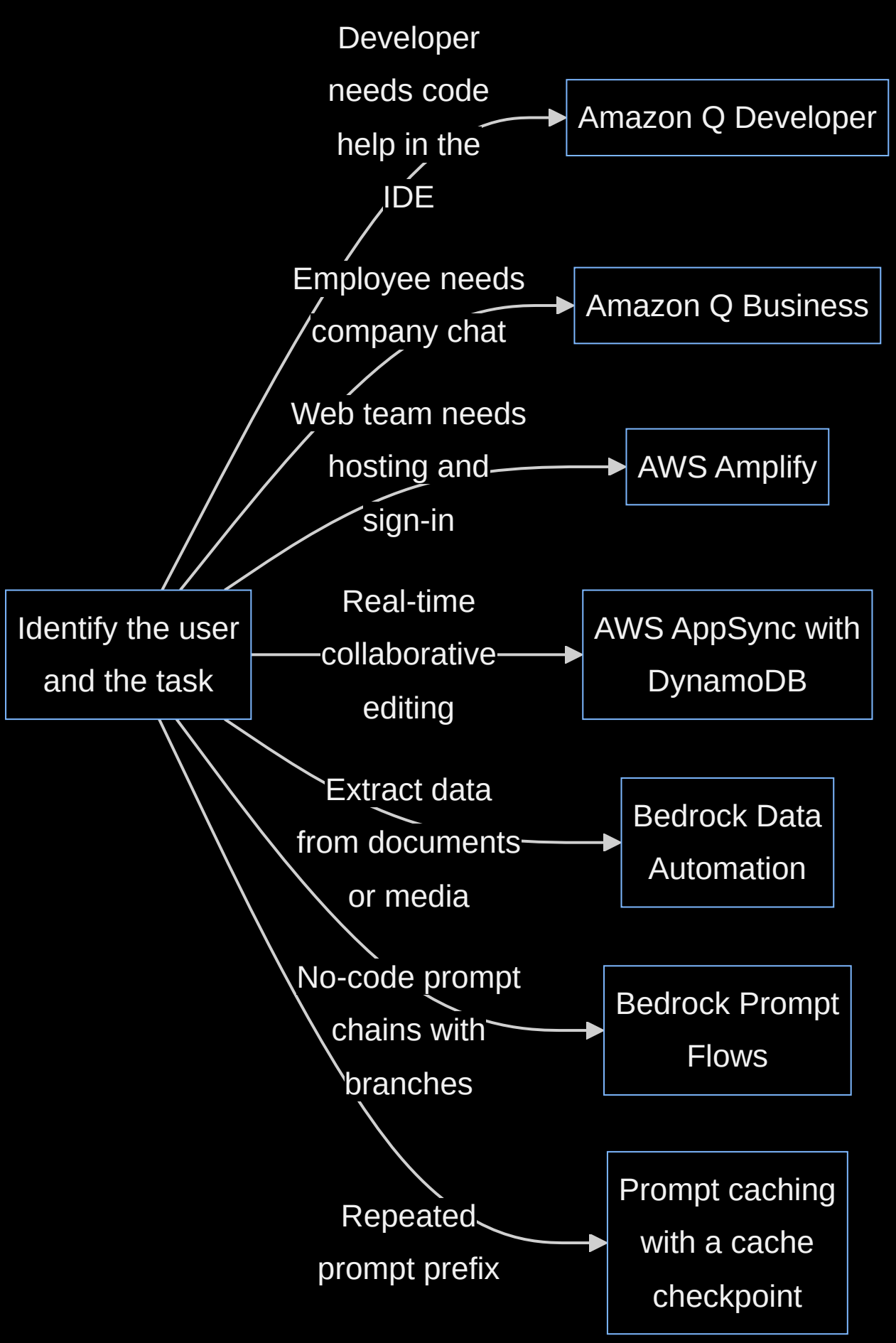


Figure: The user and the task pick the tool, not the name.

Amazon Q Developer and Amazon Q Business

Q Developer gives code help in the IDE. Q Business gives chat over company data for employees.

- Methods or APIs: Q Developer code changes and test generation. Q Business company chat.
- Pro: Each tool fits one user and one task.
- Con: Amazon Q Business is no longer open to new customers. The docs name Amazon Quick as the alternative.
- Choose it when: A developer needs code help: Q Developer. An employee needs company chat: Q Business, where the account can open it.
- Do not choose it when: The choice is made by name alone.

Decision rule: Match the tool to the user and the task. Code help goes to Q Developer.

Trap: Q Business and Q Developer have similar names. One does company chat. The other writes code.

Exam tip: A name match is a trap. Read the user and the task first.

Source: repo guide, Part 3 Domain 2, task 2.5.

Application tools: Amplify, AppSync, Data Automation, and Prompt Flows

Amplify gives managed web app hosting and sign-in. AppSync with DynamoDB gives real-time collaborative editing.

- Methods or APIs: AWS Amplify. AWS AppSync. DynamoDB. Bedrock Data Automation. Bedrock Prompt Flows.
- Pro: Each tool fits one job.
- Con: Hosting does not cover live editing. Prompt Flows is a fixed chain, not a durable workflow.
- Choose it when: A web app needs hosting and sign-in: Amplify. Many users edit one thing live: AppSync. Fields come from media: Data Automation. A no-code chain: Prompt Flows.

- Do not choose it when: The need is chat or code help. Those are Q tools. A wait or approval appears: Step Functions.
- Native pairing: DynamoDB is the store paired with AppSync for real-time editing.

Decision rule: Hosting and sign-in point to Amplify. Live shared editing points to AppSync.

Trap: Guardrails do not extract fields. Use Data Automation for extraction.

Exam tip: Several users editing live point to AppSync.

Source: repo guide, Part 3 Domain 2, task 2.5. Part 2 Domain 1, tasks 1.3 and 1.6.

Prompt variants, versions, and caching

A variant compares two wordings side by side. A version is an immutable snapshot that you can roll back to.

- Methods or APIs: A cachePoint block marks the checkpoint in the API.
- Pro: Prompt caching reuses a repeated prefix. The default time to live is 5 minutes.
- Con: Cache writes cost extra. Caching pays off only with repeat volume.
- Choose it when: A static prefix repeats across calls. The changing text goes after the checkpoint.
- Do not choose it when: The prefix changes on each call. A changing value in the prefix forces a miss.

Decision rule: Place the static prefix first. Place the changing text last.

Trap: Prompt caching does not reserve throughput. Provisioned Throughput reserves capacity.

Exam tip: Lowering the cost of a repeated prefix points to caching. Reserved capacity points to Provisioned Throughput.

Source: repo guide, Part 3 Domain 2, task 2.5.

Decision signals for Domain 2

- A need for a loop to stop points to limits in the workflow or tool layer, not to the prompt.
- A change that must reach users in a controlled step points to a version and an alias.
- Memory that must outlast a session points to a durable key-value store with a time to live.
- Steady traffic points to reserved capacity. Spiky or unknown traffic points to on-demand.
- Offline work that can wait for hours points to batch inference.
- A long job points to asynchronous inference or a durable workflow, not to a real-time endpoint.
- A fixed gateway limit on a REST path, with a need for long responses, points to a path that holds the stream or to an asynchronous job.
- Many teams or apps that share models point to one controlled gateway. On-premises data rules point to Outposts.
- A policy check on output from a model outside the platform points to ApplyGuardrail.
- A developer need for code help points to the IDE tool. An employee need for company chat points to the chat tool.
- Transient failures point to backoff with jitter, a maximum attempt count, and a fallback. Work that can finish later points to a queue.
- A repeated prompt prefix points to caching. A need for reserved throughput points to Provisioned Throughput.
- A need for hosting and sign-in points to Amplify. A need for live shared editing points to AppSync with DynamoDB.

Domain 3. AI safety, security, and governance (20 percent)

Domain 3 covers guardrails, data privacy, compliance evidence, and responsible AI. Four failure modes need four controls, and no single control covers all four. Grounding checks handle confident errors. The prompt attack category handles injection. Sensitive information filters handle PII. Content filters and denied topics handle harm.

Each policy stops one kind of failure.

Task 3.1. One guardrail, six policies

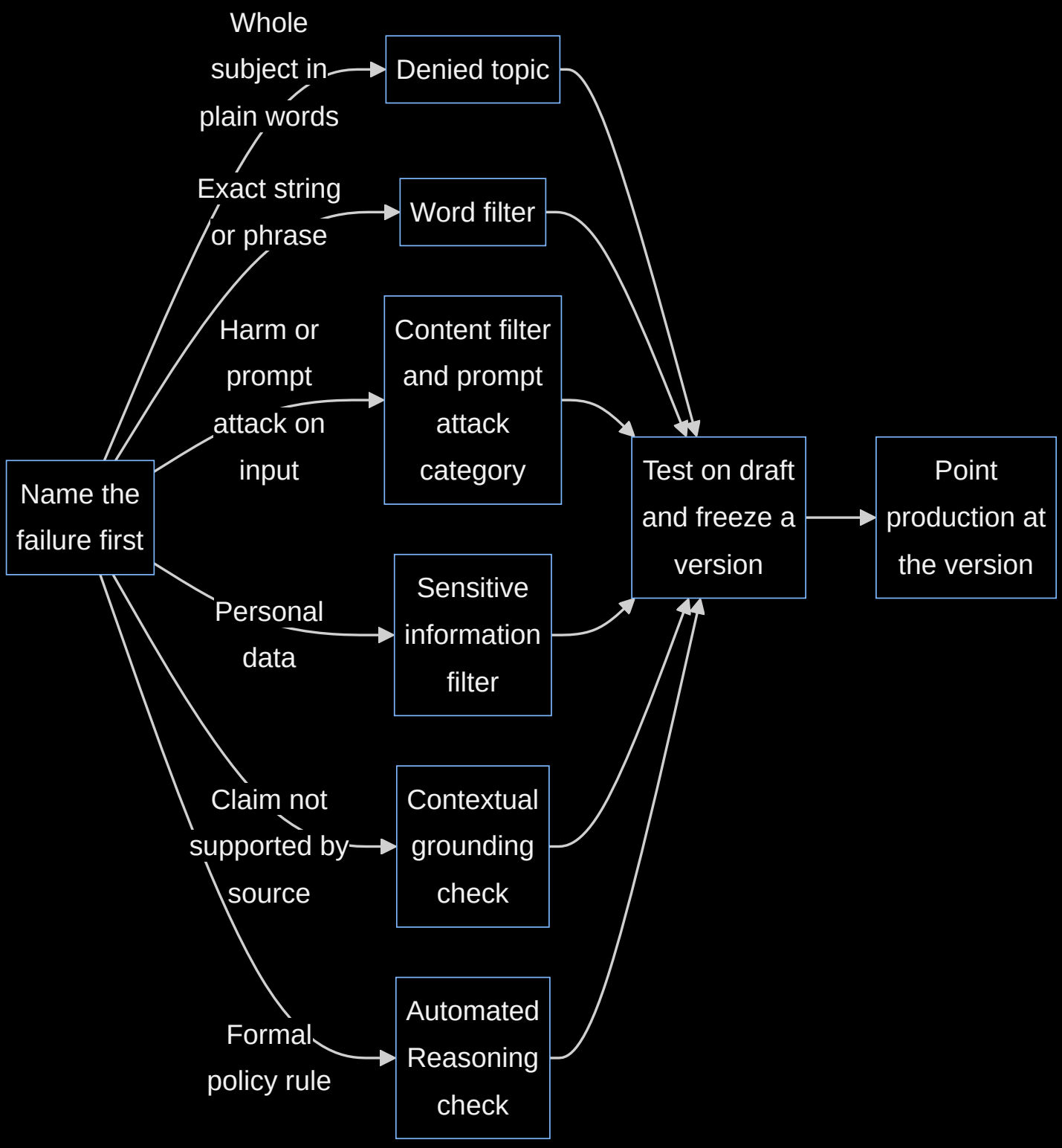


Figure: Pick the policy for the named failure. Freeze a version before production uses it.

Decision signals: a whole subject points to a denied topic. Exact terms point to a word filter. Injected instructions on input point to the prompt attack category.

Bedrock Guardrails and its six policies

Bedrock Guardrails holds six policies that check input and output.

- Methods or APIs: draft, version, production reference, separate input and output switches, Block action, Detect mode. Operation names: Not in source.
- Pro: One guardrail covers six kinds of check.
- Con: Each policy stops one kind of failure.
- Choose it when: the failure happens at the model boundary and one policy names it.
- Do not choose it when: the data sits at rest in S3. Use Amazon Macie.
- Native pairing: Lambda and API Gateway sit in the layers around the guardrail.

Decision rule: Set the policy by failure type. Turn on the switch where the failure starts.

Trap: Detect mode is not a block. A blocked message needs the Block action.

Exam tip: Refusal wording points to the Block action. Wording about measuring a new rule points to detect mode.

Source: repo guide, Part 4 task 3.1, unit 34

Denied topics and word filters

A denied topic blocks a whole subject. A word filter blocks exact words or phrases.

- Methods or APIs: denied topic policy in natural language. Word filter policy with exact strings.
- Pro: A denied topic catches rephrased requests. A word filter stops listed strings.
- Con: A denied topic does not detect prompt attacks. A word filter misses rephrased requests.
- Choose it when: the failure is a subject, or one exact term.
- Do not choose a word filter when: people phrase the subject many ways.
- Native pairing: both run as policies inside Bedrock Guardrails.

Decision rule: Use a denied topic for the subject. Use a word filter for exact strings.

Trap: A word list cannot stop every phrasing of a subject.

Exam tip: A subject named in plain words points to a denied topic.

Source: repo guide, Part 4 task 3.1, unit 34

Content filters and the prompt attack category

Content filters stop harmful text. The prompt attack category applies to input.

- Methods or APIs: harmful categories, prompt attack category, input and output switches.
- Pro: Covers harmful text, and injection or jailbreak patterns on input.
- Con: Does not show whether an answer is supported by the source.
- Choose it when: the failure is harmful text, or a jailbreak or injection on input.
- Do not choose it when: the failure is an unsupported claim. Use grounding.
- Native pairing: Lambda format checks and allow-list checks add layers around it.

Decision rule: Keep the prompt attack category on for input.

Trap: Turn off the prompt attack filter and jailbreaks reach the model.

Exam tip: Injection wording points to input-side controls, not to grounding.

Source: repo guide, Part 4 task 3.1, unit 34

Contextual grounding checks

A contextual grounding check scores the answer for grounding in the source and for relevance to the user request.

- Methods or APIs: contextual grounding check, grounding score.
- Pro: Targets claims that the retrieved source does not support.
- Con: Does not test harm. It reduces hallucination but does not remove it.

- Choose it when: the answer must stay within the retrieved source.
- Do not choose it when: the risk is harmful text. Use content filters.
- Native pairing: runs on the output. Out-of-scope golden dataset tests check it.

Decision rule: Use grounding for the support test. Use content filters for the harm test.

Trap: Grounding checks test support, not safety. No control eliminates hallucinations.

Exam tip: Confident errors point to grounding. Harm points to content filters.

Source: repo guide, Part 4 task 3.1, unit 36

Defense in depth layers

Each layer fails sometimes. Layers use different controls, so one layer can cover a gap in another.

- Methods or APIs: Amazon Comprehend PII detection, abuse classifier, Lambda format and allow-list checks, API Gateway throttling and usage plans, guardrail on input and output.
- Pro: A gap in one layer can be covered by another.
- Con: No layer is absolute.
- Choose it when: one control has a known gap and the failure is costly.
- Do not choose layers as a replacement for the guardrail.
- Native pairing: AgentCore Policy checks each tool call (unit 9).

Decision rule: Put the cheapest check first and the strictest check at the model boundary.

Trap: No single control prevents all prompt injection.

Exam tip: A request for one control that stops every attack points to layers.

Source: repo guide, Part 4 task 3.1, units 37 and 9

Strategy

Strategy: Name the failure and pick its policy. Add layers at input, output, and the API edge. Forbidden: a single control described as complete protection. Anti-pattern: a word list for a whole subject, or detect mode described as a block.

Task 3.2. Privacy: detect, then redact or block

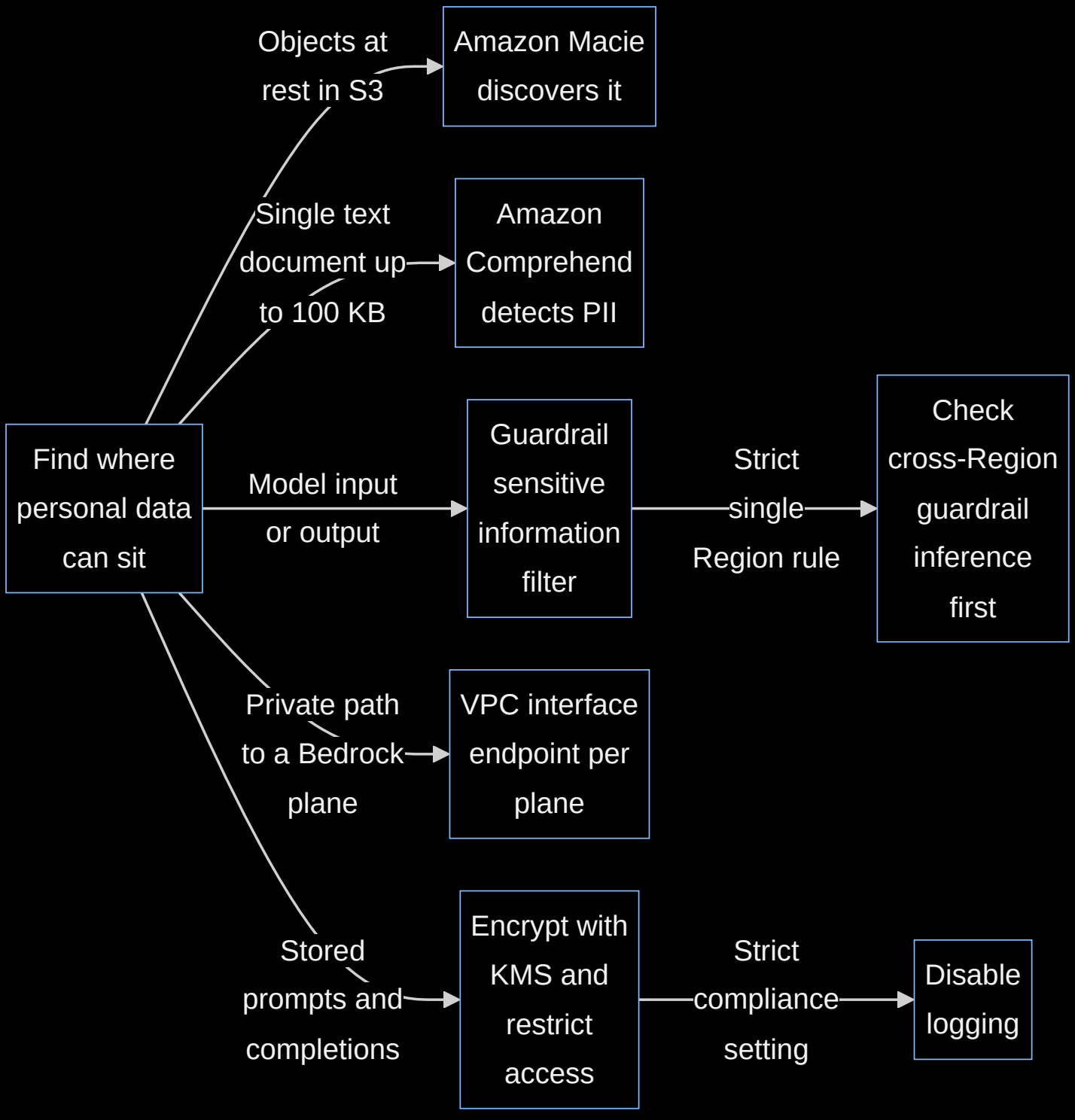


Figure: Detect personal data where it sits. Then redact or block it at the model call.

Decision signals: stored objects point to Amazon Macie. One text input points to Amazon Comprehend or the guardrail. Stored prompts point to encryption, access rules, and retention.

Amazon Macie and Amazon Comprehend

Macie discovers sensitive data in S3 objects. Comprehend detects PII in single text documents.

- Methods or APIs: Macie discovery in S3 objects. Comprehend real-time entity detection for single documents up to 100 KB. Operation names and PII-only limit: Not in source.
- Pro: Macie finds data at rest. Comprehend checks one text input in real time.
- Con: Macie does not watch live model traffic. Comprehend works on single documents only.
- Choose Macie when: data sits in S3. Choose Comprehend when: one text input needs a PII check.
- Do not choose either when: the check is a live model call. Use the guardrail.
- Native pairing: Comprehend is a pre-filter ahead of the guardrail, with an abuse classifier.

Decision rule: Use Macie for the bucket. Use Comprehend for one text input. Use the guardrail for the model call.

Trap: Macie does not watch live model traffic. Guardrails does not scan a whole bucket.

Exam tip: Stored objects point to Macie. One text input points to Comprehend.

Source: repo guide, Part 4 task 3.2, units 35 and 37

Sensitive information filters and cross-Region inference

Sensitive information filters block or mask PII at the model call.

- Methods or APIs: sensitive information filters, redact or block, mask, cross-Region guardrail inference.
- Pro: Acts on model input and output in one place.

- Con: Cross-Region guardrail inference can move prompts and outputs to another Region in the same geography.
- Choose it when: personal data must be masked or blocked at the model call.
- Do not choose cross-Region inference when: a strict single-Region rule applies.
- Native pairing: Not in source beyond the model call.

Decision rule: Check whether the guardrail allows cross-Region inference before choosing it for strict residency.

Trap: Cross-Region routing conflicts with a strict single-Region rule.

Exam tip: Strict residency wording points away from cross-Region routing.

Source: repo guide, Part 4 task 3.2, units 35 and 3

VPC interface endpoints (AWS PrivateLink)

Interface endpoints keep API traffic on the AWS network.

- Methods or APIs: VPC interface endpoints for bedrock-runtime, bedrock-agent, bedrock-agent-runtime, and bedrock (control plane).
- Pro: Traffic from the VPC stays on the AWS network.
- Con: One endpoint covers one plane only.
- Choose it when: private access from a VPC is required.
- Do not choose a runtime endpoint alone when: the workload also calls agents.
- Native pairing: IAM least privilege controls who calls each plane.

Decision rule: Create one endpoint for each Bedrock plane the workload uses.

Trap: One runtime endpoint does not cover agent calls.

Exam tip: Agent wording adds the agent planes to the endpoint list.

Source: repo guide, Part 4 task 3.2, unit 17

Model invocation logs, KMS, and retention

Invocation logs hold the prompts and completions.

- Methods or APIs: model invocation logs, CloudWatch Logs storage, AWS KMS, S3 Object Lock, S3 Lifecycle expiration.
- Pro: Keeps an exact record of content for audit.
- Con: Holds the original request even when output is masked. Can hold protected health information.
- Choose it when: content audit is needed, with KMS encryption and restricted access.
- Do not choose it when: a strict compliance setting applies. Disable logging.
- Native pairing: Object Lock gives write-once-read-many retention. Lifecycle expiration deletes objects at end of life.

Decision rule: Encrypt the log store with KMS, restrict access, then set retention.

Trap: Masked output does not mean the stored log is masked.

Exam tip: Health data wording points to encryption. A strict setting points to disabled logging.

Source: repo guide, Part 4 task 3.2, unit 38

Strategy

Strategy: Find where the data sits, detect it there, and redact or block at the model call. Protect the log store. Forbidden: Macie as a live model filter, and encryption as the answer to training use. For training use, check data terms and retention. Anti-pattern: masked output taken as proof that the stored log is masked.

Task 3.3. Data lake access and model documentation

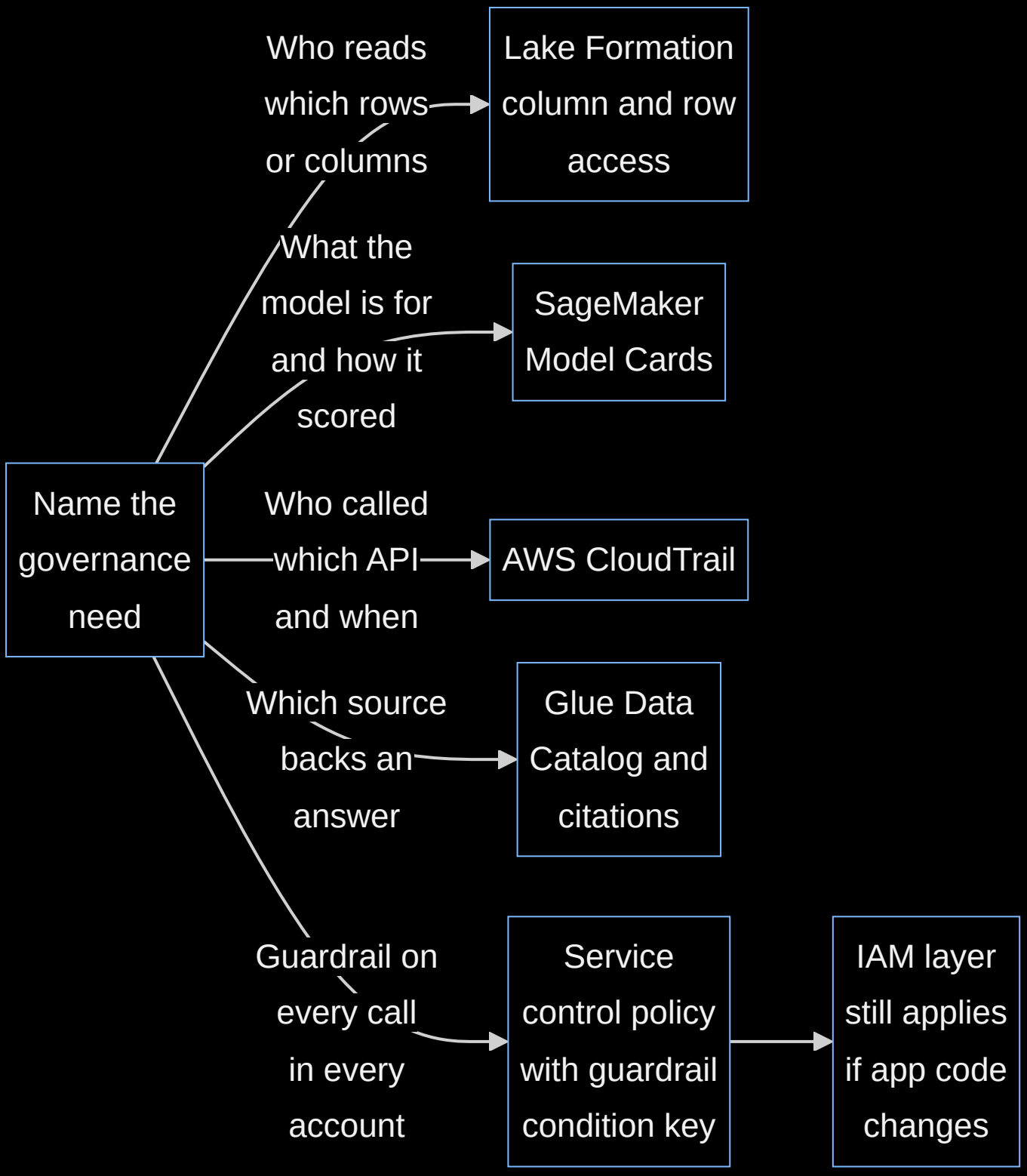


Figure: Map each governance need to one control and one proof.

Decision signals: row or column access points to Lake Formation. Intended use points to Model Cards. Who called which API points to CloudTrail. A guardrail on every call in every account points to a service control policy.

Lake Formation and Model Cards: two different needs

Lake Formation controls who reads which rows and columns. Model Cards record what a model is for and how it performed.

- Methods or APIs: column-level access, row-level access, Model Card JSON schema for the fields, linked to the model package. Permission model details: Not in source.
- Pro: Each tool serves one need with its own record.
- Con: Lake Formation does not document the model. Model Cards do not explain one decision.
- Choose Lake Formation when: access must differ by column or row. Choose Model Cards when: audit needs model documentation.
- Do not choose either when: the need is an audit of API calls. Use CloudTrail.
- Native pairing: neither replaces CloudTrail.

Decision rule: Use Lake Formation for who reads the data. Use Model Cards for what the model is.

Trap: A model card alone does not explain one individual decision.

Exam tip: Columns or rows point to Lake Formation. Intended use points to Model Cards.

Source: repo guide, Part 4 task 3.3, unit 64

AWS Glue Data Catalog and citations

Lineage shows which documents support an answer.

- Methods or APIs: catalog metadata as the source registry, citations on each output.
- Pro: Gives a registry of sources for lineage.
- Con: No single feature gives full lineage.
- Choose it when: an audit must trace an answer to its sources.
- Do not choose it when: the need is the history of API calls. Use CloudTrail.

- Native pairing: citations cover each claim, and CloudTrail covers API calls.

Decision rule: Combine catalog metadata with citations on each output.

Trap: No single feature gives full lineage.

Exam tip: Wording about which source backs an answer points to lineage plus citations.

Source: repo guide, Part 4 task 3.3, unit 16

AWS CloudTrail

CloudTrail shows who called which API, when, and from where.

- Methods or APIs: API call records with identity, time, and origin.
- Pro: Records identity and timing for audit.
- Con: Does not show row content.
- Choose it when: audit needs the identity, the time, and the API called.
- Do not choose it when: the need is the words sent or returned. Use invocation logs.
- Native pairing: CloudTrail and invocation logs both sit behind KMS encryption at rest.

Decision rule: Use CloudTrail for identity and timing. Use invocation logs for content.

Trap: CloudTrail does not show row content.

Exam tip: Who acted and when points to CloudTrail. Exact words point to invocation logs.

Source: repo guide, Part 4 task 3.3, unit 16

Service control policies and the guardrail condition key

Service control policies apply rules across accounts.

- Methods or APIs: service control policies, condition key bedrock:GuardrailIdentifier, IAM layer.

- Pro: One rule is inherited by every member account.
- Con: A condition key limits one action. Application code can still be changed or skipped.
- Choose it when: every call in every account must carry a guardrail.
- Do not choose it when: the rule covers one application only.
- Native pairing: the IAM layer still applies when application code changes.

Decision rule: Enforce a required guardrail at IAM, not in application code.

Trap: A guardrail that only the application calls can be skipped.

Exam tip: Every call in every account points to a service control policy.

Source: repo guide, Part 4 task 3.3, unit 18

Strategy

Strategy: Map each regulation to one control, one proof, and one review cadence. Forbidden: Model Cards as an access control, and CloudTrail as the content audit. Anti-pattern: relying on application code to call the guardrail on every call.

Task 3.4. Responsible AI

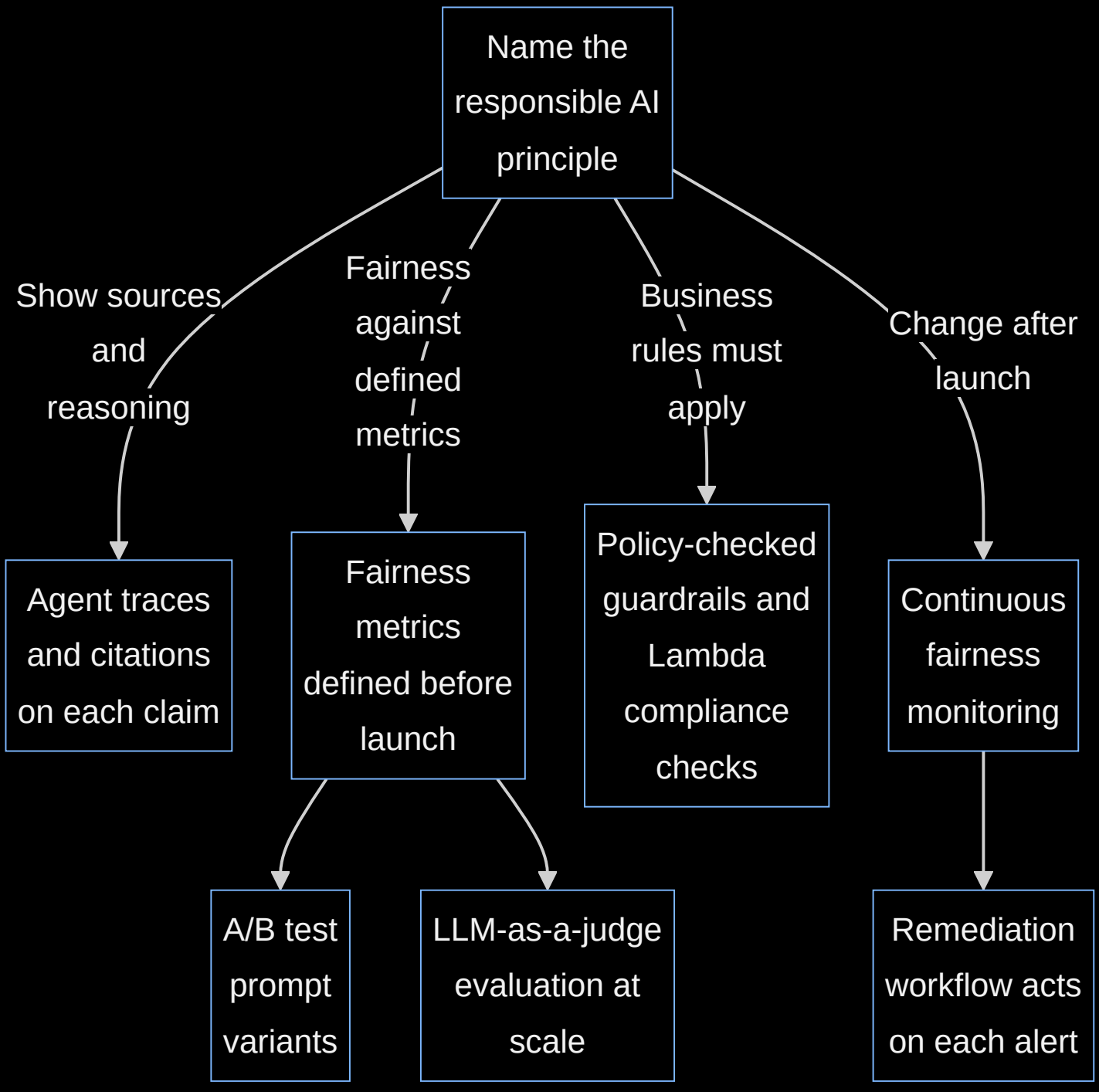


Figure: Each responsible AI principle has its own check and its own proof.

Decision signals: the reasoning behind one output points to traces and citations. Proof of fairness after launch points to continuous monitoring.

Transparency: sources, traces, and citations

Transparent means the system shows its sources and its reasoning.

- Methods or APIs: agent traces, citations on each claim.
- Pro: A reviewer can check each claim against its source.

- Con: A model card alone does not explain one decision.
- Choose it when: each output must show where its claims came from.
- Do not choose a model card alone when: one decision needs an explanation.
- Native pairing: AgentCore Observability records traces (unit 9).

Decision rule: Cite a source on each claim and keep the agent trace.

Trap: A model card alone does not explain one decision. Add citations and the trace.

Exam tip: Why one output came out as it did points to the trace and citations.

Source: repo guide, Part 4 task 3.4, units 65 and 9

Fairness metrics, A/B tests, and judge evaluation

Fairness is measured against metrics defined before launch.

- Methods or APIs: fairness metrics, A/B tests of prompt variants, LLM-as-a-judge evaluation at scale.
- Pro: The same metrics apply to each variant.
- Con: A launch test is a snapshot.
- Choose it when: a variant needs a fairness check before launch.
- Do not choose it when: the need is proof after launch. Use continuous monitoring.
- Native pairing: the judge model ID is pinned, as covered in Domain 5.

Decision rule: Define the fairness metrics before the build.

Trap: A one-time bias test before launch does not prove fairness later.

Exam tip: A launch check does not settle fairness over time.

Source: repo guide, Part 4 task 3.4, units 65 and 66

Policy-checked controls and Lambda compliance checks

Policy checks apply the rules that the business requires.

- Methods or APIs: guardrails set from policy requirements, compliance checks in Lambda.
- Pro: Ties each control to a stated rule.
- Con: The source calls this design guidance, not an AWS rule.
- Choose it when: a business rule must hold on each output.
- Do not choose it when: the need is a citation for each claim.
- Native pairing: Lambda runs compliance checks. Guardrails runs policy checks at the model boundary.

Decision rule: Set each guardrail from a written policy requirement.

Trap: Fairness needs metrics defined before launch and monitored after launch.

Exam tip: A business rule points to policy-checked controls.

Source: repo guide, Part 4 task 3.4, unit 65

Continuous fairness monitoring

A launch test is a snapshot. Data drift and demographic drift change model behavior after launch.

- Methods or APIs: fairness metrics monitored after launch, remediation workflow for each alert. Named monitoring service: Not in source.
- Pro: Shows change that a launch test cannot show.
- Con: A one-time check gives no evidence for later data.
- Choose it when: the data or the population can change after launch.
- Do not choose a one-time check alone when: the system runs over time.
- Native pairing: Not in source.

Decision rule: Monitor fairness after launch. Let a remediation workflow act on each alert.

Trap: A one-time bias test before launch does not prove fairness later.

Exam tip: Change over time points to continuous monitoring.

Source: repo guide, Part 4 task 3.4, unit 66

Strategy

Strategy: Define metrics first. Show sources. Check policy. Monitor after launch. Forbidden: a model card as proof for one decision, or one pre-launch test as proof for all later use. Anti-pattern: an uncalibrated judge used as a fairness measure.

Decision signals for Domain 3

- A whole subject named in plain words points to a denied topic.
- Exact terms or phrases point to a word filter.
- Jailbreak or injected instructions on input point to the prompt attack category.
- An answer that must stay inside the source points to contextual grounding.
- Personal data in stored objects points to Amazon Macie.
- Personal data in one text input points to Amazon Comprehend or the guardrail.
- Personal data in a live model call points to sensitive information filters.
- Access that differs by column or by row points to Lake Formation.
- Intended use or evaluation results point to Model Cards.
- Identity, timing, and API audit point to CloudTrail.
- A guardrail required on every call in every account points to a service control policy.
- Fairness after launch points to continuous monitoring.

Not in source for this domain: IAM Access Analyzer, AWS WAF rule configuration, and the Lake Formation permission model details.

Domain 4. Operational efficiency and optimization (12 percent)

Domain 4 covers cost, latency, capacity, throttling, and monitoring. Match each observed fact to one lever.

Cost equals input tokens divided by 1,000,000, times the input price, plus output tokens divided by 1,000,000, times the output price. The source uses illustrative prices of 3 USD and 15 USD per million tokens, and says they are not current AWS prices. At those prices, 4,000 input and 1,000 output tokens cost 0.027 USD.

Output usually costs more per token. Cap the output length to cap the cost.

Task 4.1. Cost levers

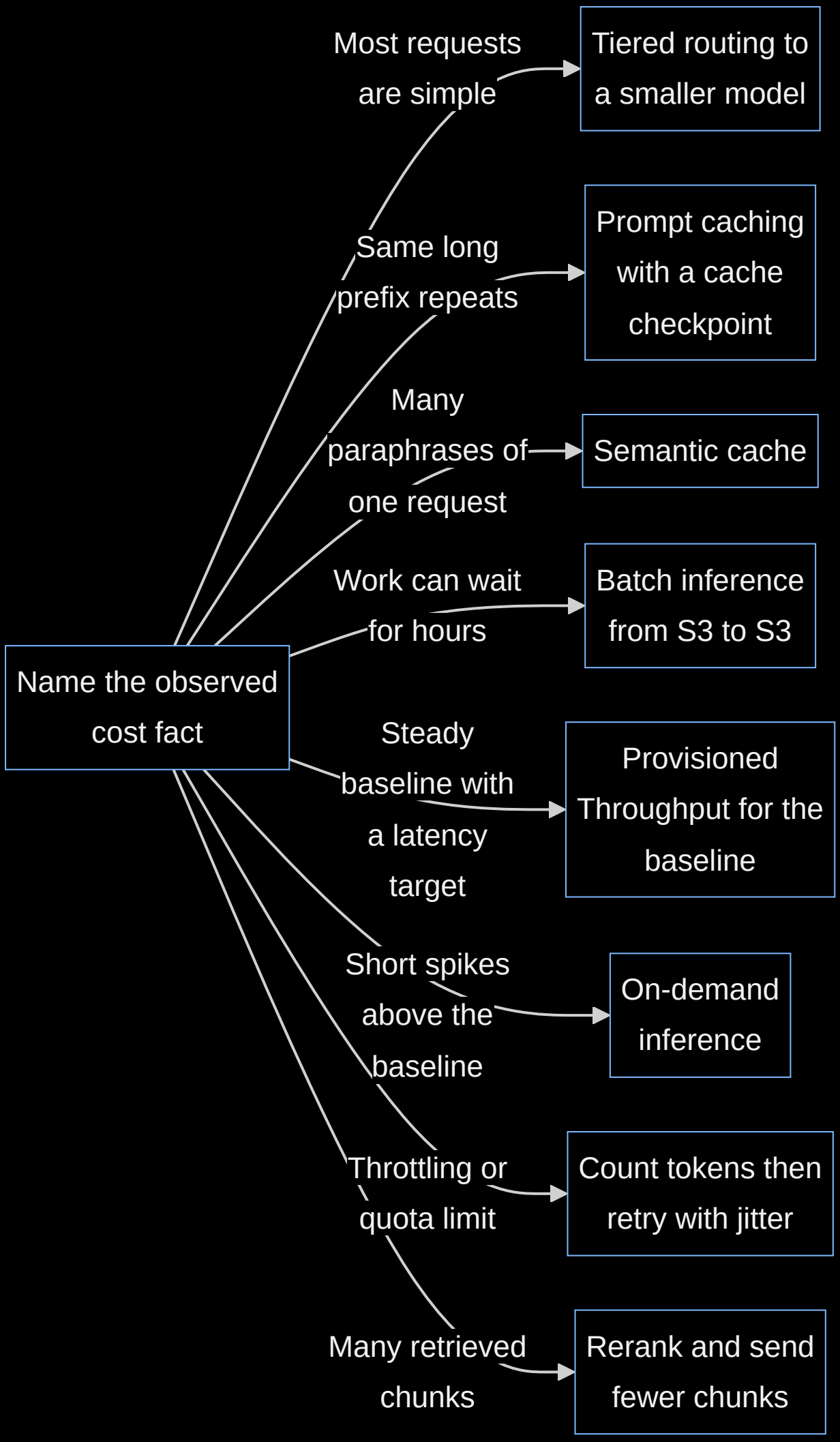


Figure: Each waste has one lever. Match the lever to the waste.

Decision signals: simple requests point to tiered routing. A long repeated prefix points to prompt caching. Work that can wait points to batch inference. Throttling points to token counts, backoff with jitter, and fallback.

Prompt caching

A cache checkpoint marks the static prefix of a prompt as cacheable.

- Methods or APIs: cache checkpoint, cachePoint block in the API, default time to live of 5 minutes, minimum prefix from 512 to 4,096 tokens by model.
- Pro: A repeated long prefix costs less.
- Con: A changing value inside the prefix breaks the cache.
- Choose it when: instructions, tools, or documents repeat across calls.
- Do not choose it when: the prefix changes on each call, or it is below the minimum.
- Native pairing: Not in source beyond the model call.

Decision rule: Put stable content first. Keep the variable user message at the end.

Trap: A date or user ID placed in the prefix forces a miss on every call.

Exam tip: A long repeated prompt head points to prompt caching.

Source: repo guide, Part 4 task 4.1, unit 4

Semantic cache

A semantic cache matches paraphrases by embedding similarity and returns the stored answer before the model runs.

- Methods or APIs: embedding similarity lookup, similarity threshold, stored answer on a hit, model call and store on a miss.
- Pro: A hit returns the stored answer with no model call.
- Con: Needs repeated, similar requests. Highly personal requests rarely hit.

- Choose it when: many requests repeat with different words.
- Do not choose it when: requests are personal or unique.
- Native pairing: embeddings come from Domain 1 task 1.4. The threshold is tested on a golden dataset.

Decision rule: Set the threshold with a golden dataset before you rely on hits.

Trap: Prompt caching reuses an exact prefix at the model. A semantic cache matches meaning before the model runs.

Exam tip: Many paraphrases of one request point to a semantic cache.

Source: repo guide, Part 4 task 4.1, unit 5

Batch inference

Batch inference reads input from S3 and writes output to S3. The lower price applies to select models. The discount amount is Not in source.

- Methods or APIs: batch job with a timeout duration in hours. Exact discount: Not in source.
- Pro: Lower price for work that can wait.
- Con: Not supported for provisioned models.
- Choose it when: the work is offline and can wait for hours.
- Do not choose it when: a user waits for a chat reply. Use on-demand inference.
- Native pairing: S3 holds the input and the output.

Decision rule: Use batch when the work can wait for hours.

Trap: Batch is not supported for provisioned models.

Exam tip: A person who waits for a reply points to on-demand inference.

Source: repo guide, Part 4 task 4.1, unit 2

Model right-sizing

Model right-sizing sends simple work to a smaller model and escalates hard work.

- Methods or APIs: cascade, classifier or router, Bedrock intelligent prompt routing, fine-tuned small model, distillation.
- Pro: Simple requests cost less.
- Con: A router needs a signal. Without one, it cannot decide.
- Choose it when: most requests are simple and a complexity or uncertainty signal exists.
- Do not choose it when: no signal can drive the routing.
- Native pairing: intelligent prompt routing picks between two models in one family.

Decision rule: Try the small model first. Escalate on uncertainty or complexity.

Trap: Sending every request to the largest model wastes cost.

Exam tip: Wording that most requests are simple points to a smaller first model.

Source: repo guide, Part 4 task 4.1, unit 20

On-demand, Provisioned Throughput, and cross-Region profiles

Provisioned Throughput reserves model units. On-demand inference pays per token.

- Methods or APIs: Provisioned Throughput, provisioned model ARN, base model ID, cross-Region inference profile ID.
- Pro: Each choice fits one traffic shape. Profiles fit availability. Reserved units fit steady traffic.
- Con: A call that uses the base model ID does not use reserved capacity.
- Choose it when: steady traffic needs reserved units, spikes need on-demand, or availability needs a profile.
- Do not choose a profile as app failover. It is not automatic failover.

- Native pairing: CloudWatch InvocationThrottles shows throttling on the call.

Decision rule: Match the purchase to the shape of the load.

Trap: Cross-Region inference is not automatic failover. Add retries and fallback in the application.

Exam tip: Steady load points to Provisioned Throughput. Throttling after a purchase points to the model ARN.

Source: repo guide, Part 4 task 4.1, units 2, 3, and 54

Quotas, retries, and pre-call checks

Quotas cap tokens per minute (TPM) and tokens per day (TPD).

- Methods or APIs: InvocationThrottles metric, CountTokens, exponential backoff with jitter, maximum number of attempts, fallback to a second model or a cached answer.
- Pro: CountTokens checks the count against the model limit before the call.
- Con: Retries add load. A tight retry loop makes an overload worse.
- Choose it when: throttling appears, or a request could overflow the context.
- Do not choose a retry loop when: it has no maximum number of attempts.
- Native pairing: CloudWatch watches InvocationThrottles.

Decision rule: Count tokens first. Retry with jitter up to a set maximum. Then fall back.

Trap: Retries that add load can make an overload worse.

Exam tip: Overload after retries points to backoff with jitter.

Source: repo guide, Part 4 task 4.1, unit 42

Strategy

Strategy: Match each waste to one lever. Count tokens before each call. Cap the output length. Forbidden: more capacity as the fix for a long prompt, and a

reserved workload that calls the base model ID. Anti-pattern: the largest model for every request, or more retrieved chunks sent without a rerank.

Task 4.2. Latency, sampling, and the slow step

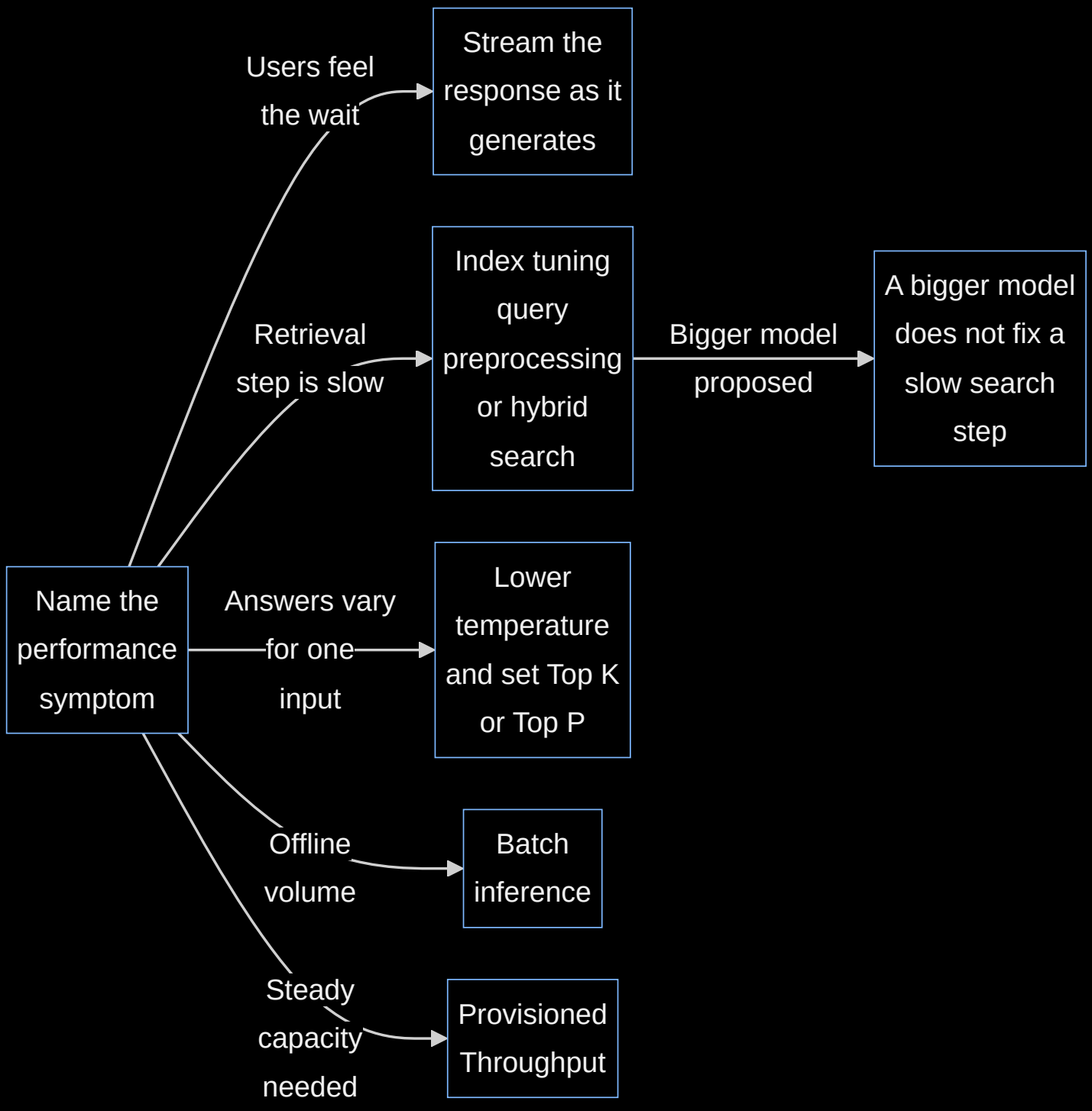


Figure: Find the slow step first. Then fix that step only.

Decision signals: a user who feels the wait points to streaming. A slow retrieval step points to index tuning or hybrid search. Answers that vary for one input point to sampling settings.

Streaming and the request timeout

Streaming improves perceived speed. It does not raise throughput.

- Methods or APIs: stream the response as it generates, WebSocket connection held open, asynchronous job that returns a job ID.
- Pro: Users see output sooner.
- Con: Streaming does not extend a REST integration timeout. The source lists 29 seconds as the default, an open check.
- Choose it when: perceived speed matters, or a long reply needs a held connection.
- Do not choose it when: throughput or a REST timeout is the problem.
- Native pairing: a standard Lambda function runs for at most 15 minutes. Step Functions Standard waits on a callback task token for up to one year.

Decision rule: Stream for perceived speed. Use an asynchronous job for slow work.

Trap: Streaming improves perceived speed. It does not extend a REST timeout.

Exam tip: A wait for a person points to Step Functions.

Source: repo guide, Part 4 task 4.2, units 11 and 10

Retrieval step tuning

For a slow retrieval step, use index optimization, query preprocessing, or hybrid search.

- Methods or APIs: index optimization, query preprocessing, hybrid search, reranker, top few chunks.
- Pro: Fixes the step that is slow. Fewer chunks also lower token cost.
- Con: A reranker orders candidates. It does not add missing chunks.
- Choose it when: the trace shows retrieval as the slow step.
- Do not choose it when: the model is the slow step. Measure first.
- Native pairing: X-Ray traces each downstream call to find the slow step.

Decision rule: Measure each step. Rerank, then keep only the top few chunks.

Trap: A bigger model does not fix a slow search step.

Exam tip: Slow answers that come from search point to retrieval tuning.

Source: repo guide, Part 4 task 4.2, units 41, 7, and 67

Sampling settings

Temperature sets how wide sampling is. Lower values give more repeatable output.

- Methods or APIs: temperature, Top K (limits candidate tokens), Top P (limits the share of the probability distribution).
- Pro: Lower values repeat output for the same input.
- Con: Sampling settings do not change the price per token. They do not fix bad retrieval.
- Choose it when: output must repeat across runs.
- Do not choose it when: the goal is lower cost or better retrieval.
- Native pairing: a golden dataset rerun shows whether output repeats. Fixed seed effect: Not in source.

Decision rule: Lower temperature when output must repeat. Compare runs on the same prompt version.

Trap: Buying capacity does not fix different answers to the same input.

Exam tip: Variance wording points to sampling settings, not to capacity.

Source: repo guide, Part 4 task 4.2, unit 53

Strategy

Strategy: Find the slow step. Stream for perceived speed. Precompute predictable requests. Set sampling for repeatable output. Forbidden: a bigger model as the fix for slow search, and streaming as a throughput fix. Anti-pattern: temperature changes used to fix retrieval.

Task 4.3. Monitoring signals

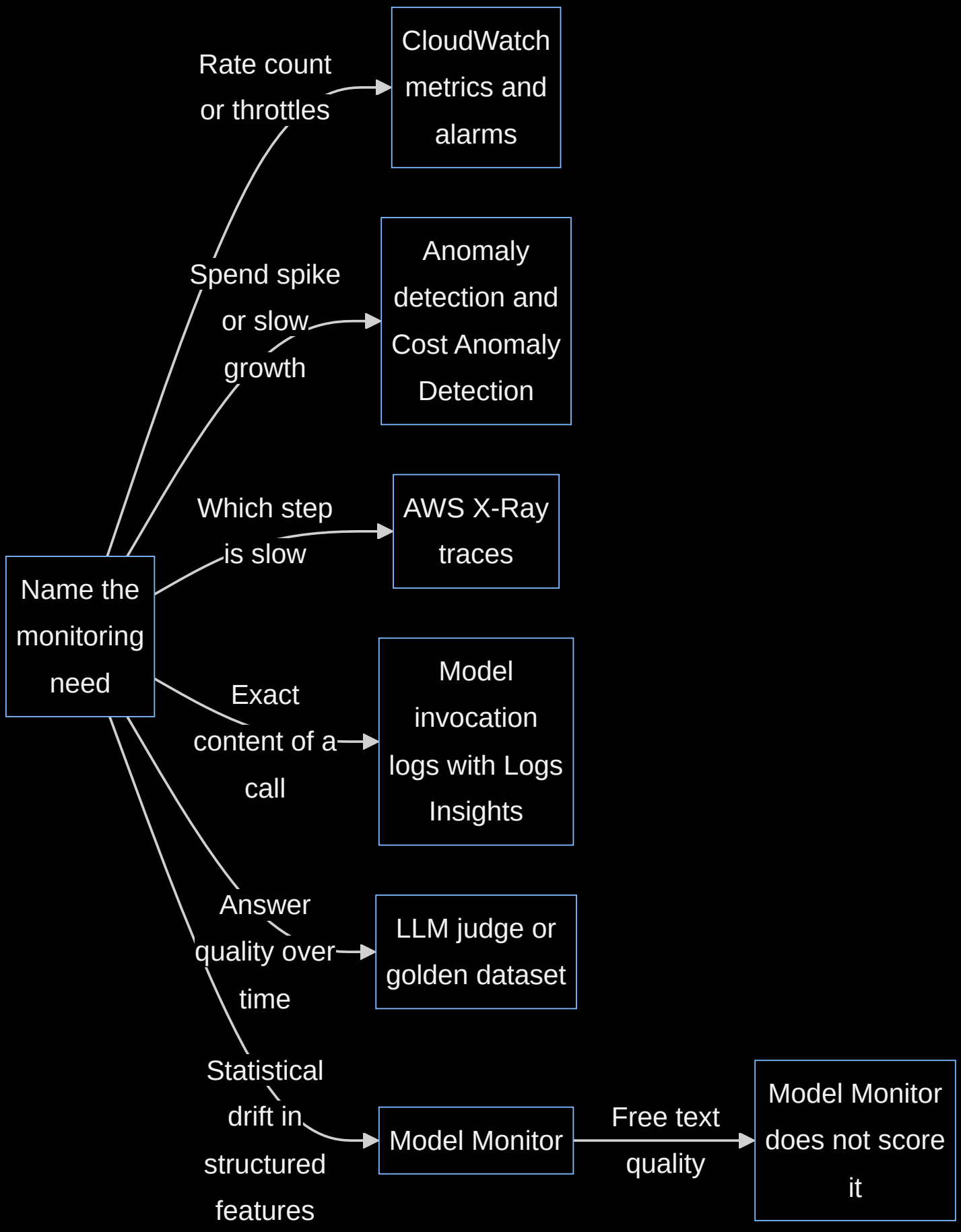


Figure: Each signal answers one need. Use the signal that answers it.

Decision signals: rate, count, or spend points to CloudWatch metrics. Where time goes points to traces. Exact words sent points to invocation logs. Answer

quality over time points to a judge or golden dataset.

CloudWatch metrics, alarms, and anomaly detection

Metrics show rate and cost. Anomaly alarms catch spikes that a fixed threshold misses.

- Methods or APIs: InputTokenCount, OutputTokenCount, Invocations, InvocationThrottles, anomaly detection and alarms, Cost Anomaly Detection for spend. AWS Cost Explorer: Not in source.
- Pro: Shows token volume, call rate, and throttles over time.
- Con: A fixed threshold alarm misses slow growth.
- Choose it when: the signal is rate, count, throttling, or spend.
- Do not choose it when: the signal is the content of a call. Use invocation logs.
- Native pairing: anomaly alarms run on the same metrics.

Decision rule: Use anomaly detection when the trend, not a fixed number, marks a problem.

Trap: A fixed threshold alarm misses slow growth.

Exam tip: Gradual spend growth points to anomaly detection.

Source: repo guide, Part 4 task 4.3, units 43 and 15

AWS X-Ray traces and invocation logs

AWS X-Ray traces requests and downstream calls. Model invocation logs hold the exact request and response.

- Methods or APIs: request traces, model invocation logs, CloudWatch Logs Insights search.
- Pro: X-Ray shows which step is slow. Logs show the exact content.
- Con: X-Ray does not show prompt text. Invocation logs hold prompts, so access must be restricted.

- Choose X-Ray when: the need is where the time goes. Choose the logs when: the need is what was sent or returned.
- Do not choose logs when: the need is a rate or spend trend.
- Native pairing: trace the slow step, then audit the exact prompts in the logs. KMS encrypts the logs.

Decision rule: Find the slow step with X-Ray. Then audit the prompts in the logs.

Trap: Metrics alone cannot rebuild a wrong answer. Keep the payloads.

Exam tip: Where time goes points to traces. Exact words point to logs.

Source: repo guide, Part 4 task 4.3, units 15 and 38

Quality checks and Model Monitor

Model Monitor watches statistical drift in structured features and predictions.

- Methods or APIs: LLM judge, golden dataset, Model Monitor for drift.
- Pro: A judge or golden dataset scores free-text answers over time.
- Con: Model Monitor does not score free-text answer quality.
- Choose it when: the need is answer quality over time. Use a judge or golden dataset.
- Do not choose Model Monitor when: the need is the quality of free text.
- Native pairing: a pinned judge model ID keeps scores comparable, as covered in Domain 5.

Decision rule: Use a judge or golden dataset for free-text quality. Use Model Monitor for drift in structured data.

Trap: Model Monitor does not score free-text answer quality.

Exam tip: Quality of generated text points to a judge, not to a drift monitor.

Source: repo guide, Part 4 task 4.3, unit 43, and Part 6 task 5.1

Strategy

Strategy: Pick the signal that answers the need. Metrics for rate, traces for the step, logs for content, and a judge for quality. Forbidden: Model Monitor as the quality measure for generated text, and a fixed threshold as the only alarm. Anti-pattern: metrics kept without the payloads needed to rebuild an answer.

Decision signals for Domain 4

- A long repeated prefix points to prompt caching.
- Many paraphrases of one request point to a semantic cache.
- Work that can wait for hours points to batch inference.
- Simple requests point to tiered routing.
- A steady baseline of traffic points to Provisioned Throughput.
- Throttling points to token counts, backoff with jitter, and fallback.
- A slow retrieval step points to index tuning, query preprocessing, or hybrid search.
- Users who feel the wait point to streaming.
- Different answers to one input point to sampling settings.
- Slow growth in spend points to anomaly detection, not to a fixed alarm.
- Free-text quality over time points to a judge or golden dataset.

Not in source for this domain: Amazon CloudWatch Synthetics, AWS Auto Scaling for GenAI workloads, and AWS Cost Explorer as a source-backed tool.

Domain 5. Testing, validation, and troubleshooting (11 percent)

Domain 5 covers evaluation methods, golden datasets, A/B tests, debugging, and release gates. Choose the cheapest evaluation method that measures the claim. Escalate only for what the cheap method cannot judge.

Task 5.1. Evaluation matrix and release gates

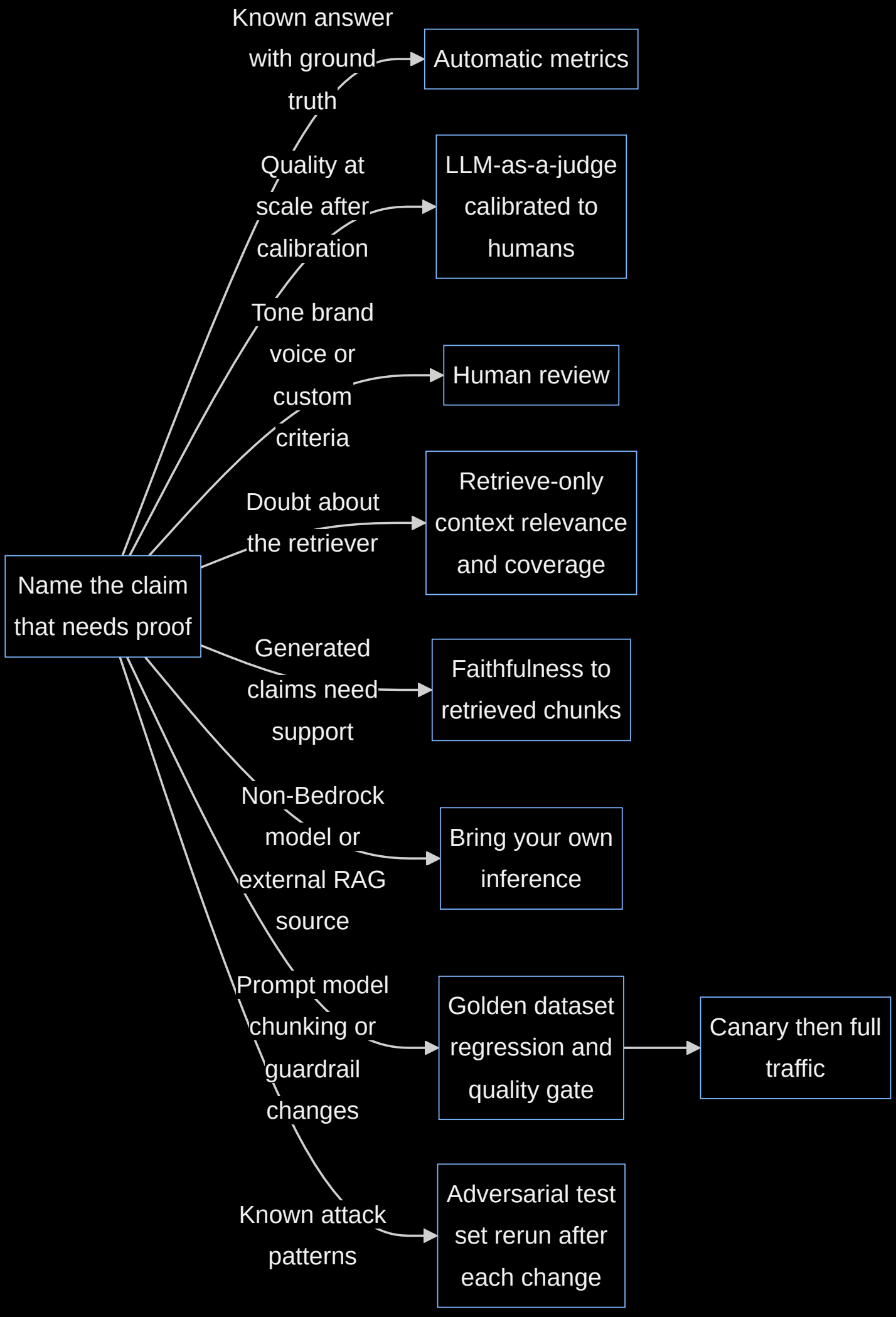


Figure: Pick the cheapest method that measures the claim. Gate every change.

Decision signals: a claim with known labels points to automatic metrics. Tone or brand voice points to human review. A change to a prompt, model, chunking setting, or guardrail points to a golden dataset gate.

Automatic and human evaluation

Automatic metrics suit claims that have known answers. Human review judges tone, voice, and custom criteria.

- Methods or APIs: automatic metrics for accuracy and toxicity, plus the third automatic metric named in the source. Human review, custom criteria, relevance rating.
- Pro: Automatic checks are objective and repeatable. Human review judges tone and voice.
- Con: Automatic metrics do not measure tone, style, or voice. Accuracy needs ground-truth labels. Human review is subjective.
- Choose automatic when: labeled answers exist. Choose human when: tone or brand voice decides the claim.
- Do not choose automatic when: the claim is tone or brand voice.
- Native pairing: setup steps for Bedrock Model Evaluations jobs: Not in source.

Decision rule: Start with automatic metrics when ground-truth labels exist. Escalate to human review for voice.

Trap: Automatic accuracy metrics do not measure tone, style, or voice.

Exam tip: Labeled answers point to automatic metrics. Voice points to human review.

Source: repo guide, Part 6 task 5.1, unit 44

LLM-as-a-judge and calibration

A judge score means something only when it agrees with people.

- Methods or APIs: LLM-as-a-judge, custom metrics, judge-human agreement, pinned judge model ID.
- Pro: Scores correctness and faithfulness at scale, for RAG and agent evaluation.
- Con: An uncalibrated judge gives opinions, not measurements.
- Choose it when: quality must be scored at scale, after calibration.
- Do not choose it when: the judge has not been checked against human ratings on the same examples.
- Native pairing: the golden dataset holds the human ratings used for calibration.

Decision rule: Rate a fixed golden dataset with humans. Run the judge on the same set. Low agreement means the judge is not valid.

Trap: A changed judge model changes the scores. Pin the judge model ID.

Exam tip: Scores that no person has checked point to calibration first.

Source: repo guide, Part 6 task 5.1, unit 13

RAG evaluation: retriever and generator

A RAG answer has two parts. The retriever finds chunks. The generator writes the answer.

- Methods or APIs: context relevance and context coverage for the retriever. Faithfulness for the generator. Citation precision and citation coverage: Not in source.
- Pro: Shows which part fails.
- Con: A good final answer can hide weak retrieval. The source for these metrics is an AWS blog post, not a reference page.
- Choose it when: the answer may rest on weak retrieval, or the generator may claim more than the chunks support.
- Do not choose the final answer alone when: the split matters. Measure both parts.

- Native pairing: bring your own inference applies the same metrics to non-Bedrock models or external RAG sources.

Decision rule: Measure both parts. A drop in retriever-only scores points at retrieval.

Trap: Fine-tuning does not fix retrieval.

Exam tip: A correct-looking answer that rests on wrong sources points to retriever metrics.

Source: repo guide, Part 6 task 5.1, unit 45 and the RAG evaluation table

Golden dataset, regression suite, canary, and A/B tests

A golden dataset holds trusted prompts with known good answers.

- Methods or APIs: golden dataset, regression suite, canary, A/B test, quality gate, versioned prompt.
- Pro: Catches drift after each change.
- Con: A passing suite with new live failures means the golden dataset lacks those cases.
- Choose it when: a prompt, model, chunking setting, or guardrail changes, or a re-sync runs.
- Do not choose a full-traffic switch when: the canary has not run first.
- Native pairing: AppConfig and aliases (Domain 2 task 2.3). CloudWatch alarms gate traffic.

Decision rule: Pass the quality gate on each change. Start the canary at a small share, such as 10 percent. If an alarm fires, roll back to the last approved version.

Trap: A re-sync or a prompt edit is a change. It needs the same gate.

Exam tip: A live rollout points to canary and rollback.

Source: repo guide, Part 6 task 5.1, units 46, 48, and 14

Adversarial and safety tests

An automated test set sends known attack patterns to the system.

- Methods or APIs: automated adversarial test sets for injection and jailbreak patterns, agent evaluations for task completion and tool usage.
- Pro: Finds gaps before users do.
- Con: Step-by-step red-team procedures: Not in source.
- Choose it when: a guardrail or prompt changes.
- Do not choose it as the only safety evidence when: red-team work is still planned.
- Native pairing: the Bedrock evaluation documentation lists the agent evaluation methods.

Decision rule: Rerun the attack set after each guardrail or prompt change.

Trap: Red-team procedures are not in the source. Plan them separately.

Exam tip: Unplanned attacks point to an automated attack set that runs on each change.

Source: repo guide, Part 6 task 5.1, unit 68

Strategy

Strategy: Pick the cheapest method that measures the claim. Calibrate the judge. Gate every change. Forbidden: automatic accuracy as proof of tone, and an uncalibrated judge as a measurement. Anti-pattern: a re-sync or prompt edit treated as a non-release.

Task 5.2. Troubleshooting by symptom

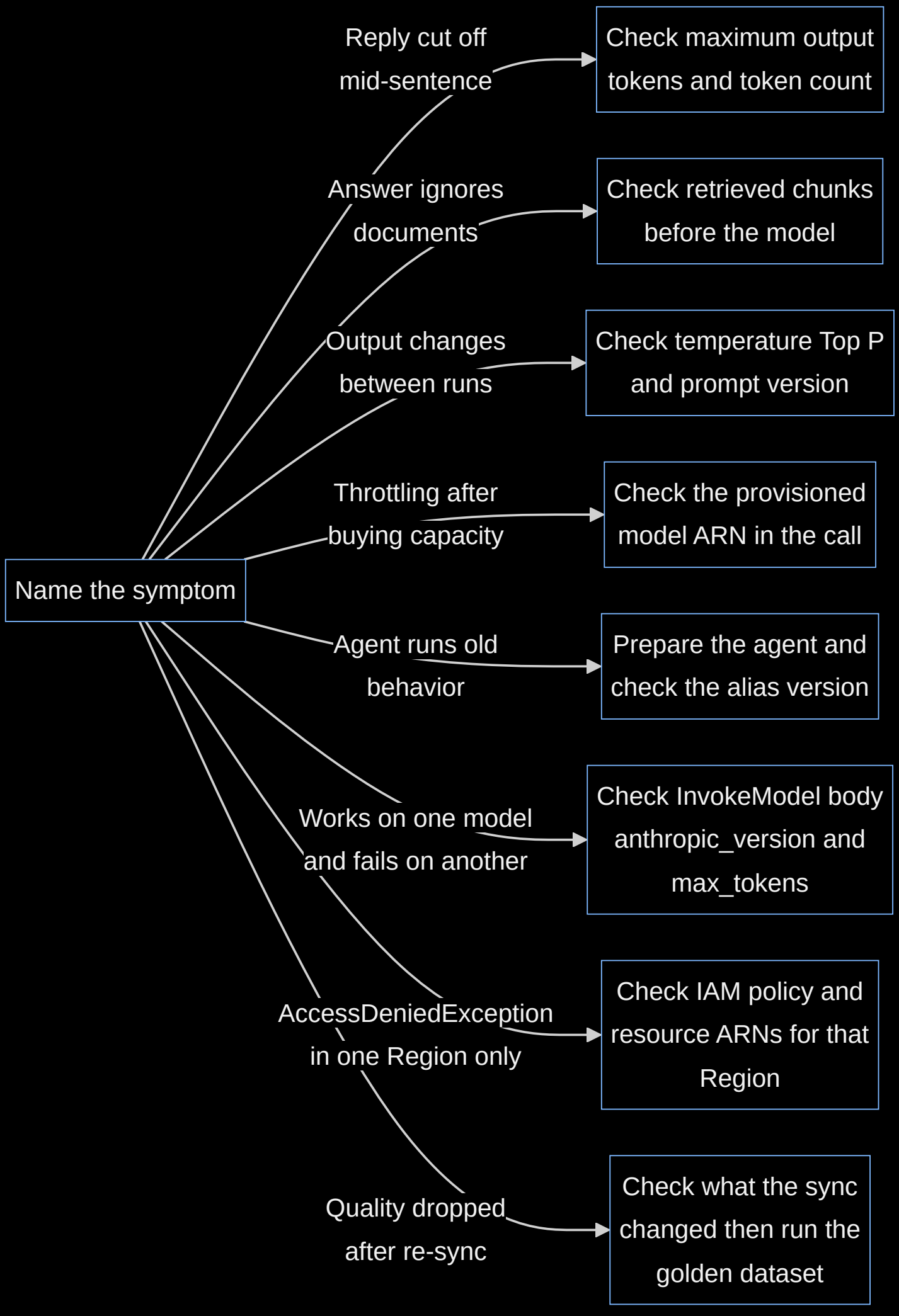


Figure: Start the debug at the step that failed.

Decision signals: a cut-off reply points to output limits. An answer that ignores documents points to retrieved chunks. Throttling after a purchase points to the model ARN. A failure in one Region points to IAM.

Output limits and model body fields

Output ends early when the maximum tokens are too low or the context overflows.

- Methods or APIs: maximum tokens setting, CountTokens, InvokeModel request body fields anthropic_version and max_tokens.
- Pro: CountTokens shows the token count against the model limit.
- Con: Body fields differ by model.
- Choose it when: a reply is cut off, or a call fails on one model only.
- Do not choose it when: the cause is the retrieved chunks. Check chunks first.
- Native pairing: Not in source.

Decision rule: Check the maximum tokens and the token count against the limit. Check the body for each model.

Trap: A body that works on one model can fail on another.

Exam tip: A reply cut off mid-sentence points to output limits.

Source: repo guide, Part 6 task 5.2, unit 47

Retrieval and chunking

When an answer ignores the documents, check the retrieved chunks before the model.

- Methods or APIs: retrieved chunks inspected before generation, chunk shapes (fixed size, hierarchical, table kept whole), hybrid search, reranker.
- Pro: Separates retrieval faults from model faults.
- Con: A split table loses its values. Each half can look correct alone.

- Choose it when: an answer ignores documents, or quality dropped after a sync.
- Do not choose it first when: sampling is the suspected cause.
- Native pairing: CloudWatch Logs shows what the model received.

Decision rule: Check the chunks first, then the prompt, then the settings.

Trap: Temperature does not fix bad retrieval.

Exam tip: A sync change points to chunking or the embedding model.

Source: repo guide, Part 6 task 5.2, units 6 and 47

Sampling and prompt version

Output that changes between runs comes from sampling settings or prompt drift.

- Methods or APIs: temperature, Top P, prompt version, output comparison across runs.
- Pro: Cheap to check before any capacity change.
- Con: Effect of a versioned prompt on variance: Not in source.
- Choose it when: the same input gives different answers.
- Do not choose it when: the symptom is throttling or a cut-off reply.
- Native pairing: a golden dataset rerun gives the output comparison.

Decision rule: Fix the prompt version. Compare outputs across runs. Then adjust temperature or Top P.

Trap: Buying capacity does not fix different answers to one input.

Exam tip: Variance wording points to sampling and prompt version.

Source: repo guide, Part 6 task 5.2, units 47 and 53

Call target: provisioned ARN and IAM per Region

Throttling after a purchase points to the model ARN in the call. An AccessDeniedException in one Region points to IAM for that call.

- Methods or APIs: provisioned model ARN, base model ID, IAM policy, resource ARNs for each Region, InvocationThrottles metric.
- Pro: Two short checks cover two symptoms.
- Con: More model units do not help a call that uses the base model ID.
- Choose it when: throttling starts after a purchase, or one Region fails only.
- Do not choose it when: the symptom is a cut-off reply or ignored documents.
- Native pairing: CloudTrail shows who called which API. It does not explain an answer.

Decision rule: Check the model ARN for throttling. Check the IAM resource ARNs for the failing Region.

Trap: More model units do not help a call that still uses the base model ID.

Exam tip: A purchase that did not reduce throttling points to the call target.

Source: repo guide, Part 6 task 5.2, units 2 and 47

Agent DRAFT, prepare, and alias

An agent runs old behavior when it is not prepared or when the alias did not move.

- Methods or APIs: DRAFT, prepare the agent, create a version, alias that points to a version, agentId and agentAliasId.
- Pro: Callers keep the alias, so traffic moves without a caller change.
- Con: A passing test on DRAFT does not prove the alias works.
- Choose it when: an existing agent runs old behavior after an edit.
- Do not choose Bedrock Agents Classic when: a new build starts. The docs say it is closed to new customers.
- Native pairing: AgentCore hosts the agent code and governs tool calls (unit 9).

Decision rule: Prepare the agent. Create a version. Point the alias to the new version.

Trap: A passing test on DRAFT does not prove the alias works.

Exam tip: Old behavior after an edit points to prepare and alias.

Source: repo guide, Part 6 task 5.2, units 8 and 9

Debugging tools

CloudWatch Logs shows what the model received.

- Methods or APIs: CloudWatch Logs, Logs Insights queries by time, caller identity, or request metadata, CloudTrail for API calls.
- Pro: Shows the payload that the model received.
- Con: CloudTrail does not show answer content.
- Choose it when: the failed step needs the payload, or a pattern across calls.
- Do not choose it when: the need is who acted and when. Use CloudTrail.
- Native pairing: X-Ray shows which step is slow or failed (unit 15).

Decision rule: Start the debug at the failed step. Read the payload the model received.

Trap: CloudTrail is for audit, not for debugging answers.

Exam tip: The payload points to CloudWatch Logs. Who called points to CloudTrail.

Source: repo guide, Part 6 task 5.2, units 15 and 16

Hallucination tests

A hallucination test needs out-of-scope requests. Without them, the test only confirms answers the documents already support.

- Methods or APIs: contextual grounding check, grounding score, golden dataset with out-of-scope requests, output comparison across runs.
- Pro: Detects claims that the source does not support, and inconsistent answers.
- Con: Without out-of-scope requests, the test is blind to unsupported claims.

- Choose it when: unsupported claims or inconsistent answers are the symptom.
- Do not choose it when: the symptom is a cut-off reply. Check output limits.
- Native pairing: contextual grounding checks run inside Bedrock Guardrails (Domain 3).

Decision rule: Add out-of-scope requests to the golden dataset. Compare answers across runs.

Trap: A passing suite with new live failures means the golden dataset lacks those cases.

Exam tip: Claims with no source point to grounding and out-of-scope tests.

Source: repo guide, Part 6 task 5.2, unit 46

Strategy

Strategy: Trace the request from input to output. Check the stage that changed last. Forbidden: temperature as the fix for bad retrieval, and more model units as the fix for a call on the base model ID. Anti-pattern: CloudTrail used as a source of answer content.

Decision signals for Domain 5

- A claim with known labels points to automatic metrics.
- Quality at scale points to a judge, after calibration.
- Tone, brand voice, or custom criteria point to human review.
- A doubt about retrieval points to retrieve-only metrics.
- A change to a prompt, model, chunking setting, or guardrail points to a golden dataset gate and a canary.
- Unplanned attack patterns point to an automated adversarial test set.
- A cut-off reply points to output limits and the token count.
- Output that changes between runs points to sampling and prompt version.
- Throttling after a purchase points to the model ARN in the call.

- Old agent behavior points to preparing the agent and the alias.
- Claims with no source point to grounding and out-of-scope tests.

Not in source for this domain: setup steps for Bedrock Model Evaluations jobs and step-by-step red-team procedures.

The exam at a glance

Source: repo guide, Part 0 The exam at a glance

The exam has 75 questions. 65 are scored and 10 are unscored.

- Question types: multiple choice with one correct answer, and multiple response with two or more correct answers.
- Passing score: 750 on a scale from 100 to 1,000.
- Scoring by section: Not in source. The pass decision uses the whole exam.
- Duration and fee: 180 minutes and 300 USD. Confirmed on the official AWS certification page.
- Target candidate: two or more years of production application experience, and one year of hands-on generative AI work.
- Assumed knowledge: AWS compute, storage, networking, security, and infrastructure as code.
- Out of scope: model training, advanced machine learning, and data engineering.

Domain weights

The weights match the official AWS exam guide. The question counts below are approximate.

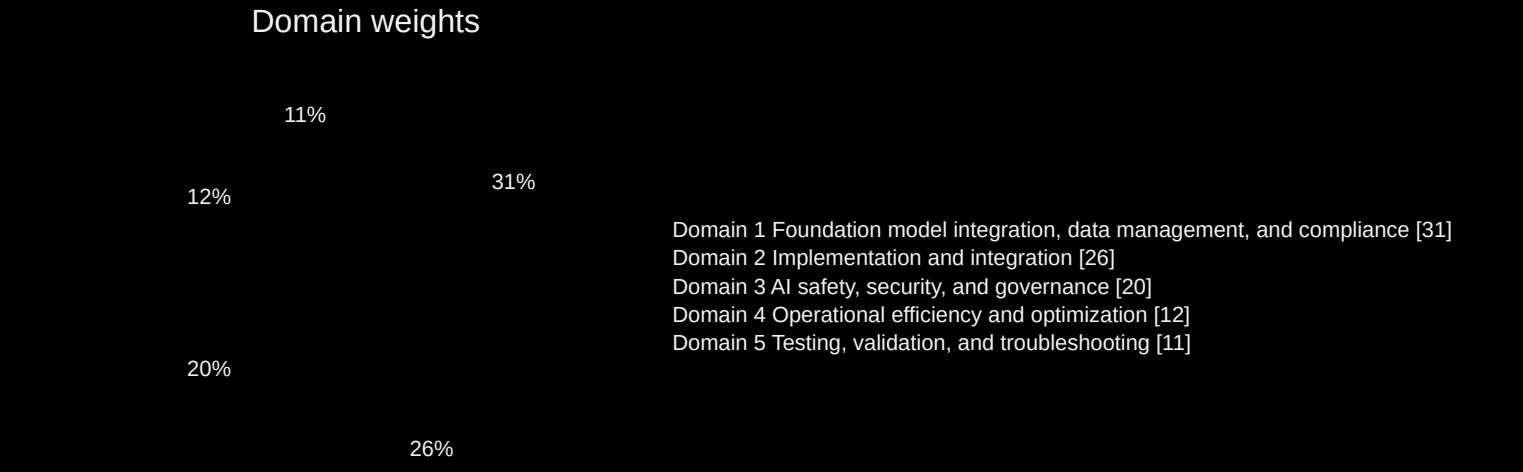


Figure: Domain weights in percent, from the official exam guide.

- Domain 1: about 20 questions (Inference).
- Domain 2: about 17 questions (Inference).
- Domain 3: about 13 questions (Inference).
- Domain 4: about 8 questions (Inference).
- Domain 5: about 7 questions (Inference).

Trap: An approximate question count used as an official count. The source labels these Inference.

Decision rule: Rank study time by domain weight, largest first.

Exam tip: Check the wording for one correct answer or two or more before you select.

Traps AWS reuses

Source: repo guide, Part 7 Traps AWS reuses

Each pair shows a wrong idea and the correct idea. The source lists 18 pairs. They are grouped here by topic.

Retrieval and storage

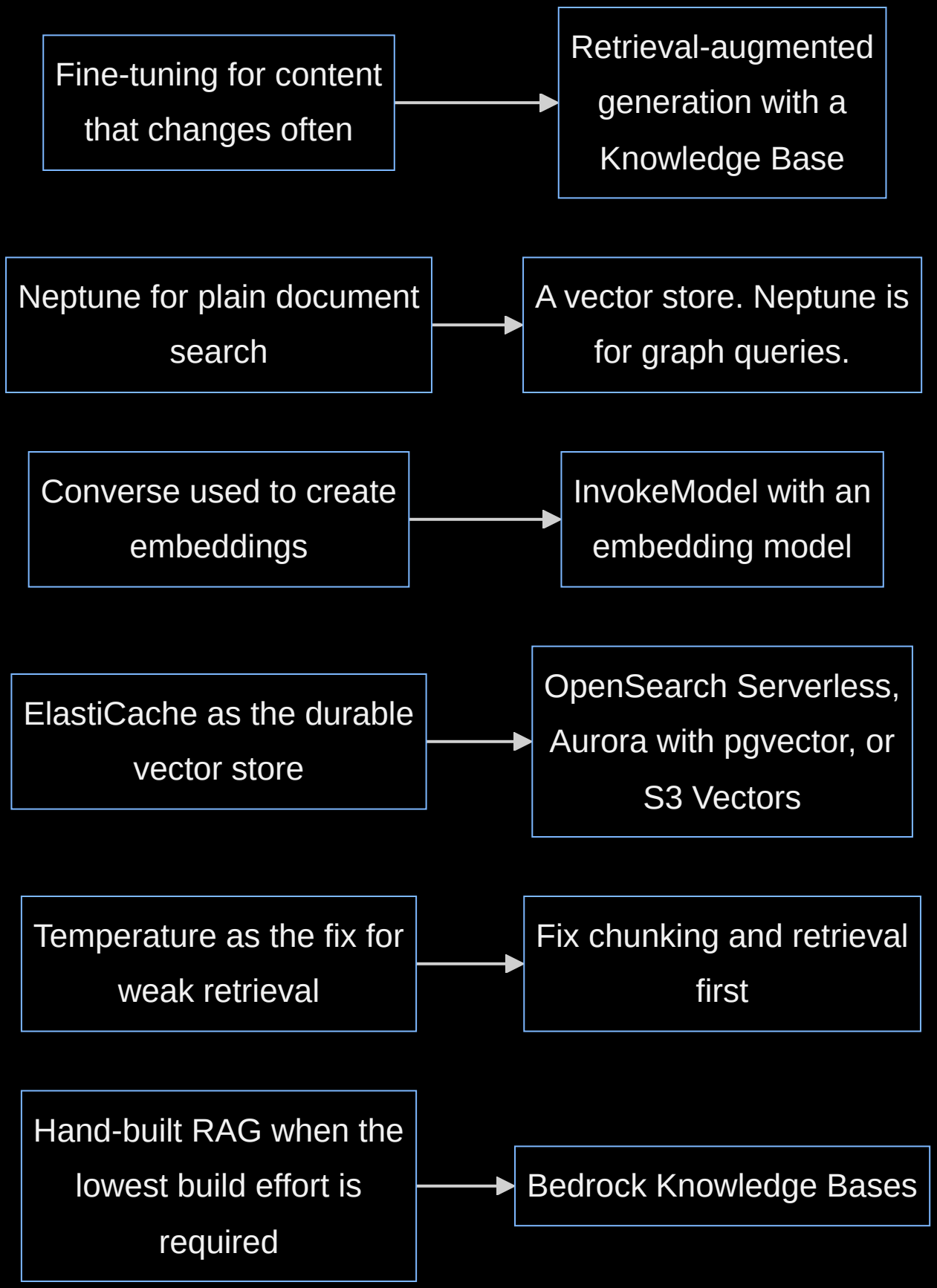


Figure: Wrong idea on the left, correct idea on the right, for retrieval and storage.

- Wrong idea: Fine-tuning for content that changes often. Correct idea: Retrieval-augmented generation with a Knowledge Base.

- Wrong idea: Neptune for plain document search. Correct idea: A vector store. Neptune is for graph queries.
- Wrong idea: Converse used to create embeddings. Correct idea: InvokeModel with an embedding model.
- Wrong idea: ElastiCache as the durable vector store. Correct idea: OpenSearch Serverless, Aurora with pgvector, or S3 Vectors.
- Wrong idea: Temperature as the fix for weak retrieval. Correct idea: Fix chunking and retrieval first.
- Wrong idea: Hand-built RAG when the lowest build effort is required. Correct idea: Bedrock Knowledge Bases.

Decision rule: Use RAG when facts are private or change often.

Extraction and triggers

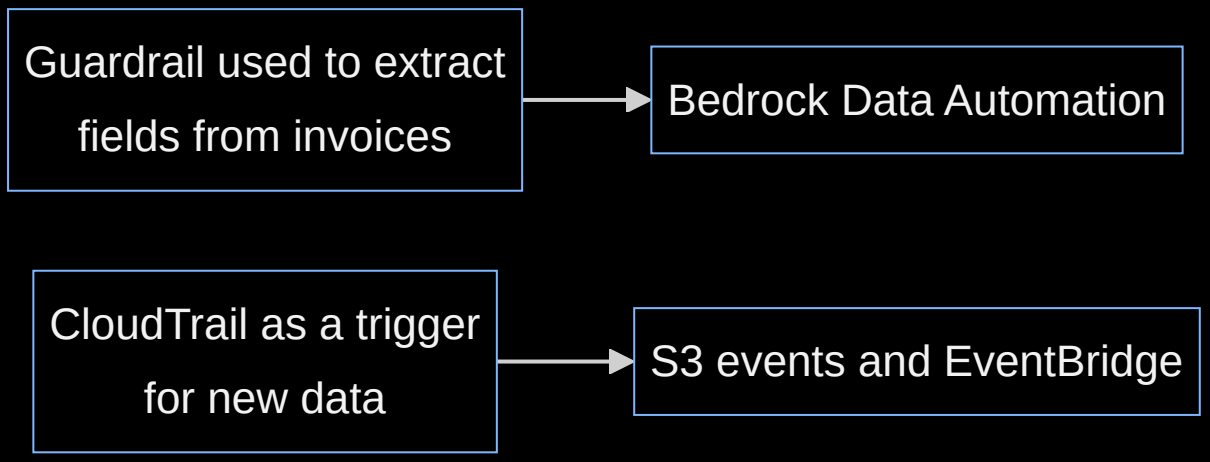


Figure: Wrong idea on the left, correct idea on the right, for extraction and triggers.

- Wrong idea: A Guardrail used to extract fields from invoices. Correct idea: Bedrock Data Automation.
- Wrong idea: CloudTrail as a trigger for new data. Correct idea: S3 events and EventBridge.

Decision rule: A Bedrock guardrail checks input and output.

Safety and governance

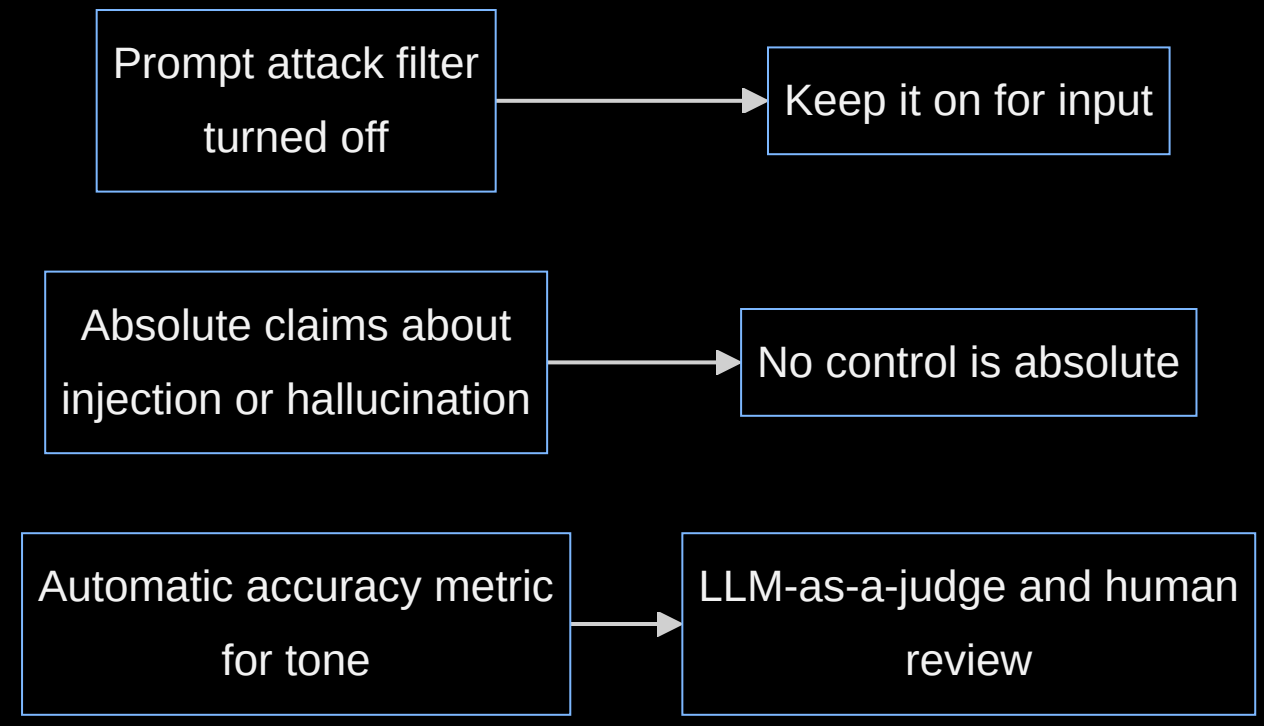


Figure: Wrong idea on the left, correct idea on the right, for safety and governance.

- Wrong idea: Prompt attack filter turned off. Correct idea: Keep it on for input.
- Wrong idea: Absolute claims about injection or hallucination. Correct idea: No control is absolute.
- Wrong idea: Automatic accuracy metric for tone. Correct idea: LLM-as-a-judge and human review.

Decision rule: Calibrate the judge before use.

Agents and versions

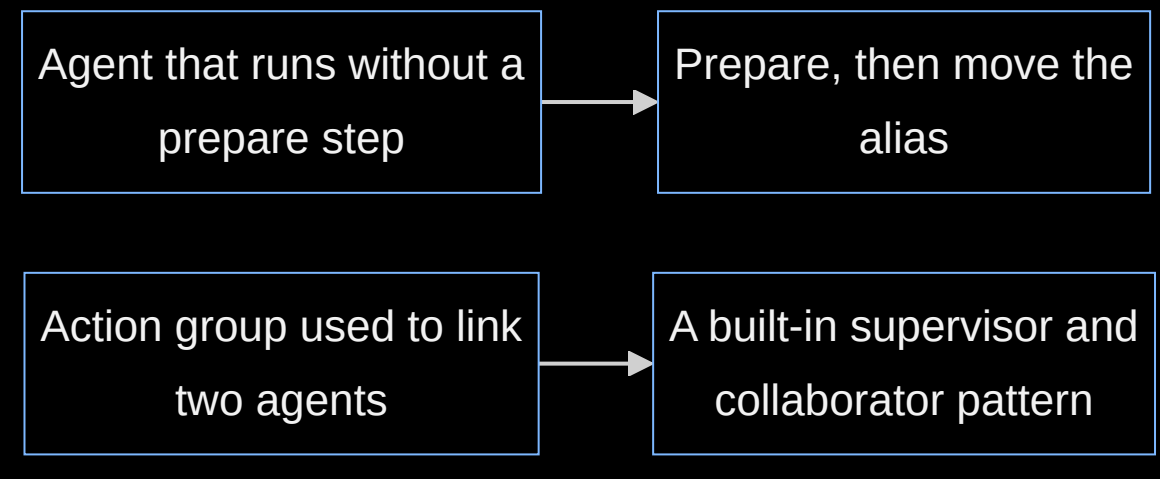


Figure: Wrong idea on the left, correct idea on the right, for agents and versions.

- Wrong idea: Agent that runs without a prepare step. Correct idea: Prepare, then move the alias.
- Wrong idea: Action group used to link two agents. Correct idea: A built-in supervisor and collaborator pattern.

Decision rule: A version is a frozen copy, and an alias points callers to a version.

Throughput and runtime

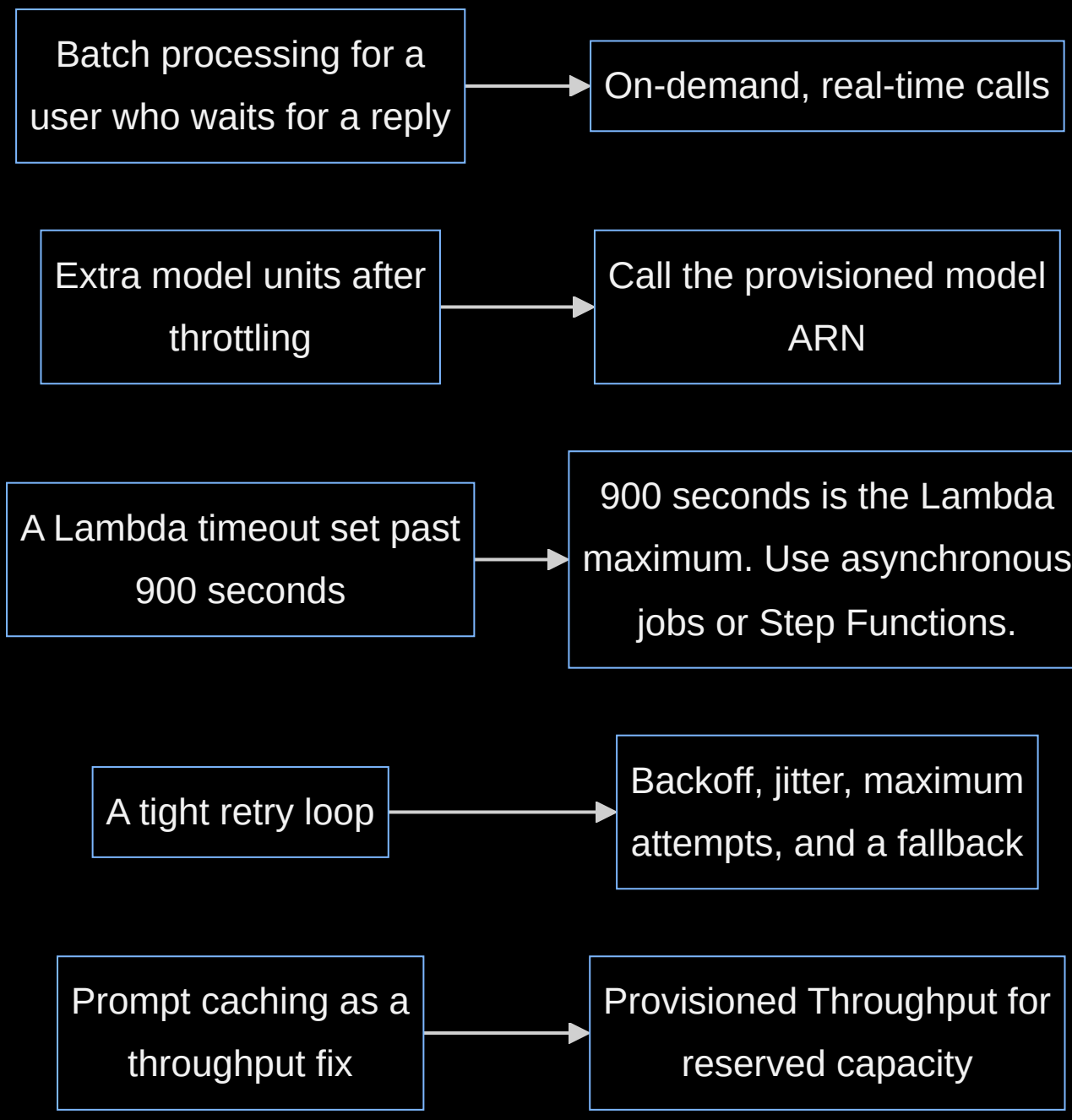


Figure: Wrong idea on the left, correct idea on the right, for throughput and runtime.

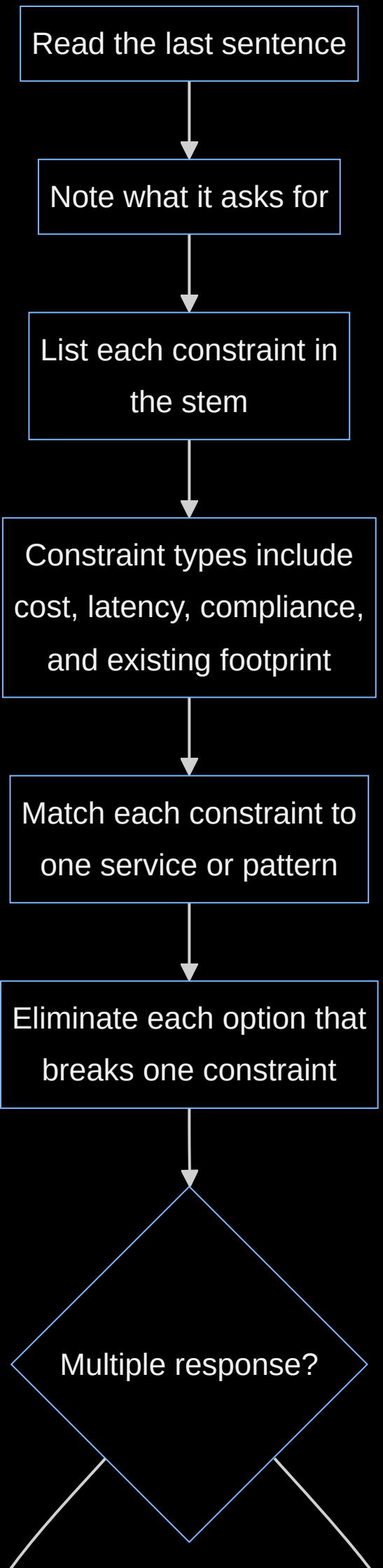
- Wrong idea: Batch processing for a user who waits for a reply. Correct idea: On-demand, real-time calls.

- Wrong idea: Extra model units after throttling. Correct idea: Call the provisioned model ARN.
- Wrong idea: A Lambda timeout set past 900 seconds. Correct idea: 900 seconds is the Lambda maximum. Use asynchronous jobs or Step Functions.
- Wrong idea: A tight retry loop. Correct idea: Backoff, jitter, maximum attempts, and a fallback.
- Wrong idea: Prompt caching as a throughput fix. Correct idea: Provisioned Throughput for reserved capacity.

Decision rule: Use Provisioned Throughput for a steady throughput. It is billed hourly.

Method for each question

Source: repo guide, Part 8 Method for each question


Read the last sentence
Note what it asks for
List each constraint in the stem
Constraint types include cost, latency, compliance, and existing footprint
Match each constraint to one service or pattern
Eliminate each option that breaks one constraint
Multiple response?

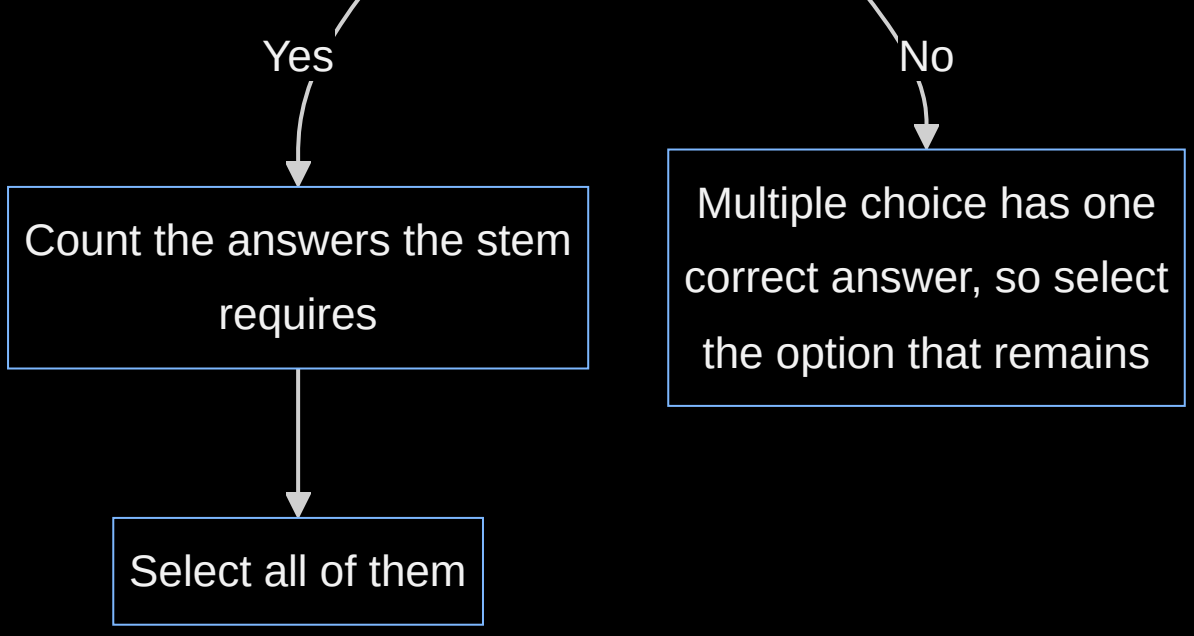


Figure: Reading steps for each question, in source order.

Decision rule: Eliminate each option that breaks one constraint.

Trap: Keep an option that breaks one listed constraint.

Trap: Fewer answers than the multiple response wording requires.

Exam tip: Read the last sentence first. It names what the wording asks for.

Glossary

Source: repo guide, Part 8C Glossary

Short definitions for the terms that the exam uses.

Token

A piece of text the model reads or writes.

Context window

The fixed limit of tokens in one call.

Embedding

A list of numbers that represents the meaning of text.

RAG (retrieval-augmented generation)

Retrieve source text first, then send it with the question to the model.

Chunking

Splitting documents into pieces before embedding.

Knowledge Base

Bedrock managed RAG that syncs a data source, chunks and embeds it, retrieves, and can return citations.

Hybrid search

Combines keyword matching and vector similarity in one query.

Reranker

A model that scores retrieved candidates for relevance before generation.

Temperature

A sampling setting where lower values make the next-token choice more deterministic.

Top K

The number of most likely candidate tokens the model considers for the next token.

Top P

The share of the cumulative probability distribution the model considers for the next token.

Max tokens

The maximum number of tokens the model returns.

Prompt caching

Reuses an exact prefix marked with a cache checkpoint.

Batch inference

Offline jobs with input and output in S3.

Provisioned Throughput

A purchase of model units for a steady throughput.

Cross-Region inference profile

A profile that routes requests across Regions in one geography for throughput and availability.

Intelligent prompt routing

Routes each request between two models in one family, by predicted response quality and cost.

Guardrail

A Bedrock control with six policy types that checks input and output.

DRAFT, version, alias

DRAFT is the editable agent state, a version is a frozen copy, and an alias points callers to a version.

MCP (Model Context Protocol)

A standard for agents to find and call tools.

Cedar

The policy language that Amazon Verified Permissions uses to decide if a request is allowed.

Cognito user pool

Authenticates users and issues JSON web tokens (JWTs).

Cognito identity pool

Exchanges a trusted token for temporary AWS credentials.

CloudFormation stack

A set of resources created and deleted together from one template file.

Textract

A document service that detects and extracts text, forms, and tables.

LLM-as-a-judge

A model scores another model's output against criteria.

Open checks before exam day

Source: repo guide, Part 9 Gaps, limits, and checks

These checks are open in this build.

- Confirm the Domain 2 API Gateway integration timeout. The quotas page does not list it. The 29 second value is Not in source.
- Confirm the Knowledge Base web crawler end date. The docs page did not show one. The date is Not in source.
- Confirm the batch discount on the Amazon Bedrock pricing page. The docs page gives no percentage.

Decision rule: Re-check each open check before exam day.

Trap: An unconfirmed timeout or date used as fact.

Gap closure: concepts to confirm before the exam

Each concept below came up in the audit as a point the guide states only in part, or states without its decision rule. Read the matching part of this guide and say the rule aloud before you move on. Concept names only. Every fact comes from the parts above or from the repo study guide.

Agents and AgentCore

- Agent loop limits: four bounds on each run (step count, time, resource scope per tool call, and circuit breaking on repeated tool failures)
- Tool schema that names each input
- Build-time agent APIs versus runtime orchestration

- Console build sequence for an agent: service role, action groups, knowledge base, test window, and multi-agent setup
- Time limit of the executor Lambda function behind an action group
- Gateway as one entry point for agents, tools, other agents, and models, and not only a model router
- Enforcement points on each gateway call: guardrail identifier, IAM policy, logs and audit from the gateway, and team cost tags
- Identity placed in front of each tool call
- Streamable HTTP as the general transport for remote MCP servers
- Cedar policy language and whether AgentCore Policy uses it
- Agent trace versus invocation log versus audit record for one agent session
- Storing trace events with each answer so one answer can be reconstructed later
- Agent draft pause for human approval through a task token, with a heartbeat
- Three tool paths: Converse client-side tool use, action groups inside an agent, and gateway tools through MCP
- Choice between a framework and a managed agent
- Agent Squad orchestration with queues and classifier-based routing for independent specialist agents
- Agent, fixed flow, or Standard workflow for each kind of step
- Actor identity, session scope, memory strategies, and flushing of batched memory writes
- Expiry check at read time for TTL records
- Gateway target types and contracts: OpenAPI operation identifiers, Lambda target input schema, HTTP target as proxy only, and outbound authorization b
- MCP primitives: tools, resources, and prompts
- One tool per OpenAPI operation, with no generic dispatcher action

- Action group schema types, parameter elicitation for required fields, and return-control mode
- Agent evaluation modes: online sampling, on-demand per trace, batch per saved session, and trajectory checks
- Optimized workflow integration that calls a deployed agent runtime and returns the final answer with usage

Bedrock model access

- Inference parameters: temperature, Top P, Top K, and maximum output tokens. Top K limits the candidate count. Top P limits the probability mass. Tempe
- Model-specific fields. additionalModelRequestFields passes a field such as top_k. additionalModelResponseFieldPaths reads a model-specific response fi
- Cascade routing. Try the small model first. Escalate when the answer is uncertain or the query is complex. The router needs a signal. Without a signal
- Implicit caching is on by default and is best effort. It works for on-demand calls only. Explicit caching uses cache checkpoints in the request
- Cost attribution. Request metadata holds up to 16 key-value pairs. Request metadata is not a cost allocation tag. An application inference profile per
- Inference phases. Prefill reads the input and sets time to first token. Decode writes the output one token at a time. Time to first token depends most
- Input and output tokens both count toward the context window. Both are billed. Output tokens usually cost more per token than input tokens
- Runtime metric names for model calls: InputTokenCount, OutputTokenCount, and Invocations. InvocationThrottles is the throttle metric
- Invocation log retention and tool choice. Logs stay until the logging configuration is deleted. For a long audit trail, use S3 with Athena. CloudWatch

- Fallback after retries. Fall back to a second model or a cached answer. A circuit breaker stops calls to a failing model. Graceful degradation returns
- The prompt attack filter does not evaluate tool results or tool definitions
- Approval and CI/CD. Approved can start CI/CD deployment when a project template is used. The approval status values are Approved and Rejected
- Topics the repo guide does not state. These are per-account or per-Region logging scope, Provisioned Throughput on cross-Region profiles, On-demand Re

Knowledge Bases and RAG

- NONE chunking and the sentence rule of default chunking
- Hierarchical chunking child and parent roles
- Deleted documents leave the vector store on sync
- Knowledge path, ingest path, and query path
- ElastiCache durability exception
- OpenSearch shard defaults and fixed primary shard count
- OpenSearch vector quantization options
- S3 event loop risk and Lambda invocation permissions
- Error codes for knowledge base API calls
- Lineage and audit trail for knowledge base answers
- Contextual grounding score for each answer
- Fine-tuning does not fix facts that change
- Retrieved documents as a route for injected instructions

Decision logic across domains

- Automated Reasoning checks as a guardrail policy with its own limits
- Prompt leakage detection is available on the Standard tier only
- Domain 2 task 2.4 routing options: static configuration, Step Functions for content-based routing, and metric-based routing

- Amazon Translate as a look-alike service
- Lambda Managed Instances asynchronous limit of 90 minutes
- Bedrock Agents closure and AgentCore as the documented alternative
- English-only scope of intelligent prompt routing in the glossary
- The six guardrail policy types named in the glossary

Guardrails and safety

- ApplyGuardrail applies a guardrail to text from any model and returns an allow or block result
- Prompt attack input tagging: user input must be tagged for InvokeModel and InvokeModelWithResponseStream, or it is not checked
- Prompt leakage detection runs on the Standard tier only
- Automated Reasoning checks: detect mode only, English only, no streaming, no prompt injection detection, and use together with content filters
- The Converse response includes a trace object with guardrail results when a guardrail runs
- Citations are attribution, not proof of truth. Contextual grounding returns a score for each answer
- Object Lock hold: while a hold is on, the retain-until-date is dynamic. After release, the date is fixed at release time plus the duration
- Object Lock minimum retention: the later of release time plus the duration and the minimum retain-until-date applies
- Object Lock retention decrease: a shorter duration does not reduce the retain-until-date
- SCP rollout: test the guardrail SCP on one organizational unit first
- StackSets deploy the guardrail, and an SCP denies model calls that omit the guardrail identifier
- SCP limits: SCPs do not affect the management account or service-linked roles, and a user still needs an identity policy that allows the action

- Defense in depth includes grounding checks and a post-processing step for sensitive audiences
- Masking the output does not mask the stored invocation log

Native compute and integration

- Retry rules by error class: 429 quota error, 503 and 529 capacity errors, 400 validation error, 403 missing IAM permissions, 404 wrong model ID or res
- Idle TCP limit of 350 seconds on NAT gateways, interface VPC endpoints, and Network Load Balancers
- Streaming permission: ConverseStream needs its own streaming permission, separate from the Converse permission
- Invocation logging covers ConverseStream and InvokeModelWithResponseStream, not only Converse and InvokeModel
- Knowledge base sync on S3 events: the guide names no built-in S3 event trigger for sync jobs
- Step Functions heartbeat (HeartbeatSeconds), SendTask, and cross-account task tokens
- Step Functions Map and Parallel states
- Step Functions payload size limit and S3 offload for large payloads
- Step Functions optimized Bedrock integration and its Body versus Input parameters
- Lambda reserved concurrency
- Lambda failure destinations and idempotency
- Lambda aliases and weighted routing
- API Gateway stage variables and canary releases
- HTTP API payload format version and HTTP API limits
- REST response STREAM transfer mode
- Lambda function URL response streaming
- WebSocket post-to-connection API for sending stream chunks to a client

- SQS event source mapping and worker concurrency
- Retry scope for a stream that fails before the first chunk
- AWS SDK retry modes

Native data and storage

- Aurora-backed knowledge base placement. The Aurora cluster must sit in the same account as the knowledge base, and a central cluster in another accoun
- Store selection for a knowledge base. Hybrid search support, binary vector support, the required engine, and SQL reporting each depend on the store ty
- Binary embeddings limit. Serverless and managed OpenSearch accept binary vectors. S3 Vectors and relational stores do not, in this setup
- OpenSearch vector field setup. The k-NN index setting must be on, and the vector field dimension must match the embedding model output
- Result size for k-NN queries. The size parameter and the k value set one global result set for each query
- Shard layout fixed at index creation. A wrong layout needs a new index with the planned primary count and a full reload
- Neural ingest and search pipelines with a Bedrock embedding connector, and a cross-account ML connector for an existing search domain
- Hybrid query with keyword and neural clauses, combined by a search pipeline that normalizes the two score sets
- An existing managed OpenSearch domain as a knowledge base store. The storage type must match, each knowledge base points to one index, and fields need
- Filterable text field in an OpenSearch Serverless collection, which hybrid matching in a knowledge base needs
- Metadata filtering on a custom OpenSearch store depends on the engine of the index

- DynamoDB Streams change capture. Insert, update, and delete records feed a derived vector index, and the source key identifies each document
- TTL deletion timing. Expired records can remain until TTL removes them, so code must check the expiry time on read
- Epoch-seconds Number attribute for the TTL key, plus conditional writes for concurrent updates to session state
- Governed lake access. Register storage locations, grant data access permission to each reader, and close direct storage read paths
- Glue Data Quality modes. Catalog evaluation generates rule recommendations, and ETL job evaluation writes row-level results and can block records
- Glue Data Quality run metrics. Metric publishing is off by default, and results can go to S3 for remediation after a run
- Nested or list-type columns. These need flattening before scalar data quality rules run
- Lineage needs more than the catalog. Catalog records and citations on each output together show which documents support an answer
- S3 Object Lock modes. Compliance and governance modes differ in who can override retention, and the bucket can set a default retention period
- Retention lock and expiry. Object Lock blocks deletion until the retain-until date, and lifecycle expiration removes objects at end of life
- Versioning alone is not write-once. A DELETE with a version ID removes that version
- CloudTrail data events for S3 object activity. These record object-level reads and writes
- Object tags as an approval state. IAM condition keys on tags gate reads of approved objects
- S3 Inventory timing. Inventory reports are too slow for freshness within minutes

- Revision identity. Version IDs identify the stored version of a source object, and entity tags do not reliably identify revisions
- Macie for data at rest. Macie finds sensitive data in S3 objects, and it does not screen live model traffic
- Semantic response cache on ElastiCache for Valkey, with the source document version in the cache key
- Reporting that joins evaluation scores with per-request metadata, shown in Athena and QuickSight
- Co-located relational vector search. Access grant tables and vector search run in one query, so permission changes apply at once

Operations, cost, and observability

- Bedrock error classes that set the retry or fix path: 429 quota, 503 and 529 capacity, 400 validation, 403 IAM permissions, 404 wrong model or resourc
- Token capacity limit that throttles below the request-count quota when prompt sizes vary
- Prompt cache usage fields and cache billing
- Prompt cache time to live rules
- Implicit and explicit prompt caching modes
- Provisioned Throughput billing and commitment terms
- Time to first token, its measurement, and its drivers
- Multi-model comparison with runtime cost and latency metrics
- Quality measured per request type before routing traffic across model tiers
- Input token reduction by pruning context
- Token-driven capacity planning instead of request counts
- Intelligent prompt routing scope and constraints
- Model invocation logging setup, destinations, and record contents
- Agent trace records for session review and per-answer lineage
- Model Monitor availability status

- Approval-gated model promotion with canary rollout and automatic rollback
- Pipeline stage conditions with a rollback outcome
- Concepts the repo guide does not support, which the fragments must not state as fact

SageMaker AI

- Hosting a model too large for one accelerator by splitting it across the GPUs of one instance in a large model inference container
- Container-wide batch and queue settings compared with per-model settings when several models share one serving container
- Startup failure of a real-time endpoint when very large weights take too long to load into the container
- Several models on one real-time endpoint with separate resource and scaling settings, compared with a multi-model endpoint
- Asynchronous endpoint that scales to zero instances based on queue backlog
- Target tracking on invocations per instance for a real-time endpoint, with tested instance limits
- Canary and linear traffic shifting on endpoint deployments with automatic rollback from CloudWatch alarms
- Promoting fine-tuned adapter packages that run in a custom inference container through the same approval gate
- Model Registry as the single record of candidate, approved, rejected, and retired versions, with object storage lacking an approval state
- Production variants or shadow variants used for live comparison of candidate models before promotion
- SageMaker Experiments for tracking training runs, which does not gate a release
- Model Monitor drift reports do not trigger rollback, which uses a CloudWatch alarm and an earlier approved version

- Bias drift monitoring for a tabular endpoint with a baseline and alarms
- Model card fields beyond intended use, limits, and evaluation results, including card creation per approved version in a release pipeline and review o
- Data lineage held in a data catalog, kept separate from the model card
- Built-in algorithms other than gradient-boosted trees
- Importing a model trained outside Bedrock into Bedrock for on-demand runtime calls, as an alternative to SageMaker hosting
- Billing for idle capacity on real-time and provisioned endpoints compared with per-request serverless hosting for sporadic traffic
- Serverless endpoints for GPU serving containers

Security, network, and identity

- Endpoint-aware IAM condition for traffic that uses an interface endpoint, instead of a source address condition
- SAML 2.0 sign-in, SCIM provisioning, and group-based account assignment from an external identity provider
- IAM conditions and SCPs restrict callers but do not change where a cross-Region profile routes
- Integration IAM role that lets an HTTP API call the EventBridge PutEvents action
- bedrock-mantle endpoint has no documented invocation logging path

Concepts listed: 165. Source: repo guide, audit of each part.

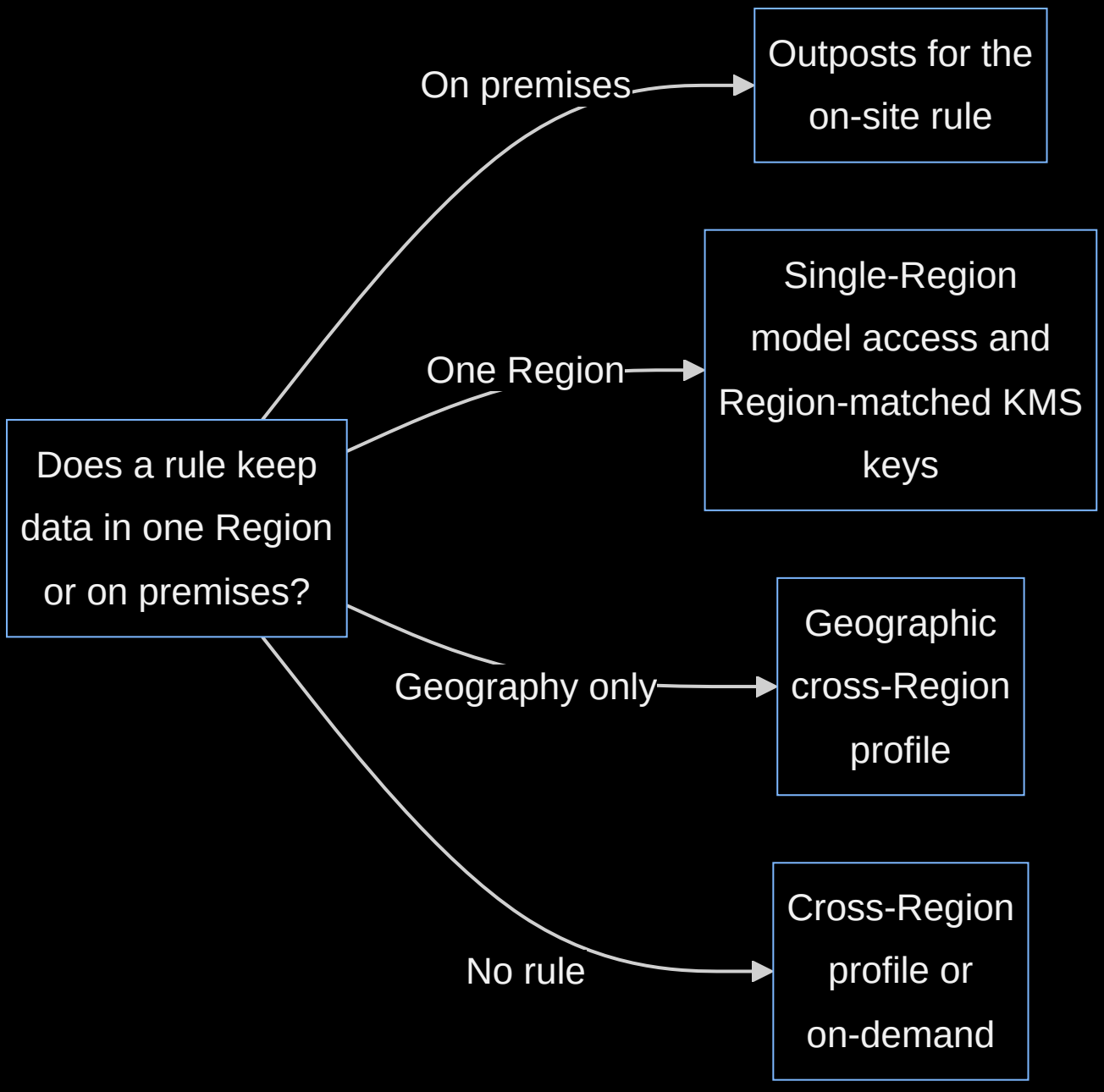
Scenario catalog: every constraint family and the choice it drives

Each family below names the constraint, the signal words in a scenario, the choice, the combination, the forbidden choice, and one trap. Read the family

first. Then match the scenario to it. Signal words are concepts. Use them to recognize the constraint, not to match a sentence.

Decision rule: a scenario can hold several families. List every family the wording triggers before you choose a service.

A. Data location and residency

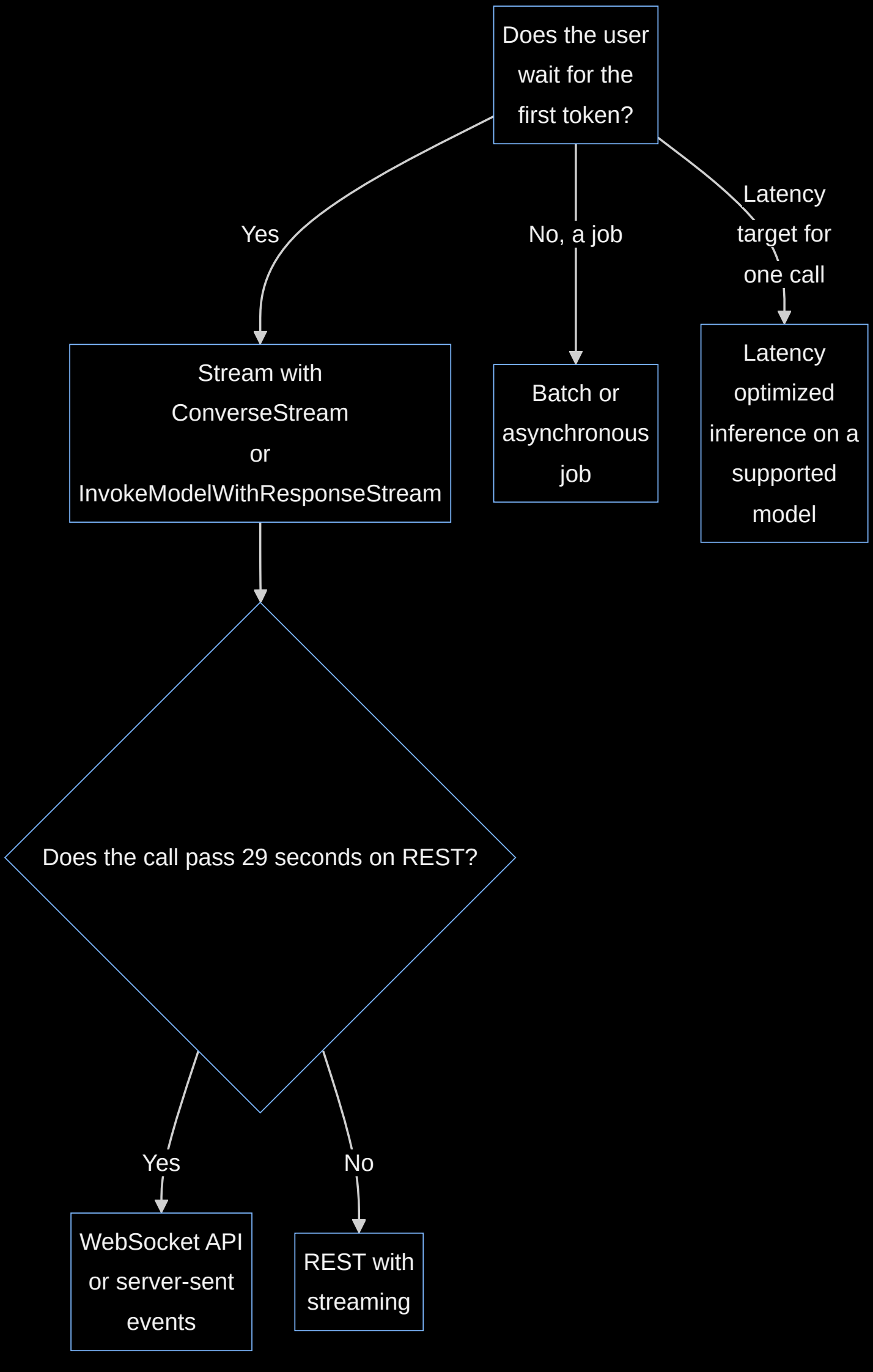


Signal words: stays in the country, sovereign, residency, on site, one Region, EU boundary.

Choose a geographic profile for a geography rule. Choose Outposts when a rule keeps data on your premises.

Trap: a global profile can route outside a strict EU boundary. It does not meet an EU residency rule.

B. Latency and streaming

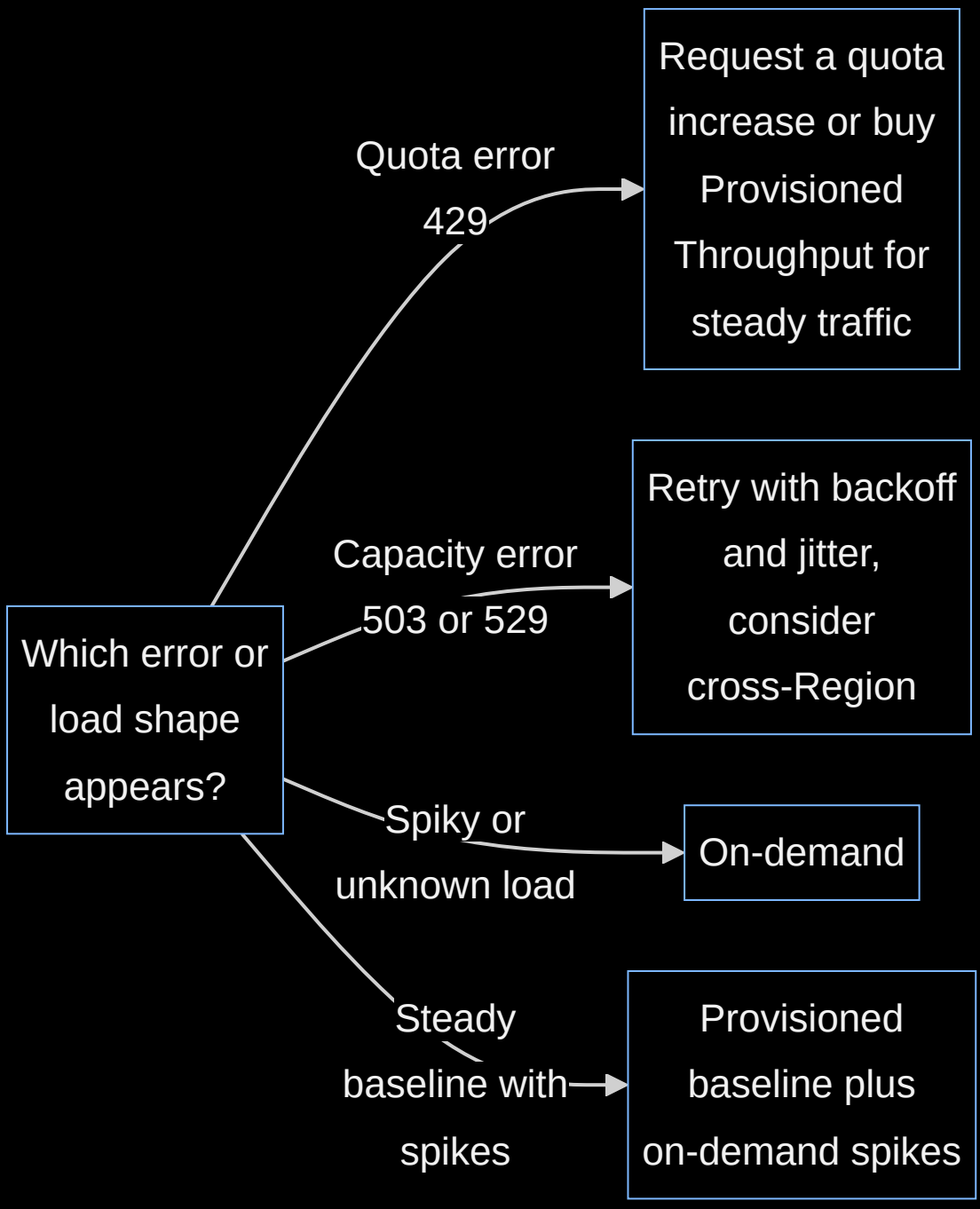

Does the user wait for the first token?
Yes
No, a job
Latency target for one call
Stream with ConverseStream or InvokeModelWithResponseStream
Batch or asynchronous job
Latency optimized inference on a supported model
Does the call pass 29 seconds on REST?
Yes
No
WebSocket API or server-sent events
REST with streaming

Signal words: first visible token, perceived speed, real-time chat, long answer, gateway timeout.

Stream for perceived speed. Use WebSocket or an asynchronous job when the call can pass the gateway limit.

Trap: streaming does not extend a REST timeout. A larger Lambda timeout does not change the gateway limit.

C. Throughput, quota, and capacity

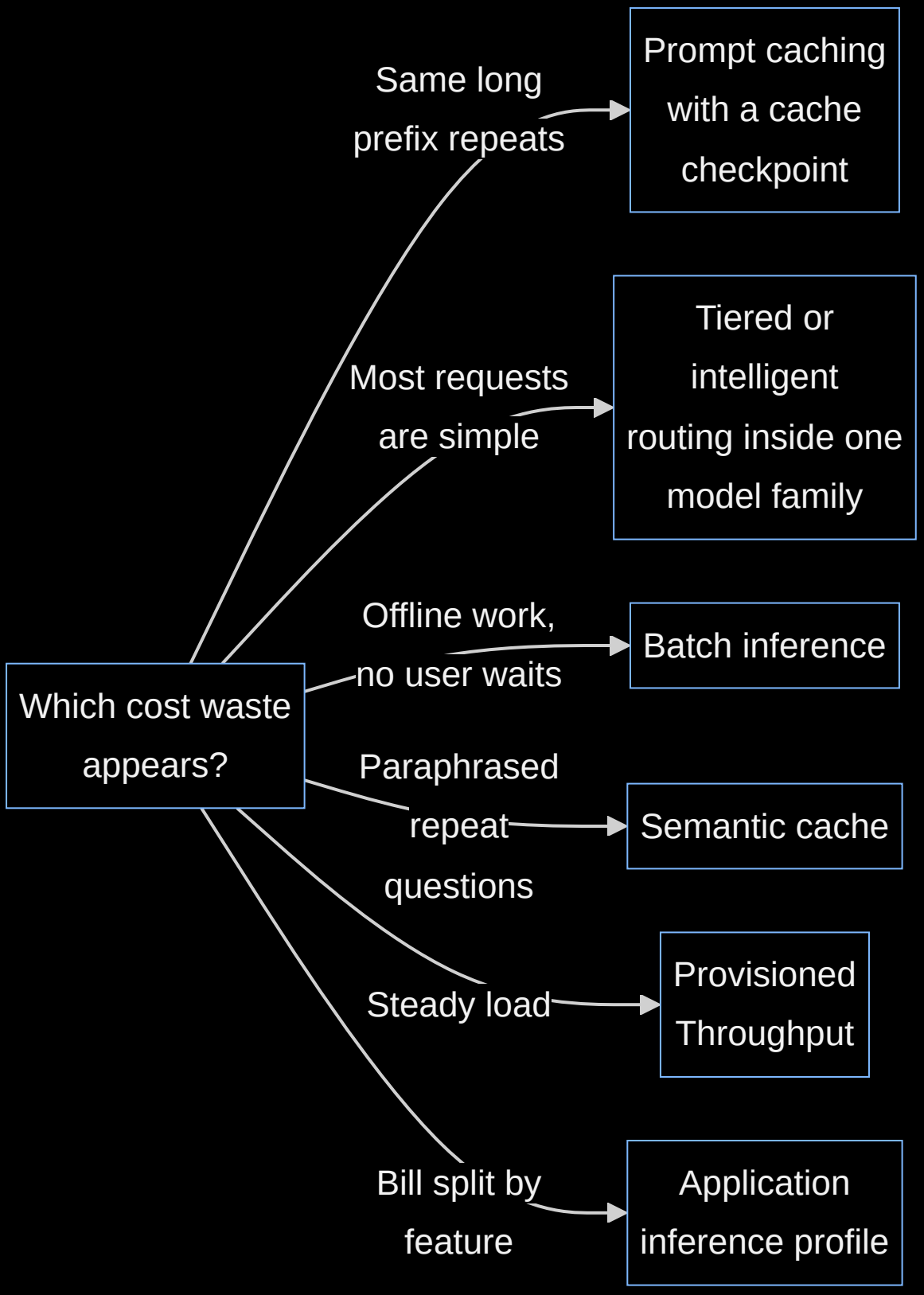


Signal words: throttled, quota, tokens per minute, spike, steady, reserved, guaranteed capacity.

A 429 needs quota or reserved capacity. A 503 or 529 needs backoff with jitter. A 400 needs a request fix and no retry.

Trap: more model units do not fix a call that still uses the base model ID. Call the provisioned model ARN.

D. Cost

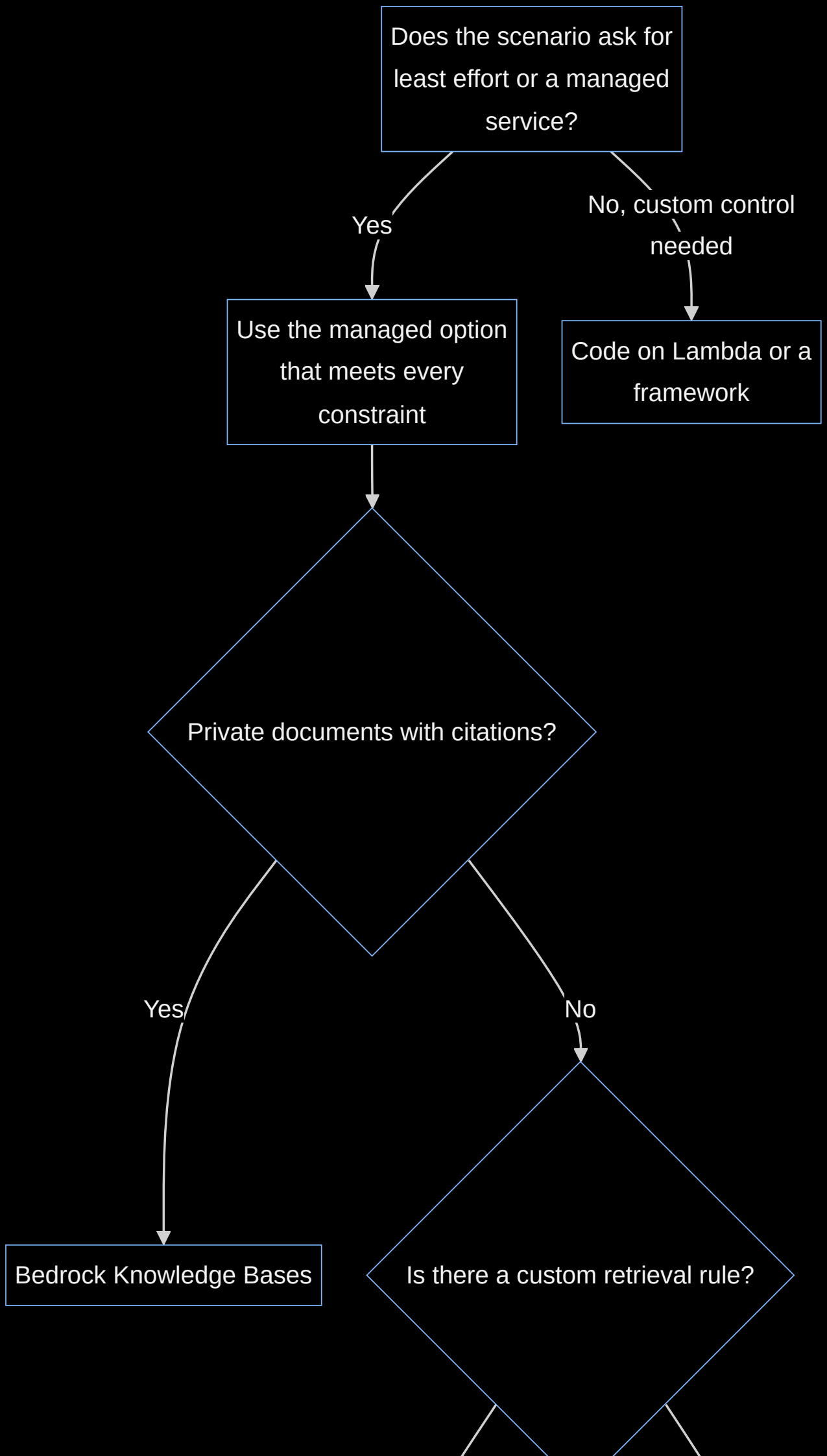


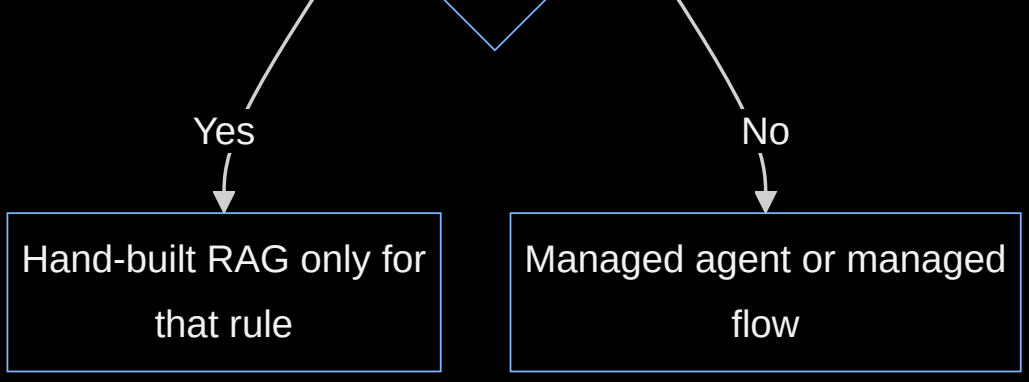
Signal words: repeated prefix, cheap enough, offline, nightly, paraphrase, bill by team.

Pick the lever that matches the waste. Do not buy capacity to fix long prompts.

Trap: request metadata is not a cost allocation tag. Use an inference profile when finance needs the bill split.

E. Least operational effort and managed preference


Does the scenario ask for least effort or a managed service?
Yes
No, custom control needed
Use the managed option that meets every constraint
Code on Lambda or a framework
Private documents with citations?
Yes
No
Bedrock Knowledge Bases
Is there a custom retrieval rule?

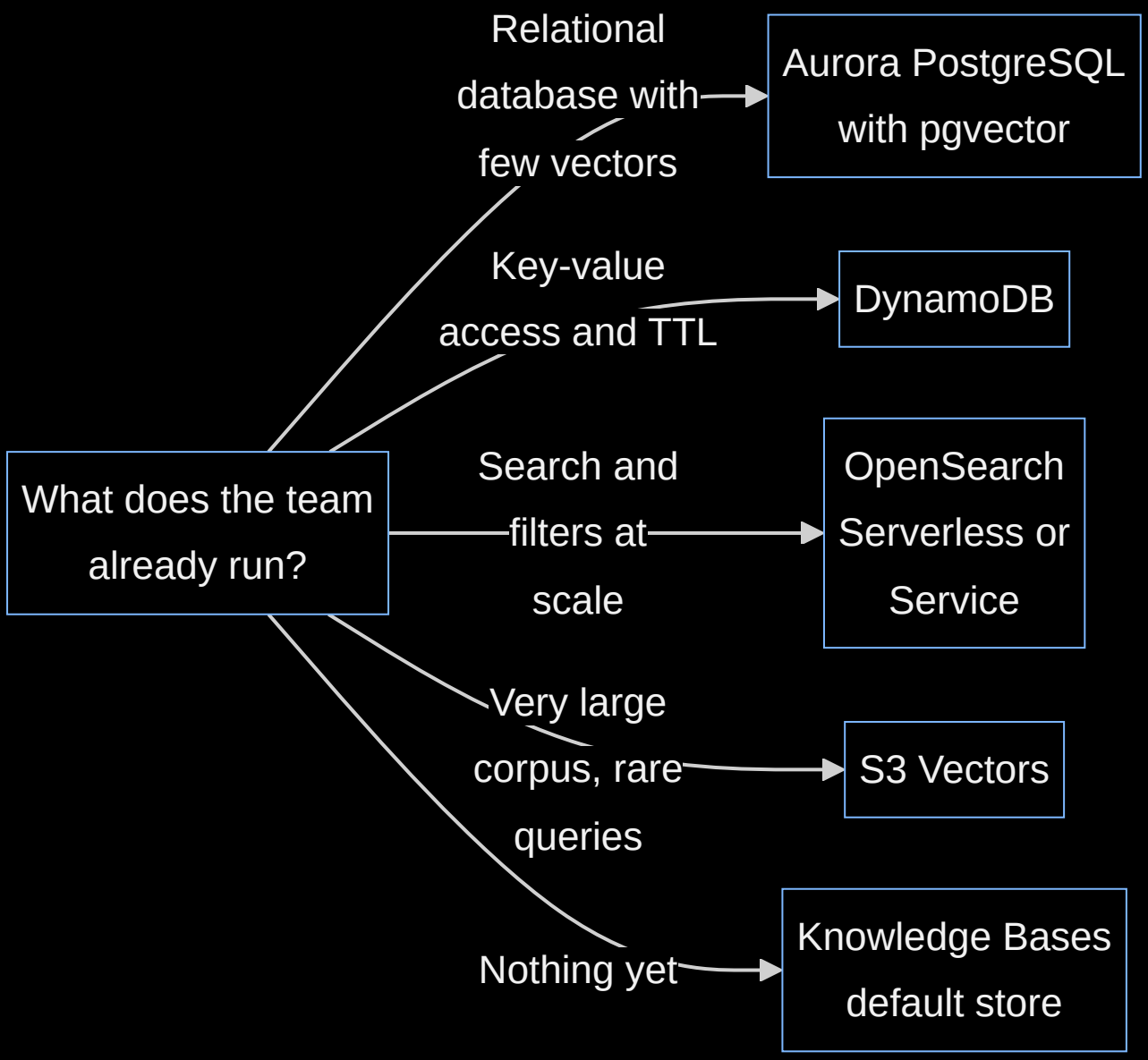


Signal words: least effort, least operational overhead, fully managed, minimal code, no infrastructure to run.

A managed option wins when it meets every stated constraint. A custom build needs a stated requirement that the managed option cannot meet.

Trap: a hand-built pipeline is wrong when the scenario asks for least effort.

F. Existing footprint

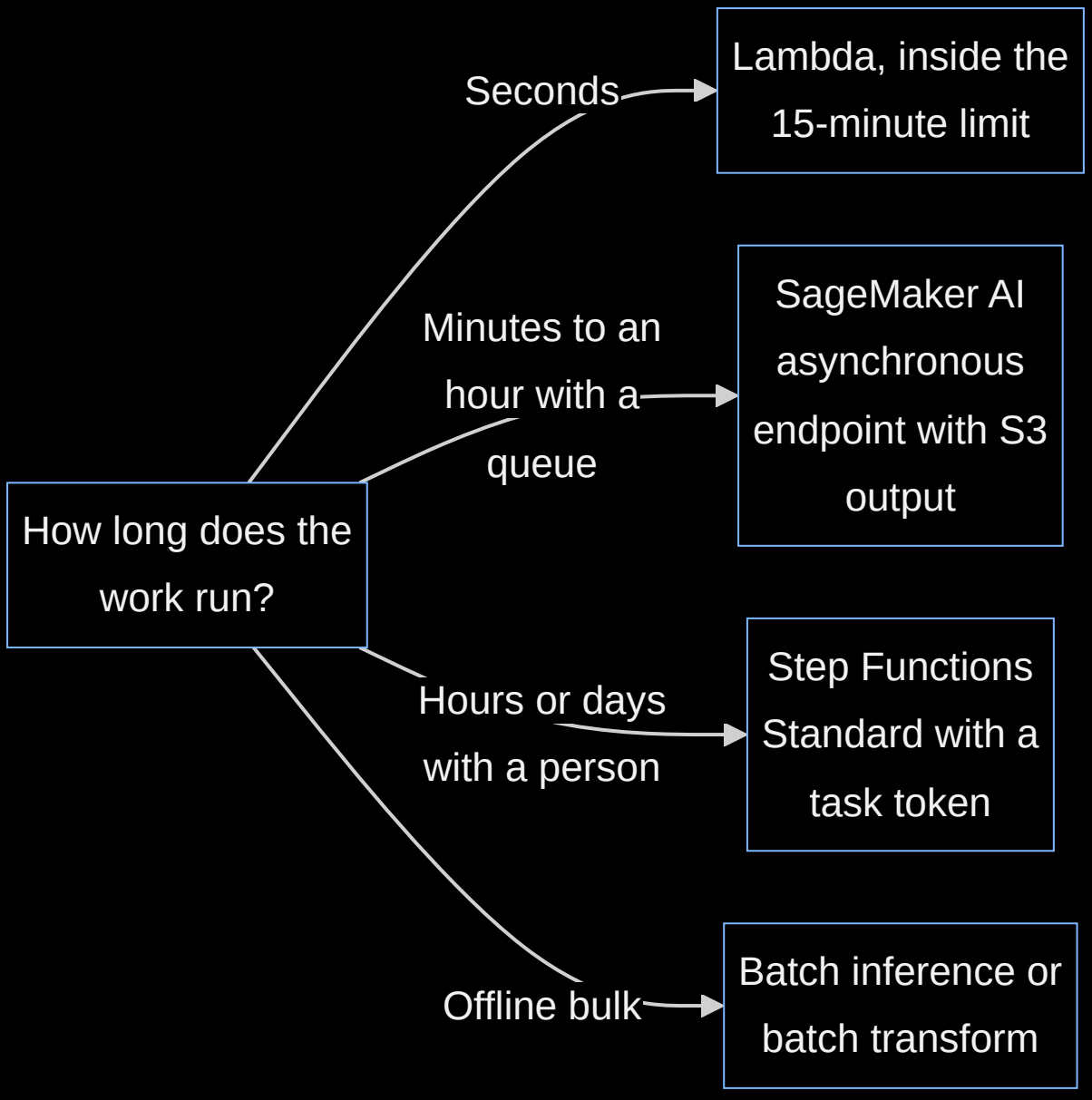


Signal words: already runs, existing Aurora, existing RDS, team knows, on-premises data center.

Reuse the existing store when it meets the access pattern. Add a new store only when the pattern fails.

Trap: ElastiCache is a cache, not a durable store of record.

G. Payload size and job duration

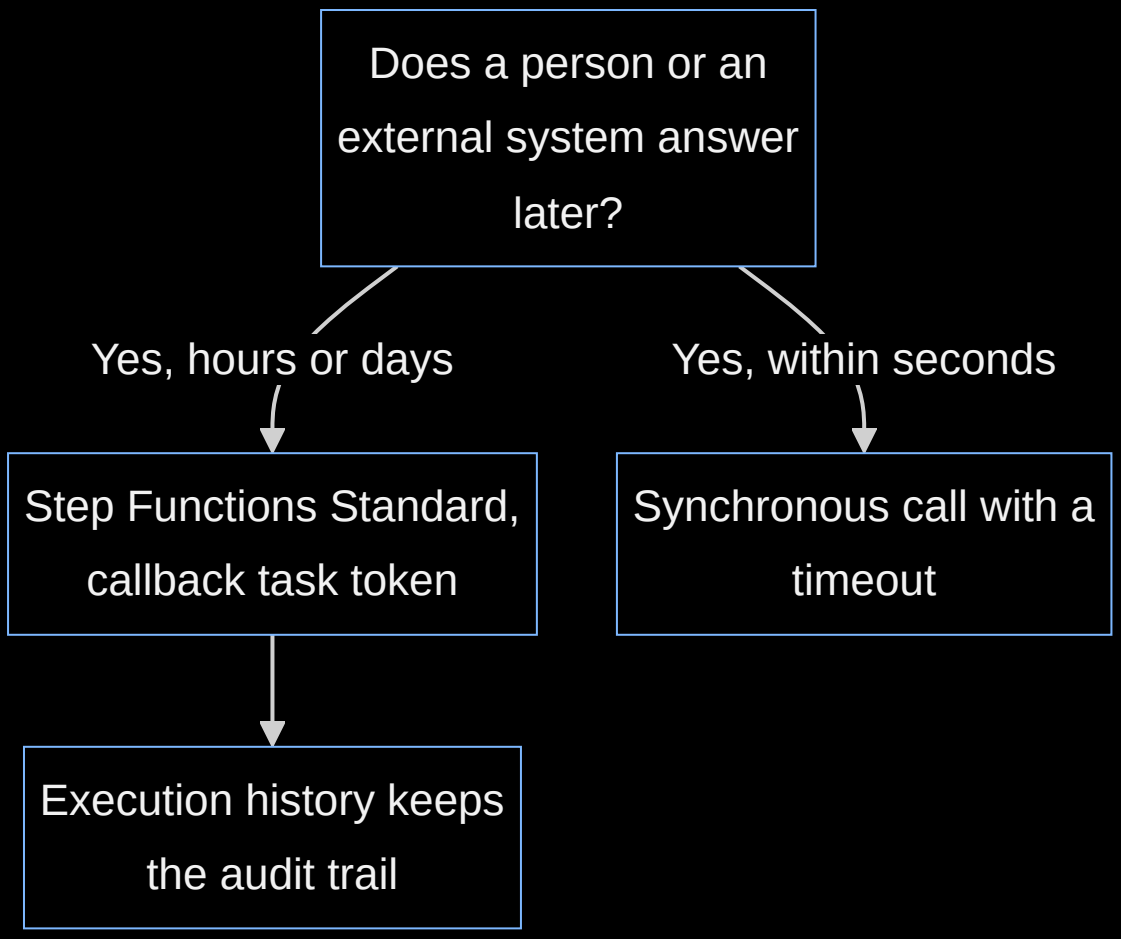


Signal words: long-running, large payload, queue, bulk, nightly, minutes.

Keep each Lambda step short. Move long flows to Step Functions or an asynchronous job.

Trap: a real-time endpoint for a 45-minute job is the wrong pick.

H. Waits and human approval

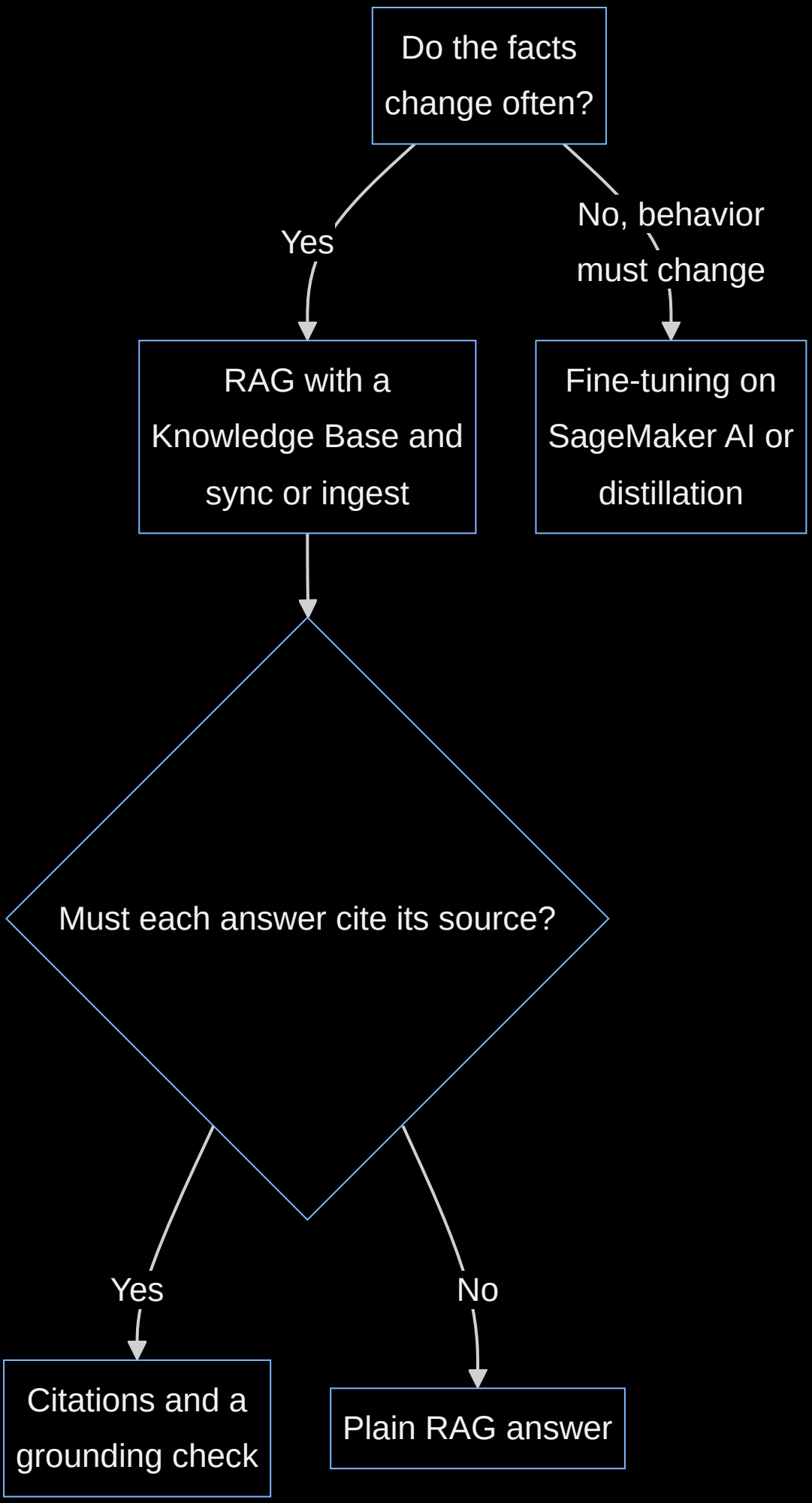


Signal words: approval, sign-off, waits for a person, days, human in the loop.

A wait for a person needs a durable workflow. A Lambda that waits pays for the wait and hits the limit.

Trap: Express workflows do not support the callback pattern.

I. Knowledge freshness and grounding

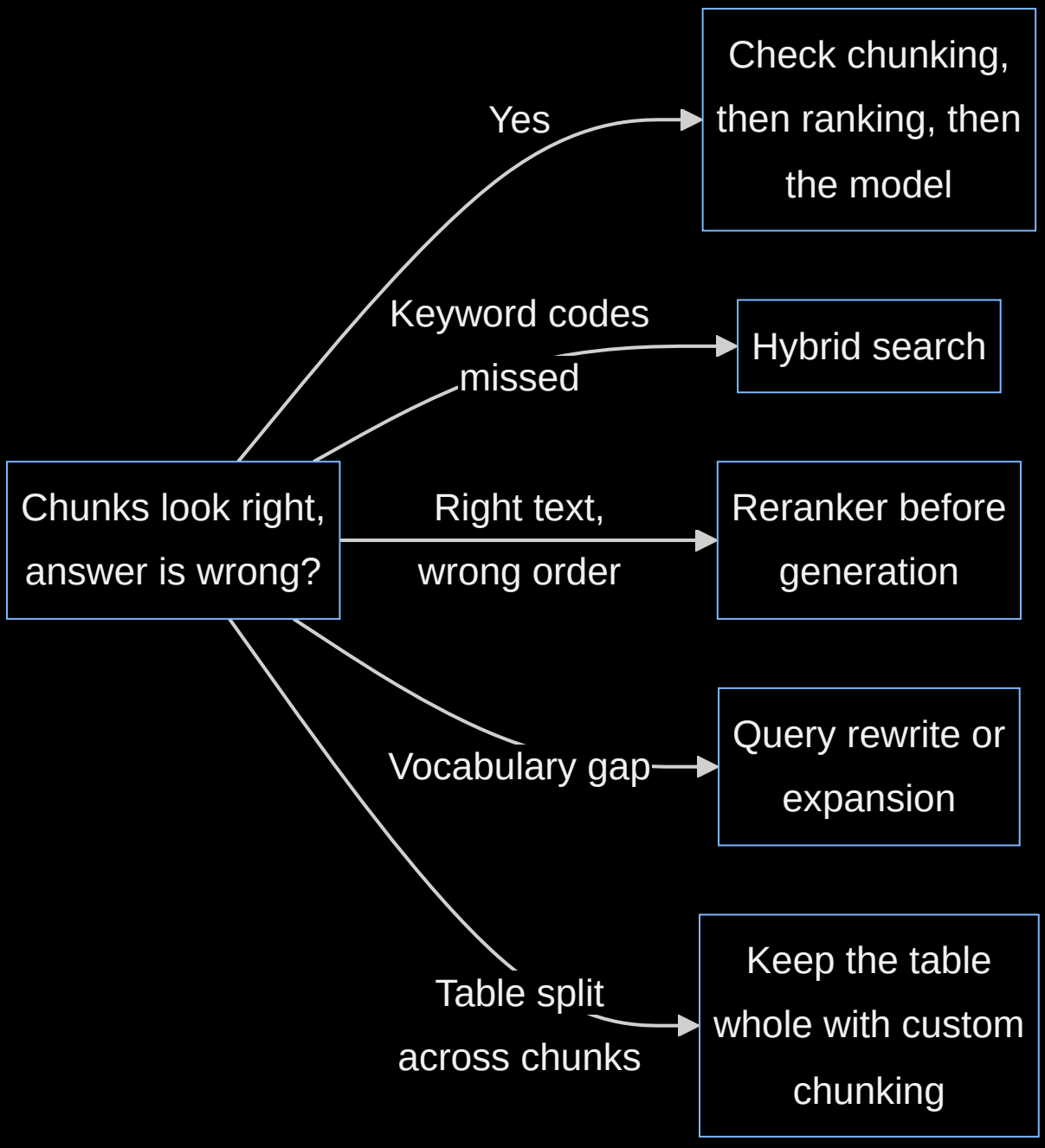


Signal words: current, changing documents, cite, source, audit trail, stable format at high volume.

Changing facts go to retrieval. Stable behavior goes to weights.

Trap: fine-tuning on documents that change is wrong. Use RAG.

J. Retrieval quality

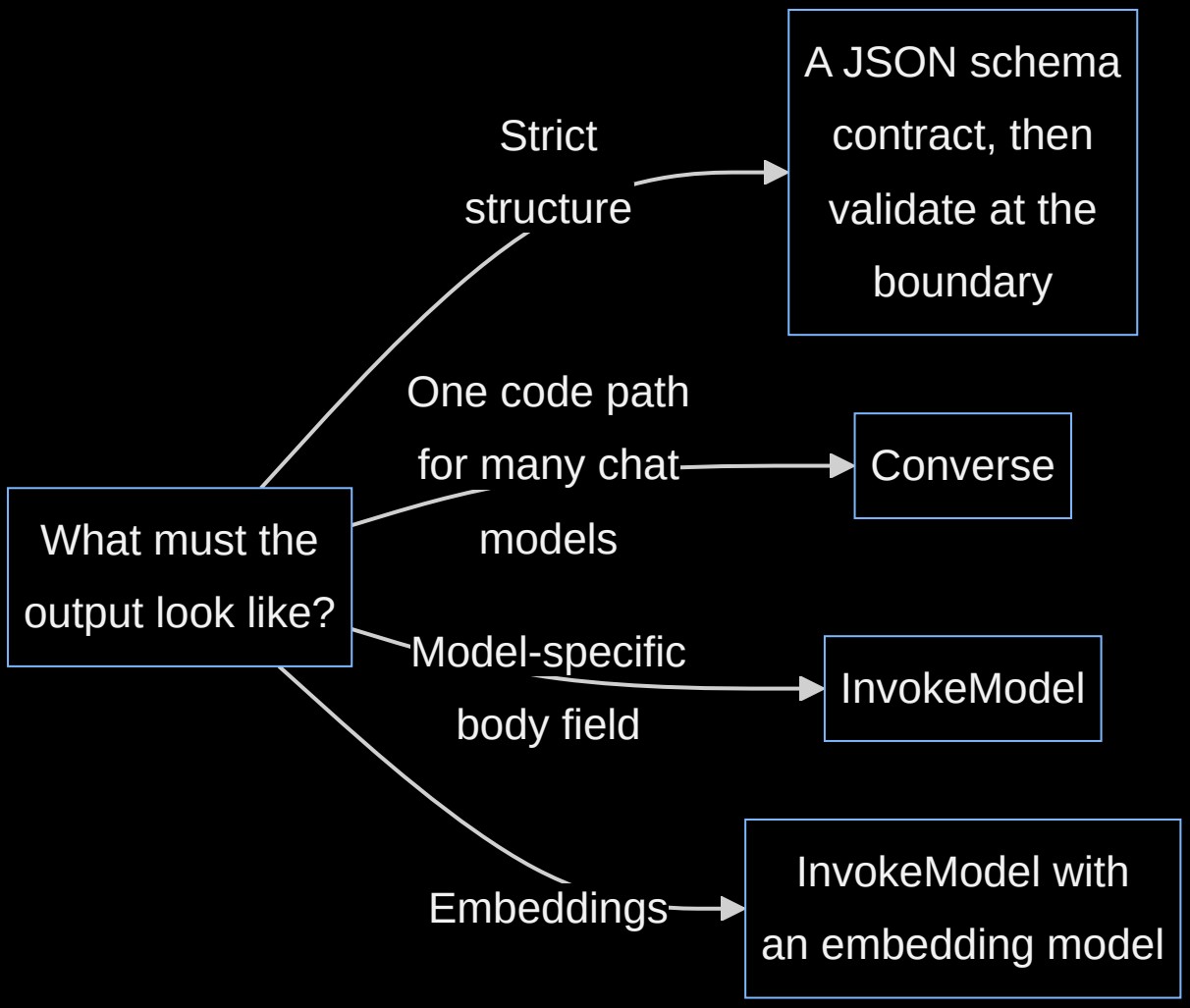


Signal words: wrong chunks, split table, exact code, paraphrase, vocabulary, multi-hop.

Fix retrieval before the generator. A bigger model does not fix broken retrieval.

Trap: temperature does not fix bad retrieval.

K. Behavior, format, and API choice

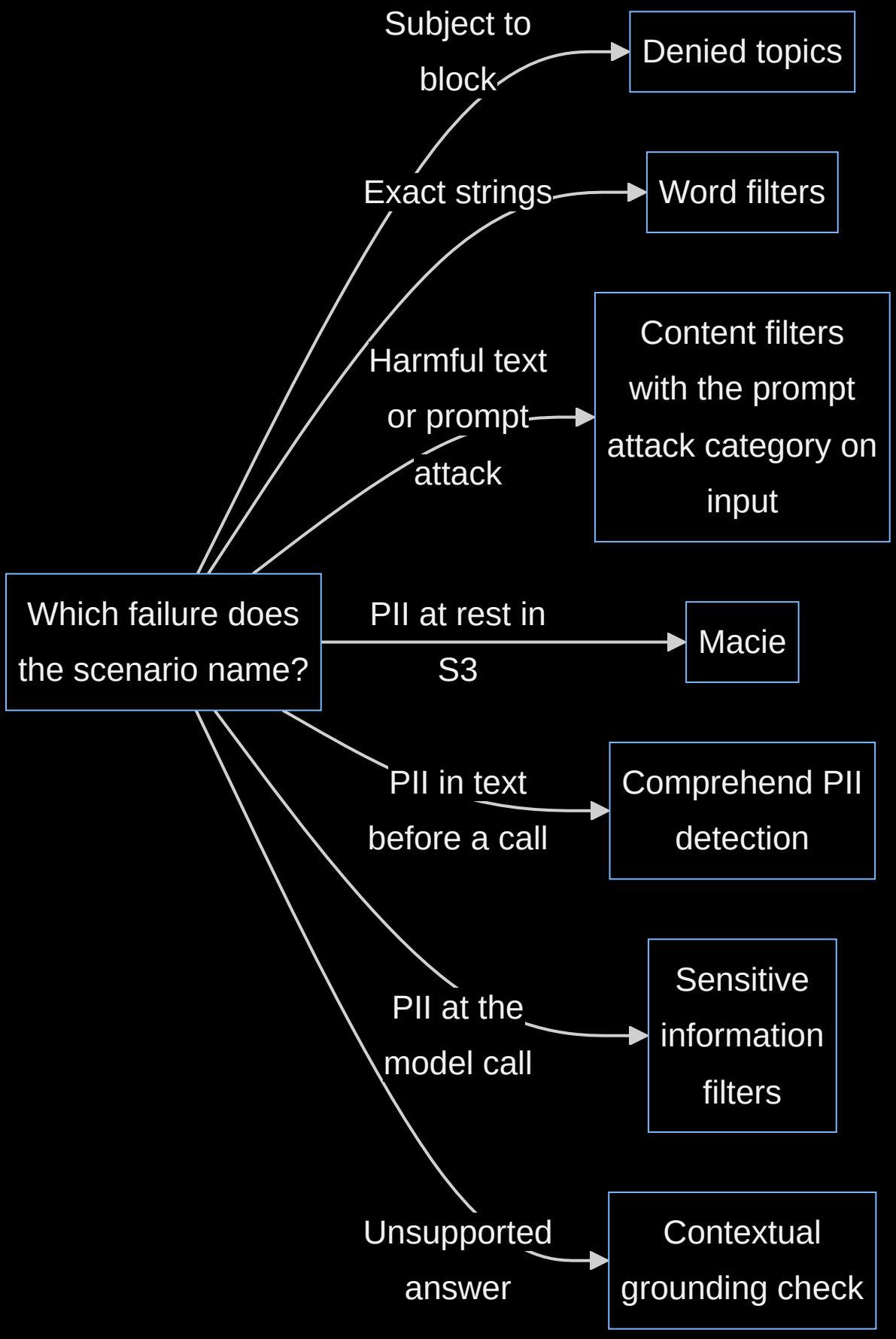


Signal words: schema, valid JSON, tool use, embeddings, provider-specific parameter.

A prompt that asks for JSON is not a contract. A schema is a contract.

Trap: Converse is not the embeddings API.

L. Safety and privacy

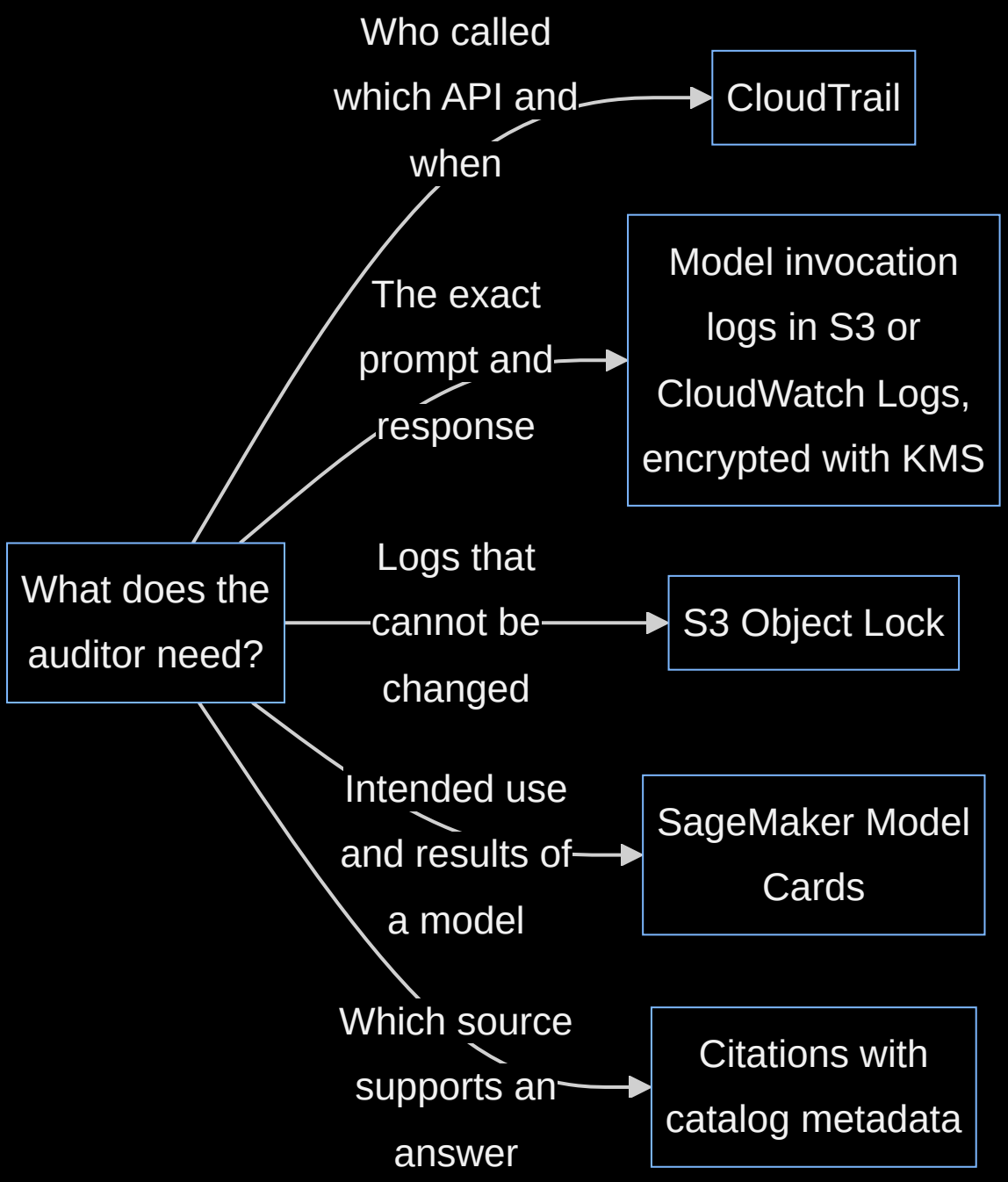


Signal words: harmful, jailbreak, injection, PII, redact, mask, unsupported claim, subject.

Pick the policy for the named failure. Turn on the prompt attack category for input.

Trap: grounding checks test support, not harm. Content filters test harm, not support.

M. Audit and evidence

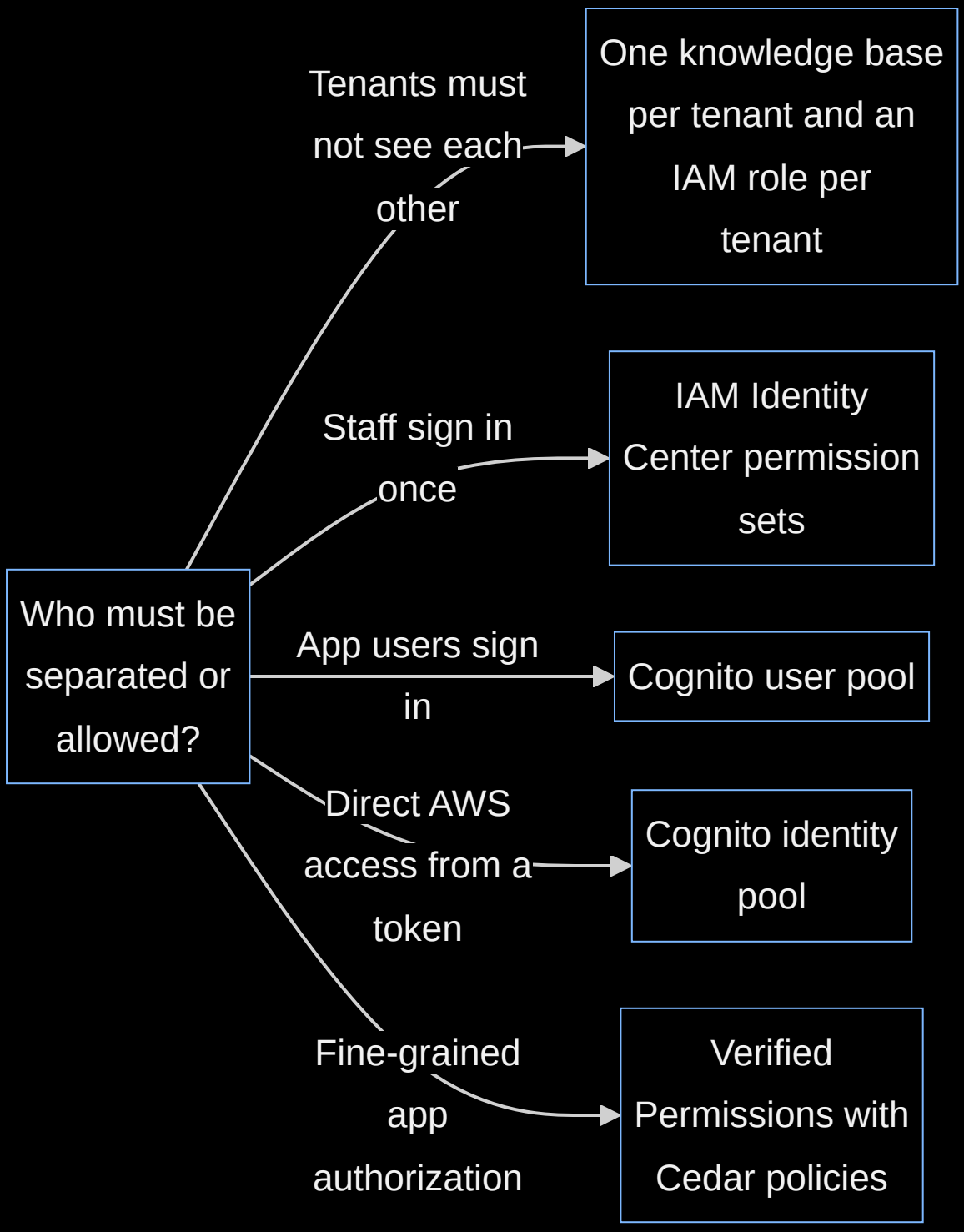


Signal words: audit, evidence, who, when, what was sent, retention, lineage.

CloudTrail answers identity and timing. Invocation logs answer content.

Trap: CloudTrail does not show row content. Masked output does not mask the invocation log.

N. Access control and tenancy

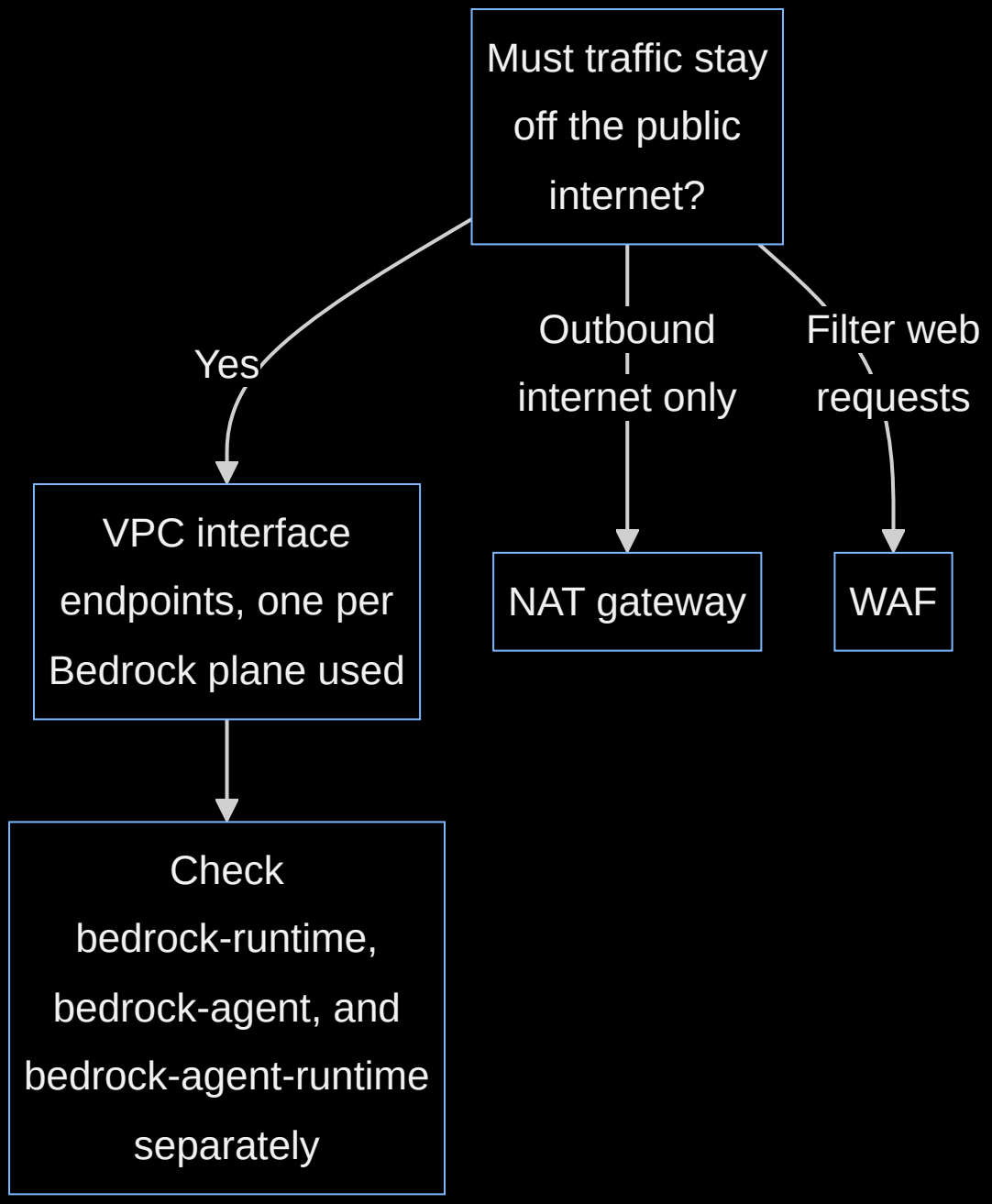


Signal words: tenant, customer, workforce, sign-in, token, least privilege, row or column access.

Isolation comes from the platform. A prompt instruction is not isolation.

Trap: a permission set does not replace the resource policy of a knowledge base or a guardrail.

O. Network path

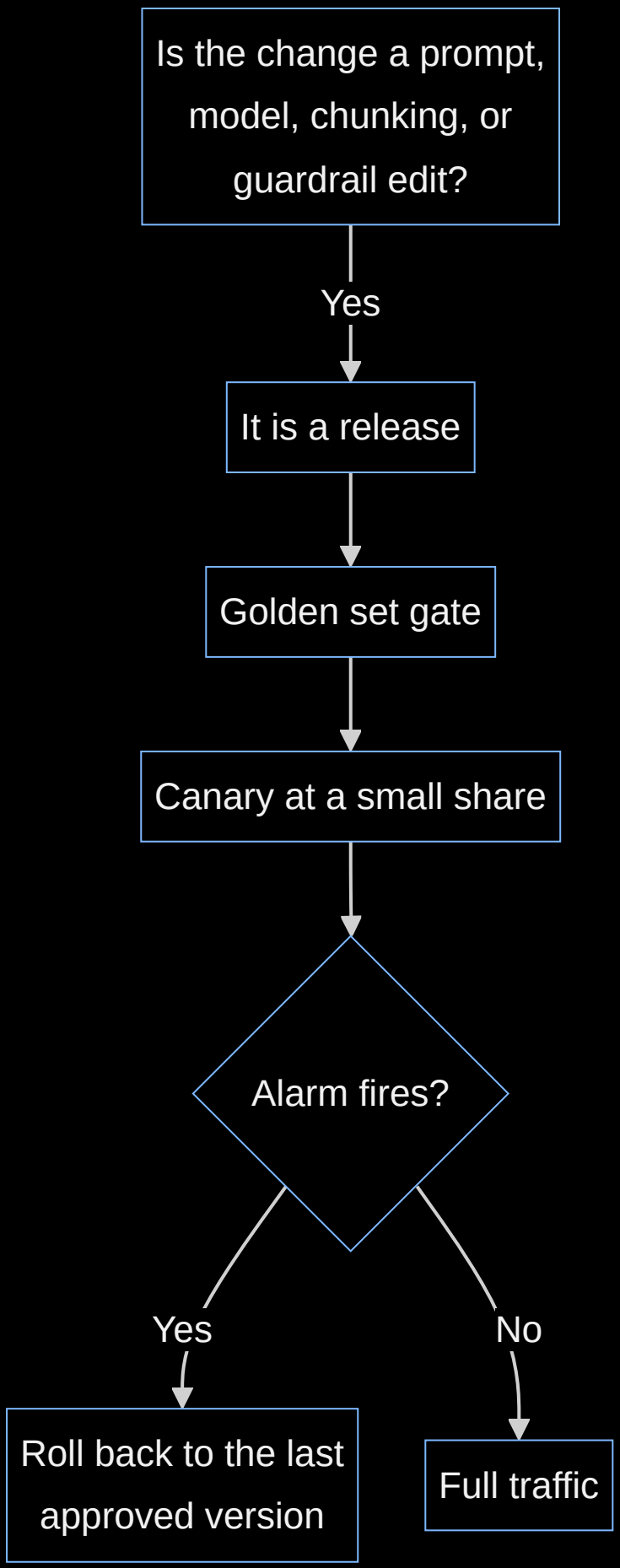


Signal words: private, no public internet, VPC, endpoint, plane.

A NAT gateway is not a private path to Bedrock. Use one endpoint for each plane.

Trap: one runtime endpoint does not cover agent calls.

P. Release and change control

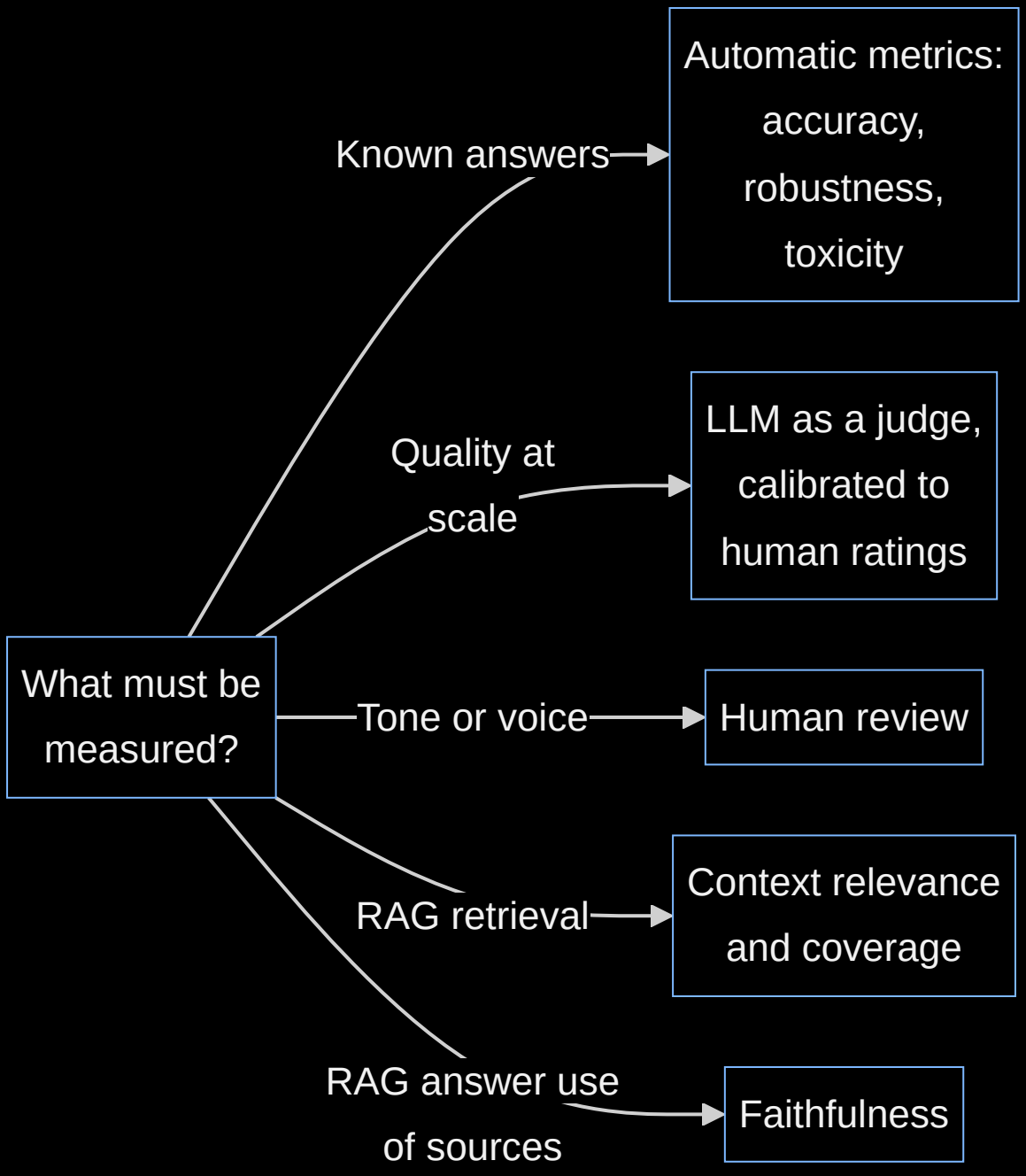


Signal words: release, rollback, canary, version, alias, last approved, gradual.

Freeze versions. Point callers at an alias. Move the alias to release.

Trap: a re-sync is a change and needs the same gate.

Q. Evaluation and troubleshooting

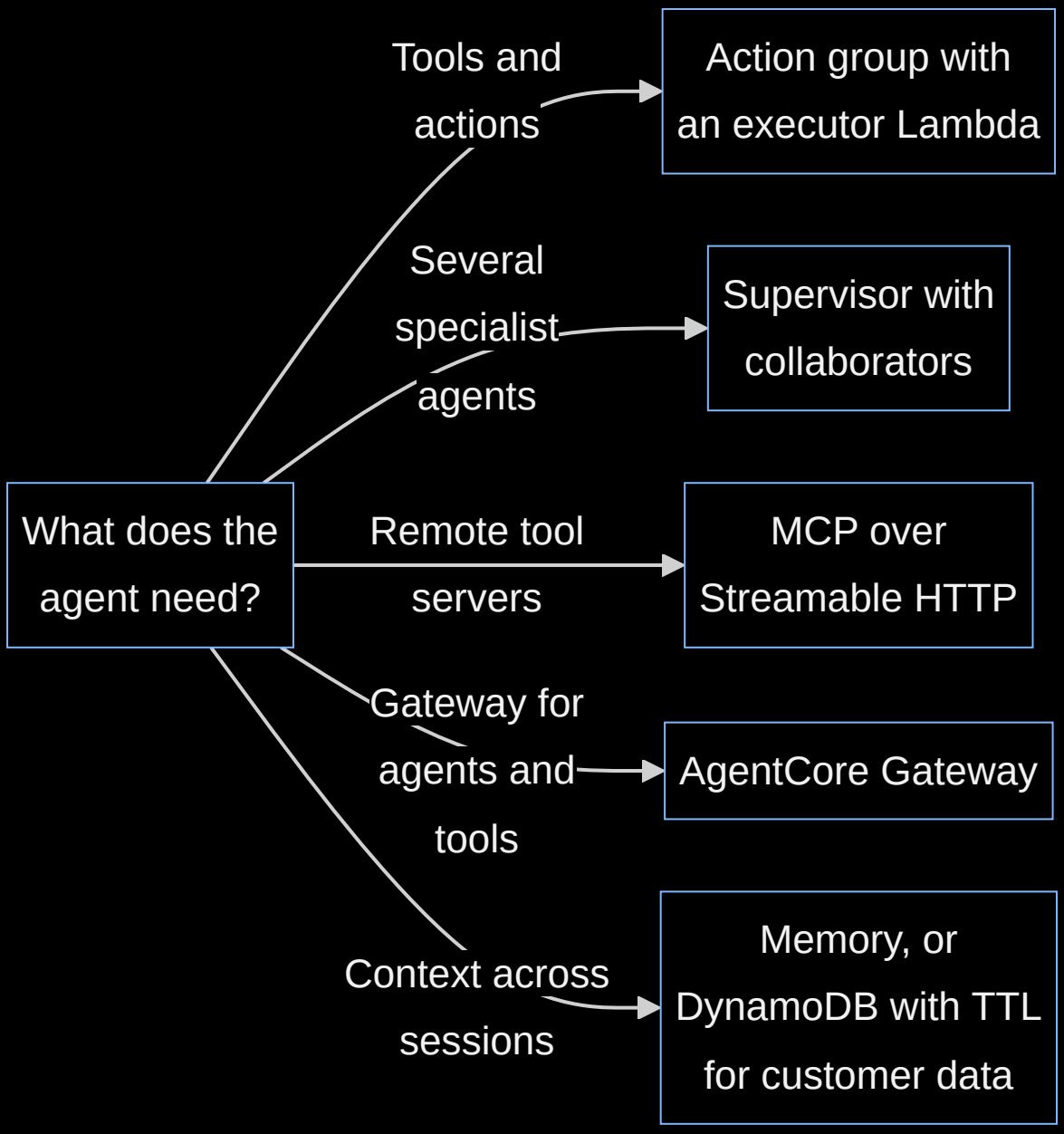


Signal words: measure, score, judge, human, faithful, out of scope, regression.

Measure retrieval and generation separately. Include out-of-scope questions in the golden set.

Trap: an uncalibrated judge gives opinions, not measurements.

R. Agents and tools

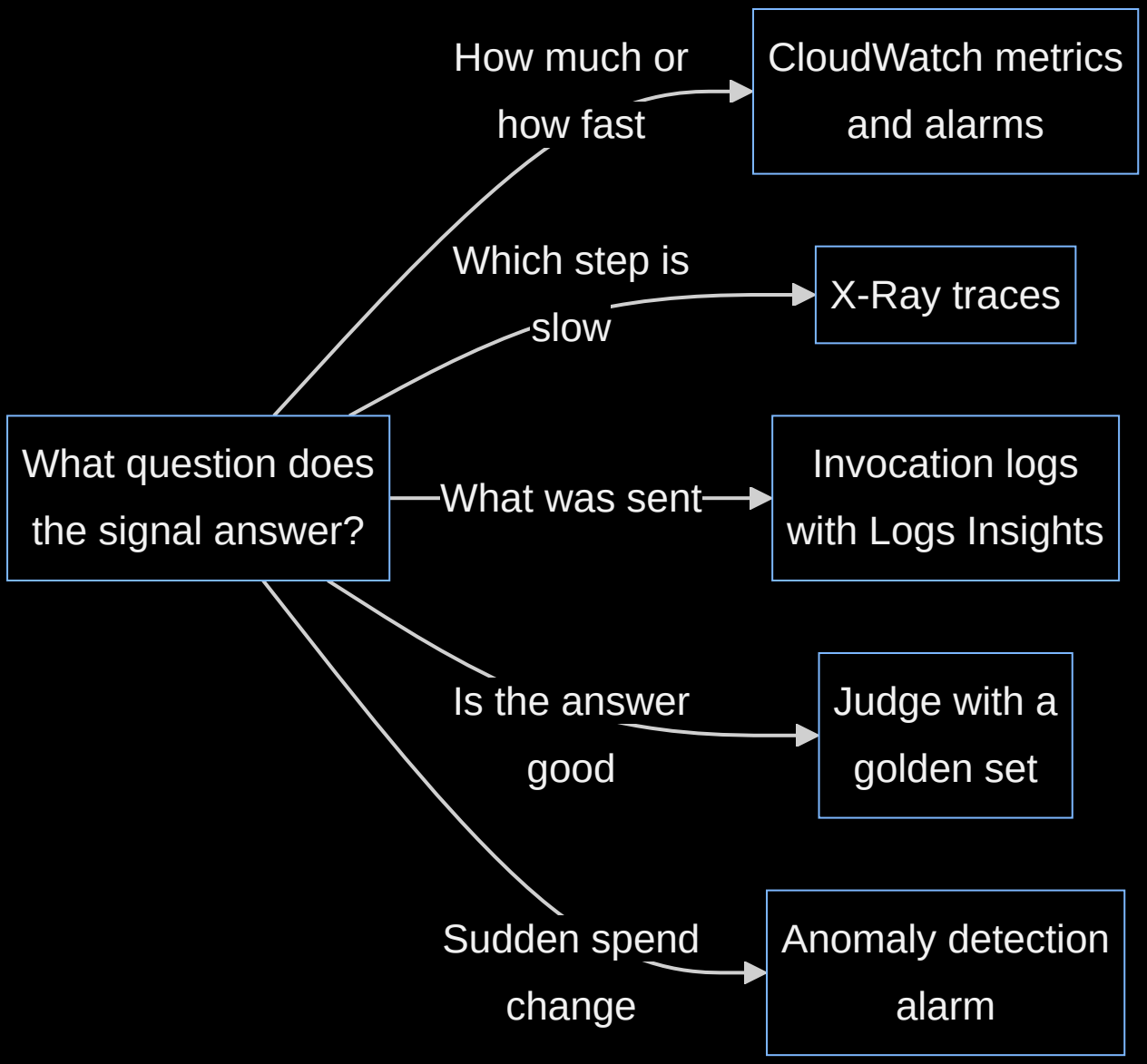


Signal words: act, call an API, specialist, tools, remember, session, customer memory.

The executor holds credentials and filters fields. The model cannot enforce either rule.

Trap: delegation is not an action group. A DRAFT collaborator cannot take delegations.

S. Monitoring signal

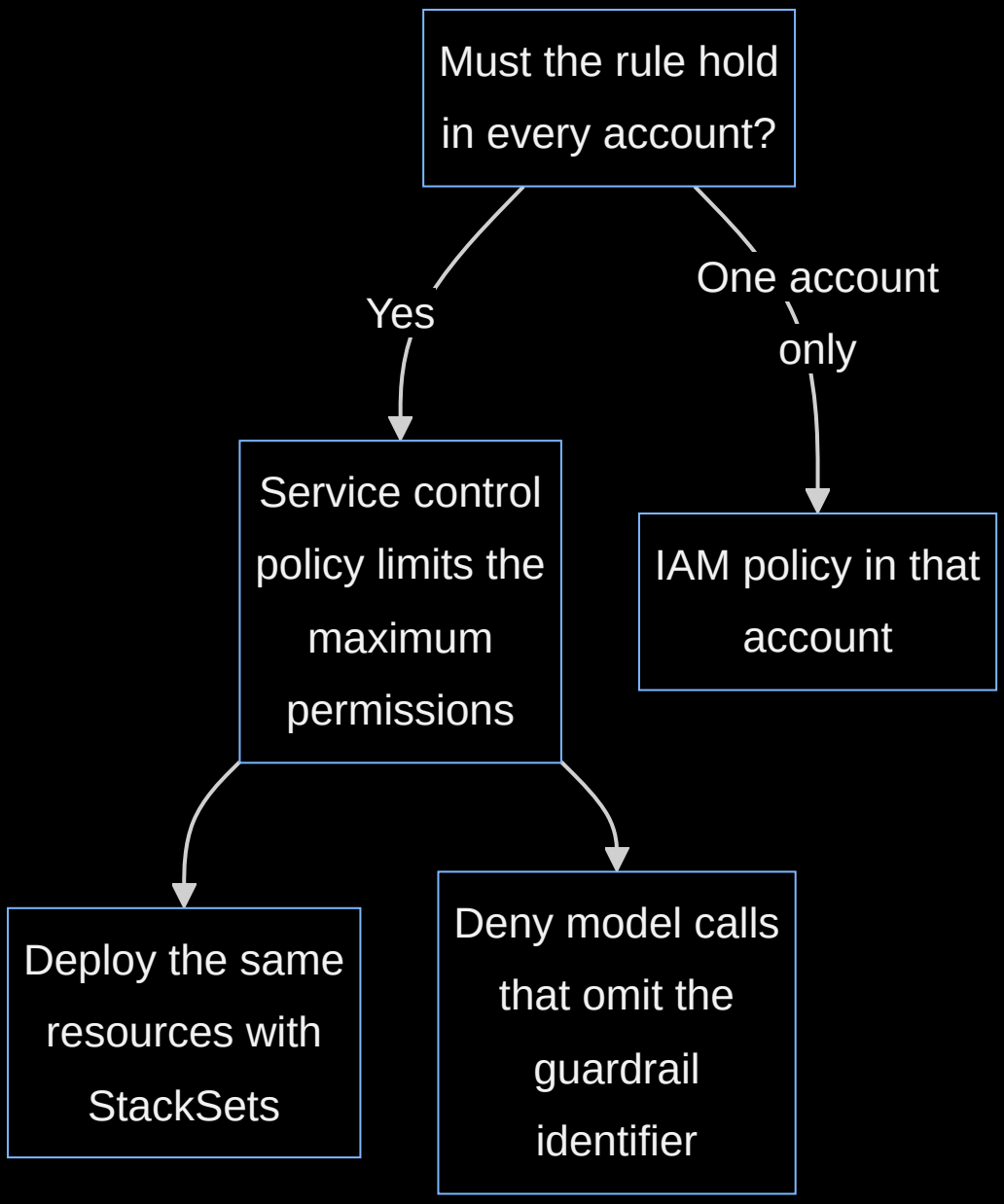


Signal words: slow step, spike, cost trend, content, quality over time.

Metrics count. Traces locate. Logs hold content. Judges score quality.

Trap: Model Monitor watches drift in structured features. It does not score free-text quality.

T. Cross-account governance

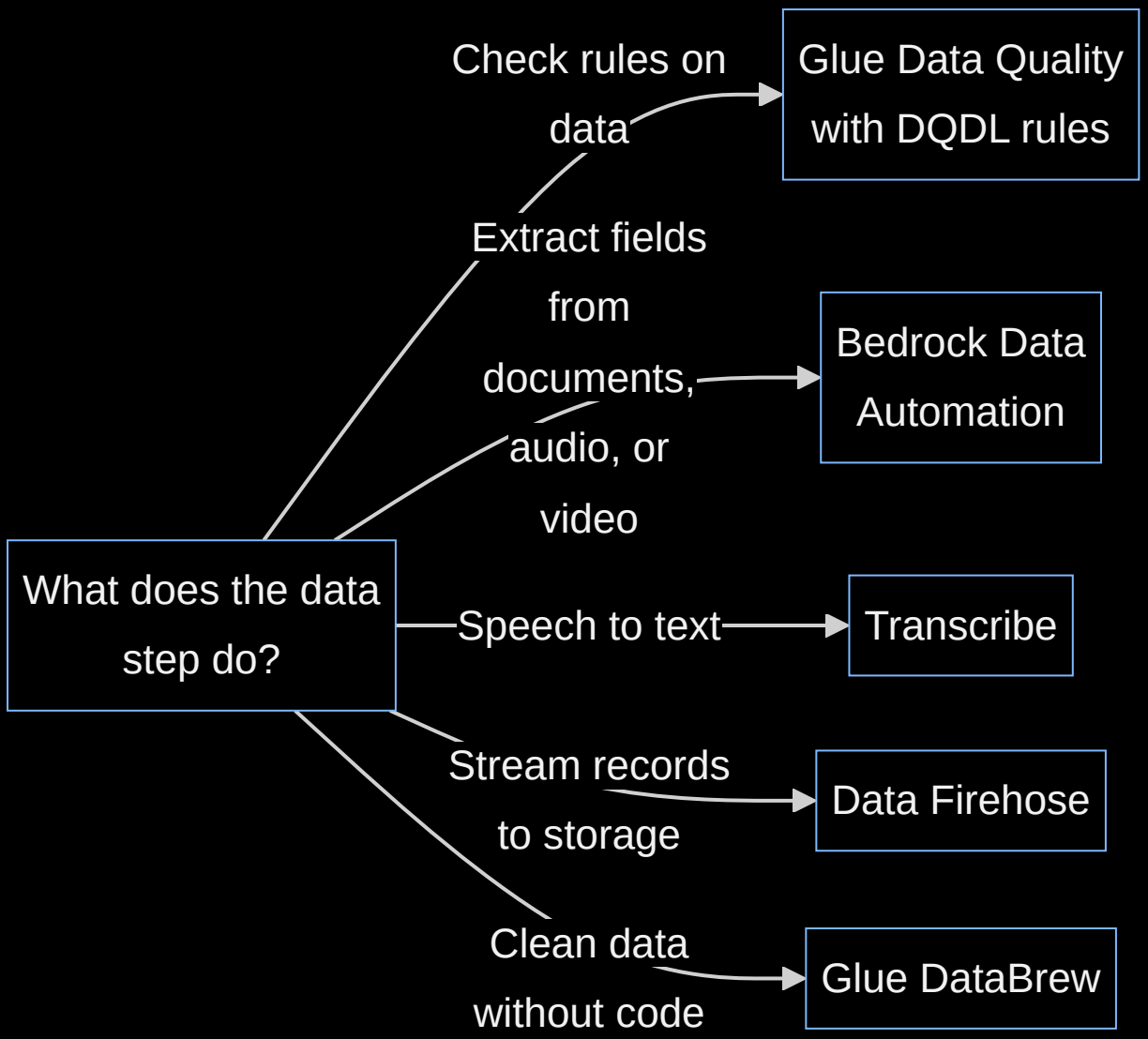


Signal words: every account, organization, central rule, all teams, cannot be skipped.

A rule that the application cannot skip belongs in IAM or an SCP.

Trap: an SCP grants no permission. A user still needs an identity policy that allows the action.

U. Data pipeline and extraction



Signal words: validate, extract fields, transcript, stream, clean, rules.

One tool does one stage. Guardrails do not validate data or extract fields.

Trap: Translate changes language. It does not detect PII.

V. Cost of the wrong fit: the forbidden choices

- A real-time endpoint for offline scoring.
- Fine-tuning for facts that change.
- On-demand for a customized model.
- Batch for a user waiting on a reply.
- A Lambda that waits for a person.
- A guardrail as the only check on data quality.

- Raising a Lambda timeout to fix a gateway timeout.
- A tight retry loop without jitter or a maximum.

Source: repo guide, Parts 1 to 5, the traps list, and the decision units. Points marked Not in source in the parts are not repeated here as facts.